目　录

第1章　异常处理 ……1
1.1　基本异常处理 ……2
1.1.1　Java中的异常 ……2
1.1.2　预定义的异常类 ……12
1.1.3　ArrayIndexOutOfBounds-Exception（另一种阅读顺序） …12
1.2　定义异常类 ……13
1.3　使用异常类 ……21
1.3.1　声明异常（推脱责任） ……21
1.3.2　不需要被捕获的异常 ……24
1.3.3　AssertionError类（选读） ……25
1.3.4　多重抛出和捕获 ……25
1.3.5　finally块（选读） ……31
1.3.6　重新抛出异常（选读） ……32
1.4　图形编程补充（选读） ……43
小结 ……46
自测题答案 ……47
编程项目 ……50

第2章　流和文件I/O ……53
2.1　流和文件I/O概览 ……54
2.1.1　流的概念 ……54
2.1.2　为什么要用文件进行I/O ……54
2.1.3　文本文件和二进制文件的区别 ……54
2.2　文本文件I/O ……55
2.2.1　用PrintWriter进行的文本文件输出 ……55
2.2.2　带有BufferedReader的文本文件输入 ……64
2.2.3　StringTokenizer类 ……70
2.2.4　FileReader和FileOutputStream类 ……73
2.3　File类 ……74
2.4　基本的二进制文件I/O ……77
2.4.1　用ObjectOutputStream向二进制文件输出 ……78
2.4.2　一些有关writeUTF的细节（选读） ……82
2.4.3　用ObjectInputStream从二进制文件读出 ……82
2.4.4　EOFException类 ……87
2.4.5　FileInputStream和FileOutputStream类 ……89
2.5　用对象流进行对象I/O ……93
2.5.1　类对象的二进制I/O ……93
2.5.2　Serializable接口 ……96
2.5.3　二进制文件中的数组对象 ……97
2.6　图形编程补充（选读） ……99
小结 ……104
自测题答案 ……104
编程项目 ……108

第3章　动态数据结构与泛型 ……110
3.1　向量 ……111
3.1.1　使用向量 ……111
3.1.2　参数化的类与泛型 ……120
3.2　链式数据结构 ……121
3.2.1　链表 ……121
3.2.2　内部类 ……129
3.2.3　节点内部类 ……130
3.2.4　迭代器 ……131
3.2.5　对链表的异常处理 ……141
3.2.6　链表的变体 ……144
3.2.7　其他链式数据结构 ……145
3.3　泛型 ……146
小结 ……152
自测题答案 ……152
编程项目 ……159

第4章　递归 ……161
4.1　递归基础 ……161
4.1.1　递归是如何工作的 ……165
4.1.2　递归及迭代定义 ……170
4.1.3　返回值的递归方法 ……171
4.2　用递归编程 ……175

小结 ……185
自测题答案 ……185
编程项目 ……187

第5章 使用Swing的视窗界面 ……189
5.1 背景知识……190
5.1.1 GUI——图形用户界面 ……190
5.1.2 事件驱动编程 ……190
5.2 基本的Swing细节 ……192
5.2.1 更多关于窗口侦听器的内容 ……197
5.2.2 屏幕对象的尺寸单位 ……198
5.2.3 更多关于setVisible的内容 …199
5.2.4 JFrame类的一些方法……207
5.2.5 布局管理器 ……208
5.3 按钮和动作侦听器……214
5.3.1 按钮 ……216
5.3.2 动作侦听器和动作事件 ……217
5.3.3 接口 ……221
5.3.4 模型—视图—控制器模式……223
5.4 容器类……225
5.4.1 JPanel类 ……225
5.4.2 Container类 ……228
5.5 GUI的文本I/O ……231
5.5.1 文本区和文本域 ……231
5.5.2 输入和输出数字 ……238
5.5.3 捕获一个NumberFormat-Exception ……243
小结 ……246
自测题答案 ……246
编程项目 ……251

第6章 applet与HTML ……253
6.1 applet ……254
6.1.1 applet基础……254
6.1.2 运行一个applet……256
6.1.3 向applet添加图标……259
6.2 HTML简介 ……261
6.2.1 HTML基础 ……261
6.2.2 插入超链接 ……263
6.2.3 显示一张图片 ……267
6.3 HTML中的applet ……267
6.3.1 将applet放入HTML文档中 ……268
6.3.2 较老的applet类（选读） ……270
6.3.3 applet与安全……271
小结 ……271
自测题答案 ……271
编程项目 ……272

第7章 更多有关Swing的内容 ……274
7.1 菜单……274
7.1.1 菜单条、菜单和菜单条目 ……275
7.1.2 嵌套式菜单 ……278
7.2 使GUI更漂亮（功能也更强） ……280
7.2.1 添加图标 ……280
7.2.2 向菜单条目添加图标 ……285
7.2.3 滚动条的JScrollPane类 ……286
7.2.4 添加边界 ……290
7.3 更多布局管理器……294
7.3.1 BoxLayout管理器类 ……295
7.3.2 strut与glue ……297
7.3.3 设置组件间的间隔 ……299
7.3.4 Box容器类 ……299
7.3.5 CardLayout管理器 ……302
7.4 内部类……306
7.5 更多关于事件和侦听器的内容……308
7.5.1 WindowListener接口 ……308
7.5.2 更多与更新GUI有关的细节 ……318
7.6 Swing类的层次关系 ……318
小结 ……320
自测题答案 ……321
编程项目 ……323

附录A 关键字（图灵网站下载）
附录B 优先规则（图灵网站下载）
附录C 统一代码（Unicode）字符集（图灵网站下载）
附录D SavitchIn（图灵网站下载）
附录E 保护（protected）修饰符与包（package）修饰符（图灵网站下载）
附录F DecimalFormat类（图灵网站下载）
附录G Iterator接口（图灵网站下载）
附录H 克隆（图灵网站下载）
附录I javadoc（图灵网站下载）
附录J JOptionPane类（图灵网站下载）
附录K C++与Java的区别（图灵网站下载）

第1章

异常处理

异常能证明规则。

——常见格言（可能是类似“异常能够检验规则”的谬传）

写程序的一种方法是首先假设不会发生错误或者不正常。例如，如果程序访问列表的条目，就可能假设该列表不为空。当程序能够在所有事情都按计划发展的核心状况下工作后，再考虑增加代码来处理例外的情况。在Java中，有一种方式反映了这种编程方法。起初，写代码时假设没有什么不寻常的事情会发生。随后，使用Java的异常处理设施为不寻常的情况添加处理代码。异常处理允许将程序或者方法的定义分成一些独立的部分，分别表示正常情况和异常情况，这样就把一个大的编程任务分解为两个更小更容易实现的编程任务。本章将向读者介绍Java中的异常处理。

异常处理最主要的用处可能是处理具有特殊情况的方法，这种特殊情况在方法的不同应用场合有不同的处理方式。方法可能会被很多程序使用，其中有些程序会对这种特殊情况采取一种处理办法，而其他程序会对这种特殊情况采取别的办法。例如，如果方法中有除以0的情况，那么对于某些调用该方法的情形，程序应当结束；而对于其他情形，会有别的处理方式。读者会看到，在这种特殊情况发生时，定义这样的方法来抛出一个异常。这种处置使得这种特殊情况在方法外部才被处理。这样一来，这种特殊情况在不同的调用中就可以做不同的处理。

目标

- 熟悉异常处理的各种记号。
- 学习Java的异常处理语法。
- 培养在自己的程序和类中有效地使用异常处理的技能。

预备知识

除了1.1.3节之外，本章需要读者先读《Java程序设计与问题解决：基础篇》的第1章~第5章以及第7章。学习1.1.3节则需要读者掌握该书第6章有关数组的内容。这一小节可以选读。

（除了1.1.3节之外）1.1节可以在读过该书的第1章~第5章之后阅读，它不需要其中第6章和第7章中讲到的知识。

1.1　基本异常处理

杰克，机灵些，
杰克，快一些，
杰克，跳过那个蜡烛。

——佚名

在Java中，**异常处理**（exception handling）如下进行：Java语言本身或者读者的代码提供一种机制，当有不正常的情况发生时，可以发出信号。这种发出信号的过程被称为**抛出异常**（throwing an exception）。在程序的另一部分——位于一个单独的类或方法中，甚至就在该程序的另外一部分代码中——可以放置代码来处理这种异常情况。这部分代码完成的任务就称为**处理异常**（handling the exception）。当然，我们还是需要详细解释Java中异常处理的各种细节。

1.1.1　Java中的异常

绝大多数小程序基本上用不到异常处理，因此异常处理不容易在这样的代码中出现。为了给出一个简单的例子，我们将用一个没什么价值的程序作为第一个示例。首先不使用Java的异常处理设施来写这个程序，然后再用异常处理重写它。

在本例中，假设牛奶是社会中的一种重要物资，人们几乎不能面对它断货的情况，不过我们还是要让程序适应极少见的牛奶短缺的情况。下面假设我们不会用完牛奶，基本代码如下所示：

```
System.out.println("Enter number of donuts:");
donutCount = keyboard.nextInt();

System.out.println("Enter number of glasses of milk:");
milkCount = keyboard.nextInt();

donutsPerGlass = donutCount/(double)milkCount;
System.out.println(donutCount + " donuts.");
System.out.println(milkCount + " glasses of milk.");
System.out.println("You have " + donutsPerGlass
                        + " donuts for each glass of milk.");
```

如果没有牛奶了（也就是说输入的牛奶的瓶数是0），这段代码会导致除以0，这是一个错误。为了妥善处理牛奶短缺的罕见情况，可以为这种异常的情形添加一个测试。图1-1中展示了增加测试此种特殊情况后的代码的完整程序。现在我们来看如何用Java的异常处理设施来重写这个程序。

```
import java.util.*;

public class GotMilk
{
    public static void main(String[] args)
    {
```

图1-1　处理问题情形的一种方法

```
int donutCount, milkCount;
double donutsPerGlass;
Scanner keyboard = new Scanner(System.in);

System.out.println("Enter number of donuts:");
donutCount = keyboard.nextInt();

System.out.println("Enter number of glasses of milk:");
milkCount = keyboard.nextInt();

if (milkCount < 1)
{
    System.out.println("No Milk!");
    System.out.println("Go buy some milk.");
}
else
{
    donutsPerGlass = donutCount/(double)milkCount;
    System.out.println(donutCount + " donuts.");
    System.out.println(milkCount + " glasses of milk.");
    System.out.println("You have " + donutsPerGlass
                + " donuts for each glass of milk.");
}
System.out.println("End of program.");
        System.out.println("End of program.");
    }
}
```

屏幕对话示例

```
Enter number of donuts:
2
Enter number of glasses of milk:
0
No Milk!
Go buy some milk.
End of program.
```

图1-1 （续）

在图1-2中，我们使用异常重写了图1-1中的程序。因为这个例子太简单，在此情况下，读者可能不愿意使用异常。但通过这个例子可以学习Java的异常处理语法。尽管整个程序没有因为使用异常而更加简单，但是在关键字try和catch之间的部分比原先的程序更简洁，这暗示了使用异常的好处。

让我们看一看try和catch之间的代码。这段代码基本上和图1-1中的代码相同，只是原先很庞杂的if-else语句（在图1-1中被高亮显示）被下面的短小的if语句所替代：

```
if (milkCount < 1)
    throw new Exception("Exception: No Milk!");
```

```
import java.util.*;

public class ExceptionDemo
{
    public static void main(String[] args)
    {
        int donutCount, milkCount;
        double donutsPerGlass;
        Scanner keyboard = new Scanner(System.in);

        try
        {
            System.out.println("Enter number of donuts:");
            donutCount = keyboard.nextInt();

            System.out.println("Enter number of glasses of milk:");
            milkCount = keyboard.nextInt();

            if (milkCount < 1)
               throw new Exception("Exception: No Milk!");

            donutsPerGlass = donutCount/(double)milkCount;
            System.out.println(donutCount + " donuts.");
            System.out.println(milkCount + " glasses of milk.");
            System.out.println("You have " + donutsPerGlass
                        + " donuts for each glass of milk.");
        }
        catch(Exception e)
        {
            System.out.println(e.getMessage());
            System.out.println("Go buy some milk.");
        }
        System.out.println("End of program.");
    }
}
```

这个程序只是用于学习异常处理基本语法的一个简单例子。

try块

throw语句

catch块

屏幕对话示例 1

```
Enter number of donuts:
3
Enter number of glasses of milk:
2
3 donuts.
2 glasses of milk.
You have 1.5 donuts for each glass of milk.
End of program.
```

屏幕对话示例 2

```
Enter number of donuts:
2
Enter number of glasses of milk:
0
Exception: No Milk!
Go buy some milk.
End of program.
```

图1-2　异常处理示例

这个if语句的意思是说，如果没有牛奶，程序应当处理这个异常。这个处理在关键字catch之后给出。其思想是正常的情况由关键字try之后的代码来处理，关键字catch之后的代码只用于处理异常的情况。下面我们来看看具体细节。

Java中的基本异常处理方法由try-throw-catch构成。try块的语法是

```
try
{
   需要尝试的代码
}
```

这个try块中含有基本算法的代码，告知计算机当所有事情都很顺利时该做些什么。它之所以被称为try块，是因为我们不能百分之百保证万事如意，但要尝试一下。

如果有些事情出了差错，就需要抛出一个异常，用以说明某些事情出问题了。因此当我们加上throw语句后，基本的框架就像下面这样：

```
try
{
   需要尝试的代码
   可能抛出的异常
   更多的代码
}
```

下面是从图1-2中摘取的代码，这是含有throw语句的try块的例子。

```
try
{
   System.out.println("Enter number of donuts:");
   donutCount = keyboard.nextInt();

   System.out.println("Enter number of glasses of milk:");
   milkCount = keyboard.nextInt();

   if (milkCount < 1)
     throw new Exception("Exception: No Milk!");

   donutsPerGlass = donutCount/(double)milkCount;
   System.out.println(donutCount + " donuts.");
   System.out.println(milkCount + " glasses of milk.");
   System.out.println("You have " + donutsPerGlass
                      + " donuts for each glass of milk.");
}
```

Exception是个预定义的类。throw语句（如下所示）创建了一个新的Exception类的对象，并把它抛出：

```
throw new Exception("Exception: No Milk!");
```

当有异常抛出时，try块中的代码停止执行，而另一块代码，即catch块开始执行。catch块的执行被称为捕获异常。当异常被抛出后，它最终会被某个catch块捕获。在图1-2中，相应的catch块紧随try块之后，其代码如下：

```
catch(Exception e)
{
    System.out.println(e.getMessage());
    System.out.println("Go buy some milk.");
}
```

快速参考：throw语句

语法：

```
throw new Exception_Class_Name(Possibly_Some_Arguments);
```

当throw语句执行时，包含它的那个try块停止执行，然后控制通常会转到一个catch块。这个catch块中的代码会随后执行。本章中后面标题为“try-throw-catch”的快速参考中有更详细的介绍。

举例：

```
throw new Exception("Unexpected End of Input.");
```

---

catch块看起来有点像有一个Exception类型参数的方法定义。尽管并不是一个方法定义，catch块在某些方面还是像一个方法。它是单独的一部分代码，当程序在之前的try块中遇到（并执行）下面的语句时，它就会被执行：

```
throw new Exception(Possibly_Some_Arguments);
```

throw语句类似一个方法调用，但它并没有去调用一个方法，而是调用了catch块，并要求执行其中的代码。

（catch块中的）下面代码行中的标识符e是什么呢？

```
catch(Exception e)
```

它看起来像一个参数并且其作用也确实很像一个参数。因此，我们称这个e为catch块参数。（但要记住，catch块并不是一个方法。）catch块参数及其之前的类名有两个作用：

（1）catch块参数之前的类名规定了这个catch块可以捕获什么样的异常。

（2）catch块参数为所捕获的异常起了一个名字，以便可以在这个catch块中写代码来操作这个异常对象。

我们将按照相反的次序来讨论catch块参数的这两个作用。在本小节中，我们讨论使用catch块参数作为所捕获异常的对象名字。在后面的1.3.4节里，将讨论哪个异常对象会匹配哪个catch块。目前的例子中只有一个catch块，所以该使用哪个catch块是毫无疑问的。catch块参数最常用的名字就是e，但也可以用任何合法的标识符来替代e。

现在来看看图1-2中的catch块是如何工作的。为了理解它的工作原理，还需要看看try块中的throw语句，摘录如下：

```
throw new Exception("Exception: No Milk!");
```

这个throw语句嵌在一个if语句中，但就我们的目的而言，我们不需要知道它是如何被执行的。只是想演示当它被执行时会发生什么。当执行这个throw语句时，一个新的Exception类的对象由下面的代码创建出来：

```
new Exception("Exception: No Milk!");
```

字符串“Exception: No Milk!”是Exception类的构造器的一个实参。这个new创建出的Exception对象，在其对象的实例变量中保存了这个字符串，因此它可以在catch块中被恢复出来。关键字throw表明这个new创建出的对象被抛出了。当某个对象被抛出时，try块中的代码就停止执行，控制传递给该try块之后的catch块（或多个这样的块）。下面的catch块来自图1-2：

```
catch(Exception e)
{
    System.out.println(e.getMessage());
```

```
        System.out.println("Go buy some milk.");
    }
```

当有异常抛出时，这个异常必须是Exception类型才能用于上面这个特定的catch块。不过，读者将会看到，所有异常都是Exception类型的，所以这个catch块能捕获任何异常。所抛出的对象被代入catch块参数e，然后catch块中的代码开始执行。因此在这个例子中，可以把e看成是下面的throw语句抛出的那个异常：

```
throw new Exception("Exception: No Milk!");
```

每个异常对象都有一个名为getMessage的方法，除非另外专门提供了代码，这个方法得到的是在抛出异常时构造器传递给该对象的那个字符串。在这个例子里，e.getMessage()返回"Exception: No Milk!"。因此，当图1-2中的catch块执行时，会在屏幕上写下这样的内容：

```
Exception : No Milk!
Go buy some milk.
```

图1-3展示的是图1-2程序的控制流。该图的第一部分假设用户输入的牛奶杯数是正数，所以没有抛出异常（如图1-2的第一个对话示例所示）。第二部分假设用户输入的牛奶杯数是零或者负数，这么一来，就抛出了一个异常（如图1-2的第二个对话示例）。

```
import java.util.*;

public class ExceptionDemo
{
    public static void main(String[] args)
    {
        int donutCount, milkCount;
        double donutsPerGlass;
        Scanner keyboard = new Scanner(System.in);

        try
        {
            System.out.println("Enter number of donuts:");
            donutCount = keyboard.nextInt();

            System.out.println("Enter number of glasses of milk:");
            milkCount = keyboard.nextInt();

            if (milkCount < 1)
                throw new Exception("Exception: No Milk!");

            donutsPerGlass = donutCount/(double)milkCount;
            System.out.println(donutCount + " donuts.");
            System.out.println(milkCount + " glasses of milk.");
            System.out.println("You have " + donutsPerGlass
                               + " donuts for each glass of milk.");

        }
        catch(Exception e)
        {
            System.out.println(e.getMessage());
            System.out.println("Go buy some milk.");
        }
        System.out.println("End of program.");
    }
```

这些箭头假设用户输入的牛奶杯数是正数。

milkCount是正数，因此这里不会抛出任何异常。

这些代码不会执行。

图1-3 异常处理的控制流

```
    }
}

import java.util.*;

public class ExceptionDemo
{
    public static void main(String[] args)
    {
        int donutCount, milkCount;
        double donutsPerGlass;
        Scanner keyboard = new Scanner(System.in);

        try
        {
            System.out.println("Enter number of donuts:");
            donutCount = keyboard.nextInt();

            System.out.println("Enter number of glasses of milk:");
            milkCount = keyboard.nextInt();

            if (milkCount < 1)
               throw new Exception("Exception: No Milk!");

            donutsPerGlass = donutCount/(double)milkCount;
            System.out.println(donutCount + " donuts.");
            System.out.println(milkCount + " glasses of milk.");
            System.out.println("You have " + donutsPerGlass
                               + " donuts for each glass of milk.");

        }
        catch(Exception e)
        {
            System.out.println(e.getMessage());
            System.out.println("Go buy some milk.");
        }
        System.out.println("End of program.");
    }
}
```

这些箭头假设用户输入的牛奶杯数是零，因此会抛出一个异常。

milkCount是零或负数，因此这里会抛出一个异常。

这些代码不会执行。

图1-3 （续）

总之，try块中假定有一部分代码包含throw语句。这条语句通常在有异常情况的时候才执行，但当它执行时，会抛出某种异常类的异常对象。（到目前为止，我们只讨论了一种异常类Exception。）当异常抛出时，try块的执行结束，块中剩下的代码被略过，控制转到合适的catch块中。catch块只对它紧随的try块起作用。若有异常抛出，异常对象被代入catch块参数，然后该块中的语句开始执行。当catch块中的代码执行完成时，程序继续处理该块后面的代码，不再返回到那个try块。因此，try块中在抛出异常之后的代码都不会执行。（以后我们会讨论若没有合适的catch块，将发生些什么。）

现在来看看当try块中没有异常抛出时会发生什么。如果try块正常执行且没有抛出异常，那么当try块执行完后，程序继续执行catch块之后的代码。换句话说，当没有异常抛出时，

catch块就被忽略。

这种解释使得try-throw-catch组合看起来和if-else语句等价。除了抛出异常时所带的消息之外，它们几乎相同。因此，try-throw-catch组合等价于if-else语句附加上往其中一个分支发送消息的能力。这个不同点可能看起来不是很显著，但是读者将会看到，因为具有能够发送消息的能力，try-throw-catch组合比if-else语句通用得多。

快速参考：catch块参数

catch块参数是catch块头部的标识符，作为可能被抛出的一个异常对象的占位符。当前面的try块中有（能匹配的）异常抛出时，那个异常对象就代换了这个catch块参数。catch块参数通常用标识符e表示，不过也可以用任何合法的（非关键字）标识符表示。

语法：

```
catch(Exception_Class_Name Catch_Block_Parameter)
{
    <如果try块中抛出的异常对象的类型是上面命名的异常类，这里的代码就执行>
}
```

*Catch_Block_Parameter*可以用任何合法的标识符表示。

举例：

在下面的代码中，e就是catch块参数：

```
catch(Exception e)
{
    System.out.println(e.getMessage());
    System.out.println("Go buy some milk.");
}
```

快速参考：getMessage方法

每个异常对象都有一个String类型的实例变量，其中包含了某些消息，一般用来表示异常的原因。例如，如果异常是像下面这样抛出的，那么作为Exception构造器实参的字符串（Quoted_String_Argument）就是这个String类型的实例变量的值：

```
throw new Exception(Quoted_String_Argument);
```

如果对象被称为e，则方法调用e.getMessage()就返回这个字符串。

举例：

假设下面的throw语句在一个try块中执行：

```
throw new Exception("Input must be positive.");
```

同时假设下面的catch块紧随那个try块：

```
catch(Exception e)
{
    System.out.println(e.getMessage());
    System.out.println("Program aborted.");
    System.exit(0);
}
```

在这个例子中，方法调用e.getMessage()返回下面的字符串：

```
"Input must be positive."
```

记住：一个异常就是一个对象

下面的代码是一个throw语句：

```
throw new Exception("Illegal character on line 57.");
```

这样的throw语句通常内嵌在分支语句中，但在简单语句中使用也是合法的。一个throw语句不只是说明了某个做过的动作，然后就忘了它。它创建了一个包含消息的对象。（在这个例子中，这个消息就是"Illegal character on line 57."。）这个对象及其消息可以供catch块使用，因此抛出异常的作用就比仅仅把控制转移到一个catch块的第一条语句要大。

为了说明在throw语句中确实创建了实实在在的对象，下面的语句，尽管不大可能实际用到，却是合法的。

```
Exception e = new Exception("Illegal character on line 57.");
```

其中的new Exception("Illegal character on line 57.")部分是对Exception类的构造器的调用，它创建了一个Exception类的对象。在之前的那个throw语句中，new Exception("Illegal character on line 57.")也是对Exception类的构造器的调用，因此它也创建了一个Exception类的对象。

快速参考：try-throw-catch

当try、throw、catch共同使用的时候，就构成了抛出和捕获异常的基本机制。throws语句抛出异常，catch块捕获异常。当异常抛出时，try块结束，然后开始执行catch块中的代码。当这个catch块执行完后，catch块（可能有一系列）后面的代码开始执行（只要这个catch块不执行结束程序或者其他特殊的任务）。

若try块中没有异常抛出，那么当try块执行完成后，程序继续执行catch块之后的代码。（换句话说，如果没有异常抛出，catch块就被忽略。）

语法：

```
try
{
    某些语句
    <或者是有些带有throw语句的代码，或者是调用了可能抛出异常的方法。>
    某些语句
}
catch(Exception_Class_Name Catch_Block_Parameter)
{
    <如果try块中抛出了这里命名的异常类的对象，就执行这里的代码。>
}
```

*Catch_Block_Parameter*可以用任何合法的标识符，通常用e。catch块中的代码可以引用*Catch_Block_Parameter*。如果有显式的throw语句，通常会嵌入到一个if语句或者if-else语句中。可以有任意多个throw语句，也可以调用任意多个可能抛出异常的方法。每个catch块只能列出一个异常，但是可以有多个catch块。

举例：

具体示例见图1-2。

自测题

1. 下面的代码的输出结果是什么？

```
int waitTime = 46;

try
{
   System.out.println("Try block entered.");
   if (waitTime > 30)
       throw new Exception("Time Limit Exceeded.");
   System.out.println("Leaving try block.");
}
catch (Exception e)
{
   System.out.println("Exception: " + e.getMessage());
}
System.out.println("After catch block");
```

2. 如果把第1题中的代码：

   ```
   int waitTime = 46;
   ```

   改成

   ```
   int waitTime = 12;
   ```

   那么输出结果又是什么？

3. 异常是什么？它是标识符么？还是变量？抑或是方法？甚或是对象？是类么？或者是其他的什么？

4. 下面的代码合法么？

   ```
   Exception myException = new Exception("Hi Mom!");
   ```

5. 图1-2的代码中，如果把如下的

   ```
   if (milkCount < 1)
       throw new Exception("Exception: No Milk!");
   ```

   替换为

   ```
   if (milkCount < 1)
   {
       Exception e = new Exception("Exception: No Milk!");
       throw e;
   }
   ```

   程序的运行过程将有何改变？

6. 第1题的代码中，throw语句是什么？

7. throw语句执行时，会发生什么？请进行一般性的讨论，不限于第1题的代码或其他某些代码。

8. 第1题的代码中，try块是什么？

9. 第1题的代码中，catch块是什么？

10. 第1题的代码中，catch块参数是什么？

11. 如果第1题中的catch块改成下面这样：

    ```
    catch(Exception messenger)
    {
        System.out.println("Exception: "+ messenger.getMessage());
    }
    ```

    代码的行为是否会有不同？

12. 写一条语句，使其当一个字符串型的变量status的值是"bad"时，抛出异常。这条语句应当在try块内部，但不需要写出整个try块。这个异常应当是Exception类的对象，并且用getMessage恢复出的字符串应当是"Exception thrown: Bad Status."，同样不需要把catch块写出来。

1.1.2　预定义的异常类

当学习预定义类的方法时，有时会了解到它们可能会抛出某些特定类型的异常。这些异常是预定义的异常类。如果使用这些方法，可以把方法调用放在try块中，随后用catch块捕获这些异常。预定义的异常类的名字设计得足够明了，某些预定义的异常类示例如下：

```
IOException
ClassNotFoundException
FileNotFoundException
```

当捕获到这些预定义的异常类对象的一个异常时，getMessage方法返回的字符串通常可以提供足够的信息来定位异常的来源。因此，如果你写一个叫SampleClass的类，其中有个方法叫doStuff，该方法抛出IOException类的异常对象，就可以按下面这样来调用它：

```
SampleClass object = new SampleClass();
try
{
    <可能有一些代码>
    object.doStuff(); //可能抛出IOException
    <可能还有一些代码>
}
catch(IOException e)
{
    <处理异常的代码，可能包括下面这样的语句：>
    System.out.println(e.getMessage());
}
```

如果觉得这个异常已经使得程序不可能继续运行，那么可以在catch块中调用System.exit来结束程序，如下所示：

```
catch(IOException e)
{
    System.out.println(e.getmessage());
    System.out.println("Program aborted");
    System.exit(0);
}
```

预定义的异常类Exception是所有异常类的祖先。每个异常类都是Exception类的子孙[①]（也就是说，它直接从Exception继承，或者从Exception类的派生类继承，或者来自更长的派生链而其源头始于Exception类）。读者可以使用Exception类本身，就像在图1-2中的那样，不过读者可能更愿意用它来定义派生类。

快速参考：Exception类

每个异常类都是Exception类的子孙，可以在类和程序中直接使用Exception类本身，不过读者可能更愿意用它来定义派生类。

1.1.3　ArrayIndexOutOfBoundsException（另一种阅读顺序）

在阅读本节之前，读者应当阅读过《Java程序设计与问题解决：基础篇》中第6章的某些部分，以了解数组。如果已经读过第6章的那些部分，本节就不应该是可选部分，应当是必读的。

如果程序试图使用一个越界的数组索引，一个ArrayIndexOutOfBoundsException异常就

① 如果还没有阅读《Java程序设计与问题解决：基础篇》中的第7章，可以略过提到子孙类和派生类的部分。

会被抛出，除非在一个catch块中把这个异常捕获，否则程序就会结束。这是一类特殊的异常，不需要捕获或者用其他方式去关注，通常也不在catch块中去捕获它。这种异常说明代码中有（逻辑）错误，需要改正代码，而不是捕获异常。因此，ArrayIndexOutOfBoundsException通常更多起到运行时错误声明的功能，而不是异常的功能。

自测题

13. 下面的语句合法吗？

```
IOException sos = new IOException("Hello Houston!");
```

14. 下面的语句合法吗？

```
IOException sos = new IOException("Hello Houston!");
throw sos;
```

1.2 定义异常类

这次我要破例了。

——我的妈妈

在本节中，我们要学习如何定义自己的异常类，并讲述何时抛出、如何抛出以及捕获异常。

定义自己的异常类

可以自行定义异常类，但是这些类必须是某些已定义的异常类的派生类。异常类可以是预定义的异常类的派生类或者其他已成功定义的异常类的派生类。示例将是Exception类的派生类。

在定义异常类时，构造器是最重要的，并且通常是仅有的不是从基类中继承的方法。例如，在图1-4中，已经定义了一个叫做DivideByZeroException的异常类，它仅有的方法包括一个默认的构造器以及有一个String类型参数的构造器。就我们的目的来说，需要定义的就是这些了。然而，这个类确实从Exception类[①]继承了其全部方法，尤其是继承了getMessage方法，该方法返回一个字符串消息。在默认构造器中，这个字符串参数用下面的语句设置，这是默认构造器的定义的第一行：

```
public class DivideByZeroException extends Exception
{
    public DivideByZeroException()
    {
        super("Dividing by Zero!");
    }
    public DivideByZeroException(String message)
    {
        super(message);
    }
}
```

在异常类的构造器中可以做更多事情，但这种形式是最常见的。

super是对基类Exception的构造器的调用。

图1-4 程序员定义的异常类

① 有些程序员喜欢从预定义的ArithmeticException类派生出DivideByZeroException，但是这会使之成为那一类不需要被代码捕获的异常类，因此就无法依靠编译器的帮助来追踪未捕获的异常。更详细的说明见1.3.2节。如果读者不理解这个脚注，可以放心地略过。这主要是为了给严谨的专家看的。

```
super("Dividing by Zero!");
```

这行代码是对基类Exception的构造器的调用。正如我们已经注意到的，当我们传递一个字符串给Exception类的构造器时，Exception类设置了一个String类型的实例变量的值，以后可以调用getMessage方法来恢复这个字符串。getMessage方法是Exception类一个普通的访问方法。DivideByZeroException类继承了这个String实例变量，以及这个访问方法getMessage。

例如，在图1-5中，给出了一个使用这个异常类的示例程序。这个异常是用默认构造器抛出的，代码如下：

```
throw new DivideByZeroException();
```

这个异常被图1-5中显示的catch块捕获。研究一下这个catch块中的这行代码：

```
System.out.println(e.getMessage());
```

它向屏幕产生了如下输出，如屏幕对话示例2和示例3中所记录的：

```
Dividing by Zero!
```

```
import java.util.*;

public class DivideByZeroExceptionDemo
{
    private int numerator;
    private int denominator;
    private double quotient;

    public static void main(String[] args)
    {
        DivideByZeroExceptionDemo oneTime =
                              new DivideByZeroExceptionDemo();
        oneTime.doIt();
    }

    public void doIt()
    {
        try
        {
            System.out.println("Enter numerator:");
            Scanner keyboard = new Scanner(System.in);
            numerator = keyboard.nextInt();
            System.out.println("Enter denominator:");
            denominator = keyboard.nextInt();

            if (denominator == 0)
                throw new DivideByZeroException();

            quotient = numerator/(double)denominator;
            System.out.println(numerator + "/"
                               + denominator
                               + " = " + quotient);
        }
        catch(DivideByZeroException e)
        {
            System.out.println(e.getMessage());
            secondChance();
        }

        System.out.println("End of Program.");
    }
```

我们将在本章后面给出本程序的一个改进版本。

图1-5　使用程序员定义的异常类

```
public void secondChance()
{
    System.out.println("Try again:");
    System.out.println("Enter numerator:");
    Scanner keyboard = new Scanner(System.in);

    numerator = keyboard.nextInt();
    System.out.println("Enter denominator:");
    System.out.println("Be sure the denominator is not zero.");
    denominator = keyboard.nextInt();

    if (denominator == 0)
    {
        System.out.println("I cannot do division by zero.");
        System.out.println("Since I cannot do what you want,");
        System.out.println("the program will now end.");
        System.exit(0);
    }

    quotient = ((double)numerator)/denominator;
    System.out.println(numerator + "/"+ denominator+ " = " + quotient);
  }
}
```

有时候采用不抛出异常的方式来处理意外情况会更好。

屏幕对话示例 1

```
Enter numerator:
5
Enter denominator:
10
5/10=0.5
End of Program.
```

屏幕对话示例 2

```
Enter numerator:
5
Enter denominator:
0
Dividing by Zero!
Try again.
Enter numerator:
5
Enter denominator :
Be sure the denominator is not zero.
10
5/10=0.5
End of Program.
```

图1-5 （续）

屏幕对话示例3

```
Enter numerator:
5
Enter denominator:
0
Dividing by Zero!
Try again.
Enter numerator:
5
Enter denominator:
Be sure the denominator is not zero.
0
I cannot do division by zero.
Since I cannot do what you want,
the program will now end.
```

图1-5　（续）

图1-4中的DivideByZeroException类定义的第二个构造器有一个String类型的参数。这个构造器允许读者抛出异常时任意设定自己的消息。如果图1-5中的throw语句使用下面这个字符串参数：

```
throw new DivideByZeroException("Oops. Shouldn't Have Used Zero.");
```

那么在屏幕对话示例2和3中，下面的语句：

```
System.out.println(e.getMessage());
```

将在屏幕上产生如下输出：

```
Oops. Shouldn't Have Used Zero.
```

请注意，在图1-5中，try块是程序在正常情况下的执行部分。如果一切顺利，它是唯一需要被执行的代码，对话将和屏幕对话示例1类似。在意外情况下，用户输入0作为分母，异常被抛出并随后被catch块捕获。catch块把异常的消息输出，然后调用secondChance方法。这个方法再给用户一次机会来输入正确的数值，然后进行计算。（如果用户第二次仍然要除以0，该方法就结束程序。）secondChance方法只是为了意外情况而存在的。我们把处理除以0这种意外情况的代码分离出来，并放在单独的方法中，这样就不会把正常情况的代码弄乱了。

□ Java提示：在定义异常类时，保留getMessage不变

对于所有预定义的异常类，getMessage将返回传递给构造器的那个字符串参数（如果构造器没有参数，则返回默认的字符串）。例如，若异常像下面的代码这样抛出，那么"This is a big exception!"将作为那个字符串类型的实例变量的值：

```
throw new Exception("This is a big exception!");
```

若这个异常对象被命名为e，那么e.getMessage()返回"This is a big exception!"。

在自定义的任何异常类中保留getMessage方法是一种好习惯。例如，假设定义了一个名为MySpecialException的异常类，像下面的代码这样抛出异常：

```
throw new MySpecialException("Wow, what an exception!");
```

若这个抛出的异常对象被命名为e，那么e.getMessage()应当返回"Wow, what an exception!"。为了确保自己定义的这个异常类也有这样的功能，一定要加上一个有字符串参数的构造器，以调用super开头，如下面的构造器所示：

```
public MySpecialException(String message)
{
    super(message);
    // 这里还可以有更多的代码，不过通常不需要。
}
```

对super的调用是对基类的构造器的调用。如果基类的构造器正确地处理了这个消息，那么这样定义的类也就能够正确处理。

读者还应当为每个异常类加入默认构造器。这个默认的构造器应当设置getMessage能取得的默认值。它也应当以对super的调用开头，如下面的构造器所示：

```
public MySpecialException()
{
    super("MySpecialException thrown.");
    // 这里还可以有更多的代码，不过通常不需要。
}
```

如果基类的getMessage像我们描述的这样工作的话，新定义的异常类的这种默认构造器就能正确工作。

记住：Exception对象的特征

异常对象最重要的两个特征是它的类型以及用String类型的实例变量承载的消息。这个消息字符串可以通过访问方法getMessage来恢复。可以通过这个字符串让异常对象传送消息，以便catch块可以用到这个消息。(即将在后面的小节解释为什么异常对象的类型很重要。)

○ 编程提示：何时需要定义异常类

一般性的准则是，如果要在自己的代码中插入throw语句，最好定义自己的异常类。这样，当自己的代码捕获一个异常时，多个catch块可以区分出自己的异常和预定义的类中的方法抛出的异常。例如，在图1-5中，使用了异常类DivideByZeroException，它是在图1-4中定义的。

尽管不是个好主意，读者可能还是很想用预定义的Exception类来抛出图1-5中的异常，如下所示：

```
throw new Exception("Dividing by Zero!");
```

然后读者就会用catch块来捕获它：

```
catch(Exception e)
{
    System.out.println(e.getMessage());
    secondChance();
}
```

虽然这种方法对图1-5中的程序能奏效，但它不是最佳技术，因为那个catch块将捕获任何异常，比如IOException。而IOException可能需要和secondChance不同的处理方法。与其使用Exception类来捕获除以0的这种情况，不如用自定义的更专用的DivideByZeroException来得更好，正如在图1-5中所作的那样。

快速参考：程序员自定义的异常类

读者可以定义自己的异常类，但每个这样的异常类都必须是某个已经存在的异常类（预定义的或者是读者已成功定义的）的派生类。

指导原则：

- 除非必须使用其他类作为基类，否则使用Exception类作为基类。
- 应当至少定义两个构造器，后面的几条中会描述。自定义的异常类应当继承getMessage。通常不需要添加其他方法，但增加其他方法也是合法的。
- 每个构造器的开头都应该调用基类的构造器，如下所示：

```
super("Tidal Wave Exception thrown!");
```

- 应当定义默认的构造器，在这种情况下调用super时应当提供一个字符串参数，告知这是什么样的异常。以后可以用getMessage方法恢复这个字符串。
- 还应当定义有一个字符串参数的构造器，在这种情况下，这个字符串应当作为调用super时的参数。这样，这个字符串就能通过调用getMessage方法恢复出来。

举例：

```
public class TidalWaveException extends Exception
{
    public TidalWaveException()
    {
        super("Tidal Wave Exception thrown!");
    }
    public TidalWaveException(String message)
    {
        super (message);
    }
}
```

super是对基类Exception的构造器的调用。

自测题

15. 定义一个名为CoreBreachException的异常类。该类应当有个无参数的构造器。如果用这个无参数的构造器来抛出一个异常，getMessage应当返回"Core Breach! Evacuate Ship!"。该类还应当有个只有一个String类型参数的构造器。如果用这个构造器抛出一个异常，那么getMessage返回传递给构造器作为参数的那个值。
16. 定义一个名为MessageTooLongException的异常类。该类应当有个无参数的构造器。如果用这个无参数的构造器来抛出一个异常，getMessage应当返回"Message Too Long!"。该类还应当有个只有一个String类型参数的构造器。如果用这个构造器抛出一个异常，那么getMessage返回传递给构造器作为参数的那个值。
17. 假设异常类ExerciseException如下这样定义：

```
public class ExerciseException extends Exception
{
    public ExerciseException()
    {
        super("Exercise Exception thrown!");
        System.out.println("Exception thrown.");
    }
```

```
    public ExerciseException(String message)
    {
        super(message);
        System.out.println(
          "ExerciseException invoked with an argument.");
    }
}
```

下面的代码将产生什么样的输出？（它只是个练习，不太可能会出现在实际的程序中。）

```
ExerciseException e = new ExerciseException("Do Be Do");
System.out.println(e.getMessage());
```

18. 假设异常类CrazyException如下这样定义：

```
public class CrazyException extends Exception
{
    public CrazyException()
    {
        super("Crazy Exception thrown!");
        System.out.println("Wow, Crazy exception thrown!!");
    }

    public CrazyException(String message)
    {
        super(message);
        System.out.println(
          "Wow,Crazy exception thrown with an argument!");
    }

    public void crazyMethod()
    {
        System.out.println("Message is " + getMessage());
    }
}
```

下面的代码将产生什么样的输出？（它只是个练习，不太可能会出现在实际的程序中。）

```
CrazyException exceptionObject = new CrazyException();
System.out.println(exceptionObject.getMessage());
exceptionObject.crazyMethod();
```

19. 假设异常类DoubleException(实际上不太可能出现)如下这样定义：

```
public class DoubleException extends Exception
{
    public DoubleException()
    {
        super("Double Exception thrown!");
    }

    public DoubleException(String message)
    {
        super(message + message);
    }
}
```

下面的代码将产生什么样的输出？（它只是个练习，不太可能会出现在实际的程序中。）

```
int number;
try
{
    System.out.println("try block entered.");
    number = 42;
    if (number > 0)
        throw new DoubleException("Double Exception thrown!");
    System.out.println("Leaving try block.");
}
catch(DoubleException exceptionObject)
{
    System.out.println(exceptionObject.getMessage());
}
System.out.println("End of code.");
```

20. 假设在自测题19中，catch块修改成下面这样：

```
catch(Exception exceptionObject)
{
    System.out.println(exceptionObject.getMessage());
}
```

（DoubleException类型被换成Exception类型。）这个改变将会怎样影响输出？

21. 假设在自测题19中，如下这行：

```
number = 42;
```

改为

```
number = -99;
```

这个改变将会怎样影响输出？

22. 尽管异常类通常只带有一个字符串消息，也可以定义异常类承载任何类型的消息。例如，下面的类型的对象就能承载一个int型的“消息”（和字符串一道）：

```
public class IntException extends Exception
{
    private int intMessage;
    public IntException()
    {
        super("IntException thrown!");
    }

    public IntException(String message)
    {
        super(message + message);
    }

    public IntException(int number)
    {
        super("IntException thrown!");
        intMessage = number;
    }

    public int getNumber()
    {
        return intMessage;
    }
}
```

下面的代码将产生什么样的输出？（它只是个练习，不太可能会出现在实际的程序中。）

```
IntException e = new IntException(42);
System.out.println(e.getNumber());
System.out.println(e.getMessage());
```

23. 能不能定义一个异常类，它派生自预定义的IOException类？或者是否自定义的异常类必须从Exception类派生？

1.3 使用异常类

buck 名词 扑克游戏时庄家面前放的小标志物。
（引申）pass the buck（意味着）推脱责任（to）。

——《袖珍牛津当代英语词典》，牛津大学出版社，1986

推诿到此为止。

——杜鲁门当总统时书桌上的座右铭

本节讨论在程序和方法中使用异常的技术。

1.3.1 声明异常（推脱责任）

有时候，推迟处理异常是有道理的。例如，可能有个方法，其代码会对尝试除以0的情况抛出异常，但可能不想在这个方法中捕获这个异常。或许某个使用这个方法的程序在这个异常抛出时应当简单地结束，而其他用此方法的程序会有别的处理方式。因此，若在该方法内部捕获这个异常，就不知道应该如何来处理它。在这种状况下，明智的做法是：不在该方法的定义里捕获这个异常，而应让任何使用这个方法的程序（或其他代码）把对这个方法的调用放在一个try块中，然后在随后的catch块中捕获此异常。

如果某方法没把一个异常捕获，它至少应该发出警告，表明任何对这个方法的调用可能会有异常抛出。这种警告在这里就是一个throws子句。例如，一个可能抛出DivideByZeroException并且没有捕获它的方法可以类似下面这样声明头部：

```
public void sampleMethod() throws DivideByZeroException
```

throws DivideByZeroException部分是throws子句，说明对sampleMethod方法的调用可能会抛出一个DivideByZeroException。

调用方法时可能会抛出的大部分异常都必须用下面两种方式之一说明：

（1）可能的异常可以被该方法定义中的一个catch块捕获。

（2）可能的异常可以用一个带有这个异常类名的throws子句在方法定义的起始部分声明（以便让这个方法的使用者在使用该方法时去操心如何处理这个异常）。

在任何方法中，都可以混合使用这两种方式，即捕获某些异常和用throws子句声明其他异常。

读者已经知晓第一种技术，也就是用catch块处理异常。第二种技术是一种“推脱责任”。例如，假设如下的methodA有个throws子句：

```
public void methodA() throws DivideByZeroException
```

在这种情况下，methodA就免除了捕获在其执行期间所抛出的任何DivideByZeroException

类型的异常的责任。可是，如果有个methodB中包含对methodA的调用，则methodB必须处理这个异常。当methodA添加throws子句时，相当于“告诉”methodB，“如果调用我，就必须处理我抛出的任何DivideByZeroException”。总之，methodA把任何DivideByZero-Exception类型的异常的处理责任推脱（“踢皮球”）给了任何调用它的方法。

当然了，如果methodA通过包含比如

```
throws DivideByZeroException
```

这样的throws子句把皮球踢给了methodB，那么methodB也可以同样在其定义中包含这个子句，再把皮球踢给任何调用它的方法。但在一个书写规范的程序中，每个抛出的异常最终都会被某个不再踢皮球的方法里的catch块捕获。

记住：抛出异常会使方法结束

如果一个方法抛出了异常，且这个异常没有在此方法中被捕获，则当异常抛出后，该方法的调用立即结束。

在图1-6中，我们重写了图1-5中的程序，在正常情况下调用normal方法。main方法中包含了对normal方法的调用且把调用放在一个try块中。因为normal方法会抛出DivideByZeroException的异常且自己没有捕获，需要把这种情况用一个throws子句在normal方法定义的开头加以声明。如果我们这样安排程序，一切顺利的情况就被独立出来。它甚至没有被try块和catch块搅浑。

```
import java.util.*;

public class DoDivision
{
   private int numerator;
   private int denominator;
   private double quotient;

   public static void main(String[] args)
   {
        DoDivision doIt = new DoDivision();

        try
        {
           doIt.normal();
        }
        catch(DivideByZeroException e)
        {
            System.out.println(e.getMessage());
            doIt.secondChance();
        }

        System.out.println("End of Program.");
   }

   public void normal() throws DivideByZeroException
```

图1-6　使用throws子句推脱责任

```
{
        System.out.println("Enter numerator:");
        Scanner keyboard = new Scanner(System.in);
        numerator = keyboard.nextInt();

        System.out.println("Enter denominator:");
        denominator = keyboard.nextInt();
        if (denominator == 0)
            throw new DivideByZeroException();
        quotient = numerator/(double)denominator;
        System.out.println(numerator + "/"
                                + denominator + " = " + quotient);
}

public void secondChance()
{
        System.out.println("Try Again:");
        Scanner keyboard = new Scanner(System.in);
        System.out.println("Enter numerator:");
        numerator = keyboard.nextInt();
        System.out.println("Enter denominator:");
        System.out.println("Be sure the denominator is not zero.");
        denominator = keyboard.nextInt();

        if (denominator == 0)
        {
            System.out.println("I cannot do division by zero.");
            System.out.println("Since I cannot do what you want,");
            System.out.println("the program will now end.");
            System.exit(0);
        }

        quotient = ((double)numerator)/denominator;
        System.out.println(numerator + "/"
                                + denominator + " = " + quotient);
    }
}
```

屏幕对话示例

输入/输出对话与图1-5中的程序相同。

图1-6 （续）

一个throws子句可以包含多个异常类型，用逗号把各个异常类型隔开即可，如下面的例子所示：

```
public int superMethod()
                    throws IOException, DivideByZeroException
```

快速参考：throws子句

如果定义了一个可能抛出某个特定异常类的方法，则通常或者这个方法定义中必须有一个catch块可以捕获这个异常，或者必须用一个throws子句声明（也就是列出）这个异常，如下所示。

语法（覆盖了通常大部分情况）：

public *Type_Or_void Method*(*Parameter_List*) throws *List_Of_Exceptions*
Body_Of_Method

举例：

```
public void methodA(int n) throws IOException, MyException
{
    .
    .
    .
}
```

常见问题：如果某个异常始终没有被捕获，将会发生什么？

如果（包括main的）每个方法都对某个异常仅仅用throws子句声明，则可能会发生这种异常被抛出但是没有被捕获的情况。此时，这个程序会结束或者表现变得不可靠。对于我们在图形编程补充之外看到的程序来说，如果有异常抛出但没有被捕获，则程序会结束。（对GUI程序——我们在图形编程补充以及第5章中讨论的那种带有交互式窗口的程序——来说，如果有异常抛出但是没有被捕获，则程序可能不结束，但是程序的行为从此之后就不可靠了。）

1.3.2　不需要被捕获的异常

到目前为止，我们一直说在大部分情况下，异常必须被catch块捕获或者被throws子句声明。这是基本的规则，但是这个规则也有例外。（关于异常的规则也碰到了反常情况，听起来蛮有道理。）有些特殊的异常不需要如此处理（尽管读者如果愿意，还是可以用catch块将其捕获）。它们是由某些类型的错误导致的一些基本的异常类。通常，不需要为这些异常写出throws语句，它们一般是被预定义的类中的方法抛出的。

那些Error类或RuntimeException类的子孙类异常不需要被catch块捕获或被throws子句声明。这种异常的一个例子是NoSuchMethodError。这种异常发生在代码去调用一个方法，但是没有找到与这个方法名对应的定义时。对这类异常，应当修改代码来排除错误，而不是添加catch块。其他不需要捕获或者声明的异常都是那些或多或少无法控制的异常。这种异常的一个例子是OutOfMemoryError。如果这个异常被抛出，程序已经耗完了内存，就意味着或者这个程序的内存效率太低，或者必须为计算机买更大内存了。在这种情况下，加上catch块没有什么用处，因此并不需要这样做[①]。

如何才能知道一个异常必须被捕获或者用throws子句声明呢？如果是预定义的类，可以查看它的文档。如果它是RunTimeException类或Error类的子孙，则不需要用catch块或

① 从技术上来说，Error类及其子孙类不是异常类，因为它们不是Exception类的子孙。但是，这些类的对象可以被抛出以及被catch块捕获，就像真正的异常一样。因此对我们来说，它们看起来就像是异常。

throws子句去处理它。对于其他情况，必须用catch块捕获它或者用throws子句列出它。不过，读者一般不需要费神去检查文档。

在大多数情况下，必须捕获或用throws子句声明通过自己的代码显式地抛出的异常，例如下面这样的：

```
if (denominator == 0)
    throw new DivideByZeroException();
```

另外，如果某个方法在其throws子句中列出了一个异常，且代码中有对该方法的调用，则代码必须捕获此异常或者在throws子句中声明它。

不必为是否需要在throws子句中声明异常操心太多。如果Java语言要求考虑某些异常而读者没有去做，编译器会负责告知，这样，读者就可以去捕获它或者把它加入到throws子句了。

快速参考：派生类的throws子句的细节

如果在派生类中重定义某方法，则重定义的方法的throws子句中不能包含基类中同一个方法的throws子句中没有列出的异常。换句话说，不能在重定义方法时，往throws子句中添加异常。当然，这就意味着不能抛出没有被捕获且没有被基类中相同方法的throws子句列出的异常。但是可以在重定义的方法的throws子句中减少声明的异常。

1.3.3 AssertionError类（选读）

在《Java程序设计与问题解决：基础篇》的第4章中讨论断言检查以及assert操作符时，提到过如果程序中有断言检查并且检查失败，程序将会打印出错消息然后结束。这个论断在一定程度上是对的，但是不够完全。实际情况是AssertionError类的异常被抛出。如果它没有被某个catch块捕获，程序就会打印错误消息然后结束。但是，如果愿意，可以用catch块将其捕获，尽管这种做法不太常见。

正如它的名字所暗示的，AssertionError类是Error类的派生类，因此不必捕获它或用throws子句声明它。

1.3.4 多重抛出和捕获

一个try块中可以抛出多个异常，并且类型各异。每个catch块只能捕获一种类型的异常，但是可以在try块后放置多个catch块来捕获多种类型的异常。例如，图1-7中的程序在try块之后有两个catch块。（NegativeNumberException类在图1-8中定义。）

```
import java.util.*;

public class TwoCatchesDemo
{
   public static void main(String[] args)
   {
      try
      {
```

这只是一个示范例子，仅供学习异常处理的基本语法。

图1-7 捕获多种异常

```
        int widgets, defective;
        double ratio;

        System.out.println("Enter number of widgets produced:");
        Scanner keyboard = new Scanner(System.in);
        widgets = keyboard.nextInt();
        if (widgets < 0)
            throw new NegativeNumberException("widgets");

        System.out.println("How many were defective?");
        defective = keyboard.nextInt();
        if (defective < 0)
            throw new NegativeNumberException("defective widgets");

        ratio = exceptionalDivision(widgets, defective);
        System.out.println( "One in every  " + ratio+ " widgets is defective.");
    }
    catch(DivideByZeroException e)
    {
        System.out.println("Congratulations! A perfect record!");
    }
    catch(NegativeNumberException e)
    {
        System.out.println("Cannot have a negative number of "+ e.getMessage());
    }

    System.out.println("End of program.");
    }

    public static double exceptionalDivision(double numerator,
                        double denominator) throws DivideByZeroException

    {
        if (denominator == 0)
            throw new DivideByZeroException();
        return (numerator/denominator);
    }
}
```

屏幕对话示例 1

```
Enter number of widgets produced:
1000
How many were defective?
500
One in every 2.0 widgets is defective.
End of program.
```

屏幕对话示例 2

```
Enter number of widgets produced:
-10
Cannot have a negative number of widgets
End of program.
```

图1-7　(续)

屏幕对话示例3

```
Enter number of widgets produced:
1000
How many were defective?
0
Congratulations! A Perfect record!
End of program.
```

图1-7 （续）

```
public class NegativeNumberException extends Exception
{
    public NegativeNumberException()
    {
        super("Negative Number Exception!");
    }
    public NegativeNumberException(String message)
    {
        super(message);
    }
}
```

图1-8 NegativeNumberException类

□ Java提示：首先捕获更明确的异常

在捕获多种异常时，catch块的顺序很重要。当异常从try块中抛出时，catch块是按照顺序被检查的，第一个能和所抛出的异常相匹配的块将被执行。因此，下面这样的catch块的顺序就不够好：

```
catch(Exception e)
{
    .
    .
    .
}
catch(DivideByZeroException e)
{
    .
    .
    .
}
```

第二个catch块始终用不上。

依照这种顺序，DivideByZeroException的catch块就一直用不上，因为所有的异常都被第一个catch块捕获了。幸运的是，编译器有可能会就此问题发出警告。正确的顺序是把它们颠倒过来，以便更明确的异常在其父异常类之前出现，如下面的代码所示：

```
catch(DivideByZeroException e)
{
```

```
        .
        .
        .
}
catch(Exception e)
{
        .
        .
        .
}
```

○ 编程提示：异常处理与信息隐藏

throw语句并不是抛出异常的唯一一种语句。方法调用也能抛出异常。（明确地说，方法调用可以抛出其throws子句中声明的异常。）每当异常被抛出，不管是什么类型的语句抛出它，处理的方式是相同的：它不是被catch块捕获就是被声明在throws子句中。当分析一个可能会抛出异常的方法调用时，不需要考虑那个throw语句究竟在这个方法中的何处定义。异常究竟是如何被抛出的并不重要。重要的是这个方法调用可能会抛出异常。异常处理采用的是同样的方式，不管在这个方法内究竟发生了什么。

自测题

24. 添加合适的throws子句以纠正下面的方法定义：

```
public void doStuff(int n)
{
    if (n < 0)
        throw new Exception("Negative number.");
}
```

25. 如果方法调用时抛出了异常，但是异常没有被该方法在内部捕获，那么会发生什么情况？

26. 假设在方法B内有对方法A的调用，方法C内有对方法B的调用。对方法C的调用导致对方法B的调用，从而也导致对方法A的调用。现在假设方法A抛出了一个自己没有捕获的异常。这个异常可能在哪里被捕获？在B中？在C中？在C之外？

27. 下列代码会产生怎样的输出（NegativeNumberException的定义在前面的介绍中已经给出了，不过回答这个问题不需要去查它的定义）？

```
int n;
try
{
    n = 7;
    if (n > 0)
        throw new Exception();
    else if (n < 0)
        throw new NegativeNumberException();
    else
        System.out.println("Hello!");
}
catch(NegativeNumberException e)
{
    System.out.println("First catch.");
```

```
}
catch(Exception e)
{
    System.out.println("Second catch.");
}
System.out.println("End of Code");
```

28. 在自测题27中，如果下面这行代码：

```
n = 7;
```

被改为

```
n = -7;
```

那么输出将会是什么？

29. 在自测题27中，如果下面这行代码：

```
n = 7;
```

被改为

```
n = 0;
```

那么输出将会是什么？

30. 下面的程序会产生怎样的输出？

```
public class CatchDemo
{
    public static void main(String[] args)
    {
        CatchDemo object = new CatchDemo();

        try
        {
            System.out.println("Trying");
            object.sampleMethod();
            System.out.println("Trying after call.");
        }
        catch(Exception e)
        {
            System.out.println("Catching");
            System.out.println(e.getMessage());
        }
    }

    public void sampleMethod() throws Exception
    {
        System.out.println("Starting sampleMethod.");
        throw new Exception("From sampleMethod with love.");
    }
    <可能还有更多的方法以及实例变量，但是我们并不需要它们来回答本题。>
}
```

△ 易犯错误：异常的过度使用

异常应当被节约着并且只以特定的方式来用。在下面的小节里，我们要讨论在哪里使用异常比较好。下面还有另外一个好的规则：如果很想包含一条throw语句，可以设想如果没有这条throw语句，将怎样写程序或类定义。若能想到一种替代方法并得出合理的代码，那很可能就不应该包含这条throw语句。 △

○ 编程提示：何时需要抛出异常

到目前为止，已经用了一些非常简单的代码来展示异常处理的基本概念。但是，这些例子有些不切实际地简单。一种更复杂但更好的指导原则是把抛出异常和捕获异常分离到单独的方法中去。在大多数情况下，应当把任意的throw语句包含在一个方法定义中，并把这个异常声明在该方法的throws子句里，然后把catch块放在不同的方法中。这样，try-throw-catch三元组的更好的用法如下面的代码所示：

```
public void methodA() throws MyException
{
      .
      .
      .
 throw new MyException("Bla Bla Bla");
      .
      .
      .
}
```

然后，在别的方法（甚至可能是别的类中的别的方法）中，可以这样写：

```
public void methodB()
{
      .
      .
      .
   try
   {
      .
      .
      .
     methodA();
      .
      .
      .
   }
   catch(MyException e)
   {
       Handle_Exception
   }
      .
      .
      .
```

此外，即使这样的throw语句也应仅在非如此不可时才用。如果可以用别的方法很容易地处理问题，就不要抛出异常。把throw语句保留用于某些情形，在这些情形中，异常条件的处理方式取决于该方法如何使用以及在哪里使用。如果处理异常条件的方式取决于该方法如何被调用以及在哪里调用，那么让调用此方法的程序员来处理异常是最理想的。在其他所有情况下，都应该尽量避免抛出异常。

预定义的方法经常把处理异常的工作留给调用它的程序员。当学习某个预定义的方法时，可能会被告知它将抛出某种特定的异常。读者，作为使用该预定义方法的程序员，将负责去处理该方法抛出的任何异常。

常见问题：代码何时应当抛出异常？

大多数情况下，throw语句应当用在方法内且在该方法的throws子句中声明。此外，应把它保留用于某些情形，在这些情形中，异常条件的处理方式取决于该方法如何使用以及在哪里使用。此时，最好的方式就是让调用该方法的程序员来处理异常。在其他所有情况下，几乎总是应该避免抛出异常。

△ 易犯错误：嵌套的try-catch块

可以把try块和其后相伴的一个或多个catch块放在一个大的try块或一个大的catch块之中。在少数情况下，这种方法可能有用处，但如果想这么用的话，应当考虑是否有更好的组织代码的方式。把内部的try-catch块放在一个方法定义内，在外面的try或catch块中放上对该方法的调用，或可能只是干脆消除一个或更多的try块，这样做几乎总是比嵌套更好。

如果把try块和其后相伴的一个或多个catch块放在一个大的catch块之中，那就必须在内部和外部的块中分别使用不同的catch块参数名，这是由Java处理各种嵌套块时的规则决定的。记住，try块和catch块都是代码块。

如果把try块和其后相伴的一个或多个catch块放在一个大的try块中，并且内部的try块抛出了一个异常，但是它没有被内部的catch块捕获，那么这个异常被抛到外部的try块来处理，并可能会被外部的某个catch块捕获。 △

1.3.5 finally块（选读）

可以在try块和其后相伴的一个或多个catch块之后添加一个finally块。finally块中的代码不管是不是有异常抛出都会执行。通用的语法如下面所示：

```
try
{
    ...
}
catch块(可能多个)
finally
{
    <不管是否有异常被抛出或捕获，这里的代码都将被执行。>
}
```

为了看出finally块的重要性和潜在用处，设想该try-catch-finally块在一个方法定义内部。（毕竟，任何一套try-catch-finally块都在某个方法内部，哪怕只是main方法内部。）当try-catch-finally块的代码执行时，有3种可能的结果：

(1) try块执行到底，没有异常抛出。这种情况下，finally块在try块之后执行。

(2) try块中抛出一个异常，且被一个位于这个try块之后的适宜的catch块捕获。此时，finally块在该catch块之后执行。

(3) try块中抛出一个异常，且该方法中没有匹配的catch块能捕获此异常。这样，该方法结束，此异常对象被抛到紧嵌在外面的方法。在这种情况下，finally块在方法结束之前执行[①]。注意，读者不能简单地用代码放在这一系列catch块之后来解释这第3种情况。

在当前的编程阶段，可能不太需要finally块，在这里介绍它是从完整性考虑。到了一

① 有多重try-catch-finally嵌套时，finally在异常被传递到紧嵌在外层的catch之前执行。

定时候，会发现它很有用。

1.3.6　重新抛出异常（选读）

在catch块内抛出异常是合法的。在极少数情况下，你可能想捕获一个异常，然后，根据getMessage产生的字符串（或根据其他条件），决定抛出同一个异常或不同的异常，以便在异常处理链的上端进一步处理。

自测题

31. 能否把try块及其相关的catch块放在更大的try块中？
32. 能否把try块及其相关的catch块放在更大的catch块中？
33. 本题是为学过1.3.5节的读者准备的。下面的程序的输出是什么？如果把sampleMethod的参数从99改为-99，输出将是什么？如果这个参数是0而不是99，输出是什么？

```
public class FinallyDemo
{
    public static void main(String[] args)
    {
        try
        {
            sampleMethod(99);
        }
        catch(Exception e)
        {
            System.out.println("Caught in main.");
        }
    }

    public static void sampleMethod(int n) throws Exception
    {
        try
        {
            if (n > 0)
                throw new Exception();
            else if (n < 0)
                throw new NegativeNumberException();
            else
                System.out.println("No Exception.");
            System.out.println("Still in sampleMethod.");
        }
        catch(NegativeNumberException e)
        {
            System.out.println("Caught in sampleMethod.");
        }
        finally
        {
            System.out.println("In finally block.");
        }
        System.out.println("After finally block.");
    }
}
```

案例分析：行式计算器

要求编写一个可以作为计算器，（类似手持式计算器）的程序。该计算器应能做加、减、乘、除运算。该程序将被用于相当老的设备上，因此它不能用视窗界面，而只能用简单的逐行文本输入和输出（像本章中此前的程序那样）。

首先需要明确定义用户界面，让用户按照提示输入运算符和数字，以便每个运算符和数字输入到同一行，如下面的例子：

```
+ 3.4
```

在运算符（比如+号）以及数字（比如3.4）的前后有没有空白都没有关系。随着用户输入更多的运算符和数字，程序记录最新的计算结果。对话过程示例如下：

```
result = 0
+ 80
result + 80 = 80
updated result = 80
-2
result - 2 = 78
updated result = 78
```

这里，程序假设初始的“结果”是0。用户输入的行已加粗。未加粗的是程序的输出。请注意输入被回显，这样，用户可以看出计算机在做什么。进一步建议用户可以通过输入E（大小写都可以）来说明一系列计算的结束。

建议的用户界面被采纳，然后开始进行项目设计。决定为计算器设计一个类。该类要有个main方法，它就是个完整的简单计算器，遵循前面细化的描述。后来，也许想为这个计算器设计一个更精美的界面。因此这个类将比main方法所需的更强大一些。

该程序记录一个数字，其用处和显示数字的手持式计算器相同。它在屏幕上显示的形式举例如下：

```
updated result = 80
```

用户可以进行加、减、乘以及除，比如：

```
+3
```

当前的结果保留在一个称为result的私有实例变量中。程序始终用当前的结果和输入的数字进行加减乘除。例如，若用户输入：

```
-9.5
```

并且result当前的值是80，那么result的值被修改为70.5。当用户输入'e'或'E'时，程序结束。如果读者提前看看图1-11中的屏幕对话示例，就能对如何使用该计算器有所了解。

决定该类至少需要下列方法：

- 一个名为reset的方法来重新设置result的值为0。
- 一个计算一次运算的结果的方法。决定将计算结果作为返回值，而不是立即更新实例变量result。这样可使这个类更通用。决定这个方法应当大致如下：

```
/**
    Returns n1 op n2,
    provided op is one of '+', '-', '*', or '/'.
*/
public double evaluate(char op, double n1, double n2)
```

- 一个名为getResult的访问方法，得到实例变量result的值。
- 一个名为setResult的设置方法，将result的值重新设定为任意给定的值。

- 一个名为doCalculation的方法，其中有一个循环，进行一系列运算，给出一个最终结果（循环的每次迭代都调用evaluate）。

首先假设一切顺利，为此而编写代码。或许注意到某些地方有可能出现异常，但是仍旧留待该类核心部分的设计完成后再来考虑。reset、setResult以及getResult这些方法的定义都是例行公事，但evaluate和doCalculation方法需要费一点心思。先来看doCalculation。

计算器的核心功能由doCalculation方法来完成。基本的循环序列应当反复执行下面的代码，直至用户输入字母'e'或者不正确的运算符：

```
nextOp = (keyboard.next()).charAt(0);
nextNumber = keyboard.nextDouble();
result = evaluate(nextOp, result, nextNumber);
```

这里，nextOp是char类型的变量，nextNumber是double类型的变量，result是实例变量。当用户输入字母'e'（不论大小写），循环结束并且计算也结束。把前面的代码转换成下面更复杂的Java循环：

```
boolean done = false;
while (! done)
{
    nextOp = (keyboard.next()).charAt(0);
    if ((nextOp == 'e') || (nextOp == 'E'))
        done = true;
    else
    {
        nextNumber = keyboard.nextDouble();
        result = evaluate(nextOp, result, nextNumber);
        System.out.println("result " + nextOp + " "
                           + nextNumber + " = " + result);
        System.out.println("updated result = " + result);
    }
}
```

下一步，需要设计evaluate方法，其原型描述如下：

```
/**
 Returns n1 op n2
 provided op is one of '+', '-', '*', or '/'.
*/
public double evaluate(char op, double n1, double n2)
```

evaluate方法的核心可以是一个大的switch语句，类似下面的代码：

```
switch (op)
{
    case '+':
        answer = n1 + n2;
        break;
    case '-':
        answer = n1 - n2;
        break;
    case '*':
        answer = n1 * n2;
        break;
    case '/':
        if (n2 == 0.0)
            throw new DivideByZeroException();
        answer = n1/n2;
```

```
            break;
    }
    return answer;
```

决定如果用户试图进行除0运算，就抛出异常，因此前面有关除法的分支包含了下面的代码：

```
if (n2 == 0.0)
    throw new DivideByZeroException();
```

这种方法在概念上是很好的，但是有一个问题：其中涉及的数字是double类型。浮点数，比如double类型的数字，只是代表了近似的数量，因此使用==来测试它们是否相等是不合理的。n2的值可能会如此接近0，以至另一个数除它时产生和除0一样的效果，但是这个测试会说它不等于0.0。因此决定只要除数非常接近0，就抛出DivideByZeroException。但是，还不确定如何定义“非常接近0”。你决定任何数量只要小于万分之一就被认为非常接近0。不过，既然你不能肯定这是一个最好的选择，所以还是决定使用一个名为precision的实例变量来作为判断是否为0的一个标准。precision的定义如下：

```
private double precision = 0.0001;
```

这样一来，测试被0除的代码就成了：

```
if ( (-precision < n2) && (n2 < precision) )
    throw new DivideByZeroException();
```

于是，你把这个switch语句改写成这样：

```
switch (op)
{
    case '+':
        answer = n1 + n2;
        break;
    case '-':
        answer = n1 - n2;
        break;
    case '*':
        answer = n1 * n2;
        break;
    case '/':
        if ( (-precision < n2) && (n2 < precision) )
            throw new DivideByZeroException();
        answer = n1/n2;
        break;
}
```

但是如果用户输入作为op的是'+'、'-'、'*'或'/'之外的字符怎么办？你决定在此情况下用抛出异常来处理。因此这个switch语句现在看起来是这样的：

```
switch (op)
{
    case '+':
        answer = n1 + n2;
        break;
    case '-':
        answer = n1 - n2;
        break;
    case '*':
        answer = n1 * n2;
        break;
```

```
        case '/':
            if ( (-precision < n2) && (n2 < precision) )
                throw new DivideByZeroException();
            answer = n1/n2;
            break;
        default:
            throw new UnknownOpException(op);
    }
```

DivideByZeroException类在图1-4中定义。UnknownOpException是个新的异常类，所以要定义它。你还需要写代码来捕获和处理所有的异常。

UnknownOpException类的代码和已经写过的其他异常类的代码很类似，在图1-9中给出。注意，当用户输入不能识别的运算符时，程序要告知用户那个特定的运算符无法识别。因此，UnknownOpException类有个构造器用一个char类型的参数来命名那个运算符。

```
public class UnknownOpException extends Exception
{
    public UnknownOpException()
    {
        super("UnknownOpException");
    }
    public UnknownOpException(char op)
    {
        super(op + " is an unknown operator.");
    }
    public UnknownOpException(String message)
    {
        super(message);
    }
}
```

图1-9　UnknownOpException类

到目前为止，程序除了处理异常的部分之外，都写好了。现在，你可以生成该程序的一个初级版本，如图1-10所示。在写异常处理的部分之前，可以用这个程序来测试和调试程序中不涉及到异常的部分。只要用户不输入无法识别的运算符以及不试图去除0，这个版本将运行正常。

```
import java.util.*;

/**
 PRELIMINARY VERSION without exception handling.
 Simple line-oriented calculator program. The class
 can also be used to create other calculator programs.
*/
public class PrelimCalculator
{
    private double result;
    private double precision = 0.0001;
    //Numbers this close to zero are treated as if equal to zero.

    public static void main(String[] args)

                                    throws DivideByZeroException,
                                           UnknownOpException
```

图1-10　没有异常的情况

```
    {
        PrelimCalculator clerk = new PrelimCalculator();

        System.out.println("Calculator is on.");
        System.out.print("Format of each line: ");
        System.out.println("operator space number");
        System.out.println("For example: + 3");
        System.out.println("To end, enter the letter e.");
        clerk.doCalculation( );

        System.out.println("The final result is "+ clerk.getValue( ));
        System.out.println("Calculator program ending.");
    }
    public PrelimCalculator()
    {
        result = 0;
    }
    public void reset()
    {
        result = 0;
    } .

    public void setResult(double newResult)
    {
        result = newResult;
    }
    public double getValue()
    {
        return result;
    }
    /**
     Returns n1 op n2, provided op is one of '+', '-', '*',or '/'.
     Any other value of op throws UnknownOpException.
     */
     public double evaluate(char op, double n1, double n2)
              throws DivideByZeroException, UnknownOpException
    {
        double answer;
        switch (op)
        {
            case '+':
                answer = n1 + n2;
                break;
            case '-':
                answer = n1 - n2;
                break;
            case '*':
                answer = n1 * n2;
                break;
    case '/':
        if ( (-precision < n2) && (n2 < precision))
            throw new DivideByZeroException();
        answer = n1/n2;
        break;
```

在本案例分析结束之前，main方法的定义会改变。

该程序的这个版本没有进行异常处理，因此不完整。不过它确实可以运行，能供调试用。

图1-10 （续）

```
            default:
                throw new UnknownOpException(op);
        }
        return answer;
    }

    public void doCalculation() throws DivideByZeroException,
                                       UnknownOpException
    {
        char nextOp;
        double nextNumber;
        boolean done = false;
        Scanner keyboard = new Scanner(System.in);
        result = 0;
        System.out.println("result = " + result);

        while (! done)
        {
            nextOp = (keyboard.next()).charAt(0);
            if ((nextOp == 'e') || (nextOp == 'E'))
                done = true;
            else
            {
                nextNumber = keyboard.nextDouble();
                result = evaluate(nextOp, result, nextNumber);
                System.out.println("result " + nextOp + " "
                                   + nextNumber + " = " + result);
                System.out.println("updated result = " + result);
            }
        }
    }
}
```

屏幕对话示例

```
Calculator is on.
Format  of each line: operator space number
For example: +3
To end, enter the letter e.
result = 0.0
+4
result +4.0 = 4.0
updated result =4.0
*2
result *2.0  = 8.0
updated result = 8.0
e
The final result is 8.0
Calculator program ending.
```

图1-10　(续)

当调试过该程序的初级版本之后，就可以开始添加异常处理部分了。最主要的异常是

UnknownOpException，应该首先来处理它。已经给出了UnknownOpException类的定义，不过除了在throws子句中声明之外，还没有用过它。当有异常抛出时，希望做些更重要和有效的事情。

为了让程序更健壮，你使用了try-throw-catch技术。evaluate方法会抛出UnknownOpException。evaluate方法被doCalculation方法调用，而doCalculation方法被main方法调用。通常有3种方式来处理这个异常：

(1) 在evaluate方法中捕获异常。

(2) 在evaluate方法的throws子句中声明UnknownOpException异常，然后在doCalculation方法中捕获这个异常。

(3) 在evaluate和doCalculation方法的throws子句中都声明UnknownOpException异常，然后在main方法中捕获这个异常。

到底选择哪一种取决于读者希望在异常抛出后发生什么。如果希望用户重新输入运算符，可以选择方法1或2；如果希望重新进行计算，可以选择方法3。

决定当UnknownOpExceptioin被抛出时就重新进行计算，因此采用方式3，把try块和catch块放在main方法中。这个决定使得main被重写，如图1-11所示。为了这样处理，你添加了两个新方法：handleUnknownOpException和handleDivideByZeroException。剩下的全部事情就是定义这两个方法来处理异常了。

```
import java.util.*;

/**
 Simple line-oriented calculator program. The class
 can also be used to create other calculator programs.
*/
public class Calculator
{
    private double result;
    private double precision = 0.0001;
    //Numbers this close to zero are treated as if equal to zero.

    public static void main(String[] args)
    {
        Calculator clerk = new Calculator();

        try
        {
            System.out.println("Calculator is on.");
            System.out.print("Format of each line: ");
            System.out.println("operator space number");
            System.out.println("For example: + 3");
            System.out.println("To end, enter the letter e.");
            clerk.doCalculation();
        }
        catch(UnknownOpException e)
        {
            clerk.handleUnknownOpException(e);
        }
        catch(DivideByZeroException e)
        {
            clerk.handleDivideByZeroException(e);
        }
```

图1-11 完整的行式计算器

```
        System.out.println("The final result is "+ clerk.getValue());
        System.out.println("Calculator program ending.");
    }
    public Calculator()
    {
        result = 0;
    }

    public void reset()
    {
        result = 0;
    }

    public void setResult(double newResult)
    {
        result = newResult;
    }

    public double getValue()
    {
        return result;
    }

    /**
     The heart of a calculator. This does not give
     instructions. Input errors throw exceptions.
     */
     public void doCalculation() throws DivideByZeroException,
                                        UnknownOpException
    {
        char nextOp;
        double nextNumber;
        Scanner keyboard = new Scanner(System.in);
        boolean done = false;
        result = 0;
        System.out.println("result = " + result);
        while (! done)
        {
            nextOp = (keyboard.next()).charAt(0);
            if ((nextOp == 'e') || (nextOp == 'E'))
                done = true;
            else
            {
                nextNumber = keyboard.nextDouble();
                result = evaluate(nextOp, result, nextNumber);
                System.out.println("result " + nextOp + " "
                                   + nextNumber + " = " + result);
                System.out.println("updated result = " + result);
            }
        }
    }
```

reset、setResult和getResult在这个程序中都没有用到，但是其他使用这个类的应用可能会需要它们。

图1-11　(续)

```
/**
 Returns n1 op n2,
 provided op is one of '+', '-', '*',or '/'.
 Any other value of op throws UnknownOpException.
 */
 public double evaluate(char op, double n1, double n2)
          throws DivideByZeroException, UnknownOpException
{
    double answer;
    switch (op)
    {
        case '+':
            answer = n1 + n2;
            break;
        case '-':
            answer = n1 - n2;
            break;
        case '*':
            answer = n1 * n2;
            break;
        case '/':
            if ( (-precision < n2) && (n2 < precision))
                throw new DivideByZeroException();
            answer = n1/n2;
            break;
        default:
            throw new UnknownOpException(op);
    }
    return answer;
}

 public void
      handleDivideByZeroException(DivideByZeroException e)
 {
    System.out.println("Dividing by zero.");
    System.out.println("Program aborted");
    System.exit(0);
 }

 public void handleUnknownOpException(UnknownOpException e)
 {
    System.out.println(e.getMessage());
    System.out.println("Try again from the beginning:");

try
{
    System.out.print("Format of each line: ");
    System.out.println("operator number");
    System.out.println("For example: + 3");
    System.out.println("To end, enter the letter e.");
    doCalculation();
}
```

当第一次抛出UnknownOpException时，执行这个代码块。

图1-11 （续）

```
            catch(UnknownOpException e2)
            {
                System.out.println(e2.getMessage());
                System.out.println("Try again at some other time.");
                System.out.println("Program ending.");
                System.exit(0);
            }

            catch(DivideByZeroException e3)
            {
                handleDivideByZeroException(e3);
            }
        }
    }
}
```

当UnknownOpException再次被抛出时，这块代码捕获它。

屏幕对话示例

```
Calculator is on.
Format of each line: operator space number
For example:+3
To end, enter the letter e.
result = 0.0
+ 80
result + 80.0 = 80.0
undated result = 80.0
- 2
result - 2.0 =78.0
updated result = 78.0
% 4
% is an unknown operator.
Try again from the beginning:
Format of each line is: operator space number
For example:+3
To end, enter the letter e.
result = 0.0
+ 80
result +80.0 = 80.0
updated result = 80.0
- 2
result - 2.0 = 78.0
updated result = 78.0
* 0.04
result * 0.04 = 3.12
updated result = 3.12
e
The final result is 3.12
Calculator program ending.
```

图1-11　（续）

如果察看main方法中的catch块，就会看到当UnknownOpException抛出时，会被handleUnknownOpException方法处理。设计handleUnknownOpException方法时给了用户第二次机会来进行计算（从头开始）。如果用户第二次输入了无法识别的运算符，则另一个UnknownOp-Exception会被抛出，但这次它在handleUnknownOpException方法内部被捕获。这种情况下的代

码请看handleUnknownOpException方法中的catch块（图1-11）。如果UnknownOpException被再次抛出，程序就会结束。（还有其他处理UnknownOpException的好方法，但是这种方法也令人满意。）

请注意在doCalculation方法的定义中（图1-11），尽管在其方法体中没有包含任何throw语句，还是需要包含一个有UnknownOpException和DivideByZeroException类的throws子句。这是因为doCalculation方法中含有对evaluate方法的调用，而evaluate方法可以抛出UnknownOpException或DivideByZeroException。

你决定如果用户试图除0，就简单地结束程序。（可能打算在将来的版本中更认真地处理它，但现在这样的做法也行。）因此，handleDivideByZeroException方法非常简单。

自测题

34. 为图1-11中的Calculator类写一个名为getPrecision的访问方法，返回实例变量precision的值。同时，写一个名为setPrecision的设置方法，把实例变量precision的值改为任意指定的值。
35. 如果运行图1-10中的程序，而用户输入了一个未知的操作符（比如%或#），程序并没有考虑如何处理它，这时会发生什么情况？

1.4 图形编程补充（选读）

那种颜色很有趣，它叫什么呀？

——一个朋友看到我新粉刷的餐厅时问

在这部分补充材料里，我们将学习一个使用了异常的JFrame GUI的例子。

GUI中的异常

只要在所有含有异常抛出的方法的throws子句中声明所抛出的异常，就可以不用烦神去捕获异常了。但是，这不是好的编程实践。在普通的（不是GUI的）Java应用程序中没有捕获的异常会使程序在显示一个错误消息之后结束。在GUI程序中（不论是JFrame GUI或者applet）的未捕获异常不会使程序结束。但是，除非你去捕获异常，否则GUI程序可能无法应付该异常，用户也会很茫然。因此，在GUI程序中，确保所有抛出的异常都被捕获尤其重要[①]。

编程示例：使用异常的JFrame

图1-12中含有一个JFrame GUI，它允许用户看颜色的样例。用户在文本域中输入颜色的名称，然后点击“Show Color”按钮。如果该程序知晓这种颜色，其背景就会变成这种颜色。如果该程序不认识这种颜色，则文本域显示消息“Unknown Color”，背景变为灰色。

用户输入的字符串被getColor方法转换为Color类的一个对象。如果所要求的颜色不在其已知之列，比如"Orange"，getColor方法会抛出异常。该异常没有被getColor方法捕获，不过列在其throws子句中。在actionPerformed方法中把对getColor方法的调用放在一个try块中。如果有异常抛出，就被actionPerformed方法中的一个catch块捕获。

① 那些不需要在catch块中捕获或在throws子句中声明的异常是特殊情况，它们并不总是需要被捕获。

运行这个GUI程序需要ColorDemo.java以及UnknownColorException.java这两个文件，都在图1-12中给出，还需要《Java程序设计与问题解决：基础篇》的图7-20中的WindowDestroyer.java（可到网上资源下载源代码），以及图1-13中的ShowColorDemo.java文件。

△ 易犯错误：actionPerformed中不允许有throws子句

当为动作侦听器类定义actionPerformed方法时，不能添加throws子句。actionPerformed方法的头部已经完全规定好了，不能改变它（修改那个参数的名称除外）。因此如果在actionPerformed方法定义中抛出异常，就必须在actionPerformed方法中捕获它。例如，在图1-12中，不能通过用在actionPerformed方法头部的throws子句中声明的办法来避免捕获UnknownColorException，UnknownColorException必须在actionPerformed方法体内被捕获。

与此类似，如果要重新定义窗口侦听器类(比如WindowDestroyer)中的windowClosing方法，则不能给windowClosing方法的头部添加throws子句。 △

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class ColorDemo extends JFrame implements ActionListener
{
    public static final int WIDTH = 400;
    public static final int HEIGHT = 300;
    public static final int NUMBER_OF_CHAR = 20;

    private JTextField colorName;

    public ColorDemo()
    {
        setSize(WIDTH, HEIGHT);
        WindowDestroyer listener = new WindowDestroyer();
        addWindowListener(listener);

        Container contentPane = getContentPane();
        contentPane.setBackground(Color.GRAY);
        contentPane.setLayout(new FlowLayout());

        JButton showButton = new JButton("Show Color");
        showButton.addActionListener(this);
        contentPane.add(showButton);

        colorName = new JTextField(NUMBER_OF_CHAR);
        contentPane.add(colorName);
    }

    public void actionPerformed(ActionEvent e)
    {
        Container contentPane = getContentPane();
```

图1-12　使用异常的JFrame GUI程序

```
try
{
    contentPane.setBackground(
            getColor(colorName.getText()));
}
catch(UnknownColorException exception)
{
   colorName.setText("Unknown Color");
   contentPane.setBackground(Color.GRAY);
   }
}

public Color getColor(String name) throws UnknownColorException
{
    if (name.equalsIgnoreCase("RED"))
        return Color.RED;
    else if (name.equalsIgnoreCase("WHITE"))
        return Color.WHITE;
    else if (name.equalsIgnoreCase("BLUE"))
        return Color.BLUE;
    else if (name.equalsIgnoreCase("GREEN"))
        return Color.GREEN;
    else
        throw new UnknownColorException();
}
}
```

这是ColorDemo.java文件。

这是UnknownColorException.java文件。

```
public class UnknownColorException extends Exception
{
   public UnknownColorException()
   {
      super("Unknown Color!");
   }

   public UnknownColorException(String message)
   {
      super(message);
   }
}
```

图1-12 （续）

自测题

36. 重写图1-12中的ColorDemo类，使之成为一个applet。

```
public class ShowColorDemo
{
    public static void main(String[] args)
    {
        ColorDemo gui = new ColorDemo();
        gui.setVisible(true);
    }
}
```

得到的GUI程序

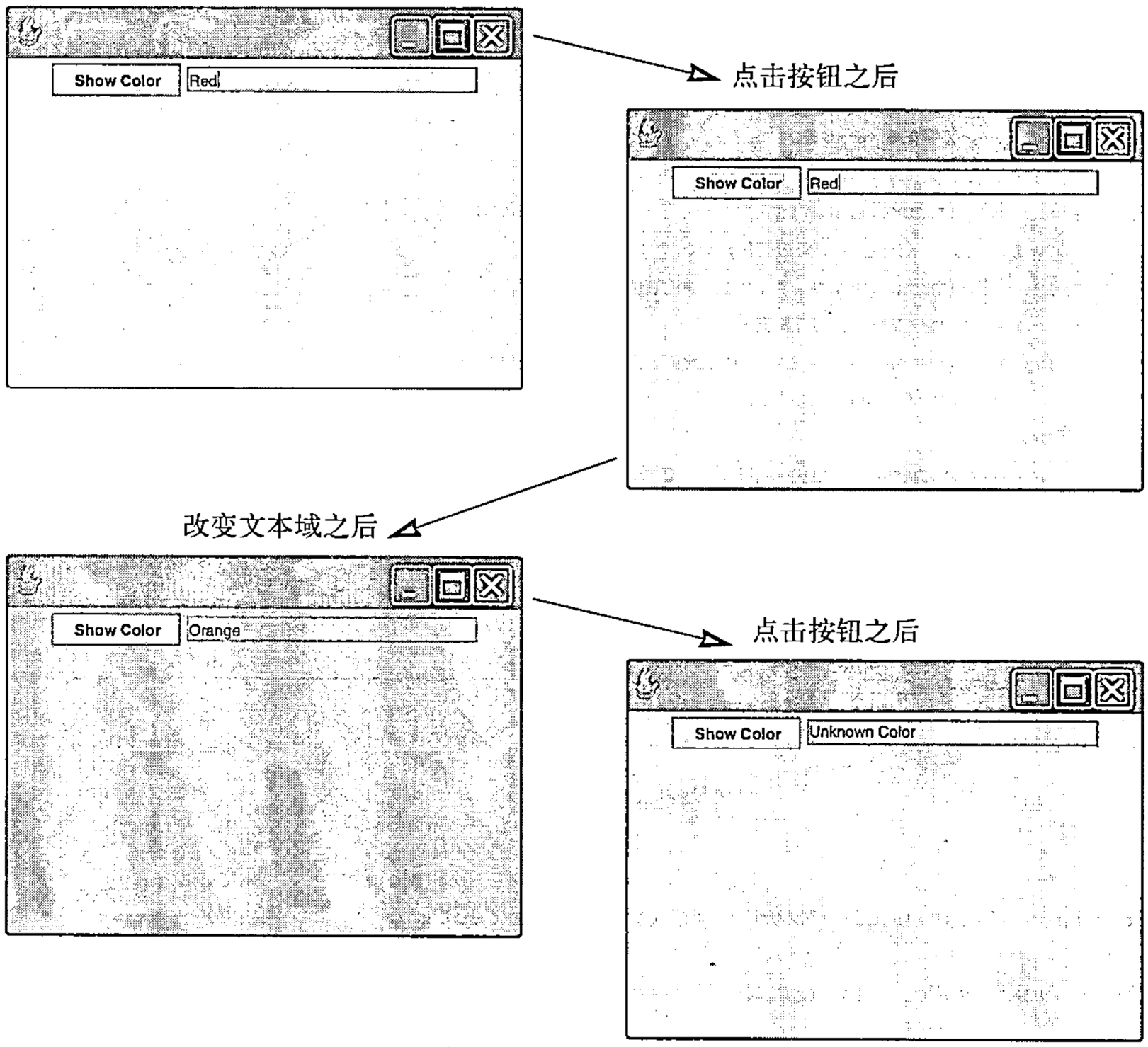


图1-13　运行GUI程序ColorDemo

小　结

- 一个异常就是Exception类的子孙类的一个对象。(Error类的子孙类并不是异常，但是其表现类似异常，因此也把它们看作异常。)
- 异常处理允许读者为程序的正常情况单独设计和编写代码，并与不正常情况下的处理代码分开。
- 有一些预定义的异常类。读者也可以定义自己的异常类。
- 一些Java语句自身可能会抛出异常。类库中的方法可能会抛出异常。可以在自己的代码中用throw语句显式地抛出异常。
- 异常可以在try块中抛出。或者，异常也可以从不包含try块的方法定义中抛出，这种情况下，对

此方法的调用可以放在一个try块中。

- 当某方法要抛出异常，却没有捕获它时，该异常类通常必须列在该方法的throws子句中。
- 异常在catch块中被捕获。
- 一个try块后可以有不止一个catch块。此时，应把更专用的异常类的catch块放在更通用的异常类的catch块的前面。
- 每个异常对象都有getMessage方法，可以恢复对异常的描述。
- 不要滥用异常。

✓自测题答案

1.
```
Try block entered.
Exception: Time Limit Exceeded.
After catch block.
```

2.
```
Try block entered.
Leaving try block.
After catch block.
```

3. 一个异常是一个对象。例如，在下面的代码中，Exception是Exception类的构造器：

```
throw new Exception("Time Limit Exceeded.");
```

下列语句创建了一个异常对象：

```
new Exception("Time Limit Exceeded.");
```

4. 是的，在合法性方面它无懈可击，尽管读者可能没有充足的理由这样使用。
5. 行为不会改变。新的代码和它替换的旧代码是等价的。
6.
```
throw new Exception("Time Limit Exceeded.");
```

注意下面的代码是if语句，不是throw语句，尽管它含有一个throw语句：

```
if (waitTime > 30)
   throw new Exception("Time Limit Exceeded.");
```

7. 当throw语句执行时，包含它的try块终止。该try块内的其他代码都不再执行，控制转向随后的catch块（可能有多个）。当我们说控制转向随后的catch块时，是指那个被抛出的异常对象代换了catch块参数，并且catch块的代码开始执行。
8.
```
try
{
   System.out.println("Try block entered.");
   if (waitTime > 30)
    throw new Exception("Time Limit Exceeded.");
   System.out.println("Leaving try block.");
}
```

9.
```
catch(Exception e)
{
   System.out.println("Exception: " + e.getMessage());
}
```

10. e是catch块参数。
11. 不会。catch块参数e只是一个占位符，可以被任何（非关键字的）标识符替换，比如用messenger。
12.
```
if (status.equals("bad"))
     throw new Exception("Exception thrown: Bad Status.");
```

13. 是的，完全合法，尽管可能找不到充足的理由来这么用它。

14. 是的，完全合法，不过几乎在任何情况下，都可以用下面更好的方式来替代它：

```
throw new IOException("Hello Houston!");
```

在实际编程中，前面这个throw语句通常被包含在某些分支语句中，比如if语句。

15.
```
public class CoreBreachException extends Exception
{
    public CoreBreachException()
    {
        super("Core Breach! Evacuate Ship!");
    }

    public CoreBreachException(String message)
    {
        super(message);
    }
}
```

16.
```
public class MessageTooLongException extends Exception
{
    public MessageTooLongException ()
    {
        super("Message Too Long!");
    }

    public MessageTooLongException (String message)
    {
        super(message);
    }
}
```

17.
```
ExerciseException invoked with an argument.
Do Be Do
```

18.
```
Wow, Crazy exception thrown!!
Crazy exception thrown!
Message is Crazy exception thrown!
```

19.
```
try block entered:
Double Exception thrown! Double Exception thrown!
End of code.
```

20. 输出根本不会改变。修改后的程序同原来的执行效果完全一样。

21. 输出将变为这样：

```
try block entered:
Leaving try block.
End of code.
```

22.
```
42
IntException thrown!
```

23. 异常类可以是任何预定义的或程序员定义的异常类的派生类。特别地，可以从预定义的IOException类派生出自己的异常类。

24.
```
public void doStuff(int n) throws Exception
{
    if (n < 0)
        throw new Exception("Negative number.");
}
```

25. 如果一个方法抛出了异常，而且该异常没有在这个方法内部被捕获，那么异常抛出后，方法调用立

刻结束。如果这个方法调用在try块的内部，那么这个异常被抛给合适的catch块（假如有的话）。如果没有能与该异常匹配的catch块，则这个异常就是未捕获的，此方法调用在异常抛出后立刻结束。

26. 它可能在方法B中被捕获。如果在方法B中没有捕获它，它可能在方法C中被捕获。如果在方法C中没有被捕获，它可能在方法C的外部被捕获。

27.
```
Second catch.
End of Code.
```

28.
```
First catch.
End of Code.
```

29.
```
Hello!
End of Code.
```

30.
```
Trying
Starting sampleMethod.
Catching
From sampleMethod with love.
```

31. 是的，可以将一个try块和相应的catch块放到另一个更大的try块之中。不过，可能更好的方法是把内部的try块及其catch块放在一个单独的方法定义中，然后在那个大的try块中调用这个方法。

32. 是的，可以将一个try块和相应的catch块放到另一个更大的catch块之中。不过，可能更好的方法是把内部的try块及其catch块放在一个单独的方法定义中，然后在那个大的catch块中调用这个方法。

33. 参数是99时的输出如下：

```
In finally block.
Caught in main.
```

参数是-99时的输出如下：

```
Caught in sampleMethod.
In finally block.
After finally block.
```

参数是0时的输出如下：

```
No Exception.
Still in sampleMethod.
In finally block.
After finally block.
```

34.
```
public double getPrecision()
{
    return precision;
}

public void setPrecision(double newPrecision)
{
    precision = newPrecision;
}
```

35. 在抛出UnknownOpException之后，程序立即结束。

36. 只需要这样修改ColorDemo.java文件：

把

```
extends JFrame
```

替换为

```
extends JApplet
```

把

```
public ColorDemo()
```

替换为

```
public void init()
```

删除下面3行：

```
setSize(WIDTH, HEIGHT);
WindowDestroyer listener = new WindowDestroyer();
addWindowListener(listener);
```

在applet版本中，不需要WindowDestroyer类，但是仍旧需要Unknown-ColorException类。网站上本书源码中的applet文件ColorDemoApplet.java就是这样修改得来的。

○ 编程项目

1. 写一个程序把24小时制的时间转换为12小时制的时间。下面是示例对话过程：

```
Enter time in 24-hour notation:
13:07
That is the same as
1:07 PM
Again?(y/n)
y
Enter time in 24-hour notation:
10:15
That is the same as
10:15 AM
Again?(y/n)
y
Enter time in 24-hour notation:
10:65
There is no such time as 10:65
Try Again:
Enter time in 24-hour notation:
16:05
That is the same as
4:05 PM
Again?(y/n)
n
End of program
```

要定义一个名为TimeFormatException的异常类。如果用户输入了不合法的时间，比如10:65，甚或是无意义的东西，比如8&*68，程序将会抛出、捕获并处理一个TimeFormatException。

2. 使用图1-11中的Calculator类来编写程序，创建一个功能更强大的计算器。该计算器允许用户在内存中保存一个结果，以后可以调出来。该计算器用的命令如下：

   e 结束

   c 清除；设置结果为0

   m 保存到内存；把内存中的内容设置为result的值

   r 从内存中取出；显示内存中的值，但是不改变result的值

   应当定义一个Calculator类的派生类，它包括：另外一个实例变量，用来保存结果；一个新的

main方法，用来运行这个改良的计算器；重定义的handleUnknownOpException方法；以及任何你认为需要的新的或重写的部分。下面是一个示例对话。你的程序不必产生完全一样的对话，但应当类似，并且同样清晰甚或更加清晰。

```
Calculator on:
result = 0.0
+4
result + 4.0 = 4.0
updated result = 4.0
/2
result / 2.0 = 2.0
updated result = 2.0
m
result saved in memory
c
result = 0.0
+99
result + 99.0 = 99.0
updated result = 99.0
/3
result / 3.0 = 33.0
updated result = 33.0
r
memory recall
memory value = 2.0
result = 33.0
+2
result + 2.0 = 35.0
updated result = 35.0
e
End of Program
```

3. 在程序中使用自测题16中的异常类MessageTooLongException，要求用户输入一行不超过20个字符的文本。如果用户输入的字符数不超过20，程序应当打印出消息："You entered *x* characters, which is an acceptable length"（*x*应被替换为实际的字符数）。否则，一个MessageTooLongException对象会被抛出并被捕获。不论如何，程序应当循环下去，并询问用户是否要输入另外一行或者要退出程序。
4. 写一个程序进行日期转换，从数字式的月-日格式转为英文名称式的月-日格式（例如，1/31或01/31转为January 31）。对话方式应当类似编程项目1。定义两个异常类：一个名为MonthException，另一个名为DayException。如果用户输入的不是合法的月份数（1~12的整数），程序应当抛出并捕获一个MonthException对象。类似地，如果用户输入的不是合法的月内日数（1~29，1~30或1~31间的数字，根据月份不同而不同），则程序应当抛出并捕获一个DayException对象。为了让问题简单些，始终允许二月份有29天。
5. 定义一个名为DimensionException的异常类，把它用在《Java程序设计与问题解决：基础篇》第7章的编程项目6的驱动程序中。修改那个驱动程序，当用户输入的维数小于或等于0时，抛出并捕获一个DimensionException的对象。
6. 修改《Java程序设计与问题解决：基础篇》第7章的编程项目5的驱动程序，使用3个异常类，分别叫做CylinderException、LoadException和TowingException。汽缸数必须是1~12的整数，载重量必须是1~10之间的数（可能带有小数部分），拖曳重量必须是1~20之间的数（可能带有小数部分）。

对于其他不在这些数字范围内的输入，程序将会抛出并捕获相应的异常。也需要定义CylinderException、LoadException 及TowingException类。

7. 写个程序将Employee数据，包括社会安全号码以及薪酬，输入到一个数组中。（员工数的最大值是100，不过程序应当对任何小于100的员工数也能正常工作。）程序应当使用两个异常类：一个叫做SSNLengthException，用在输入的社会安全号码(SSN)的字符数目不是9的时候；另一个叫作SSNCharacterException，用在输入的社会安全号码中有不是数字字符的时候。(SSN输入时只有数字，没有破折号和空白。）当异常抛出时，提醒用户她或他输入的是什么，告知为何不合适，并请求重新输入数据。当数据全部输入完之后，程序应当输出所有员工的记录，并给出注解说明该员工的薪酬是高于还是低于平均数。还需要定义Employee、SSNLengthException以及SSNCharacterException这些类。Employee类将是《Java程序设计与问题解决：基础篇》第7章图7-1中Person类的派生类。除此之外，Employee类应当有输入输出方法，以及构造器、访问方法，还有设置方法。每个Employee对象会记录员工的名字、薪酬以及社会安全号码（还可以增加任何需要的或认为合适的数据)。

8. (本项目需要阅读本章的图形编程补充。）重写图1-11中的Calculator类，使之成为一个applet。让用户在一个文本域中输入，比如+80。用一个标记为“Update”的按钮让applet执行所要求的运算，比如+80。在另一个文本域中给出结果。并且用一个“Reset”按钮重新启动计算（把result设置为0.0)。(在第5章中，我们将讨论制作一个计算器，包括各种运算的按钮，不过在这里请按照所描述的去做。)

第2章

流和文件I/O

我会记住你的。

——威廉·莎士比亚，《亨利六世》，第二幕

I/O是指对输入和输出的编程。输入可以来自键盘或文件。类似地，输出也可以发送到屏幕或文件中。本章将介绍如何编写程序，从文件中读取输入，并将输出发送到文件中。

目标

- 熟悉I/O流的概念。
- 理解二进制文件和文本文件之间的区别。
- 学习怎样用Java程序将数据保存到一个文件中。
- 学习如何用Java程序从一个文件中读取数据。
- 学习怎样分别使用ObjectOutputStream类和ObjectInputStream类向二进制文件中写入或从中读取类对象。

预备知识

要阅读本章，只需要读过第1章关于异常处理的那部分内容就可以了。除了2.5节之外，本章其余的部分都不需要关于数组和继承的知识，2.5节介绍了向二进制文件中写入和从中读取类及数组对象的内容。如果想在学习数组之前学习该节的内容，可以略过2.5.3节关于数组I/O的部分。

如果愿意的话，可以在2.4和2.5节之后学习有关File类的2.3节。很多读者可能会选择跳过二进制文件那部分内容。本书的其余部分都不需要使用与二进制文件或本章介绍的其他内容有关的知识。

与预备知识有关的细节如下所示：

小节	预备知识
2.1节	《Java程序设计与问题解决：基础篇》第1章~第5章
2.2节	《Java程序设计与问题解决：基础篇》第1章~第5章，本书1.1节和2.1节
2.3节	《Java程序设计与问题解决：基础篇》第1章~第5章，本书1.1节，2.1节和2.2节
2.4节	《Java程序设计与问题解决：基础篇》第1章~第5章，本书1.1节和1.2节，以及2.1节和2.2节。不需要2.3节
2.5节	《Java程序设计与问题解决：基础篇》第1章~第7章，本书1.1节和1.2节，以及2.1节、2.2节和2.4节。不需要2.3节

2.1　流和文件I/O概览

鱼儿们说，它们有了溪流（stream）和池塘，可是还有没有别的呢？

——鲁伯特·布鲁克，《天堂》

本节对文件I/O进行了概略的介绍。特别对文本文件和二进制文件之间的区别进行了解释。本章的后继小节给出了文件I/O语句的Java语法。

2.1.1　流的概念

你已经在用文件存储Java类和程序了。还可以用文件来存储一个程序的输入或者输出。在Java中，文件I/O以及简单的键盘和屏幕I/O都是由**流**（stream）来处理的。流是一个对象，既可以用来将数据送到目的地，比如文件或屏幕上，又可以用来从文件或键盘这样的源端接收数据，并将数据传送到程序中。

对象`System.out`是到目前为止我们用过的唯一一个输出流。用于键盘输入的`Scanner`类对象也是流。本章对将程序与文件连接起来的流进行了讨论。

常见问题：什么是流？

流就是数据流。数据可能是字符、数字或者由二进制数字组成的字节。如果数据是流入程序的，这个流就被称为**输入流**（input stream）。如果数据是流出程序的，这个流就被称为**输出流**（output stream）。例如，如果将一个输入流连接到键盘，数据就会从键盘流入你的程序。如果将一个输入流连接到文件，数据就会从文件流入程序。在Java中，流是作为特定的流类对象实现的。对象`System.out`就是输出流的一个例子。用于键盘输入的`Scanner`类对象就是输入流。

2.1.2　为什么要用文件进行I/O

到目前为止我们用过的键盘输入和屏幕输出处理的都是暂时性的数据。程序结束时，从键盘输入的数据和留在屏幕上的数据都会消失。文件则提供了一种永久储存数据的方式。文件的内容会保持不变直到有人或程序修改了文件为止。

输入文件可以被不同的程序反复使用，不需要为每个程序分别输入数据。文件还提供了一种方便的、处理大量数据的方式。程序从一个大的输入文件中获取输入时，不需要用户进行大量的键入就可以接收到很多数据了。

2.1.3　文本文件和二进制文件的区别

任意文件中的所有数据都是以二进制数字（或比特）的形式存储的——也就是说，是作为（可能很长的）零一序列存储的。但是，在很多情况下，我们都没有把文件的内容当成二进制数字序列。相反，我们认为文件的内容是由一系列字符组成的。有些文件被当作字符序列，并拥有一些使二进制数字对程序和编辑器来说就像字符一样的流和方法，这样的文件被称为**文本文件**（text file）。而那些内容必须作为二进制数字序列处理的文件则被称为**二进制文件**（binary file）。

尽管从技术角度来说不太严格准确，但是，可以安全地认为文本文件中包含的是字符序列，二进制文件中包含的是二进制数字序列。Java程序是存储在文本文件中的。理解二进制文件和文本文件之间区别的另一种方式是：注意到文本文件是为人类使用而设计，而二进制文件则是为仅由程序读取而设计。

文本文件的优点之一是：通常在所有的计算机上它们看起来都是一样的，因此，可以将文本文件从一台计算机上移到另一台计算机上去，这个过程中只会出现极少的问题，甚至不会出现问题。但是，二进制文件的实现通常会随计算机的不同而不同，因此，二进制文件通常只能在同一种类型的计算机上，用最初创建这个文件的应用程序来读取。

二进制文件的优点是它们的处理效率比文本文件高。与其他编程语言不同，Java还将文本文件的某些优点赋予了二进制文件。特别是，Java的设计者将二进制文件设计成与平台无关的。也就是说，对Java来说，可以将二进制文件从一种类型的计算机移动到另一种类型的计算机中去，Java程序仍然能够读入二进制文件。这种特性将文本文件的便携性与二进制文件的高效性结合起来。

文本文件一个很大的优点是可以用文本编辑器将其读入或写出。对二进制文件来说，所有的读入和写出通常都必须由程序来完成。

记住：文本文件和二进制文件

可以用编辑器写入或读出的文件被称为**文本文件**（text file）。（有时也将文本文件称为ASCII文件，因为其中包含的是用ASCII码体系编码的数据。）另一类文件被称为**二进制文件**（binary file）。二进制文件表达数据的方法不便于文本编辑器读入，但可以通过程序非常高效地写出或读入。

记住：输入及输出术语

为了避免混淆，请记住单词输入（input）表示的是数据进入到程序（而不是文件）中。单词输出表示的是数据从程序（而不是文件）中取出。

自测题

1. 为什么要编写向文件而不是向屏幕输出数据的程序呢？
2. 讨论输入时，指的是数据从程序传向文件还是从文件传向程序呢？
3. 二进制文件和文本文件之间有什么区别？

2.2 文本文件I/O

风格的准确定义就是在适当的场合说适当的话。

——乔纳森·斯威夫特，给一名年轻牧师的信（1720年1月9日）

本节描述了在Java中执行文本文件I/O的最常见方式。

2.2.1 用PrintWriter进行的文本文件输出

编写一个向文本文件发送输出的程序时，要使用一个名为println的方法。这个方法的

表现与System.out.println一样，但它是PrintWriter类中的一个方法。PrintWriter类是向文本文件写入时常用的流类。图2-1包含了一个向文本文件写入数据的简单程序。

注意，图2-1中的程序是以下列代码行开始的：

```
import java.io.*;
```

这条语句告诉Java程序要使用java.io库，这个库中包含了PrintWriter类的定义以及本章中讨论的其他文件I/O类。所有用本章给出的技术进行文件I/O的程序或类都必须在文件起始处附近包含前面的import语句。

记住：import语句

用本章给出的任意技巧进行文件I/O的程序或类都必须在文件起始处附近包含下列import语句：

```
import java.io.*;
```

这条语句告诉Java：程序要使用java.io包（库），包中包含了PrintWriter这样的类定义，以及本章中讨论的其他文件I/O类。

图2-1中的程序创建了一个名为out.txt的文本文件，可以通过编辑器来读取这个文件，另一个Java程序可以用稍后要讨论的BufferedReader类来读取这个文件。注意文件是怎样打开的：

```
outputStream =new PrintWriter(new FileOutputStream("out.txt"));
```

（变量outputStream是PrintWriter类型的，它是在try块之外声明的。）前面的两行代码将名为outputStream的流连接到一个名为out.txt的文件。进行这样的连接被称为**打开文件**（opening the file）。用这种方式将一个文件连接到一个流时，程序都是以空文件开始的。如果文件out.txt已经存在了，那么out.txt中原有的内容就会丢失。如果文件out.txt不存在，就会创建一个新的、名为out.txt的空文件。

注意：文件的名字，在这个例子中就是out.txt，是作为一个String值给出的，因此要用引号。PrintWriter类没有可以将文件名作为实参的构造器。因此用FileOutputStream类创建了一个可以用作PrintWriter构造器实参的流。表达式

```
new FileOutputStream("out.txt");
```

创建了一个FileOutputStream类的对象，然后按下列形式将其用作PrintWriter类构造器的实参：

```
new PrintWriter(new FileOutputStream("out.txt"))
```

这条语句生成了一个PrintWriter类的对象，并通过（图2-1中的）下列代码将名字outputStream赋给它：

```
outputStream = new PrintWriter(new FileOutputStream("out.txt"));
```

但是，可以将整个表达式

```
output_Stream_Name = new PrintWriter(new FileOutputStream(File_Name));
```

当作一项很难拼写的操作，这项操作将一个文件名作为实参，在PrintWriter类中生成了一个输出流，并将这个输出流连接到已命名的文件，以便程序可以将输出发送到那个文件中去。

```
import java.io.*;
import java.util.*;

public class TextFileOutputDemo
{
    public static void main(String[] args)
    {
        PrintWriter outputStream = null;
        try
        {
    outputStream =
            new PrintWriter(new FileOutputStream("out.txt"));
}
catch(FileNotFoundException e)
{
    System.out.println("Error opening the file out.txt.");
    System.exit(0);
}

System.out.println("Enter three lines of text:");
String line = null;
Scanner keyboard = new Scanner(System.in);
int count;
        for (count = 1; count <= 3; count++)
        {
            line = keyboard.nextLine();
            outputStream.println(count + " " + line);
        }
        outputStream.close();
        System.out.println("Those lines were written to out.txt.");
    }
}
```

创建一个FileOutputStream类对象，作为PrintWriter类的构造器实参使用。

屏幕对话示例

```
Enter three lines of text:
A tall tree
in a short forest is like
a big fish in a small pond.
Those lines were written to out.txt.
```

（用给定的对话运行程序之后的）文件out.txt

```
1 A tall tree
2 in a short forest is like
3 a big fish in a small pond.
```

可以用一个文本编辑器来阅读这个文件。

图2-1 将输出发送到一个文本文件中去

快速参考：将文本文件连接到流以便写入（为输出打开一个文本文件）

可以按下列方式创建一个PrintWriter类的流，并将其连接到一个文本文件以便写入。

语法：

```
PrintWriter output_Stream_Name =
          new PrintWriter(new FileOutputStream(File_Name));
```

举例：

```
PrintWriter outputStream =
          new PrintWriter(new FileOutputStream("out.txt"));
```

之后，可以用方法println和print向文件写入。（实际上，可能会像图2-1那样，将流变量的声明和构造器的调用分隔开来。）

以这种方式使用时，FileOutputStream构造器就可以抛出一个IOException类型的FileNotFoundException，因此，PrintWriter构造器调用也可以抛出此异常。

记住：一个文件有两个名字

程序中用到的每个输入文件和输出文件都有两个名字：（1）操作系统使用的真实文件名；（2）连接到文件的流的名字。流名字是作为文件的临时名在程序内部使用的。将文件连接到流之后，程序通常会用流名字来表示这个文件。

快速参考：文件名

文件名的拼写规则取决于操作系统，而不是Java。将文件名赋给一个流的Java构造器时，给构造器的并不是一个Java标识符，而是一个对应于文件名的字符串。大多数常见的操作系统都允许在拼写文件名时使用字母、数字和点符号。很多操作系统还允许使用其他的字符，但对大多数情况来说，字母、数字和点符号就够用了。output.txt中.txt这样的后缀对Java程序来说没有特殊的意义。用后缀.txt来说明一个文本文件，但这只是一种惯例。可以使用操作系统允许的任意文件名。

按照前面的描述打开一个文本文件，将其连到一个PrintWriter类型的流上去时，可能会抛出一个FileNotFoundException，而且应该在一个catch块中捕获所有这些可能存在的异常。（实际上，可能抛出FileNotFoundException的应该是FileOutputStream构造器，但效果是一样的。）

注意，图2-1的try块只是打开了文件。这个块是唯一一个可能抛出异常的地方。还要注意：变量outputStream是在try块之外声明的，因此可以在try块之外使用变量outputStream。记住，在块中（甚至在try块中）声明的任何东西都是局部于那个块的。在名为“try块是一个块”的易犯错误部分中对这个概念进行了更完整的解释。

常见问题：为什么是FileNotFoundException？

打开一个文本文件将输出写入时，构造器可能会抛出一个FileNotFoundException。但在这种情况下，是想创建一个用于输出的新文件，那为什么要关心没有找到这个文件呢？答案非常简单：这个异常的名字起得不太好。FileNotFoundException并不表示文件没有找到。它实际上表示的是无法创建这个文件。如果无法创建文件，——例如，已经将这个文件名用作目录（文件夹）名的时候——就会抛

出FileNotFoundException。

FileNotFoundException是一种IOException，因此IOException的catch块也能工作，而且看起来更合理一些。但是，通常最好是去捕获能够捕获到的最具体的异常，这样可以给出更多的信息。

注意，如图2-1所示，PrintWriter类中的println方法向文本文件中写入时，工作方式与方法System.out.println向屏幕写出的方式一样。PrintWriter类中也有方法print，它的行为和System.out.print一样，只是输出是传送到一个文本文件中去的。图2-2对PrintWriter类中的一些方法进行了描述。

快速参考：final[①]

图2-2对很多方法的描述中都出现了关键字final。定义一个派生类时，不能重新定义任何被标记为final的方法。

PrintWriter(OutputStream streamObject)

如下所示，尽管可以用很多方式来使用它，但这可能是唯一一个需要用到的构造器。没有哪个构造器会接受一个文件名作为实参。如果想用文件名创建一个流，可以使用

new PrintWriter(new FileOutputStream(*File_Name*))

用这种方式使用构造器时，会创建一个空白文件。如果已经有一个名为*File_Name*的文件，文件中原来的内容就会丢失。如果想在原文件内容的末尾追加新文本，可以使用

new PrintWriter(new FileOutputStream(*File_Name*,true))

(对实参true的解释，请参见Java提示“向文本文件追加数据”)

以任意一种方式使用时，FileOutputStream构造器都可以抛出FileNotFoundException，因此PrintWriter构造器调用也可以抛出这个异常，这个异常是IOException类型的，可以在IOException的catch块中捕获。

File类会在2.3节中介绍。包含下列内容是为了使这个图中包含更完整的参考，但可以忽略这个关于构造器的条目中的其余内容，直到读过2.3节为止。

如果想用File类的对象创建一个流，可以使用

new PrintWriter(new FileOutputStream(*File_Object*))

以这种方式使用时，FileOutputStream构造器可以抛出IOExcpetion类型的FileNotFoundException，因此，PrintWriter构造器调用也可以抛出这个异常。注意，不能对File对象使用那种带有两个实参的附加形式。

public final void println(*Almost_Anything*)

实参可以是字符串、字符、整数、浮点数、布尔值或以+号连接的这些符号的任意组合。实参被发送到与流相连的文件中。输出实参之后，这一行就结束了，接下来的输出会被发送到下一行。以任意具有适当toString()方法的类对象作为实参时，这种方法也能正确地工作。

public final void print(*Almost_Anything*)

除了不会结束一行的输出之外，这个方法与println一样，因此接下来的输出会出现在同一行上。

public void close()

关闭流到文件的连接。这个方法在关闭文件之前会调用flush。

public void flush()

刷新输出流。这会强制那些将被缓存但还未物理地写入文件的所有数据实际写入文件。通常都不需要调用flush。

图2-2 PrintWriter类中的一些方法

① 如果没有读过《Java程序设计与问题解决：基础篇》中第7章关于继承的内容，可以安全地略过所有关于final的内容。

程序完成向一个文件的写入时，应该**关闭**（close）连接到那个文件的流。在图2-1中，用语句

```
outputStream.close();
```

关闭了连接到文件out.txt上的流。

PrintWriter类以及所有其他用于文件输入或输出的流，都有一个名为close的方法。调用这个方法时，系统会释放所有用来将流连接到文件，以及进行所有其他内部处理所需的资源。如果在程序结束前没有关闭文件，Java会在程序结束时将其关闭，但最安全的做法是用一个显式的close调用来关闭文件。

快速参考：关闭文本文件

当程序结束向文件的写入（或从文件的读出）时，应该通过调用close方法关闭连接到那个文件的流。

语法：

Stream_Name.close();

举例：

```
outputStream.close();
inputStream.close();
```

常见问题：为什么要关闭文件？

如果程序正常终止，但没有关闭文件，系统会自动地将其关闭。那为什么还要用对close方法的显式调用来关闭文件呢？原因至少有两个。首先，如果程序非正常终止，那么Java可能无法将文件关闭，文件会打开着但没有程序与之连接。这种情况可能会破坏文件。越早关闭文件，发生这种破坏的可能性就越小。第二，如果程序向一个文件中写入，稍后又从同一个文件中读出，它就必须在向文件写入之后关闭这个文件，然后再次打开文件进行读出。（Java中确实有一个类允许文件同时为输入输出打开，但在本书中我们不讨论那个类。）

△ 易犯错误：try块是一个块

再去看看图2-1中的程序。在try块之外声明变量outputStream并不是一个意外或小的风格问题。如果将那个声明移到try块内部，就会收到一条编译器错误消息。我们来看看其中的细节。

假设用下列代码

```
try
{
     PrintWriter outputStream =
       new PrintWriter(new FileOutputStream("out.txt"));
}
```

取代图2-1中的代码：

```
PrintWriter outputStream = null;
try
{
```

```
        outputStream = new PrintWriter(new FileOutputStream("out.txt"));
    }
```

这种替换看起来完全没有问题，但它将变量outputStream当成了try块的局部变量，这可能意味着不能在try块之外使用outputStream了。如果进行了这种修改，并尝试着编译修改过的程序，就会得到一条错误消息，表明在try块之外使用outputStream时，它是一个未定义的标识符。 △

△ 易犯错误：覆盖一个文件

以下列形式，用PrintWriter类将一个流连到文本文件上去时，通常会生成一个空文件：

```
PrintWriter outputStream = new PrintWriter(new FileOutputStream("out.txt"));
```

如果没有名为out.txt的文件，这条语句就会创建一个名为out.txt的空文件。如果已经存在一个名为out.txt的文件了，这条语句就会删除那个文件，创建一个新的、名为out.txt的空文件。因此，如果在执行上面这行代码之前，存在一个名为out.txt的文件，那么，那个文件中所有的数据都会被删除。2.3节会介绍如何测试一个文件是否存在，这样就可以避免无意中覆盖文件了。接下来的Java提示说明了如何在不丢失文件中已存数据的情况下向文本文件中添加数据。 △

□ Java提示：向文本文件追加数据

图2-1中用来打开输出文件的方法生成的是一个空文件。如果命名的文件已经存在了，原来的内容就会丢失。有时，并不希望这样，只是想将程序输出添加到文件末尾上去。这个过程被称为向文件**追加数据**（appending to a file）。如果想将程序输出追加到文件out.txt中去，就应该按下列方式将文件连接到outputStream流上去：

```
outputStream = new PrintWriter(new FileOutputStream("out.txt", true));
```

如果文件out.txt不存在，Java会创建一个以此为名的空文件，并将输出追加到这个空文件的末尾上去。（因此如果没有名为out.txt的文件存在，效果就和图2-1一样。）但是，如果文件out.txt已经存在了，那么原来的内容会保留，程序的输出会放在这些内容之后。

用这种方式向文本文件追加数据时，仍然应该使用与图2-1一样的try和catch块。

现在，可能会奇怪为什么要使用第二个参数true呢？为什么不使用字符串"append"之类的词呢？原因在于FileOutputStream类的这个构造器版本的设计使你可以用一个布尔变量（或表达式）来确定是向一个已存文件追加数据还是创建一个新文件。例如，可以使用下列代码：

```
System.out.println("Enter Append for append or New for a new file:");
Scanner keyboard = new Scanner(System.in);
String ans = keyboard.next();
boolean append = (ans.equalsIgnoreCase("append"));
outputStream =new PrintWriter(new FileOutputStream("out.txt", append));
```

从现在开始，程序向文件中写入的方式就与图2-1的程序完全一样了。如果用户回答“Append”，那么所有输入都会添加到原来的文件内容后面去。如果用户回答“New”，那么文件原来的所有内容都会丢失。Web上本书的源代码中包含的程序AppendTextFile.java就是使用前面的方式打开文件out.txt的图2-1程序版本。

快速参考：将一个文本文件连接到流上以追加数据（打开一个文本文件附加数据）

要创建一个PrintWriter类的流，并将其连到一个文本文件上，以便向文件中已存文本的末尾追加文本，可以按下列形式操作：

语法：

```
PrintWriter Output_Stream_Name =
    new PrintWriter(new FileOutputStream(File_Name, True_Boolean_Expression));
```

举例：

```
PrintWriter outputStream =
   new PrintWriter(new FileOutputStream("out.txt", true));
```

在这条语句之后，可以用方法println和print向文件写入，新文本会写在文件原有文本之后。（在实际编程中，可能希望像图2-1那样，将流变量的声明和构造器的调用分开。）

以这种方式使用时，FileOutputStream构造器可以抛出一个IOException类型的FileNotFoundException，因此，PrintWriter构造器调用也可以抛出此异常。

□ Java提示：用toString进行文本文件输出

在类中包含一个toString()方法是很常见的[①]。这个方法会生成一个直观上代表了对象中数据的字符串值。例如，在图2-3中，重新定义了《Java程序设计与问题解决：基础篇》中第4章的Species类，使其拥有一个toString()方法。要弄清楚toString()是怎样工作的，可以参见下列代码：

```
Species oneRecord = new Species("Calif. Condor", 27, 0.02);
outputStream.println(oneRecord.toString());
```

这段代码会产生输出

```
Name = Calif. Condor
Population = 27
Growth rate = 0.02%
```

方法toString有一项特殊的性质。如果不将其包含在System.out.println调用中，它会被自动调用，因此下列两行代码是等价的：

```
System.out.println(oneRecord.toString());
System.out.println(oneRecord);
```

这两条语句会生成下列输出：

```
Name = Calif. Condor
Population = 27
Growth rate = 0.02%
Name = Calif. Condor
Population = 27
Growth rate = 0.02%
```

对于toString来说，PrintWriter类中方法println（和print）的表现与System.out.println相同。图2-4对此概念进行了说明。

① 如果读过《Java程序设计与问题解决：基础篇》中第7章的话，这个方法听起来会很熟悉。那里对方法toString的讨论比这儿更详细一些。

```
import java.util.*;

/**
 Class for data on endangered species.
 */
public class Species
{
    private String name;
    private int population;
    private double growthRate;

    public Species( )
    {
        name = null;
        population = 0;
        growthRate = 0;
    }

    public Species(String initialName, int initialPopulation,
                                   double initialGrowthRate)
    {
        name = initialName;
        if (initialPopulation >= 0)
            population = initialPopulation;
        else
        {
            System.out.println("ERROR: Negative population.");
            System.exit(0);
        }
        growthRate = initialGrowthRate;
    }

    public String toString()
    {
        return ("Name = " + name + "\n"
              + "Population = " + population + "\n"
              + "Growth rate = " + growthRate + "%");
    }
      <其他方法与《Java程序设计与问题解决：基础篇》第4章图4-19中一样，但本章不需要对它们进行讨论。>
}
```

只有那些在这里没有列出来的方法需要这条语句。

这是一个新的、改进了的Species类定义，取代了《Java程序设计与问题解决：基础篇》中第4章的定义。

图2-3 带有toString方法的Species类

```
import java.io.*;

public class TextFileObjectOutputDemo
{
    public static void main(String[] args)
    {
        PrintWriter outputStream = null;
        try
        {
            outputStream =
```

图2-4 用toString来输出一个对象

```
            new PrintWriter(new FileOutputStream("species.records"));
        }
        catch(FileNotFoundException e)
        {
            System.out.println("Error opening species.records.");
            System.exit(0);
        }

        Species oneRecord = new Species("Calif. Condor", 27, 0.02);
        outputStream.println(oneRecord.toString());
        outputStream.println();
        outputStream.println(oneRecord);
        outputStream.close();
        System.out.println("End of Program.");
    }
}
```

这些代码行是等价的。

（程序运行之后的）文件species.records

```
Name = Calif.Condor
Population = 27
Growth rate = 0.02%

Name = Calif.Condor
Population = 27
Growth rate = 0.02%
```

方法toString是自动调用的，因此，省略它的话，结果也是一样的。

图2-4　（续）

自测题

4. 编写一些代码，创建一个作为PrintWriter类成员的、名为outStream的流，然后将这个流连接到一个名为sam的文本文件上去，这样程序就可以向这个文件发送输出了。以下列方式来实现代码：如果文件sam已经存在了，就将sam原有的内容删除，然后程序启动一个新的、名为sam的空文件。（如果文件sam不存在，当然还是会创建文件，并以空文件开始。）
5. 像自测题4那样，编写一些代码，创建一个作为PrintWriter类成员的、名为outStream的流，然后将这个流连接到一个名为sam的文本文件上去，这样程序就可以向这个文件发送输出了。但这一次，用这样一种方式来实现：如果文件sam已经存在了，sam原来的内容不会丢失，程序的输出会写在文件原有的内容之后。（如果文件sam不存在，还是会创建文件，并以空文件开始。）
6. 下列代码会抛出哪种异常，如果抛出了这个异常能说明什么问题？

```
PrintWriter outputStream =new PrintWriter(new FileOutputStream("out.txt"));
```

7. PrintWriter类有可以接受（表示文件名的）字符串作为实参，以使下列代码合法的构造器吗？

```
PrintWriter outputStream = new PrintWriter("myFile.txt");
```

2.2.2　带有BufferedReader的文本文件输入

图2-5包含了一个简单的程序，它从文本文件中读入数据，并将其写回到屏幕上去。文件

data.txt是一个人们可以用文本编辑器创建，或者Java程序可以用PrintWriter类创建的文本文件。

注意文件是怎样打开的：

```
BufferedReader inputStream = new BufferedReader(new FileReader("data.txt"));
```

BufferedReader类是从文本文件中读取数据时通常会使用的流类。

就像PrintWriter类一样，BufferedReader类没有可以将文件名作为实参的构造器，因此需要使用另一个类——在这种情况下，就是FileReader类——来帮忙打开这个文件。FileReader类可以接受一个文件名作为构造器实参，并会生成一个作为Reader对象的流。Reader是一个抽象类，它包含了所有名字中含有“Reader”的流。但是，与其去弄清楚这么复杂的解释，不如直接把下列代码当作一个长的、经过特别措辞的表达式，这个表达式将一个文本文件连接到了BufferedReader类的流上，使程序可以从文件中读出：

```
BufferedReader Stream_Name = new BufferedReader(new FileReader(File_Name));
```

注意，图2-5中的程序捕获了两种类型的异常：FileNotFoundException和IOException。打开文件可能会抛出一个FileNotFoundException，所有对inputStream. readLine()的调用都可能会抛出一个IOException。因为FileNotFoundException是一种IOException，所以，可以只为IOException使用catch块。但是，这样做的话，抛出异常时获得的信息就比较少。如果只用了一个catch块，而且有异常抛出，就无法得知问题是在打开文件时出现的，还是在打开文件后从中读出时出现的。

```
import java.io.*;

public class TextFileInputDemo
{
    public static void main(String[] args)
    {
       try
       {
           BufferedReader inputStream =
               new BufferedReader(new FileReader("data.txt"));

           String line = null;
           line = inputStream.readLine();
           System.out.println("The first line in data.txt is:");
           System.out.println(line);

           line = inputStream.readLine();
           System.out.println("The second line in data.txt is:");
           System.out.println(line);
           inputStream.close();
       }
       catch(FileNotFoundException e)
       {
```

图2-5 从文本文件获取输入

```
            System.out.println("File data.txt was not found");

            System.out.println("or could not be opened.");
        }
        catch(IOException e)
        {
            System.out.println("Error reading from file data.txt.");
        }
    }
}
```

文件data.txt

```
1 2
buckle my shoe.
3 4
shut the door.
```

可以用文本编辑器或另一个Java程序生成这个文件。

屏幕输出

```
The first line in data.txt is:
1 2
The second line in data.txt is:
buckle my shoe.
```

图2-5　（续）

快速参考：将文本文件连接到流上进行读取（打开一个文本文件进行读取）

按下列形式创建一个BufferedReader类的流，并将其连接到一个文本文件上进行读取：

语法：

```
BufferedReader Stream_Name =new BufferedReader(new FileReader(File_Name));
```

举例：

```
BufferedReader inputStream =new BufferedReader(new FileReader("data.txt"));
```

这条语句之后，就可以用方法readLine和read从文件中读取数据了。

以这种方式使用时，FileReader构造器可以抛出IOException类型的FileNotFoundException，因此BufferedReader构造器调用也可以抛出这个异常。

快速参考：FileNotFoundException类

如果程序试图打开一个文件进行读入，但这个文件并不存在，就会抛出一个FileNotFoundException。如在本章前面所见，在其他一些特定的情况下也会抛出FileNotFoundException。

注意，BufferedReader类的方法readLine从文本文件中读入的方式与Scanner类的方法nextLine从键盘上读入一行文本的方式相同。但是，BufferedReader类中没有Scanner中任何其他的读方法，比如nextInt和nextDouble。图2-6对BufferedReader类中的一些方法进行了描述。

BufferedReader(Reader readerObject)

这是可能会用到的唯一一个构造器。没有可以接受文件名作为实参的构造器。如果想用文件名创建一个流，可以编写

new BufferedReader(new FileReader(*File_Name*))

以这种方式使用时，FileReader构造器可以抛出IOException类型的FileNotFoundException，因此BufferedReader构造器调用也可以抛出这个异常。

File类会在2.3节中介绍。这里包含的其他内容是为了使这个图中的介绍更完整，但可以忽略本条目对构造器的其余介绍，留待读过2.3节之后再来阅读。

要想用File类的对象创建一个流，可以编写代码

new BufferedReader(new FileReader(*File_Object*))

以这种方式使用时，FileReader构造器可以抛出一个IOException类型的FileNotFoundException，这样BufferedReader构造器调用也可以抛出此异常。

public String readLine() throws IOException

从输入流中读取一行输入，并将其返回。如果读操作超出了文件的末尾，就返回null。(注意，在文件末尾不会抛出EOFException。文件的末尾是通过返回null来说明的。)

public int read() throws IOException

从输入流中读取单个字符，并将那个字符作为一个int值返回。如果读操作超出了文件的末尾，就返回-1。注意，值是作为int类型返回的。要获取一个char，必须对返回值执行强制类型转换。通过返回的-1来说明文件的末尾。(所有“真的”字符都会返回一个正整数。)

public void close() throws IOException

关闭流与文件的连接。

图2-6 BufferedReader类中的一些方法

由于BufferedReader中没有nextInt这样可以读取数字的方法，所以用BufferedReader类从文本文件中读入一个数字的唯一方法就是将其作为一个字符串读入，然后将字符串转换成数字。本章2.2.4节对一些将字符串转换成数字的技术进行了讨论。

BufferedReader类中确实有一个名为read的方法，可以读入单个字符。但是，它有个很复杂的问题：方法read返回的是对应于读入字符的int类型的值；而不是字符自身。因此，要想获得单个字符，就要使用强制类型转换，比如

```
char next = (char)(inputStream.read());
```

如果inputStream在BufferedReader类中，且与一个文本文件相连，这条语句就会将next设置为文件中还未被读出的第一个字符。

自测题

8. 编写一些代码，创建一个PrintWriter类型、名为textStream的流，并将这个流连到一个名为dobedo的文本文件上去，这样程序就可以将输出发送到文本文件dobedo中去了。
9. 假设运行了一个通过自测题8中定义的流向文本文件dobedo写入的程序。编写一些代码，创建一个名为inputStream的流，按照本节讨论的方式用这个流从文本文件dobedo中进行读取。
10. BufferedReader有一个名为readLine的方法，这个方法本质上与Scanner类中的nextLine方法相同，只是前者是从文本文件中获取其输入的。BufferedReader中有nextInt或nextDouble这样的方法吗？
11. BufferedReader类中的方法read和readLine会抛出异常吗？如果会，它们会抛出什么类型的异常？

12. 注意图2-1和图2-7中try块之间的一个区别：图2-1中的try块只打开了文件，而图2-7中的try块则包含了程序中的大部分动作。为什么图2-7中的try块比图2-1中的try块大呢？
13. BufferedReader类中的readLine方法返回的值是什么类型的？BufferedReader类中的read方法返回的值又是什么类型的？

编程示例：从键盘读入一个文件名

到目前为止，我们在程序中一直用引用字符串作为文件名。但是，在编写程序时，可能不知道文件名是什么，因此，可能希望用户在程序运行时从键盘输入文件名。这项任务很容易完成。只要通过程序将文件名读入一个String类型的变量中，并用那个String变量来取代文件名就行了。图2-7对这种技术进行了说明。

注意，图2-7中的程序从两个不同的地方读取了输入。名为keyboard的Scanner对象用来从键盘读取文件名。名为inputStream的流被连接到文件data.txt上去，并从这个文件中读取其输入。

```
import java.io.*;
import java.util.*;

public class TextFileInputDemo2
{
    public static void main(String[] args)
    {
        System.out.println("Enter file name:");
        Scanner keyboard = new Scanner(System.in);
        String fileName = keyboard.next();

        try
        {
            BufferedReader inputStream =
                new BufferedReader(new FileReader(fileName));

            String line = null;
            line = inputStream.readLine();
            System.out.println("The first line in " + fileName + " is:");
            System.out.println(line);
            line = inputStream.readLine();
            System.out.println("The second line in " + fileName + " is:");
            System.out.println(line);

            inputStream.close();
        }
        catch(FileNotFoundException e)
        {
            System.out.println("File " + fileName + " not found.");
        }
        catch(IOException e)
        {
            System.out.println("Error reading from file " + fileName);
```

图2-7　读入一个文件名

```
        }
    }
}
```

文件data.txt

```
1 2
buckle my shoe.
3 4
shut the door.
```

可以用文本编辑器或另一个Java程序来生成这个文件。

屏幕对话示例

```
Enter file name:
data.txt
The first line in data.txt is:
1 2
The second line in data.txt is:
buckle my shoe.
```

图2-7 （续）

□ Java提示：使用路径名

将一个文件名作为构造器实参给出后，以我们讨论过的任意一种方式打开文件，可以使用一个简单的文件名，在这种情况下就假定文件所在的目录（文件夹）与运行程序的目录（文件夹）是一样的。还可以使用全称或相对路径名。**路径名**（path name）不仅给出了文件的名字，而且还说明了文件所处的目录（文件夹）。如其名字所示，**全路径名**（full path name）给出了从根目录开始的完整路径名。**相对路径名**（relative path name）给出了从程序所处目录开始，直到文件的路径。指定路径名的方式取决于特定的操作系统，在这里我们只对部分细节进行了讨论。

典型的UNIX路径名为

```
/user/smith/home.work1/data.txt
```

要创建一个连接到此文件的输入流，可以使用

```
BufferedReader inputStream =
      new BufferedReader(new FileReader("/user/smith/home.work1/data.txt"));
```

Windows在路径名中使用的是\而不是/。典型的Windows路径名为

```
D:\homework\hw1\data.txt
```

要创建一个连接到此文件的输入流，可以使用

```
BufferedReader inputStream =
      new BufferedReader(
        new FileReader("D:\\homework\\hw1\\data.txt"));
```

注意，要用\\取代\，否则Java会将反斜杠和一个字符作为一对来解释，比如\h就会被解释为换码符。尽管通常在引用字符串中使用反斜杠时需要非常小心，但从键盘读入名字时不存在这个问题。假设运行了一个图2-7那样的程序，并假设对话部分如下所示，其中第二行是由用户输入的。

```
Enter file name:
D:\homework\hw1\data.txt
```

程序可以理解这个路径名。用户不需要键入

```
D:\\homework\\hw1\\data.txt
```

而且，尽管不一定会这样，但实际上在输入中使用\\可能造成文件名的错误读入。用户从键盘输入时，Java“知道”\h肯定是一个反斜杠符号后面跟着一个h，而不是一个换码符。因为用户输入的所有内容都是由字符组成的，所以Java“知道”这一点。

一种可以完全避免这些换码符问题的方式就是，在编写路径名时总是使用UNIX的惯用写法。即使是在操作系统语法与之不符的计算机上运行，Java程序也可以接受以Windows或UNIX格式书写的路径名。因此，创建连接到Windows文件

```
D:\homework\hw1\data.txt
```

上的输入流的另一种方式为

```
BufferedReader inputStream =
      new BufferedReader(new FileReader("D:/homework/hw1/data.txt"));
```

2.2.3 StringTokenizer类

用BufferedReader类从文本文件读取输入时，可以读入整行内容，也可以读入单个字符。通常可能希望读入单词，但BufferedReader没有读入单个单词的方法。但是，通常可以首先读入整行文本，然后用预定义类StringTokenizer将这行内容分成单个的单词，用这种方式来实现基本上等价的功能。

下面的例子说明了StringTokenizer类一种简单、但典型的使用方式：

```
StringTokenizer wordFinder = new StringTokenizer("We love you madly.");
while(wordFinder.hasMoreTokens())
{
    System.out.println(wordFinder.nextToken());
}
```

代码会产生下列输出：

```
We
love
you
madly.
```

（new后面的那部分）构造器生成了一个新的StringTokenizer类对象。这个对象可以生成作为构造器实参的字符串中的单个单词。这些单个的单词被称为标记（token）。

第一次调用方法nextToken时，它会返回第一个标记（单词），第二次调用时会返回第二个标记，以此类推。只要nextToken没有返回字符串中的所有标记，方法hasMoreTokens就会返回true，当方法nextToken返回了字符串中的所有标记之后，它就会返回false。

StringTokenizer类位于java.util［“实用工具（utility）”的缩写］包中。因此，任何使用StringTokenizer类的类或程序都必须在文件起始处包含下列代码行：

```
import java.util.*;
```

像前面的例子那样，用单个实参使用StringTokenizer的构造器时，标记就是无空白字符的子字符串，将空白字符作为标记间的分隔符。一个或多个空白字符组成的任意字符串都被当作一个分隔符。（回想一下空白字符包括空格、新行字符以及其他在纸上作为空白打印的字符。）因此，在前面的例子中，最后一个单词是包含句号在内的“madly.”。包含句号是因为它不是空白字符，因此也就不是分隔符。

如果想指定自己的分隔字符集，而不是简单地接受包含空白字符在内的默认集合，就应该在用new创建字符串标记生成器（Tokenizer）时给出构造器的第二个实参。第二个实参是一个由所有的分隔字符组成的字符串。因此，如果希望分隔符中包含空白、新行字符、句号和逗号的话，可以按照下面的例子来实现：

```
StringTokenier secondWordFinder =
            new StringTokenizer("Love you, madly.", " \n.,");
while(secondeWordFinder.hasMoreTokens())
{
    System.out.println(secondWordFinder.nextToken());
}
```

这段代码会产生如下输出：

```
Love
you
madly
```

一定要注意，句号和逗号不是所生成标记的一部分，因为它们现在是标记分隔符了。还要注意到的是标记分隔符字符串是构造器的第二个实参。

图2-8对StringTokenizer类中的一些方法进行了总结。

`public StringTokenizer(String theString)`
用空白字符在theString中寻找标记的标记生成器构造器。

`public StringTokenizer(String theString, String delimiters)`
将字符串delimiters中的字符作为分隔符在theString中寻找标记的标记生成器构造器。

`public boolean hasMoreTokens()`
测试这个标记生成器的字符串中是否还有其他标记。与nextToken联合使用时，只要nextToken还没有返回字符串中的所有标记，它就返回true，否则，就返回false。

`public String nextToken()throws NoSuchElementException`
返回这个字符串标记生成器中的下一个标记。（如果没有更多的标记可返回，就抛出一个NoSuchElement-Exception。）

`public int countTokens()`
返回nextToken还要返回的剩余标记的数目。

图2-8 StringTokenizer类中的一些方法

□ Java提示：对文件的末尾进行测试

使用BufferedReader类时，如果程序试图用方法readLine或read去读取超出文件末尾的内容，方法都会返回一个特定的值来说明已经抵达文件的末尾了。当readLine试图去读取超出文件末尾的内容时，会返回null值。因此，程序可以通过测试readLine是否返回null来测试程序的末尾。图2-9对这种技术进行了说明。方法read试图读取超出文件末尾的内容时，会返回值-1。由于每个普通的字符所对应的int值都是正的，所以可以用此条件来测试是否达到文件的末尾。

```
import java.io.*;
import java.util.*;

/**
```

图2-9 查看文本文件的末尾

```
 Makes storylines.txt the same as story.txt,
 but with each line numbered.
*/
public class TextEOFDemo
{
   public static void main(String[] args)
   {
      try
      {
         BufferedReader inputStream =
               new BufferedReader(new FileReader("story.txt"));
         PrintWriter outputStream =
               new PrintWriter(new FileOutputStream("storylines.txt"));
         int count = 0;
         String line = inputStream.readLine();
         while (line != null)
         {
            count++;
            outputStream.println(count + " " + line);
            line = inputStream.readLine();
         }
         System.out.println(count + " lines written to storylines.txt.");
         inputStream.close();
         outputStream.close();
      }
      catch(FileNotFoundException e)
      {
         System.out.println("File opening problem.");
      }
      catch(IOException e)
      {
         System.out.println("Error reading from file story.txt.");
      }
   }
}
```

文件story.txt

```
Once upon a time
there were three little
auto mechanics: Click,
Clack, and Joe.
```

Story.txt可以由文本编辑器或另一个Java程序生成。

（运行程序之后的）文件storylines.txt

```
1 Once upon a time
2 there were three little
3 auto mechanics:Click,
4 Clack, and Joe.
```

如果你的storylines.txt版本中4后面有几个编了号的空行，也不必担心。那是由于story.txt的末尾有一些空行。

图2-9　（续）

快速参考：查看文本文件的末尾

BufferedReader类的方法readLine试图读取超出文本文件末尾的内容时，会返回null。BufferedReader类的方法read试图读取超出文本文件末尾的内容时，会返回-1。(两种方法都不会抛出EOFException。)

自测题

14. BufferedReader类的方法readLine试图读取超出文本文件末尾的内容时，会发生什么情况？如何利用这种条件来测试文件是否结束？
15. BufferedReader类的方法read返回的值是哪种类型？
16. BufferedReader类的方法read试图读取超出文本文件末尾的内容时，会发生什么情况？如何利用这种条件来测试是否达到文件的末尾？

2.2.4 FileReader和FileOutputStream类

每次创建BufferedReader或PrintWriter类的流时，都会分别使用流类FileReader或FileOutputStream。例如，图2-9中使用了下列代码行：

```
BufferedReader inputStream = new BufferedReader(new FileReader("story.txt"));
PrintWriter outputStream = new PrintWriter(new FileOutputStream("storylines.txt"));
```

我们使用FileReader和FileOutputStream类是因为它们可以接受文件名作为构造器实参，比如前面例子中的实参"story.txt"和"storylines.txt"。正如前面所解释的那样，BufferedReader和PrintWriter都不接受文件名作为实参。因此，用一个字符串名将BufferedReader对象与一个文件连接起来时，必须分两步来实现。首先，可以用new FileReader("story.txt")创建一个FileReader类的对象，然后用这个FileReader类对象通过

```
BufferedReader inputStream = new BufferedReader(new FileReader("story.txt"));
```

创建一个BufferedReader类的对象。如果我们说这种将inputStream连接到文件story.txt上去的方式等价于：

```
FileReader aFileReaderObject = new FileReader("story.txt");
BufferedReader inputStream = new BufferedReader(aFileReaderObject);
```

可能对理解这个过程的工作方式会有所帮助。与之类似，要分两步用FileOutputStream根据文件名生成一个PrintWriter输出流。

既然new FileReader("story.txt")生成了一个输入流，而这个输入流连到了一个由字符串"story.txt"命名的文件上，那么为什么我们不使用那个流，而要创建一个BufferedReader对象呢？原因在于FileReader类中没有read和readLine这样的、我们想要的好的输入方法。要获得BufferedReader类中的好方法，需要将FileReader对象转换成一个BufferedReader对象。每次用这种方式将流链接起来时，得到的流通常都拥有最后一个命名的流（即，最左边的流）中的方法。例如，在

```
BufferedReader inputStream =
        new BufferedReader(new FileReader("story.txt"));
```

中，inputStream流是BufferedReader类的对象，因此，拥有BufferedReader类中所有的方法，但不拥有FileReader类中的方法。

我们没有直接使用FileOutputStream构造器生成的流，而是创建了PrintWriter流也是出于类似的原因。

由于只会在BufferedReader和PrintWriter这样的流类的构造器实参中使用FileReader和FileOutputStream类，所以实际上，除了FileReader和FileOutputStream类的构造器方法之外，我们并不太关心它们拥有或者不拥有哪些方法，而是关心这些类有哪些构造器。因为在前面的例子中和类似的情况下，我们用new使用的是它们构造器。尤其是，需要了解FileReader和FileOutputStream类都有一个可以接受文件名的构造器。

快速参考：FileReader和FileOutputStream

本书只是使用了FileReader和FileOutputStream的构造器，而没有使用它们的其他部分。这两个类都有一个将文件名作为其实参的构造器。我们用这些构造器来生成BufferedReader和PrintWriter这样的流类的构造器实参，这些类的构造器不能将文件名作为其实参。下面就是FileReader和FileOutputStream这种用法的示例：

```
PrintWriter outputStream = new PrintWriter(new FileOutputStream("stuff.txt"));
BufferedReader inputStream = new BufferedReader(new FileReader("story.txt"));
```

FileReader和FileOutputStream的构造器可以抛出FileNotFoundException类异常，这种异常是IOException类型的一种。

自测题

17. 下面哪些类拥有可以接受文件名作为实参的构造器？PrintWriter，BufferedReader，FileReader，FileOutputStream。

18. 下列代码合法吗？

```
FileReader readerObject = new FileReader("myFile.txt");
BufferedReader inputStream = new BufferedReader(readerObject);
```

2.3 File类

一盎司的预防相当于一磅的治疗。

——谚语

本节将对File类进行描述。File类不是一个I/O流类，但在进行文件I/O时非常有用。

使用File类

可以用File类中的方法来查看文件的特性。可以查看是否存在具有特定名称的文件以及文件是否可读等内容。图2-10给出了一个示例程序，这个程序在读入一个文本文件时使用了File类。（File类在二进制文件和文本文件中的工作方式相同。）

```
/**
 Demonstrates use of the class File with text files.
*/
import java.io.*;
import java.util.*;

public class FileClassDemo
{
    public static void main(String[] args)
    {
String name = null;
System.out.println("I will show you the first line");
System.out.println("in a text file you name.");
System.out.println("The file must contain one or more lines.");

System.out.println("Enter file name:");

Scanner keyboard = new Scanner(System.in);
name = keyboard.next();
File fileObject = new File(name);
while (( ! fileObject.exists())
                  || ( ! fileObject.canRead()))
{
   if ( ! fileObject.exists())
      System.out.println("No such file");
   else // ! fileObject.canRead()

      System.out.println("That file is not readable.");
      System.out.println("Enter file name again:");
      name = keyboard.next();
      fileObject = new File(name);
 }

 try
 {
      BufferedReader fileInput =
            new BufferedReader(new FileReader(name));
      System.out.println("The first line in the file is:");
      String firstLine = fileInput.readLine();
      System.out.println(firstLine);
      fileInput.close();
}
catch(IOException e)
{
   System.out.println("Problem reading from file.");
         }
      }
}
```

如果愿意的话，可以用fileObject取代name作为FileReader的实参。

图2-10 在处理文本文件时使用File类

File类像文件名字的包装类一样。"treasure.txt"这样的字符串可能是个文件名，但它只具有字符串的特性，Java并没有把它当作一个文件名。另一方面，对象

```
new File("treasure.txt")
```

就不仅仅是一个字符串了。它是一个对象，并且“知道”它应该命名了一个文件。

假设用下列代码创建了一个File对象，并将其命名为fileObject：

```
File fileObject = new File("treasure.txt");
```

然后，就可以用File类中名为exists的方法来测试是否存在名为treasure.txt的文件了。比如，可以编写代码

```
if (!fileObject.exists())
    System.out.println("No file by that name.");
```

如果存在那个名字的文件，可以用方法canRead来确定操作系统是否允许从那个文件中进行读取。比如，可以编写代码

```
if (!fileObject.canRead())
    System.out.println("Not allowed to read from that file.");
```

大多数操作系统都允许将一些文件指定为不可读或仅对部分用户可读的。方法canRead是查看你或其他人是否无意中（或有意）将文件设置为不可读的一种很好的方式。

方法canWrite与canRead类似，只是前者是查看操作系统是否允许向文件中写入的。大多数操作系统都允许指定一些文件不可写或仅对部分用户可写。

注意图2-10中的注解。注解说明可以用File对象取代文件名作为FileReader构造器的实参。只要可以将文件名作为实参，通常都（但不总是）可以用File类型的对象作为流构造器的实参。尤其是，FileReader、FileOutputStream和FileInputStream都可以接受File对象作为实参。（FileInputStream类是用于二进制文件的，将在2.4节讨论。）因此，在通常用来打开一个文件的这部分代码中，可以用File类对象来取代文件名，只有一个很重要的例外：向一个文件追加数据时，必须用文件名来打开文件；不能用File对象取代文件名。

图2-11列出了File类中的部分方法。

```
public boolean exists()
```

测试在创建File对象时，是否存在一个文件与构造器实参同名。

```
public boolean canRead()
```

测试程序是否能从文件读出。

```
public boolean canWrite()
```

测试程序是否能向文件写入。

```
public boolean delete()
```

试图删除文件。如果能够删除文件，就返回true。不能删除文件就返回false。

```
public long length()
```

返回以字节为单位的文件长度。

```
public String getName()
```

返回文件的名字。（注意，这个名字不是路径名，只是文件名。）

```
public String getPath()
```

返回文件的路径名。

图2-11　File类中的一些方法

快速参考：File类

File类就像文件名的包装类一样。File类的构造器将字符串作为实参，并生成一个对象，可以将这个对象当成具有那个名字的文件。可以用File对象和File类中的方法来回答下面这样的问题：文件存在吗？程序有读取文件的权限吗？程序有向文件写入的权限吗？图2-11对File类中的一些方法进行了总结。

举例：

```
File fileObject = new File("stuff.txt");
if (!fileObject.exists())
   System.out.println("There is no file named stuff.txt");
else if(!fileObject.canRead())
   System.out.println("File stuff.txt is not readable.");
```

自测题

19. 编写一个完整的Java程序，请求用户输入一个文件名；测试那个文件是否存在；并且如果文件确实存在，询问用户是否应该将其删除，然后按照用户的要求去做。

2.4 基本的二进制文件I/O

小白兔戴上眼镜，问道："我该从哪儿开始呢？陛下。""从开始的地方开始吧，一直读到末尾，然后停止。"国王郑重地说。

——刘易斯·卡莱尔《爱丽丝漫游奇境记》

二进制文件存储数据的格式与（在《Java程序设计与问题解决：基础篇》的第1章中描述的）计算机主存中存储数据的格式相同。每个数据条目，比如一个整数，都是作为一系列字节存储的。Java程序读入这些字节的方式与从计算机主存中读入整数这样的数据条目的方式完全一样。二进制文件存储数据的格式与计算机主存中存储数据所用格式相同的原因在于，这样可以非常有效地处理二进制文件。

Java程序创建的二进制文件可以从一台计算机移到另一台计算机上去，并仍能被Java程序读取——但也只能被Java程序读取。通常不能用文本编辑器读取，用除Java之外的编程语言编写的程序通常也无法读取这些文件。

处理二进制文件时常用的流类为ObjectInputStream和ObjectOutputStream。每个类都有一些可以一次读或写一个字节的方法。这些流也可以自动地将数字和字符转换成可以存储在二进制文件中的字节。这就使得在向文件写入，或从文件中读出时，文件中放置的数据好像不仅仅是字节，而是字符串，或int、char和double这样任意的Java基本数据类型条目，甚至是定义的类对象一样。如果不需要通过编辑器来访问文件，那么从文件中读取、或向文件中写入数据的最简单有效的方式就是用ObjectOutputStream向二进制文件写入，用ObjectInputStream从二进制文件中读出。

本节介绍了String类型和基本类型的二进制文件I/O。在2.5节中，会介绍类类型对象的二进制文件I/O。

记住：import语句

这里讨论的二进制文件I/O需要与文本文件I/O一样的import语句，即

```
import java.io.*;
```

2.4.1　用ObjectOutputStream向二进制文件输出

图2-12显示了一个向二进制文件中写入整数的程序。我们来看看那个程序的细节。

```
import java.io.*;
import java.util.*;

public class BinaryOutputDemo
{
   public static void main(String[] args)
   {
      try
      {
         ObjectOutputStream outputStream =
             new ObjectOutputStream(new FileOutputStream("numbers.dat"));

         int n;
         Scanner keyboard = new Scanner(System.in);
         System.out.println("Enter nonnegative integers.");
         System.out.println("Place a negative number at the end.");
         do
         {
            n = keyboard.nextInt();
            outputStream.writeInt(n);
         }while (n >= 0);

         System.out.println("Numbers and sentinel value");
         System.out.println("written to the file numbers.dat.");
         outputStream.close();
      }
      catch(IOException e)
      {
         System.out.println("Problem with output to file numbers.dat.");
      }
   }
}
```

二进制文件的关闭方式与文本文件相同。

屏幕对话示例

```
Enter nonnegative integers.
Place a negative number at the end.
1 2 3 -1
Numbers and sentinel value
written to the file numbers.dat.
```

（程序运行之后的）文件numbers.dat

```
1
2
3
-1
```

注意，文件中有-1。它不一定要在文件中，但在这个程序中，希望将它作为一个标记值使用。

这个文件是个二进制文件。不能用文本编辑器来读取这个文件。

图2-12　用ObjectOutputStream向文件写入

注意，这个程序的实质性内容位于try块中。以我们所讨论的方式进行二进制文件I/O的所有代码都可以抛出IOException。程序可以在catch块中捕获任意的IOException，这样，如果抛出了一个IOException，就会得到一条错误消息，然后程序正常结束。

向文件numbers.dat写入的输出流是用下列代码创建并命名的：

```
ObjectOutputStream outputStream =
 new ObjectOutputStream(new FileOutputStream("numbers.dat"));
```

像文本文件一样，这个过程被称为**打开**（opening）（二进制）文件。如果文件numbers.dat事先不存在，这条语句就会创建一个名为numbers.dat的空文件。如果文件numbers.dat已经存在了，这条语句会清除文件的内容，以空文件开始。这种情况基本上与文本文件一样，只是在这里使用了不同的类。

与文本文件和屏幕输出不同，ObjectOutputStream类没有名为println的方法。但是，如图2-12所示，ObjectOutputStream类的对象确实有一个名为writeInt的方法，可以将单个int值写入到文件中去，它还有其他一些输出方法，稍后会对这些方法进行讨论。

一旦ObjectOutputStream类中的流被连到文件上去，就可以用方法writeInt向文件写入整数了，如图2-12中的下列代码行所示：

```
outputStream.writeInt(n);
```

在图2-12中，前面的代码行将变量n中的int值写入到文件numbers.dat中去了。去看看图2-13，你就会发现还有一些类似的方法writeLong、writeDouble和writeFloat可以将其他类型的数字写入到文件中去。还要注意，所有这些方法都可以抛出IOException。

```
public ObjectOutputStream(OutputStream streamObject)
```

没有可以将文件名作为实参的构造器。如果想用文件名创建一个流的话，可以编写代码

```
new ObjectOutputStream(new FileOutputStream(File_Name))
```

这条语句会创建一个空文件。如果已经有一个名为*File_Name*的文件了，文件中原来的内容就会丢失。
如果想用File类的对象创建一个流，可以使用语句

```
new ObjectOutputStream(new FileOutputStream(File_Object))
```

FileOutputStream的构造器可能会抛出IOException类型的FileNotFoundException。如果FileOutputStream构造器成功执行，那么ObjectOutputStream的构造器可能会抛出一个不同的IOException。

```
public void writeInt(int n) throws IOException
```

将int值n写入输出流中。

```
public void writeLong(long n) throws IOException
```

将long值n写入输出流中。

```
public void writeDouble(double x) throws IOException
```

将double值n写入输出流中。

```
public void writeFloat(float x) throws IOException
```

将float值n写入输出流中。

```
public void writeChar(int n) throws IOException
```

将char值n写入输出流中。注意，它希望实参是个int值。可以使用强制类型转换将char值转换成int值。例如，可以编写代码

```
outputStream.writeChar((int)'A');
```

图2-13 ObjectOutputStream类中的一些方法

实际上，不需要将其强制转换为int，因为Java会自动将char值转换为int值。因此，下列代码与前面的writeChar调用是等价的：

```
outputStream.writeChar('A');
```

```
public void writeBoolean(boolean b) throws IOException
```

将boolean值b写入输出流中。

```
public void writeUTF(String aString) throws IOException
```

将String值aString写入输出流中。UTF是一种特殊的对字符串编码的方法。要从文件中将字符串读回，应该使用ObjectInputStream类中的readUTF方法。

```
public void writeObject(Object anObject) throws IOException,
        NotSerializableException, InvalidClassException
```

将其实参写入输出流中。对象实参应该是一个可串行化类的对象，本章稍后会对这个概念进行讨论。如果anObject的类不是可串行化的，就抛出一个NotSerializableException。如果串行过程出了问题，就抛出一个InvalidException。2.5节对writeObject方法进行了介绍。

```
public void close() throws IOException
```

关闭流和文件的连接。这个方法在关闭文件之前会调用flush。

```
public void flush() throws IOException
```

刷新输出流。这个方法会强制将所有已经缓存但还未物理写入文件中的数据实际写入文件中去。通常都不需要调用flush。

图2-13　（续）

在图2-12中，我们使文件numbers.dat中的数字看起来好像都是一行一个以人类可读的形式写出来的。但是，实际上不是这样的。数字之间没有分行或其他的分隔符。相反，数字是一个紧接着一个写入文件中的，而且是按照数字在计算机主存中的编码方式，将它们编码成字符序列的。不能用编辑器读取这些经过编码的int值。实际上，它们只能由另一个Java程序来读取。

图2-13中包含了ObjectOutputStream类中一些方法的列表。

快速参考：将二进制文件连接到流上进行写入（打开一个输出文件）

创建一个ObjectOutputStream类的流，并按下列形式将其连接到一个二进制文件上去：

语法：

```
ObjectOutputStream Output_Stream_Name =
    new ObjectOutputStream(new FileOutputStream(File_Name));
```

FileOutputStream的构造器可能会抛出一个IOException类型的FileNotFoundException。如果FileOutputStream构造器成功执行，那么ObjectOutputStream的构造器可能会抛出一个不同的IOException。一个用于IOException的catch块就可以涵盖所有的情况。

举例：

```
ObjectOutputStream myOutputStream =
                    new ObjectOutputStream(new FileOutputStream("myfile.dat"));
```

打开文件之后，可以用（图2-13中）ObjectOutputStream 类的方法向文件中写入。

至此对图2-12中示例程序的描述就结束了，但还没有对图2-13中列出的所有ObjectOutputStream方法进行讨论。

可以用ObjectOutputStream类的流来输出任意基本类型的值，以及写出String类型的数据。在ObjectOutputStream类中，每种基本数据类型都有一个相应的写入方法。已经提到过那些用来输出数字的写入方法了。方法writeChar可以用来输出单个字符。方法writeBoolean可以用来输出单个布尔值。例如，下列代码会向连接到名为outputStream的流的文件输出字符'A'和一个布尔值false：

```
outputStream.writeChar((int)'A');
outputStream.writeBoolean(false);
```

方法writeChar有一个可能令人吃惊的特性：它希望它的实参是int类型的。因此，如果有一个char类型的值，可以在将其赋于方法writeChar之前，用强制类型转换将这个char值转换成int值。例如，要想输出名为symbol的char变量中的内容，可以使用

```
outputStream.writeChar((int)symbol);
```

实际上，不需要编写向int的强制类型转换，因为Java会自动地将char值转换成一个int值，在这儿利用自动强制类型转换的优点是有意义的。因此，下列代码等效于前面的writeChar调用：

```
outputStream.writeChar(symbol);
```

这就实现了基本类型的输出。要输出String类型的值，可以使用方法writeUTF。例如，如果outputStream是ObjectOutputStream类型的流，下列代码就会将字符串"Hi Mom"写入与那个流相连的文件中去：

```
outputStream.writeUTF("Hi Mom");
```

当然，对writeUTF或任意的写入方法来说，都可以用适当类型（在这个例子中就是String类型）的变量作为方法的实参。

可以将不同类型的输出写入同一个文件中去。因此，可以写入不同值的组合，比如int、double和String值的组合。但是，将不同类型的数据混合放入文件中需要特别小心，以便能够从文件中读出这些类型。要将其读出，就需要知道各种不同类型在文件中的出现顺序，因为，如你所知，从文件中读出数据的程序要用不同的方法来读取每种不同类型的数据。

常见问题：UTF表示什么意思？

（使用ObjectOutputStream类时）要写出一个int值，可以使用writeInt；要写出double值，可以使用writeDouble；以此类推。但是，要写出一个字符串，使用的不是writeString，而是writeUTF。在ObjectOutputStream中没有名为writeString的方法。为什么要使用writeUTF这个奇怪的名字呢？UTF表示Unicode文本格式（Unicode Text Format）。这不是一个描述性很强的名字。完整的故事是这样的：

回想一下，Java使用的是Unicode字符集，这个字符集中包含了很多亚洲语言及其他语言中的字符。这些语言的字符集与英语字符集有很大区别。大多数编辑器和操作系统都使用ASCII字符集，这是英语和典型的Java程序中通常使用的字符集。ASCII字符集是Unicode字符集的一个子集，因此Unicode字符集中有大量你不需要的字符。有一种标准的方式来对所有的Unicode字符进行编码，但对说英语国家，这不是一种很有效的编码策略。UTF编码策略是一种替换策略，它仍然是对所有的Unicode字符进行编码，但比较照顾ASCII字符集。UTF编码方法为ASCII字符使用了短的、高效的编

码。代价是要为其他Unicode字符使用长的、低效编码。但是，因为很可能不会用到其他Unicode字符，所以还是合算的。

2.4.2 一些有关writeUTF的细节（选读）

方法writeInt将整数写入文件，并用相同的字节数——即，同样的0，1数——来存储任意整数。类似地，方法writeLong用同样数量的字节来存储每个long类型的值。（但是方法writeInt和writeLong使用的字节数不同。）对所有其他向二进制文件写入基本类型的写入方法来说，情况也是一样的。但是，方法writeUTF在文件中用不同的字节数存储不同的字符串。较长的字符串比较短的字符串需要更多的字节。因为二进制文件中数据条目之间是没有分隔符的，所以这种情况对Java来说可能会有问题。Java在每个字符串的起始处写入了一些额外的信息以使这种方法能够工作起来。这些额外的信息说明了写入字符串需要多少字节，这样readUTF就知道需要读入并转换多少字节了。（本章稍后将对方法readUTF进行讨论，但正如你可能已经猜到的那样，它会读入一个String值。）

writeUTF的情况甚至比在上一段中讨论的情况还要稍微复杂一点。注意，我们说过，文件中字符串代码起始处的信息说明了要读入多少字节，而不是字符串中有多少字符。这两个数字是不一样的。使用UTF编码方式，不同的字符会被编码成不同数量的字节。但是，所有ASCII字符都是存储在一个字节中的，而在本书中毫无疑问只会使用ASCII字符，因此这种区别主要是在理论上而不是实际上的。名为“UTF表示什么意思？”的常见问题对这种变长编码进行了更多的讨论。

自测题

20. 如何创建一个名为toFile的ObjectOutputStream类型的输出流，并将其连接到一个名为stuff.data的二进制文件上去？
21. 给出三条语句，将三个double变量x1、x2和x3的值写入文件stuff.data中去。使用你创建的、作为自测题20答案的toFile流。
22. 给出一条语句，关闭作为自测题20答案的toFile流。
23. 按照本节所述进行I/O操作时，需要使用什么import语句？

2.4.3 用ObjectInputStream从二进制文件读出

如果用ObjectOutputStream向文件写入，就可以用ObjectInputStream流类从二进制文件中读出。图2-14给出了这个类中最常用的一些方法。如果将这些方法与图2-13中给出的ObjectOutputStream方法进行比较，就会发现ObjectOutputStream中的每个输出方法在ObjectInputStream中都有一个相应的输入方法。例如，如果用ObjectOutputStream中的writeInt方法向一个文件写入了一个整数，就可以用ObjectInputStream中的readInt方法将那个整数读出。如果用ObjectOutputStream中的writeDouble方法向一个文件写入了一个数字，就可以用ObjectInputStream中的readDouble方法将那个数字读出，以此类推。图2-15给出了readInt这种用法的一个实例。

ObjectInputStream(InputStream streamObject)
没有可以将文件名作为实参的构造器。如果想用文件名创建一个流，可以编写代码

new ObjectInputStream(new FileInputStream(*File_Name*))

如果想用File类的对象创建一个流，可以使用代码

new ObjectInputStream(new FileInputStream(*File_Object*))

FileInputStream的构造器可能会抛出IOException类型的FileNotFoundException。如果FileInputStream构造器成功执行，那么ObjectInputStream的构造器可能会抛出一个不同的IOException。

public int readInt() throws IOException

从输入流中读入一个int值，并将那个int值返回。如果readInt试图从文件中读出一个值，但那个值不是用ObjectOutputStream类中的方法writeInt（或一些等价的方式）写入的，就会出现问题。如果读操作超出了文件的末尾，就会抛出一个EOFException。

public long readLong() throws IOException

从输入流中读入一个long值，并将那个long值返回。如果readLong试图从文件中读出一个值，但那个值不是用ObjectOutputStream类中的方法writeLong（或一些等价的方式）写入的，就会出现问题。如果读操作超出了文件的末尾，就会抛出一个EOFException。

注意，用writeLong写入一个整数，然后稍后用readInt读出那个整数，或者用writeInt写入一个整数，稍后用readLong读出那个整数，都是不正确的。

public double readDouble() throws IOException

从输入流中读入一个double值，并将那个double值返回。如果readDouble试图从文件中读出一个值，但那个值不是用ObjectOutputStream类中的方法writeDouble（或一些等价的方式）写入的，就会出现问题。如果读操作超出了文件的末尾，就会抛出一个EOFException。

public float readFloat() throws IOException

从输入流中读入一个float值，并将那个float值返回。如果readFloat试图从文件中读出一个值，但那个值不是用ObjectOutputStream类中的方法writeFloat（或一些等价的方式）写入的，就会出现问题。如果读操作超出了文件的末尾，就会抛出一个EOFException。

注意，用writeDouble写入一个浮点数，稍后用readFloat读出那个数字，或者用writeFloat写入一个浮点数，稍后用readDouble读出那个数字，都是不正确的。其他的类型不匹配，比如用writeInt写入，用readFloat或readDouble读出，也都是不正确的。

public char readChar() throws IOException

从输入流中读入一个char值，并将那个char值返回。如果readChar试图从文件中读出一个值，但那个值不是用ObjectOutputStream类中的方法writeChar（或一些等价的方式）写入的，就会出现问题。如果读操作超出了文件的末尾，就会抛出一个EOFException。

public boolean readBoolean() throws IOException

从输入流中读入一个boolean值，并将那个boolean值返回。如果readBoolean试图从文件中读出一个值，但那个值不是用ObjectOutputStream类中的方法writeBoolean（或一些等价的方式）写入的，就会出现问题。如果读操作超出了文件的末尾，就会抛出一个EOFException。

public String readUTF() throws IOException

从输入流中读入一个String值，并将那个String值返回。如果readUTF试图从文件中读出一个值，但那个值不是用ObjectOutputStream类中的方法writeUTF（或一些等价的方式）写入的，就会出现问题。如果读操作超出了文件的末尾，就会抛出一个EOFException。

Object readObject() throws ClassNotFoundException,
InvalidClassException, OptionalDataException, IOException

从输入流中读入一个对象。如果无法找到串行化对象的类，就抛出ClassNotFoundException。如果可串行化类有问题，就抛出InvalidClassException。如果在流中找到的是基本数据项而不是对象，就抛出OptionalDataException。如果存在其他I/O问题，就抛出IOException。2.5节对方法readObject进行了介绍。

public void close() throws IOException
关闭流与文件的连接。

图2-14 ObjectInputStream类中的一些方法

```
import java.io.*;

public class BinaryInputDemo
{
  public static void main(String[] args)
  {
     try
     {
         ObjectInputStream inputStream =
           new ObjectInputStream(new FileInputStream("numbers.dat"));
         int n;
         System.out.println("Reading the nonnegative integers");
         System.out.println("in the file numbers.dat.");
         n = inputStream.readInt();
         while (n >= 0)
         {
             System.out.println(n);
             n = inputStream.readInt();
         }

         System.out.println("End of reading from file.");
         inputStream.close();
     }
     catch(FileNotFoundException e)
     {
         System.out.println("Cannot find file numbers.dat.");
     }
     catch(IOException e)
     {
         System.out.println("Problem with input from file numbers.dat.");
     }
  }
}
```

屏幕输出

```
Reading the nonnegative integers
in the file numbers.dat.
1
2
3
End of reading from file.
```

假设已经用这里显示的对话运行过图2-12中的程序了。

注意，从文件中读出了标记值-1，但并没有将其输出到屏幕上去。

图2-15　用ObjectInputStream从文件中读出

用ObjectInputStream打开一个二进制文件进行读操作的方式与ObjectOutputStream的使用方式类似。在图2-15中，以下列方式将文件numbers.dat打开，并将其连接到一个名为inputStream的流上去：

```
ObjectInputStream inputStream =
    new ObjectInputStream(new FileInputStream("numbers.dat"));
```

注意，这条语句与在图2-12中对ObjectOutputStream的使用方法相同，只是在这儿，分别用ObjectInputStream和FileInputStream类取代了ObjectOutputStream和FileOutputStream。使用FileInputStream的原因与对ObjectOutputStream使用FileOutputStream的原因相同。

ObjectInputStream允许从同一个文件中读取不同类型的输入。因此，比如说，可以读取int值、double值和String值的组合。但是，如果文件中的下一个数据条目不是读方法所期待的类型，结果很可能会一塌糊涂。例如，如果程序用writeInt写入了一个整数，那么任何读取那个整数的程序都应该用readInt来读它。如果使用了readLong或readDouble，程序就会出问题。

快速参考：将二进制文件连接到流上进行读入（打开一个输入文件）

可以按下列形式创建一个ObjectInputStream类的流，并将其连接到一个二进制文件上去：

语法：

```
ObjectInputStream Input_Stream_Name =
      new ObjectInputStream(new FileInputStream(File_name));
```

FileInputStream的构造器可能会抛出一个IOException类型的FileNotFoundException。如果FileInputStream构造器成功执行，那么ObjectInputStream的构造器可能会抛出一个不同的IOException。

举例：

```
ObjectInputStream myInputStream =
                        new ObjectInputStream(
                            new FileInputStream("myfile.dat"));
```

这段代码之后，就可以用（图2-14）ObjectInputStream 类中的方法从文件中读入了。

△ 易犯错误：对文本文件使用ObjectInputStream

二进制文件和文本文件是以不同的方式对它们的数据进行编码的。因此，希望读入一个二进制文件的流，比如ObjectInputStream类中的流，在读取文本文件时会遇到一些问题。如果试图用ObjectInputStream类中的流去读取一个文本文件，程序要么会读入“一些垃圾值”，要么会遇到一些其他的错误情况。类似的，如果试图（用BufferedReader）像读取文本文件那样去读取一个二进制文件，也会遇到麻烦的。 △

自测题

24. 如何创建一个名为fromFile的ObjectInputStream类型的输入流，并将其连接到一个名为stuff.data的文件上去？
25. 给出三条语句，从文件stuff.data中读取三个double类型的数。使用在自测题24中创建的fromFile流。声明三个变量来装载这三个数字。
26. 给出一条语句，关闭自测题24中创建的流fromFile。
27. 一个程序用writeInt向文件中写入一个数字，另一个程序可以用readLong读出那个数字吗？如果用readDouble呢？

28. 可以用readUTF从一个文本文件中读取一个字符串吗？

△ 易犯错误：定义一个方法去打开流

下列代码看起来好像是一个可以包含在某个类中的、很合理的方法，但是，它有个问题：

```
//This method does not do what we want it to do.
public static void openFile(ObjectOutputStream streamName)throws IOException
{
    System.out.println("Enter file name:");
    Scanner keyboard = new Scanner(System.in);
    String fileName = keyboard.next();
    streamName =
       new ObjectOutputStream(new FileOutputStream(fileName));
}
```

方法可以很好地编译，也可以被调用，但不会像所希望的那样执行。例如，考虑下列代码（尽管在这儿有关null的具体细节是无关紧要的，但可以回想一下，null是一个可以赋给任意类类型变量的值）：

```
ObjectOutputStream outputStream = null;
openFile(outputStream);
```

执行了这段代码之后，outputStream的值仍然是null。当方法openFile结束时，在该方法中打开的文件消失了。这个问题有些微妙，并不会在很多情况下出现，但这就是那些比较少见的情况之一。

这个问题与Java如何处理类类型实参有关。这些实参是作为一个无法改变的内存地址传递给方法的。内存地址命名的通常是一些可以改变的东西，但内存地址自身是无法改变的。考虑这个概念的一种等价方式是：回想一下，前面的outputStream这样的对象变量是一个名字。方法可以改变outputStream命名的文件的内容。例如，它可以向流发送输出，这样就会改变连接到流上去的文件。但是，一个方法无法改变流的名字使其命名一个不同的文件。使用new时，就是在改变outputStream使其命名一个不同的文件，这是不允许的。

一定要注意这只是在非常有限的环境中适用的一种很窄的限制条件。一旦将流连到文件上去，就可以将流的名字作为一个实参传递给方法了，这个方法就可以修改文件了。

避免这个问题最好的办法就是在将流对象连接到文件之后，再使用那个会返回流对象的方法。下面的方法就没有任何问题：

```
public static ObjectOutputStream openFile() throws IOException
{
   ObjectOutputStream tempStreamName;
   System.out.println("Enter file name");
   Scanner keyboard = new Scanner(System.in);
   String fileName = keyboard.next();
   tempStreamName =
      new ObjectOutputStream(new FileOutputStream(fileName));
   return tempStreamName;
}
```

Web上本书源代码中包含的OpenFileDemo.java文件是一个简单的程序，对这个方法的使用进行了演示。　△

自测题

29. 编写一些Java代码，创建一个ObjectOutputStream类型的流。将这个流命名为writer，并将其连

接到以String变量theFile为名的二进制文件上去。

30. 编写一些Java代码，创建一个ObjectInputStream类型的流。将这个流命名为reader，并将其连接到以String变量theFile为名的二进制文件上去。

31. 编写一个完整的Java程序，请求用户输入一个二进制文件名，并将这个文件中第一个数据条目输出到屏幕上去。假设第一个数据条目是用方法writeUTF写入文件中的一个字符串。

2.4.4 EOFException类

我们讨论过的所有从二进制文件中读取数据的方法，都会在试图读取超出文件末尾的内容时抛出一个EOFException。具体来说，图2-14中显示的所有ObjectInputStream方法，在试图读取超出文件末尾的内容时，都会抛出一个EOFException。

如图2-16所示，使用ObjectInputStream时，可以用EOFException类来测试是否达到文件的末尾。在那个示例程序中，读操作被放在一个“无限循环”中，这个循环是用true作为while循环的循环变量来实现的。循环不是真正无限的，因为到达文件末尾时，会抛出一个异常，这会结束整个try块，并将控制权传递给catch块。

```
import java.io.*;

public class EOFExceptionDemo
{
    public static void main(String[] args)
    {
        try
        {
            ObjectInputStream inputStream =
              new ObjectInputStream(new FileInputStream("numbers.dat"));
            int n;

            System.out.println("Reading ALL the integers");
            System.out.println("in the file numbers.dat.");

            try
            {
                while (true)
                {
                    n = inputStream.readInt();
                    System.out.println(n);
                }
            }
            catch(EOFException e)
            {
                System.out.println("End of reading from file.");
            }
            inputStream.close();
        }
        catch(FileNotFoundException e)
        {
            System.out.println("Cannot find file numbers.dat.");
```

抛出一个异常时，循环就结束了。

图2-16 使用EOFException

```
        }
        catch(IOException e)
        {
            System.out.println("Problem with input from file numbers.dat.");
        }
    }
}
```

屏幕输出（假设图2-12中的程序已经用那儿显示的对话运行过了）

```
Reading ALL the integers
in the file numbers.dat.
1
2
3
-1
End of reading from file.
```

注意，使用EOFException时，对-1的处理就和任意其他整数一样了。使用EOFException就允许文件中包含任意类型的整数。

图2-16　（续）

比较一下图2-16中的程序与图2-15中的类似程序是很有益的。图2-15中的程序通过对负数的检测来测试文件是否结束。这种方式很好，但这意味着无法在文件中存储负数（除了将其作为标记值使用）。图2-16中的程序用EOFException来测试文件是否结束。这样，它就可以处理存储了包括负数在内的任意整数类型的文件了。

快速参考：EOFException类

如果程序是用图2-14中列出的、ObjectInputStream类的任意一个方法从二进制文件中读出的，而且程序试图读取超出文件末尾的内容，那么就会抛出一个EOFException。这个异常可以结束用来读取文件中所有数据的循环。

不是所有类中的所有方法都会在试图读取超出文件末尾的内容时，抛出EOFException的。对本书中讨论的类，规则如下：如果程序从二进制文件中读取数据，就会在试图读取超出文件末尾的内容时，抛出一个EOFException。但是如果程序是从文本文件中读取数据，就不会在试图读取超出文件末尾的内容时抛出EOFException（或任意其他异常）。

EOFException类是IOException类的派生类。因此每个EOFException类型的异常也都是IOException类型的异常。

△ 易犯错误：忘了检查文件是否结束

如果程序没有准备检查文件是否结束，那么抵达文件末尾时，发生的情况就取决于程序中的细节——但无论发生什么情况，都不会是什么好事。如果程序试图读取超出文件末尾的内容，可能会进入一个无限循环或异常终止。一定要确保程序检查了文件是否结束，并在其抵达文件末尾时做一些正确的事情。即使认为程序不会读取超出文件末尾的内容，也应该为这种可能做好准备，以防事情没有完全按照计划发展。△

△ 易犯错误：用错误的方式检查文件是否结束

（通常位于不同类中的）不同的文件读取方法会以不同的方式来检查文件是否结束。有些方法会在试图读取超出文件末尾的内容时，抛出EOFException类中的一个异常。其他方法会返回一个特殊的值，比如null。从文件中读出时，一定要注意，要通过对所用方法都是正确的方式来测试文件是否结束。如果用错误的方式来测试文件是否结束，很可能会发生下列两种情况之一：程序要么会进入一个非计划的无限循环，要么会非正常终止。

下列规则对本书所讨论的类都适用：如果程序从二进制文件中读取，那么在读取的内容超出文件末尾的时候，就会抛出一个EOFException。如果程序从文本文件中读取，那么在读取的内容超出文件末尾的时候，就会返回一个特殊的值，比如null，而不会抛出EOFException。 △

2.4.5 FileInputStream和FileOutputStream类

每次创建ObjectInputStream和ObjectOutputStream类的流时，都会分别使用FileInputStream和FileOutputStream流类。例如，我们分别在图2-12和图2-16中使用了下列语句：

```
ObjectOutputStream outputStream =
                    new ObjectOutputStream(new FileOutputStream("numbers.dat"));
ObjectInputStream inputStream =
                    new ObjectInputStream(new FileInputStream("numbers.dat"));
```

使用FileOutputStream类和FileInputStream类是因为它们可以接受文件名作为构造器实参，比如前面例子中的实参"numbers.dat"。ObjectInputStream和ObjectOutputStream都不接受文件名作为实参。因此，使用字符串名将一个ObjectInputStream连接到一个文件上去时，必须分两步来实现。首先，要用new FileOutputStream("numbers.dat")创建一个FileOutputStream类对象。然后，通过

```
ObjectOutputStream outputStream =
                    new ObjectOutputStream(new FileOutputStream("numbers.dat"));
```

用这个FileOutputStream类对象来创建一个ObjectOutputStream类对象。类似地，要用文件名创建一个ObjectInputStream对象，也要使用FileInputStream分两步实现。

快速参考：FileInputStream与FileOutputStream

在本书中，只是使用了FileInputStream和FileOutputStream的构造器，没有使用这两个类的其他东西。这两个类都有一个可以将文件名作为实参的构造器。我们用这些构造器为ObjectInputStream和ObjectOutputStream这样不能将文件名作为实参的流类的构造器生成实参。下面是一些使用FileInputStream和FileOutputStream的例子：

```
ObjectOutputStream fileOutput =
                    new ObjectOutputStream(
                        new FileOutputStream("nicestuff.dat"));

ObjectInputStream fileInput =
                    new ObjectInputStream(
                        new FileInputStream("rawstuff.dat"));
```

FileInputStream和FileOutputStream的构造器会抛出一个IOException类型的FileNotFoundException。

编程示例：处理一个二进制数据文件

图2-17包含了一个可以进行一些简单数据处理的程序。要求用户输入两个文件名，然后将一个文件中的所有数字复制到另一个文件中去，但将每个数字都乘以2，使得输出文件中的数字都是输入文件中值的两倍。这个编程任务不是很复杂，但它确实用到了很多处理文件I/O的标准编程技术。尤其要注意，连接到文件上去的流对象变量是实例变量，还要注意，这项任务被分割成了一些子任务，分派给了不同的方法。

```
import java.io.*;
import java.util.*;

public class Doubler
{
    private ObjectInputStream inputStream = null;
    private ObjectOutputStream outputStream = null;

    /**
     Doubles the integers in one file and puts them in another file.
    */
    public static void main(String[] args)
    {
        Doubler twoTimer = new Doubler();
        twoTimer.connectToInputFile();
        twoTimer.connectToOutputFile();
        twoTimer.timesTwo();
        twoTimer.closeFiles();
        System.out.println("Numbers from input file");
        System.out.println("doubled and copied to output file.");
    }

    public void connectToOutputFile()
    {
        String outputFileName = getFileName("Enter output file name:");
        try
        {
            outputStream = new ObjectOutputStream(
                        new FileOutputStream(outputFileName));
        }
        catch(IOException e)
        {
            System.out.println("Error opening output file " + outputFileName);
            System.out.println(e.getMessage());
            System.exit(0);
        }
    }

    public void connectToInputFile()
```

图2-17　处理一个二进制数据文件

```
{
    String inputFileName = getFileName("Enter input file name:");
    try
    {
        inputStream = new ObjectInputStream(
                          new FileInputStream(inputFileName));
    }
    catch(FileNotFoundException e)
    {
        System.out.println("File " + inputFileName + " not found.");
        System.exit(0);
    }
    catch(IOException e)
    {
        System.out.println("Error opening input file " + inputFileName);
        System.exit(0);
    }
}

private String getFileName(String prompt)
{
    String fileName = null;
    System.out.println(prompt);
    Scanner keyboard = new Scanner(System.in);
    fileName = keyboard.next();
    return fileName;
}

public void timesTwo()
{
    int next;
    try
    {
        while (true)
        {
            next = inputStream.readInt();
            outputStream.writeInt(2*next);
        }
    }
    catch(EOFException e)
    {
        //Do nothing. This just ends the loop.
    }
    catch(IOException e)
    {
        System.out.println("Error: reading or writing files.");
        System.out.println(e.getMessage());
        System.exit(0);
    }
}

public void closeFiles()
{
```

实际应用程序中使用的类可能还包含其他方法，可以从输入文件中获取数据，用某种方式转换数据，并将修改过的数据写到输出文件中去。

图2-17 （续）

```
            try
            {
                inputStream.close();
                outputStream.close();
            }
            catch(IOException e)
            {
                System.out.println("Error closing files " + e.getMessage());
                System.exit(0);
            }
        }
    }
```

图2-17　（续）

注意，将try块设置的很小，这样，抛出一个异常时，就可以在邻近的catch块中将其捕获。如果减少try块，就很难确定是哪部分代码抛出了异常。

自测题

32. 假设想创建一个输入流，并将其连接到一个名为mydata.dat的二进制文件上去。下面给出的代码能完成任务吗？如果不行，怎样编写出类似的、能完成任务的代码？

```
ObjectInputStream inputStream = new ObjectInputStream("mydata.dat");
```

33. FileInputStream类中有名为readInt的方法吗？有名为readDouble的方法吗？有名为readUTF的方法吗？

34. FileOutputStream类有可以接受文件名作为实参的构造器吗？

35. ObjectOutputStream类有可以接受文件名作为实参的构造器吗？

36. 用本章讨论的方法打开一个二进制文件进行输出时，可能抛出异常吗？会是哪种类型的异常？用本章讨论的方法打开一个二进制文件进行输入时，可能抛出异常吗？会是哪种类型的异常？

37. 假设一个二进制文件中包含了三个用ObjectOutputStream类中的writeDouble方法写入文件的数字。再进一步假设程序通过对ObjectInputStream类中readDouble方法的三次调用将这三个数字都读取出来。如果程序调用readDouble试图从文件中读取第四个数字，会发生什么情况？

38. 下列代码出现在图2-16的程序中：

```
try
{
    while(true)
    {
        n = inputStream.readInt();
        System.out.println(n);
    }
}
catch(EOFException e)
{
    System.out.println("End of reading from file.");
}
```

为什么说这段代码中实际上没有包含无限循环？

39. 编写一个完整的Java程序，读取一个名为temperatures的二进制文件中的所有数字，并将数字一行一个地写到屏幕上去。假设这个文件完全是由方法writeDouble写入的数字组成的。

2.5 用对象流进行对象I/O

本节将介绍使用ObjectInputStream和ObjectOutputStream为类和数组对象进行二进制文件I/O。

2.5.1 类对象的二进制I/O

可以用ObjectOutputStream类的writeObject方法向二进制文件输出类对象，然后用ObjectInputStream类的readObject方法从文件中将对象读出。但是，要这样做，代码所写入及读出的类必须是**可串行化的**（serializable）。使一个类可串行化是很简单的。需要做的只是像下面的例子一样，在类定义的头部添加两个单词implements Serializable：

```
public class Species implements Serializable
{
```

另外，要使Serializable接口对代码可用，需要使用下列import语句：

```
import java.io.*;
```

在图2-18中，将这些条目添加到了本章前面用过的Species类中。将在稍后的小节中回过头来讨论串行化的含义，但现在用Species类来说明如何对二进制文件进行对象的读和写。

```
import java.io.*;                          <-- Serializable所需的。
import java.util.*;

/**
 Class for data on endangered species. This class is serialized.
 */
public class Species implements Serializable      <-- 除了这两个单词和import语句之外，这个定义与图2-3中的定义是一样的。
{
    private String name;
    private int population;
    private double growthRate;

    public Species()                       <-- 这段代码是一个新的、改进了的Species类定义，取代了图2-3中的定义。
    {
        name = null;
        population = 0;
        growthRate = 0;
    }

    public Species(String initialName, int initialPopulation,
                                       double initialGrowthRate)
    {
        name = initialName;
        if (initialPopulation >= 0)
            population = initialPopulation;
        else
        {
            System.out.println("ERROR: Negative population.");
            System.exit(0);
        }
        growthRate = initialGrowthRate;
```

图2-18 为二进制文件I/O串行化了的Species类

```
    }

    public String toString()
    {
        return ("Name = " + name + "\n"
                + "Population = " + population + "\n"
                + "Growth rate = " + growthRate + "%");
    }
      <其他方法与《Java程序设计与问题解决：基础篇》中第4章的图4-19一样，但本章的讨论不需要这些方法。>
}
```

图2-18　（续）

图2-19说明了怎样才能从二进制文件中写入，或从中读取类对象。要向二进制文件中写入一个Species类的对象，只要使用ObjectOutputStream类的writeObject方法就行了。writeObject的使用方法与ObjectOutputStream类中其他方法，比如writeInt的用法一样，但要用对象作为实参，就像

```
outputStream.writeObject(oneRecord);
```

一样。其中outputStream是ObjectOutputStream类型的流，而oneRecord是Species类型的。

```
import java.io.*;

public class ClassIODemo
{
    public static void main(String[] args)
    {
        ObjectOutputStream outputStream = null;

        try
        {
            outputStream =
                  new ObjectOutputStream(
                      new FileOutputStream("species.records"));
        }
        catch(IOException e)
        {
            System.out.println("Error opening file species.records.");
            System.out.println("for writing.");
            System.exit(0);
        }
        Species oneRecord = new Species("Calif. Condor", 27, 0.02);
        Species secondRecord = new Species("Black Rhino", 100, 1.0);
        try
        {
            outputStream.writeObject(oneRecord);
            outputStream.writeObject(secondRecord);
            outputStream.close();
        }
```

图2-19　类对象的文件I/O

```
catch(IOException e)
{
    System.out.println("Error writing to file species.records.");
    System.exit(0);
}

System.out.println("Records sent to file species.record.");
System.out.println("Now let's reopen the file and echo the records.");
ObjectInputStream inputStream = null;

try
{
    inputStream =
        new ObjectInputStream(
            new FileInputStream("species.records"));
}
catch(IOException e)
{
    System.out.println("Error opening file species.records.");
    System.out.println("for reading.");
    System.exit(0);
}

Species readOne = null, readTwo = null;

try
{
    readOne = (Species)inputStream.readObject();
    readTwo = (Species)inputStream.readObject();
    inputStream.close();
}
catch(Exception e)
{
    System.out.println("Error reading from file species.records.");
    System.exit(0);
}
System.out.println("The following were read\n"
                   + "from the file species.record:");
System.out.println(readOne);
System.out.println();
System.out.println(readTwo);
System.out.println("End of program.");
    }
}
```

注意这个强制类型转换。

为每种类型的异常使用单独的catch块会更好一些。在这儿只用了一个catch块以节省空间。

图2-19 (续)

屏幕对话示例

```
Records sent to file species.record.
Now let's reopen the file and echo the records.
The following were read
from the file species.record:
Name = Calif.Condor
Population = 27
Growth rate = 0.02%

Name = Black Rhino
Population = 100
Growth rate = 1.0%
End of program.
```

图2-19　(续)

记住，要以这种方式将类写入文件，类必须是串行化的。到文档中查找预定义类时，应该去查找那些说明类实现了Serializable的文档。

如下列来自图2-19的例子所示，可以用ObjectInputStream流类中的readObject方法将对象读回到一个记录中去：

```
readOne = (Species)inputStream.readObject();
```

在这里，inputStream是一个ObjectInputStream类型的流，这个流连接到了一个已经填充了Species记录的文件上去。这些记录是通过ObjectOutputStream类的writeObject方法写入的。变量readOne是Species类型的。

注意，readObject会将其值作为一个Object类型的对象返回。如果想将其作为一个Species类型的对象使用，就必须把它强制类型转换成Species类型。

在结束这个例子之前，我们来澄清一个潜在的错误概念。Species类有一个toString方法。要用System.out.println正确地向屏幕输出，以及用println正确地向文本文件输出，都需要这个toString方法。但是，方法toString与二进制文件的对象I/O没有任何关系。即使Species类定义中没有toString方法，二进制文件的对象I/O也能工作的很好。

△ 易犯错误：异常、异常、异常

在很多对文本文件I/O来说不会抛出异常的情况下，二进制文件I/O都会抛出异常。因此，使用二进制文件时需要进行的异常处理，比使用文本文件时需要进行的异常处理要多。例如，关闭一个连接到PrintWriter类型的流上去的文本文件不会抛出一个异常。但是关闭一个我们讨论过的那些类型的二进制文件流就会抛出一个IOException。像我们所说的那样进行二进制I/O时，几乎所有事情都会抛出一个异常。同样，writeObject和readObject方法可以抛出的异常也有很长一列，可以通过查看图2-13和图2-14来弄清楚有哪些异常。　△

2.5.2 Serializable接口

像对图2-18中的Species类所做的那样，使一个类成为可串行的，会有什么影响呢？从某

种意义上来说，对那个类没什么直接的影响，但对Java如何对那个类的对象执行文件I/O会有影响。如果类是可串行的，Java会为它写入ObjectOutputStream类型的流中的每个类对象分配一个串行码。如果多次将同一个对象写入流中，那么第一次写入之后，Java只要写入那个对象的串行码就行了，不需要写入对象数据的整体描述。这种特性使文件I/O更加高效，并使文件变得更小。用ObjectInputStream类型的流将其读回时，会将重复的串行号作为对同一个对象的引用返回。注意，这个条件意味着，如果两个变量中包含了对同一个对象的引用，将这些对象写入文件，稍后又将其从文件中读出，那么读出的两个对象仍然是对同一个对象的引用。因此，将对象写入文件并在稍后将其读出时，对象数据的结构中没有丢失任何东西。

可串行化类拥有类类型的实例变量时，那些实例变量的类也应该是可串行化的，对类内部所有级别的实例变量来说，也都是如此。

串行化听起来很棒。那么，为什么不将所有的类都设置成可串行化的呢？这是出于对安全性的考虑。串行码系统使程序员可以更容易地对写入二级存储器的对象数据进行访问。同样，对于某些类，将对象写入二级存储器是毫无意义的，因为它们稍后被再次读出时已经改变原来的含义了。例如，如果对象中包含了与系统有关的数据，那么，稍后将其读出时，这些数据可能就毫无意义了。

△ 易犯错误：在同一个文件中混和多种类类型

用ObjectOutputStream和ObjectInputStream来写入和读取数据的最好方式，就是在任何一个文件中只存储一种类类型的数据。如果存储了多种类类型的对象，甚至将一种类类型的对象与基本类型的数据混合在一起，以我们的经验来看，系统会出现混乱，你可能会丢失数据。 △

2.5.3 二进制文件中的数组对象

可以用objectWrite将整个数组保存到一个二进制文件中去，稍后再用objectRead将其读出。这样做的时候，如果数组的基类型是类，这个类就应该是可串行化的。这就意味着，如果将一个可串行化类的所有数据都存储在单个数组中，就可以通过一次objectWrite调用将所有数据都输出到一个二进制文件中去。

图2-20对这种将一个数组存储在二进制文件中的方式进行了说明。注意，基本的类类型Species是可串行化的。还要注意用数组类型Species[]进行的强制类型转换。由于readObject会将其值作为一个Object类型的对象返回，所以必须对它进行强制类型转换，将其转换成正确的数组类型。

```
import java.io.*;

public class ArrayIODemo
{
    public static void main(String[] args)
    {
        Species[] oneArray = new Species[2];
        oneArray[0] = new Species("Calif. Condor", 27, 0.02);
```

图2-20 数组对象的文件I/O

```
        oneArray[1] = new Species("Black Rhino", 100, 1.0);

        try
        {
            ObjectOutputStream outputStream =
                  new ObjectOutputStream(
                      new FileOutputStream("array.file"));
            outputStream.writeObject(oneArray);
            outputStream.close();
        }
        catch(IOException e)
        {
            System.out.println("Error writing to file array.file.");
            System.exit(0);
        }

         System.out.println("Array sent to file array.file.");

        System.out.println("Now let's reopen the file and echo the array.");
        Species[] anotherArray = new Species[2];


        try
        {
            ObjectInputStream inputStream =
注意强制类型转换。  new ObjectInputStream(new FileInputStream("array.file"));
            anotherArray = (Species[])inputStream.readObject();
            inputStream.close();
        }
        catch(Exception e)
        {
            System.out.println("Error reading file array.file.");
            System.exit(0);
        }

        System.out.println("The following were read\n"
                           + "from the file array.file:");
        int i;
        for (i = 0; i < anotherArray.length; i++)
        {
            System.out.println(anotherArray[i]);
            System.out.println();
        }
        System.out.println("End of program.");
    }
}
```

为每种类型的异常使用单独的catch块会更好一些。在这儿只用了一个catch块以节省空间。

图2-20　(续)

屏幕对话示例

```
Records sent to file array.file.
Now let's reopen the file and echo the array.
The following were read
from the file array.file:
Name = Calif.Condor
Population = 27
Growth rate = 0.02%

Name = Black Rhino
Population = 100
Growth rate = 1.0%

End of program.
```

图2-20 （续）

自测题

40. 如何将一个类设置为可串行化的？
41. ObjectInputStream类的readObject方法的返回值类型是什么？
42. ObjectOutputStream类的writeObject方法可能会抛出哪种异常？
43. ObjectInputStream类的readObject方法可能会抛出哪种异常？

2.6 图形编程补充（选读）

“真实的世界”是唯一的世界。

——格拉菲蒂

GUI不仅适用于一些可爱的小型演示程序。GUI可以作为实际应用程序的I/O接口使用。在本节中将给出这种用法的一个小例子：操纵文件的JFrame GUI实例。

编程示例：操纵文件的JFrame GUI

图2-21中包含了一个JFrame GUI，可以用它对文本文件进行一些整理。可以在标记了"Enter file name here."的文本域中输入一个文本名，并点击"Show first line"按钮。如果文件不存在或不可读，会在另一个文本域中收到一条消息对这种情况进行解释。如果文件存在且可读，就会将文件的第一行显示在另一个文本域中。（如果文件不是文本文件，就会得到一行无用的数据。）通过这种方式可以提醒你文件中存储的是什么内容。

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.io.*;

public class FileOrganizer extends JFrame implements ActionListener
```

图2-21 文件组织器GUI

```
{
    public static final int WIDTH = 400;
    public static final int HEIGHT = 300;
    public static final int NUMBER_OF_CHAR = 30;

    private JTextField fileNameField;
    private JTextField firstLineField;

    public static void main(String[] args)
    {
        FileOrganizer gui = new FileOrganizer();
        gui.setVisible(true);
    }

    public FileOrganizer()
    {
        setSize(WIDTH, HEIGHT);
        WindowDestroyer listener = new WindowDestroyer();
        addWindowListener(listener);
        Container contentPane = getContentPane();
        contentPane.setLayout(new FlowLayout());

        JButton showButton = new JButton("Show first line");
        showButton.addActionListener(this);
        contentPane.add(showButton);

        JButton removeButton = new JButton("Remove file");
        removeButton.addActionListener(this);
        contentPane.add(removeButton);

        JButton resetButton = new JButton("Reset");
        resetButton.addActionListener(this);
        contentPane.add(resetButton);

        fileNameField = new JTextField(NUMBER_OF_CHAR);
        contentPane.add(fileNameField);
        fileNameField.setText("Enter file name here.");

        firstLineField = new JTextField(NUMBER_OF_CHAR);
        contentPane.add(firstLineField);
    }

    public void actionPerformed(ActionEvent e)
    {
        String actionCommand = e.getActionCommand();

        if (actionCommand.equals("Show first line"))
            showFirstLine();
        else if (actionCommand.equals("Remove file"))
            removeFile();
        else if (actionCommand.equals("Reset"))
        {
            fileNameField.setText("Enter file name here.");
```

图2-21　(续)

```
            firstLineField.setText("");
        }
        else
            firstLineField.setText("Unexpected error.");
    }

    public void showFirstLine()
    {
        BufferedReader fileInput;
        String firstLine;
        String fileName = fileNameField.getText();
        File fileObject = new File(fileName);

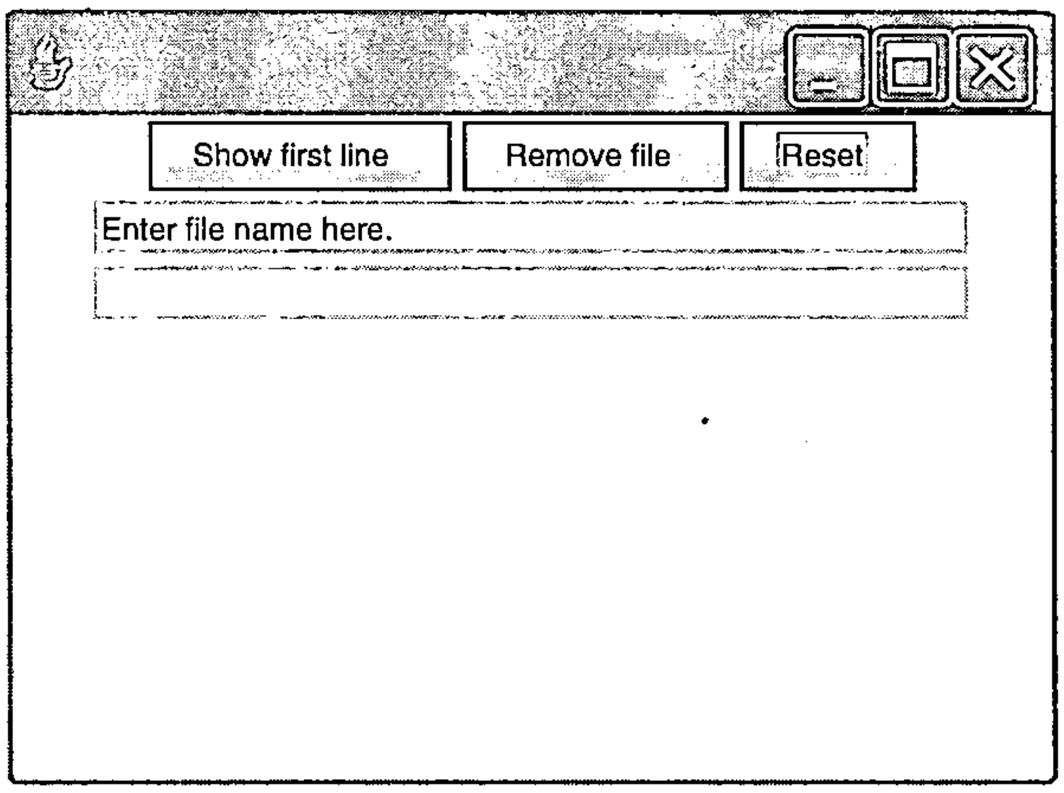
        if ( ! fileObject.exists( ))
            firstLineField.setText("No such file");
        else if ( ! fileObject.canRead( ))
            firstLineField.setText("That file is not readable.");
        else
        {
            try
            {
                fileInput = new BufferedReader(new FileReader(fileObject));
                firstLine = fileInput.readLine();
                firstLineField.setText(firstLine);
                fileInput.close();
            }
            catch(IOException e)
            {
                firstLineField.setText("Problem reading from file.");
            }
        }
    }

    public void removeFile()
    {
        BufferedReader fileInput;
        String firstLine;
        String fileName = fileNameField.getText();
        File fileObject = new File(fileName);

        if ( ! fileObject.exists())
            firstLineField.setText("No such file");
        else if ( ! fileObject.canWrite())
            firstLineField.setText("Permission denied.");
        else
        {
            if (fileObject.delete())
                firstLineField.setText("File deleted.");
            else
                firstLineField.setText("Could not delete file.");
        }
    }
}
```

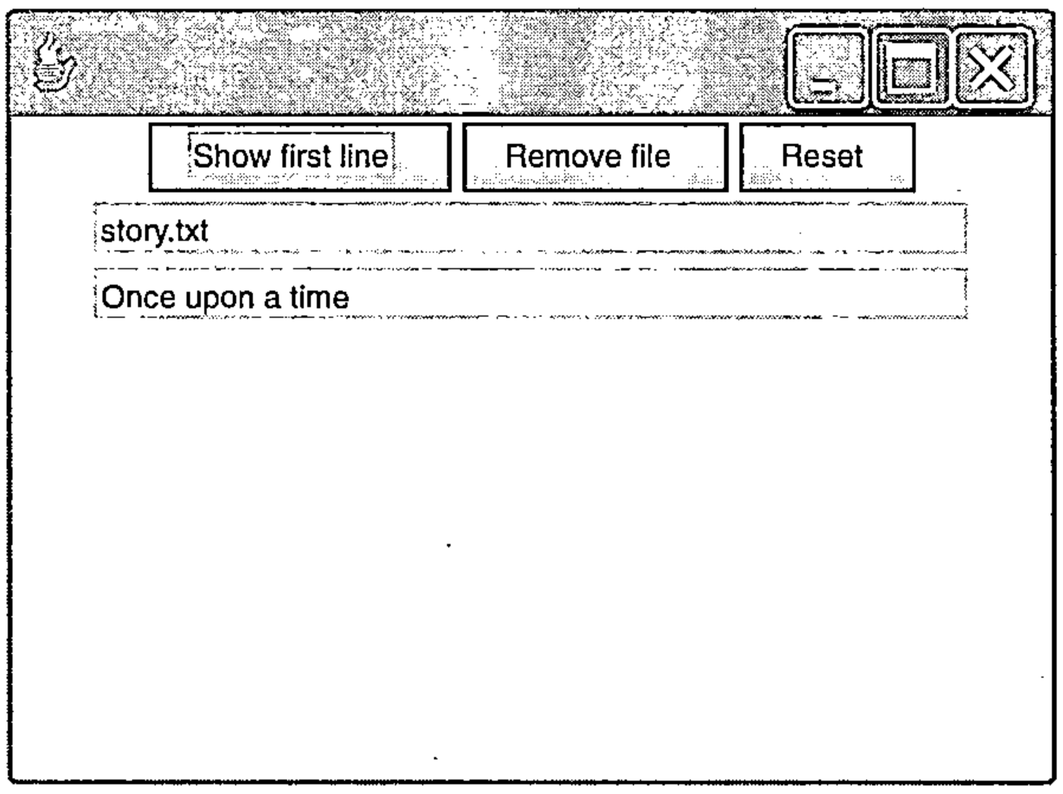
图2-21 （续）

得到的GUI



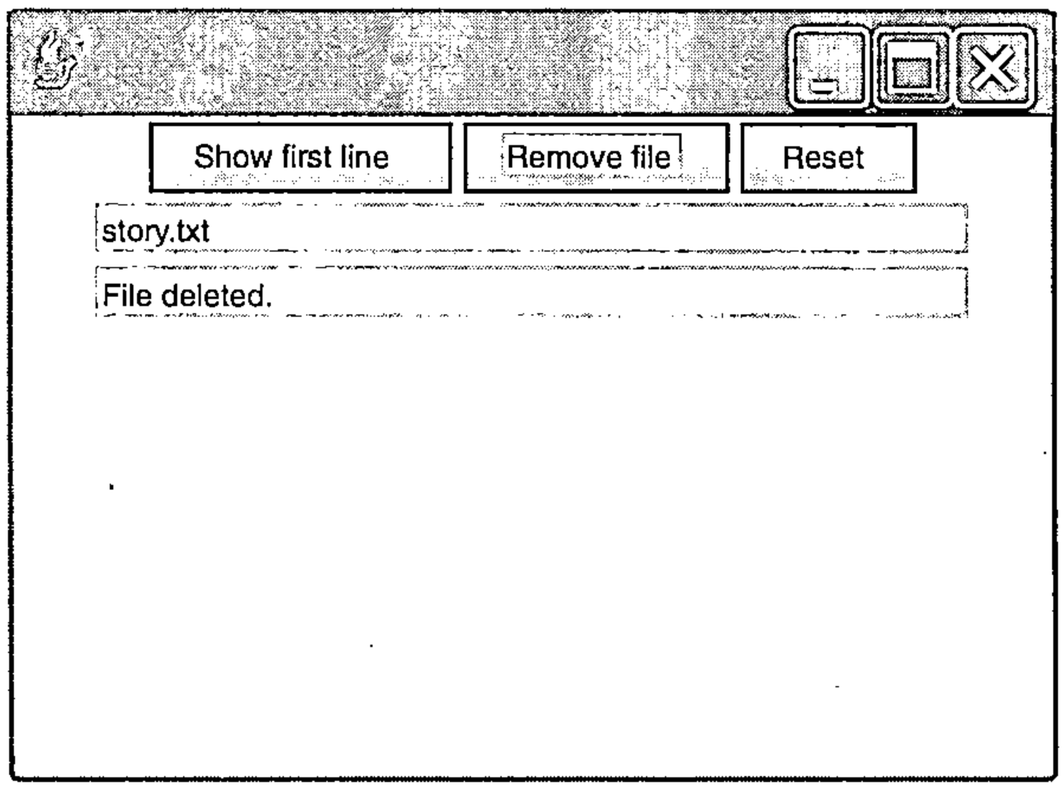
第一次显示GUI时，或点击了"Reset"按钮之后的情形。

得到的GUI



假设文件的第一行内容如图所示，这是输入文件名并点击"Show first line"按钮之后的情形。

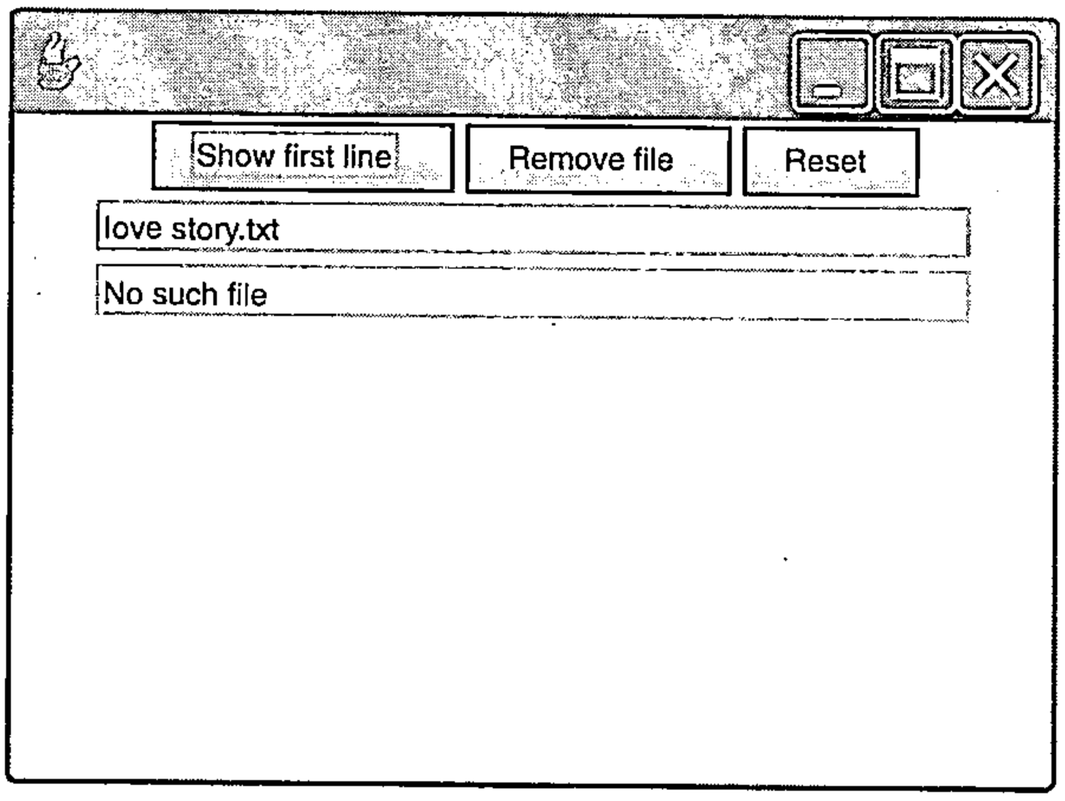
得到的GUI



假设文件可以被删除，这是点击"Remove file"按钮之后的情形。

图2-21　（续）

得到的GUI



假设不存在指定名字的文件，这是输入文件名并点击"Show first line"按钮之后的情形。

图2-21 （续）

如果想删除一个文件，可以在标记了"Enter file name here."的文本域中输入这个文本名，并点击"Remove file"按钮。如果文件不存在或不可写（通常这就意味着无法删除它），另一个文本域就会给出一条适当的消息。如果文件存在且可写，就将其删除，并显示消息"File deleted"。这是由方法remove中的下列代码实现的：

```
if (fileObject.delete())
   firstLineField.setText("File deleted.");
else
   firstLineField.setText("Could not delete file.");
```

如果可以删除文件，调用fileObject.delete()就返回true，如果不能删除文件，就返回false。因此，如果由于某些原因不能删除文件，就会给出一条适当的消息。

有两个文本域，因此，在另一个文本域中给出任意输出信息时，文件名仍然是可见的。为了使用户知道哪个文本域是用于文件名的，用构造器中的下列代码行对存储文件名的文本域进行了初始化：

```
fileNameField.setText("Enter file name here.");
```

在第5章中会学到如何将带有这类指令的标签附加到文本域上去，这样就不用在文本域中给出这种指令了。

输入另一个文件名之前，用户应该点击"Reset"按钮，这个按钮按下列方式对两个文本域进行了设置：

```
fileNameField.setText("Enter file name here.");
firstLineField.setText("");
```

前面两行代码中的第二行将文本域中的文本设置为""，也就是中间什么也没有的两个引号。空字符串就是这样写的，空字符串就是不包含字符的字符串。将一个文本域设置为空字符串时，就是将其设置为空了。（即使不点击"Reset"按钮，GUI也会给出正确的答案，但如果点击一下"Reset"按钮，事情就会更清晰一些。）

在这个例子中，将创建及显示GUI的main方法放在了GUI的类文件中。这就意味着不需要第二个应用程序来显示GUI了。选择用第二个应用程序文件来显示GUI，还是用GUI类的main

方法来显示GUI，要取决于具体的情况，某种程度上还要取决于个人的喜好。

自测题

44. 图2-21的GUI类FileOrganizer中包含了一个创建及显示GUI的main方法。（像前面那个例子那样）从一个独立的应用程序中创建及显示GUI，并且在不删除文件FileOrganizer.java中的main方法的情况下这么做，是合法的吗？
45. 在图2-21中，可以简单地通过向所有可能抛出IOException的方法中添加throws IOException来取代对IOException的捕获吗？

△ 易犯错误：applet无法操纵另一台计算机上的文件

出于安全的原因，applet所能做的工作是有限的。由于applet是设计为嵌套在Web页面中，并从万维网上其他某个地方的另一台计算机上运行的，所以，不能期望applet通过Web去操纵那台计算机上的文件。因此，无法编写图2-21中JFrame GUI的applet版本，以读取浏览它的计算机上的文件，并将其删除。（可以编写一个在用你计算机中的applet查看程序运行时能够操纵你计算机上文件的applet版本，但在这种情况下也可以使用JFrame GUI。applet通常是应该在万维网上的另一台计算机上运行的。）　△

小　结

- 被当作字符串，且对程序和编译器来说都像是字符的文件被称为文本文件。内容必须作为二进制数字串处理的文件被称为二进制文件。
- 程序可以用PrintWriter类向文本文件写入，用BufferedReader类从文本文件中读出。
- 从文件中读出时，应该总是查看是否达到文件的末尾，如果抵达了文件的末尾，还应该采取一些适当的措施。测试文件末尾的方式取决于程序是从文本文件还是二进制文件中读出的。
- 可以从键盘上将一个文件名读入一个String类型的变量中，并用那个变量来取代文件名。
- 可以用File类来确定是否存在指定名字的文件。还可以用它来确定是否允许程序读取文件或向文件中写入。
- 程序可以用ObjectOutputStream类向二进制文件写入，并可用ObjectInputStream类从二进制文件中读出。
- 程序可以用ObjectOutputStream类的writeObject方法方便地将类对象写入一个二进制文件。可以用ObjectInputStream类的readObject方法将这些对象读回。
- 要使用ObjectOutputStream类的writeObject方法和ObjectInputStream类的readObject方法，那些对象被写入文件中去的类必须实现Serializable接口。

✓ 自测题答案

1. 如果程序将其输出发送到屏幕上去，程序结束时（或以后），输出就会消失。将其输出发送到文件中去的程序就为其输出制作了一份（或多或少）永久性的副本。文件为你提供了一种永久存储数据的方式。文件的内容会一直保存下去，直到有人或程序修改了文件为止。如果程序将其输出发送到一个文件中去，程序结束运行之后，输出文件仍然会保留下来。
2. 从文件传向程序。
3. 文本文件是可以用编辑器写入并读出的文件。可以认为它们包含的是字符。二进制文件用一种不太便

于编辑器读取的方式来表示数据，但程序可以用一种非常有效的方式读出和写入二进制文件。二进制文件表示数据的格式与计算机主存中存储数据时所用的格式相同。

4. `PrintWriter outStream = new PrintWriter(new FileOutputStream("sam"));`
5. `PrintWriter outStream = new PrintWriter(new FileOutputStream("sam", true));`
6. 如果由于已经存在一个名为out.txt的目录（文件夹）而无法打开这个文件的话，会抛出一个FileNotFoundException。注意，如果文件不存在，但可以被创建，就不会抛出异常。如果回答IOException，也没错，因为FileNotFoundException是一个IOException。但是，更具体的异常类，即FileNotFoundException是更好的答案。
7. 没有。这就是使用FileOutputStream类对象作为实参的原因。问题中所示代码的正确表达方式如下所示：

   ```
   PrintWriter outputStream = new PrintWriter(new FileOutputStream("myFile.txt"));
   ```

8. `PrintWriter textStream =  new PrintWriter(new FileOutputStream("dobedo"));`
9. `BufferedReader inputStream = new BufferedReader(new FileReader("dobedo"));`
10. 没有。
11. BufferedReader类中的read和readLine都可以抛出IOException。
12. 图2-7中的try块比较大，包含了对方法readLine的调用，这个调用可能会抛出IOException。图2-1中的方法println不会抛出任何异常。
13. 方法readLine返回一个String类型的值。方法read读入了单个字符，但将其作为一个int类型的值返回。要获取类型char的值，需要进行一次强制类型转换。
14. 方法readLine试图读取超出文本末尾的内容时，会返回值null。因此，可以通过测试null来检测是否达到文件的末尾。
15. 方法read读入单个字符，但将其作为一个int类型的值返回。要获取char类型的值，需要进行一次强制类型转换。
16. 方法read试图读取超出文本末尾的内容时，会返回-1。因此，可以通过测试值-1来检测是否达到文件的末尾。由于所有真正的字符都会返回一个正的int值，所以这种方法才得以工作。
17. FileReader和FileOutputStream类都有可以将文件名作为实参的构造器。PrintWriter和BufferedReader类没有可以将文件名作为实参的构造器。
18. 合法，它等价于下列代码（只是在下列代码中，没有给对象readerObject起名）：

    ```
    BufferedReader inputStream = new BufferedReader(new FileReader("myFile.txt"));
    ```

19. 下列程序的代码包含在Web上本书的源代码中：

    ```
    import java.io.*;
    import java.util.*;

    public class FileClassExercise
    {
        public static void main(String[] args)
        {
            String name = null;
            File fileObject = null;

            System.out.print("Enter a file name and I will ");
            System.out.println("tell you if it exists.");
            Scanner keyboard = new Scanner(System.in);
            name = keyboard.next();
    ```

```
fileObject = new File(name);
if (fileObject.exists())
{
    System.out.println("I found the file " + name);
    System.out.println("Delete the file?");
    String ans = keyboard.next();

    if (ans.equalsIgnoreCase("yes"))
    {
        System.out.println("If you delete the file " + name);
        System.out.println("all data in the file will");
        System.out.println("be lost. Delete?");
        ans = keyboard.next();
        if (ans.equalsIgnoreCase("yes"))
        {
            if (fileObject.delete())
                System.out.println("File deleted.");
             else
                System.out.println(
                            "Cannot delete file.");
        }
        else
            System.out.println("File not deleted.");
}
else
    System.out.println("File not deleted.");
        }
        else
            System.out.println("I cannot find " + name);
    }
}
```

20. `ObjectOutputStream toFile =`
    `new ObjectOutputStream(`
    `new FileOutputStream("stuff.data"));`

21. `toFile.writeDouble(x1);`
    `toFile.writeDouble(x2);`
    `toFile.writeDouble(x3);`

22. `toFile.close();`

23. `import java.io.*;`

24. `ObjectInputStream fromFile =`
    `new ObjectInputStream(new FileInputStream("stuff.data"));`

25. `double x1, x2, x3;`
    `x1 = fromFile.readDouble();`
    `x2 = fromFile.readDouble();`
    `x3 = fromFile.readDouble();`

26. `fromFile.close();`

27. 如果用writeInt将一个数字写入文件，就只能用readInt将其读出。如果用readLong或readDouble读取数字，就会出问题。

28. 不能用readUTF从文本文件中读取字符串。只能用readUTF从二进制文件中读取字符串。而且，字

符串应该是用writeUTF写入文件中去的。

29.
```
ObjectOutputStream writer =
    new ObjectOutputStream(new FileOutputStream(theFile));
```

30.
```
ObjectInputStream reader =
    new ObjectInputStream(new FileInputStream(theFile));
```

31. 下列程序包含在Web上本书的源代码中。

```
import java.io.*;
import java.util.*;

public class FileNameExercise
{
   public static void main(String[] args)
   {
      String fileName = null;//to use fileName in catch block,
                        //must declare it outside of try block.
      try
      {
         System.out.println("Enter file name:");
         Scanner keyboard = new Scanner(System.in);
         fileName = keyboard.next();
         ObjectInputStream inputStream =
                        new ObjectInputStream(
                              new FileInputStream(fileName));
         System.out.println("The first thing in the file");
         System.out.println(fileName + " is");
         String first = inputStream.readUTF();
         System.out.println(first);
         inputStream.close();
      }
      catch(IOException e)
      {
         System.out.println("Problem with input from file " + fileName);
      }
   }
}
```

32. 代码不能工作，因为ObjectInputStream没有使用String类型参数的构造器。实现期望效果的正确方式是使用

```
ObjectInputStream  inputStream =
            new ObjectInputStream(
                  new FileInputStream("mydata.dat"));
```

33. FileInputStream类中没有方法readInt、readDouble及readUTF。

34. 有。

35. 没有。

36. 用本章讨论的方法打开一个二进制文件进行输出或输入时，都可能会抛出FileNotFoundException及其他IOException。

37. 会抛出一个EOFException。

38. 因为抵达文件末尾时，会抛出一个异常，这个异常会结束整个try块。

39. 下列程序包含在Web上本书的源代码中。

```
import java.io.*;

public class TemperatureShow
{
   public static void main(String[] args)
   {
      try
      {
         ObjectInputStream inputStream =
               new ObjectInputStream(new FileInputStream("temperatures"))
double t;
System.out.println("Numbers from the file temperatures:");
try
{
   while(true)
   {
      t = inputStream.readDouble();
      System.out.println(t);
   }
}
catch(EOFException e)
{
   //Do nothing
}

System.out.println("End of reading from file.");
inputStream.close();;
         }
         catch(IOException e)
         {
             System.out.println("Problem reading from file.");
         }
      }
   }
```

40. 在类定义的起始处添加两个单词implements Serializable。
41. 返回类型为Object，这意味着通常要对返回的值进行强制类型转换，将其转换成它“真正的”类。
42. IOException、NotSerializableException及InvalidClassException。
43. ClassNotFoundException, InvalidClassException, OptionalDataException和IOException。
44. 是的，完全合法。试试吧。
45. 这并不是一种好的风格，并且在这种情况下，甚至不可以这样做。可以向方法showFirstLine中添加throws子句，并删除那个方法中的try和catch。但是，这样就得在方法actionPerformed中捕获异常。不能向方法actionPerformed中添加throws子句。请参看第1章1.4节中名为“actionPerformed中不允许使用throws子句”的易犯错误部分。(没有其他可以删除的catch块了。)

○ 编程项目

本小节没有独立的编程项目。下列任意一个编程项目都可以以命令行I/O界面的方式，或者以

JFrame GUI界面方式实现以供用户使用。

1. 编写一个程序，搜索一个由数字组成的文件，并将文件中最大的数字、最小的数字以及文件中所有数字的平均值输出。应该将输出发送到屏幕上去。不要假定文件中的数字是以任何特殊规律排列的。程序应该从用户那儿获取文件名。可以用文本文件或二进制文件来实现这个项目。对文本文件版本，文件应该是个每行一个数字的文本文件。对二进制版本，文件应该是完全由writeDouble写入的double类型的数字组成的。
2. 编写一个程序，读入一个由int类型数字组成的文件，并将所有的数字输出到另一个文件中去，但不要输出重复的数字。假设输入文件是从小到大排序的。运行程序之后，新文件中会包含原来文件中的所有数字，但文件中没有哪个数字会出现多次。输出文件中的数字也应该是从小到大排列的。程序应该从用户处获取这两个文件名。对文本文件版本，文件应该是每行一个数字的文本文件。对二进制版本，文件应该完全是由writeInt写入的int类型的数字组成的。
3. 编写一个程序，对文本文件的格式及标点等方面的问题进行检查。程序会请求一个输入文件名和一个输出文件名，然后将输入文件中的所有文本都拷贝到输出文件中去，但要进行下列两点修改：（1）两个或多个空格组成的字符串都被单个空格所取代；（2）所有的句子都以大写字母开头。将后面跟着一个或多个空白字符的句号、问号或感叹号之后的位置定义为一个句子的开始。
4. 编写一个类似于图2-19的程序，允许向一个文件中写入任意数量的濒危物种记录。文件名和记录的数据都包含了与单个濒危物种有关的信息，这些内容是从键盘读入的。然后再编写另一个程序，这个程序可以搜索第一个程序创建的文件，并将所请求的记录展示给用户。用户给出文件名，然后输入物种的名字。程序显示那个物种的完整记录，或者给出一条消息，表明它没有那个物种的记录。允许用户输入其他的物种名，直到用户表示要结束为止。
5. 编写一个程序，从编程项目4的程序创建的文件中读取数据，并将下列信息发送到屏幕上去：数量最少的物种的记录，以及数量最多的物种的记录。不要假设文件中的记录是以任何特定的规律排列的。由用户给出文件名。
6. 除了其他内容之外，编程项目4请你编写一个程序，创建一个包含了Species类对象的文件。编写一个程序，从那个程序创建的文件中读出，并将记录输出到另一个文件中去。但是，输出文件中的记录所显示的种群规模应该与输入文件中的不同，它应该给出在指定的物种增长率下，100年之后的种群规模。可能要使用Species类的projectedPopulation方法。
7. 编写一个程序，允许向一个文件中写入（《Java程序设计与问题解决：基础篇》第5章图5-20中的）PetRecord类型的记录，或从文件中将其读出。程序询问用户是否想向文件中写入，或从文件中读出。在任何一种情况下，程序接下来都要询问文件名。如果用户要向文件中写入，就允许用户想输入多少记录就输入多少记录。如果用户想从文件中读出，就向其展示文件中的所有记录。确保记录不会滚动的太快使用户无法阅读。重新定义PetRecord类，使其成为可串行化的。
8. 编写一个程序，从一个（用在编程项目7中编写的程序创建的）文件中读取PetRecord类型的记录，并将下列信息输出到屏幕上去：最大的宠物的名字和重量、最小的宠物的名字和重量、最年轻的宠物的名字和年龄以及最老的宠物的名字和年龄。

第3章 动态数据结构与泛型

所有的事情都是变化的，没有任何事情是静止不动的。

——赫拉克里特斯

数据结构（data structure）是一种以特定的方式来组织数据的构造。比如，数组就是一种数据结构。每种数据结构都以其特定的方式来组织数据。本章讨论了两种尺寸会随程序的运行而变大（及缩小）的数据结构：向量和链式数据结构。向量与数组类似，但在某些情况下提供了更强的灵活性。

链式数据结构是一组数据结构的统称，这些数据结构可以通过各种不同的方式变大或缩小。尽管有很多不同类型的链式数据结构，但我们会将重点放在一种简单，但很实用的链式数据结构——链表上。在介绍链表的过程中，会介绍内部类的概念，内部类是在另一个类定义的内部定义的。

从5.0版开始，Java允许定义中带有表示类型的参数。这被称为泛型（generic）。我们将在3.1节展示如何使用这样的定义。在3.3节展示如何编写带有类型参数的类定义。

目标

- 熟悉向量的概念，以及如何在Java中使用它们。
- 了解什么是链式数据结构，以及如何在Java中实现它们。
- 弄清楚如何操纵链表。
- 学会在定义链式数据结构时使用内部类。
- 了解什么是迭代器，以及如何创建和使用它们。
- 了解Java中泛型（用来表示类型的参数）的概念。

预备知识

可以以任意顺序阅读3.1节关于向量的内容以及3.2节关于链式数据结构的内容。应该在阅读3.3节关于泛型的内容之前阅读3.1节和3.2节。

要想完全理解本章的内容，需要阅读《Java程序设计与问题解决：基础篇》第2章~第6章的内容。还需要熟悉基本的继承和基本的异常处理概念。细节如下所示：

节　号	预备知识
3.1节	《Java程序设计与问题解决：基础篇》的第1章~第6章，以及本书的1.1节
3.2节	《Java程序设计与问题解决：基础篇》的第1章~第6章

3.2.5节	《Java程序设计与问题解决：基础篇》的第1章~第6章以及本书的第1章
3.3节	3.1节及3.2.5节之前的部分，不包括3.2.5节

3.1 向量

“好，我就吃了它，”爱丽丝说，“如果它使我变大，我就可以够着钥匙了；如果它使我变得更小，我就能从门缝下面爬过去，反正不管怎样，我都可以到那个花园里去了。”

——刘易斯·卡莱尔《爱丽丝漫游奇境记》

可以把**向量**（vector）当成在程序运行时长度可以变大（及缩小）的数组。在Java中，可以在程序运行时读入数组的长度，但是一旦程序创建了那个长度的数组，数组的长度就无法改变了。例如，假设编写了一个程序来记录一个邮购商店的客户订单，并将一个顾客的所有订单都放在一个名为OrderItem的类对象数组中。可以询问用户要订购多少件物品，并将这个数字存储在一个名为numberOfItems的变量中，然后用下列语句创建数组item：

```
OrderItem[] item = new OrderItem[numberOfItems];
```

但是如果客户输入了numberOfItems件物品，然后又决定再订购另一件物品会怎么样呢？数组item的长度是无法增加的。有一些方法可以解决数组的这个问题，但这些方法都相当复杂，而且需要创建一个新数组。除了向量可以在程序运行时改变长度之外，向量和数组的用途是一样的。因此，可以用向量来处理客户的额外订单，不会有任何问题。

如果向量与数组相似，但前者具有更好的、可以改变长度的附加特性，为什么不干脆用向量取代数组呢？光明好像总是会伴随着黑暗，对向量来说也是如此。向量有两个主要问题：(1)它们的效率比数组低；(2)向量的基类型必须是类类型，不能是int、double或char这样的基本类型。例如，如果需要一个包含int值的向量，就必须用一个包含Integer值的向量来模拟这个结构，其中，Integer是一个包装类，它的对象对int值进行了模拟。（如《Java程序设计与问题解决：基础篇》第5章所讨论的那样）自动装箱和拆箱使(2)不成什么问题，因为具有基类型，比如Integer类型的向量，实际上是可以存储int类型的值的。

3.1.1 使用向量

向量的用法和数组非常相似，但又有一些重要的区别。首先，Vector类的定义不是自动提供的。定义位于java.util包中，所有使用Vector类的代码都必须包含下列语句，通常位于文件的起始处：

```
import java.util.*;
```

除了要用一种新记法来指定基类型之外，向量的创建和命名方式与所有的类对象都是一样的。例如：

```
Vector<String> v = new Vector<String>(20);
```

这条语句将v作为存储String类对象的向量的名字，向量的初始（initial）**容量**为20个条目。说一个向量具有特定的容量，表示为它分配了那么多个条目所需的内存，但如果需要装载更多的条目，系统会自动为它分配更多的内存。仔细地为向量选择初始容量，通常可以使代码

的效率更高，但这个容量对向量可以装载多少个条目没有影响。如果容量选择的足够大，系统就不需要经常重新分配内存，这样，程序就可以运行得更快了。另一方面，如果容量设置的太大，就会浪费存储空间。但是，不管选择什么样的容量，都可以用向量来做任何想做的事情。

前面那个向量示例中的String类型是向量类的**基类型**（base type）。向量——即向量类的一个对象——中存储的是其基类型的对象。可以用任意类类型作为一个向量类的基类型，但是，不能用int或double这样的基本类型作为向量类的基类型。

快速参考：创建及命名一个向量

除了需要指定向量的基类型之外，Vector类对象的创建和命名方式与所有其他对象一样。

语法：

```
Vector<Base_Type> v = new Vector<Base_Type>();
Vector<Base_Type> v2 = new Vector<Base_Type>(Initial_Capacity);
```

*Base_Type*必须是个类类型，不能是int或double这样的基本类型。将一个数字作为构造器的实参时，这个数字就决定了向量的初始容量。

举例：

```
Vector<String> v = new Vector<String>();
Vector<Double> v2 = new Vector<Double>(30);
```

可以像使用数组那样使用向量，但向量没有数组那种方括号记法。如果对字符串数组a使用

```
a[index] = "Hi Mom!";
```

那么，对向量v来说，类似的语句就是

```
v.setElementAt("Hi Mom!", index);
```

如果对字符串a使用

```
String temp = a[index];
```

那么，对向量v来说，类似的语句就是

```
String temp = v.elementAt(index);
```

setElementAt和elementAt这两个方法对向量所起的作用与方括号对数组所起的作用基本相同。但你需要弄清楚的很重要的一点是：方法调用

```
v.setElementAt("Hi Mom!", index);
```

不总是完全类似于下面这行代码：

```
a[index] = "Hi Mom!";
```

方法setElementAt可以取代任意现存的元素，但无法像对数组所做的那样，用setElementAt将一个元素放到任意的索引位置上去。方法setElementAt是用来修改元素值，而不是用来首次设置这些元素值的。首次设置元素时，通常要使用方法addElement。方法addElement会按照索引位置0、位置1、位置2等这样的顺序来添加元素。这就意味着向量必须总是按此顺序添加。然后，代码可以回过头去，对任意单独的元素进行修改，就像可以对数组所做的那样。

可以用方法size来确定有多少个索引位置包含了元素。如果v是一个向量，v.size()就

会返回那个向量的**大小**（size），也就是存储在v中的元素的数量。这些元素的索引从0开始，到v.size()减一为止。

快速参考：addElement

可以用方法addElement将元素添加到向量中去。元素会被添加到索引位置0、然后是位置1，以此类推。

举例：

```
v.addElement("Zero");
v.addElement("One");
v.addElement("Two");
```

对象v是一个基类型为String的向量。

快速参考：访问一个索引

如果v是个向量，可以按下列方式来访问它的元素：

举例：

```
Vector<String> v = new Vector<String>();
int index;
   ...
v.setElementAt("Here", index);      //Sets the element
                                    //at index to "Here".
String temp = v.elementAt(index);  //The expression
   //v.elementAt(index) returns the element at position index.
Vector<Integer> v2 = new Vector<Integer>();
   ...
v2.setElementAt(42, index);//Sets the element at index to
   //new Integer(42). This example relies on automatic boxing.
Integer temp2 = v2.elementAt(index);//The expression
   //v.elementAt(index) returns the element at position index.
```

index必须大于或等于0且小于向量v的当前尺寸。

快速参考：方法size

方法size会返回向量中的元素个数。

举例：

```
for (index = 0; index < v.size(); index++)
   System.out.println(v.elementAt(index));
```

v是个向量，而index是int类型的。

对数组来说，方括号和实例变量length是自动提供给程序员的仅有工具。如果想用数组做其他一些事情，就必须编写代码来操纵数组。另一方面，向量有大量强有力的方法可供选择，这些方法可以实现很多对数组来说需要编写代码才能实现的功能。比如，Vector类有一个方法可以在向量中的两个元素之间插入一个新元素。图3-1对这些方法中的大部分进行了描述。

构 造 器

public Vector<*Base_Type*>(int initialCapacity, int capacityIncrement)

用指定的*Base_Type*以及指定的初始容量和容量增量构造一个空向量。当向量需要增大时，会添加capacityIncrement个条目的空间。*Base_Type*必须是类类型，不能是int或double这样的基本类型。

public Vector<*Base_Type*>(int initialCapacity)

用指定的*Base_Type*以及指定的初始容量创建一个空向量。向量需要增加其容量时，容量会加倍。*Base_Type*必须是类类型，不能是int或double这样的基本类型。

public Vector<*Base_Type*>()

用指定的*Base_Type*以及初始容量10创建一个空向量。向量需要增加其容量时，容量会加倍。*Base_Type*必须是类类型，不能是int或double这样的基本类型。

类似数组的方法

public void setElementAt(*Base_Type* newElement, int index)

将指定index上的元素设置为newElement。那个位置上原来的元素被丢弃。如果要在向量和数组a之间找到相似处的话，这条语句类似于将a[index]设置为值newElement。index必须是一个大于或等于0且小于向量当前大小的值。如果index不在这个范围内，就抛出ArrayIndexOutOfBoundsException。

public *Base_Type* elementAt(int index)

返回指定索引位置的元素。这条语句类似于为数组a返回a[index]。index必须是一个大于或等于0且小于向量当前大小的值。如果index不在这个范围内，就抛出ArrayIndexOutOfBoundsException。

添加元素的方法

public void addElement(*Base_Type* newElement)

在调用向量的末尾添加指定元素，并将向量的大小加一。如果需要的话就增加向量的容量。

public void insertElementAt(*Base_Type* newElement, int index)

在调用向量的指定索引位置插入newElement作为一个元素。向量中索引大于或等于index的每个元素都向上移，拥有一个比它原来拥有的索引值大一的索引。index的值必须大于或等于0且小于或等于向量当前的大小。如果index不在这个范围内，就抛出ArrayIndexOutOfBoundsException。注意，可以用这个方法在当前最后一个元素后面添加一个元素。如果需要的话，向量的容量会增加。

删除元素的方法

public void removeElementAt(int index)

删除指定索引位置的元素。向量中索引号大于index的每个元素都会改为拥有一个比它原来的索引值小一的索引。index必须是一个大于或等于0且小于向量当前大小的值。如果index不在这个范围内，就抛出ArrayIndexOutOfBoundsException。

public boolean removeElement(Object theElement)

从调用向量中删除第一次出现的theElement。如果在向量中找到了theElement，那么向量中索引号大于theElement的每个元素都会改为拥有一个比原来索引值小一的索引。如果找到（且删除）了theElement，就返回true。如果在调用向量中没有找到theElement，就返回false。

public void removeAllElements()

从调用向量中删除所有元素，并将向量大小设置为零。

搜索方法

public boolean contains(Object target)

如果target是调用向量中的一个元素，就返回true；否则就返回false。

图3-1　Vector类中的一些方法

public int indexOf(Object target)

返回第一个等于target的元素的索引。使用对象target的equals方法来测试相等性。如果没找到target，就返回-1。

public int indexOf(Object target, int startIndex)

返回第一个等于target元素的索引，但只考虑大于或等于startIndex的索引值。使用对象target的equals方法对相等性进行测试。如果没找到target，就返回-1。

public int lastIndexOf(Object target)

返回最后一个等于target的元素的索引。使用对象target的equals方法来测试相等性。如果没找到target，就返回-1。

public *Base_Type* firstElement()

返回调用向量的第一个元素。如果向量为空就抛出NoSuchElementException。

public *Base_Type* lastElement()

返回调用向量的最后一个元素。如果向量为空就抛出NoSuchElementException。

内存管理（大小和容量）

public boolean isEmpty()

如果调用向量为空（即，大小为0）就返回true；否则，就返回false。

public int size()

返回调用向量中元素的数量。

public int capacity()

返回调用向量的当前容量。

public void ensureCapacity(int newCapacity)

增加调用向量的容量，以确保向量至少可以装载newCapacity个元素。使用ensureCapacity有时可以提高效率，但除了这个原因外，一般不会使用它。

public void trimToSize()

将调用向量的容量削减为向量的当前大小。可以用这个方法来节省存储空间。

public void setSize(int newSize)

将调用向量的大小设置为newSize。如果newSize大于当前的大小，新元素的值就为null。如果newSize小于当前的大小，索引大于及等于newSize的所有元素都会被丢弃。如果newSize为负，就抛出ArrayIndexOutOfBoundsException。

制作副本

public Object clone()

返回调用向量的一份克隆。克隆是调用向量的一份完全相同的副本。克隆是一份独立的副本：对克隆所做的后继修改不会对调用对象产生影响，反之亦然。

相 等 性

public boolean equals(Object other)

如果other是另一个（任意基类型的）向量，那么当且仅当这两个向量大小相同，且以相同的顺序包含了相同的元素列表时，equals才会返回true。

图3-1 （续）

常见问题：为什么有些参数是*Base_Type*类型的，而其他一些则是Object类型的？

看看图3-1中的那些方法。在有些情况下，当参数自然是一个基类型的对象时，参数类型就是基类

型，但在其他情况下，就是Object类型的了。比如，来看看addElement方法和removeElement方法。第一个方法有一个基类型的参数，第二个方法有一个Object类型的参数。为什么参数的类型会有所不同呢？在这种情况下可以这样解释：只有添加基类型的元素才是有意义的，但从某种意义上来说，删除任意类型的元素都是有意义的。用一个向量中没有的元素作为实参来调用removeElement时，方法对向量没有任何影响。参数类型中这些微小的细节差异使方法可以适用于包含了不同类型的集合之间交互的更大的领域。遗憾的是，这个话题超出了本书的范围（向量只是Java库中各种不同种类的集合中的一种。）

○ 编程提示：向向量中添加元素

首次将元素放入（位于一个向量索引上的）向量位置时，通常可以使用addElement方法。方法addElement会按照索引位置0、1、2等的顺序来添加元素。然后代码可以回过头去，用setElementAt对任意单独的元素进行修改。但是，setElementAt只能对在索引位置上已经存在的元素进行重置。不能用setElementAt向向量中以前未用过的索引位置添加元素。

还可以用insertElementAt向向量中添加元素。例如，假设v是一个基类型为String的向量。要在向量v的索引位置3插入字符串"Do Be Do"，可以使用下列语句：

```
v.insertElementAt("Do Be Do", 3);
```

如果3大于v.size()，运行这行代码时，就会得到一条说明ArrayIndexOutOfBoundsException的错误消息。错误消息（异常）中提到的是数组，但在这种情况下，它也适用于向量。注意，这是一个运行时的错误消息，不是编译器的错误消息。

一般来说，要使下列语句有效，index的值必须小于或等于v.size()：

```
v.insertElementAt(elementToInsert, index);
```

使用的最后一个索引位置为v.size()-1，这一约定意味着可以在第一个未用的索引位置（以及任意使用过的索引位置）插入元素，但不能在任何更高的索引位置插入元素。这种限制确保了向量中的元素总是位于位置0、1等等，直到最后的索引位置，中间没有空隙。

用方法insertElementAt在索引位置插入一个新元素时，位于那个索引位置以及更高位置的元素的索引值都会加一，这样就可以在不丢失任何原有元素的情况下，腾出空间来插入新元素了。与向数组中插入元素的过程不同，这个过程是自动发生的，不需要编写任何额外的代码来移动元素。

自测题

1. 假设v是Vector<String>类的一个对象。如何才能将字符串"Hello"添加到向量v中去？
2. 假设instruction是Vector<String>类的一个对象，在索引位置5上包含了字符串"Stop"。如何才能将索引位置5的字符串改成"Go"？
3. 可以用方法setElementAt将一个元素放到向量中的任意位置上去吗？
4. 可以用方法insertElementAt将一个元素放到向量中的任意位置上去吗？可以用方法insertElementAt将一个元素插入到无法用setElementAt设置的任意位置（索引）上去吗？
5. 如果用下列语句创建一个向量，向量中可以包含多于20个的元素吗？

```
Vector<Double> v = new Vector<Double>(20);
```

6. 假设v是一个Vector<String>类的对象。给出可以将向量v中所有元素输出到屏幕上去的代码。
7. 编写一个类，根据《Java程序设计与问题解决：基础篇》第6章图6-13中SelectionSort类的轮廓，将字符串按照词典序排列。但是，定义中要使用Vector<String>类的向量，而不是元素类型为int的数组。对单词来说，如果所有的单词都是小写或大写字母的，词典序就可以简化为字母序。可以用

String的方法compareTo来比较两个字符串，看看按照词典序哪个在前面。对字符串s1和s2来说，如果按照词典序s1在s2之前，s1.compareTo(s2)就会返回一个负数；如果s1等于s2，就返回0；如果按照词典序s1在s2之后，就返回一个正数。将类命名为StringSelectionSort。(提示：后面是一个可以用来对类进行测试的测试程序。)(Web上本书的源代码中包含了这个程序。)

```
import java.util.*;
public class StringSelectionSortDemo
{
   public static void main(String[] args)
   {
      Vector<String> b = new Vector<String>();
      b.addElement("time");
      b.addElement("tide");
      b.addElement("clouds");
      b.addElement("rain");

      System.out.println("Vector values before sorting:");
      int i;
      for( i = 0; i < b.size(); i++)
          System.out.print(b.elementAt(i) + " ");
      System.out.println();

      StringSelectionSort.sort(b);
      System.out.println("Vector values after sorting:");
      for(i = 0; i < b.size(); i++)
          System.out.print(b.elementAt(i) + " ");
      System.out.println();
   }
}
```

△ 易犯错误：使用capacity而不是size

在图3-2中，使用了下列代码来打印向量toDoList中的所有字符串列表：

```
int vectorSize = toDoList.size();
for (position = 0; position < vectorSize; position++)
     System.out.println(toDoList.elementAt(position));
```

```
import java.util.*;

public class VectorDemo
{
   public static void main(String[] args)
   {
      Vector<String> toDoList = new Vector<String>(10);
      System.out.println("Enter items for the list, when prompted.");
      boolean done = false;
      String next = null;
      String ans;
      Scanner keyboard = new Scanner(System.in);

      while (! done)
      {
          System.out.println("Input an entry:");
```

图3-2 向量演示

```
        next = keyboard.nextLine();
        toDoList.addElement(next);
        System.out.print("More items for the list? ");

        ans = keyboard.next();
        keyboard.nextLine();
        if (!(ans.equalsIgnoreCase("yes")))
              done = true;
    }
        System.out.println("The list contains:");
        int position;
        int vectorSize = toDoList.size();
        for (position = 0; position < vectorSize; position++)
             System.out.println(toDoList.elementAt(position));
    }
}
```

用作void函数时，nextLine读入的字符串就被丢弃了。

屏幕对话示例

```
Enter items for the list , when prompted.
Input an entry
Buy milk.
More items for the list? yes
Input an entry
Wash car.
More items for the list? yes
Input an entry
Do assignment.
More items for the list? no
The list contains:
Buy milk.
Wash car.
Do assignment.
```

图3-2　（续）

如果错误地使用了方法capacity而不是size，程序就会试图写出垃圾值。在这种情况下，我们很可能会发现错误，因为如果用capacity取代了size，程序就会以一条说明它抛出了ArrayIndexOutOfBoundsException的消息作为结束（除非size和capacity碰巧相等）。△

Java提示：用trimToSize来节省内存

当程序需要向量拥有额外的容量时，向量会自动增加其容量。但是，容量的增加可能会超出程序的需求。而且，当程序需要向量具有较小容量时，向量是不会自动缩减的。如果向量有大量额外的容量，可以用方法setSize和trimToSize来缩减向量的容量以节省内存。如果v是一个向量，调用v.setSize(n)会将v的大小设置为n，并将位置n及更高位置的所有元素都丢弃。调用v.trimToSize()会将向量v的容量缩减为v的大小，这样，v中就不会有未用的容量了。通常，只有在知道向量以后不再需要其额外容量时，才能使用trimToSize。

△ 易犯错误：使用方法clone

与其他类对象以及数组一样，不能使用赋值语句生成向量的副本。例如，考虑下列代码：

```
Vector<String> v = new Vector<String>(10);
<一些用来填充向量v的代码。>
Vector<String> otherV;
otherV = v;
```

这段代码只是将otherV设置成了向量v的另一个名字，这样就有了两个名字，但只有一个向量。如果想使otherV等于v，但又有别于v的一个副本，就应该按下列方式使用方法clone：

```
otherV = (Vector<String>)v.clone();
```

一定要注意前面那行代码中向类型Vector<String>进行的强制类型转换。需要进行一次这样的强制类型转换，因为方法clone返回的是一个Object类型的值，下列代码行会产生一条错误消息：

```
Vector otherV = v.clone(); //Incorrect form
```

clone返回一个Object类型值的原因与语言的特性有关，而与向量本身无关，在这儿解释有些太复杂了，但对付这种不便特性的方式却是简单明了的：只要如前面那个例子所示，插入一个强制类型转换就行了。

类可以有向量类型的私有实例变量，比如Vector<String>。但是，向量类型的私有实例变量与我们讨论过的数组类型的私有实例变量的复杂性相似。假设一个类中带有一个数组类型的私有实例变量。在《Java程序设计与问题解决：基础篇》第6章的易犯错误部分“返回数组实例变量”中曾提到，为了防止程序员直接访问私有的数组实例变量，访问方法应该返回数组的一个副本。同样的道理也适用于向量类型的私有实例变量，比如Vector<String>。访问方法不应该返回私有实例向量自身，而应该返回向量的一个副本。要生成向量的一个副本，可以使用方法clone来生成私有实例向量的一个克隆（一个副本），并返回那个克隆。

但是方法clone的使用有点儿复杂，而且会产生一些陷阱。首先，方法clone的返回类型为Object；它不是个向量类型。为了弄清楚这种情况可能带来的问题，假设有一个按下列形式声明的、名为v的私有实例变量：

```
public class SampleClass
{
    private Vector<String> v;
          .
          .
          .
}
```

将下列访问方法添加到SampleClass中去时，会产生一个编译器错误消息：

```
public Vector<String> getVector()
{
    return v.clone();
}
```

问题在于v.clone是Object类型的，为了使其与Vector类型的指定的返回值相匹配，需要进行强制类型转换。一种避免产生编译器错误消息的方式是按下列形式编写访问方法：[①]

① 这会引发一个编译器警告，因为编译器不会去查看强制类型转换的安全性。你的责任是只在那些有意义的地方，比如这里，使用强制类型转换。如果能确定强制类型转换没有问题，就可以忽略那个警告。

```
public Vector<String> getVector()
{
    return (Vector<String>)v.clone();
}
```

另一个问题是在对大多数类，而不仅仅是向量类使用clone方法时都经常会遇到的一般问题。如果存储在向量中的对象自身没有行为良好的clone方法，那么向量的克隆只会简单地复制向量中元素的内存地址，而不会复制向量中实际的元素。这样仍然可以对私有数据进行访问。即使用到的所有其他类都拥有行为良好的clone方法，仍然需要做一些额外的工作，使SampleClass中的访问方法正确地工作。精确的细节超出了本书的范围，但这至少提醒了你，可能会存在一些潜在的问题。 △

3.1.2　参数化的类与泛型

Vector类是一个**参数化的类**（parameterized class）。它有一个参数，用*Base_Type*来表示，可以用任意类类型来取代这个参数，以获得具有指定基类型的向量类。现在，Vector只是Java库（即java.util）中定义的一个类。从1.5版开始，Java允许在定义中带有类型参数。这些带有类型参数的定义被称为**泛型定义**（generic definition），或更简单地称为**泛型**（generic）。你已经知道如何使用这些泛型定义了。（就像使用参数化类Vector那样使用泛型定义。）3.3节给出了一个简单泛型定义的轮廓。

□ Java提示：较老的Vector类

早期的Java版本中有一个名为Vector的类，这个类不是参数化的，其用法与所有其他类一样。例如：

```
Vector v = new Vector(10);
```

在Java的近期版本中仍然有这个较老的向量类，它基本上等价于Vector<Object>类。因此，前面那行代码基本上等价于

```
Vector<Object> v = new Vector<Object>(10);
```

Vector<Object>类及较老的Vector类不是完全等价的，但对简单的应用，可以认为它们是等价的。只有在处理与强制类型转换或向量的子类化有关的问题时，才涉及这些类之间的区别。

这个较老的向量类与较新的参数化向量类一样，位于java.util包中。

□ Java提示：较新的集合类（选读）

Java添加了一组名为**集合类**（collection class）（因为它们实现了Collection接口）的新类。如果继续学习并使用集合类，就可以继续使用Vector类。Java近期版本中的Vector定义已经被改进成一个集合了。在现阶段的Java学习中，这种改变的影响不大。但如果继续学习更多有关Java的内容，你会很高兴地发现所学的向量知识仍然适用于这种新的框架结构。

自测题

8. 没有指定基类型的Vector类的基类型是什么？
9. 向量可以是int类型的吗？
10. 假设v是一个向量，v.capacity()和v.size()之间有什么区别？
11. 假设v是一个向量，而且v.size()返回了10。现在假设程序中有下列调用语句：

```
v.setsize(20);
```

索引号从10到19的新元素中的值为多少？（垃圾值？某些默认值？什么样的默认值？还是其他什么值？）

3.2 链式数据结构

因指见月指非月[①]。

——禅宗谚语

链式数据结构（linked data structure）是对象［被称为**节点**（node）］的集合，每个对象中都包含数据及对另一个节点的引用。我们对链式数据结构的讨论大部分都局限于对一种简单（但被广泛使用）的、名为链表的结构的讨论。

3.2.1 链表

Java的java.util包中有一个预定义的LinkedList类。使用这个预定义类是很明智的，因为这个预定义类是由专家定义、经过良好测试的，可以节省大量的工作。但是，这个类不会教你如何在Java中实现链式数据结构。要实现这一点，需要去看一个简单的、至少构建了一个链式数据结构的例子。链表是一个既简单又典型的链式数据结构。因此，为了弄清楚链式数据结构是如何工作的，我们将自己构建一个简化的链表实例。

图3-3以图形方式显示了**链表**（linked list）的结构。与所有链式数据结构一样，链表由名为节点的对象组成。在图中，节点就是那些被水平线一分为二的方框。每个节点都有存储数据的空间和存储至另一个节点的链接的空间。**链接**（link）是作为箭头显示的，箭头指向它们所“链接”的节点。在Java中，链接是作为节点的引用实现的，实际上，它们是节点类型的实例变量。但是，第一次看到链表时，可以简单地把链接当成箭头。在链表中，每个节点都只包含一个链接，一个节点跟在另一个节点的后面，形成一个链，如图3-3所示。从直觉上看，你，或更恰当地说，你编写的代码可以沿着链接，从一个节点移到另一个节点。标识为head的链接不在节点链表上，实际上，它甚至不是一个节点，而是一个使你进入第一个节点的链接。（实现中，head会包含一个对节点的引用。因此，实际上，head是一个节点类型的变量。）沿着“箭头”，程序可以很容易地按顺序在链表上移动，从第一个节点移到最后一个节点。现在，来看看在Java中到底该如何实现一个链表。

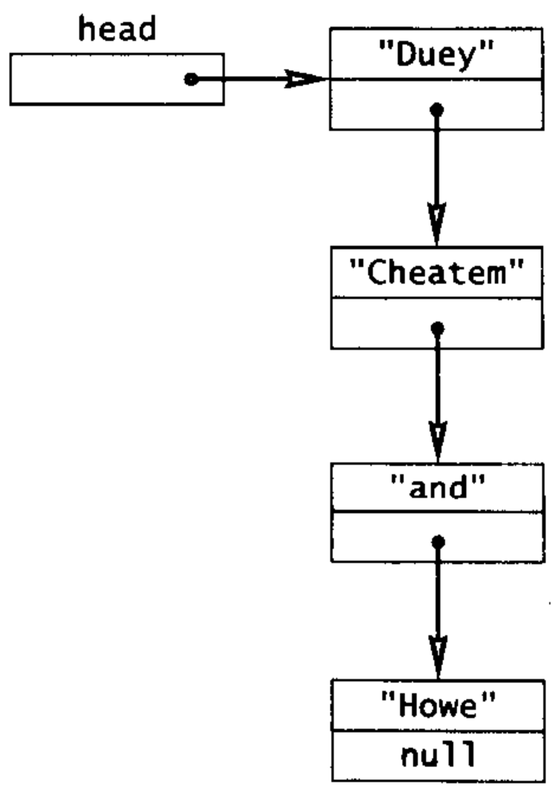


图3-3 链表

① 原文如下：Do not mistake the pointing finger for the moon.

快速参考：链表

链表（linked list）是一种由名为**节点**（node）的对象组成的数据结构。每个节点都既包含数据，又包含对另一个节点的引用，这样，这些节点就可以链接在一起形成一个链，如图3-3所示。链中的第一个节点被称为**头节点**（head node）。

在Java中，每个节点都是一个类的对象，这个类包含两个实例变量，一个用于表示数据，一个用于表示链接。图3-4给出了一个Java类的定义，这个类定义可以作为图3-3那样的链表的节点类使用。在这个例子中，每个节点的数据都是由单个String值组成的[①]。（如图3-4所述，稍后将给出这个节点类及其相关链表类的一个更好的定义。但很多程序员就这样使用这个版本，这个版本对于学习链式数据结构所需的基本技巧已经足够了。）

```
public class ListNode
{
    private String data;
    private ListNode link;

    public ListNode()                                    本章稍后将给出这个
    {                                                    类的更好的定义。
        link = null;
        data = null;
    }

    public ListNode(String newData, ListNode linkValue)
    {
        data = newData;
        link = linkValue;
    }

    public void setData(String newData)
    {
        data = newData;
    }

    public String getData()
    {
        return data;
    }

    public void setLink(ListNode newLink)
    {
        link = newLink;
    }

    public ListNode getLink()                            这种方法可以工作，但会出现
    {                                                    一个问题。我们稍后将对这个
        return link;                                     问题进行讨论，并提供一个更
    }                                                    好的替换方法。
}
```

图3-4　节点类

注意，图3-4中ListNode类的link实例变量是ListNode类型的。这种关系听起来是循环

① 从技术上来讲，节点中不包含字符串，而只包含对字符串的引用，就像所有String类型的变量那样。但对我们的目的而言，可以认为节点中包含了字符串，因为我们决不会将这个字符串引用作为“箭头”使用的。

的，而且在某种意义上，它确实是循环的。但是，这种类型的类定义在Java中是完全合法的。回想一下，类类型的变量中装载的是对那个类的对象的引用。所以，ListNode类对象的link实例变量中会包含对另一个ListNode类对象的引用。因此，在Java中，图3-3图形中显示的箭头是作为引用来实现的。链表的每个节点对象（在其link实例变量中）都包含了对另一个ListNode类对象的引用，而这个对象包含的则是对另一个ListNode类对象的引用，以此类推，直到链表结束为止。

处理链表时，代码应该能够“进入”第一个节点，而且需要通过某种方式来查看什么时候会到达最后一个节点。要使代码进入第一个节点，可以使用包含了对第一个节点的引用的ListNode类型的变量。在图3-3中，带有对第一个节点引用的变量被称为head，并由标记为head的方框来表示。链表中的第一个节点被称为**头节点**（head node），通常包含第一个节点引用的变量都被称为head。

如图3-3所示，在Java中，可以将一个节点对象的link实例变量设置为null来表示链表的结束。这样代码就可以通过测试节点的link实例变量中包含的是否为null来判断该节点是否是链表中的最后一个节点了。在图3-3以及图3-4的Java节点定义中，data实例变量是String类型的，通常可以用equals方法来测试两个这样的String实例变量是否相等。但是，要记住，测试link是否“等于”null时，使用的是==。

记住：大多数节点都没有名字

变量head中包含了对链表中第一个节点的引用。因此head可以被用作第一个节点的名字。但链表中的其他节点都没有包含了其引用的命名变量，因此，链表中的其他节点都是没有名字的。命名其中一个节点的唯一方式就是通过非直接引用，比如head.link.link，或使用另一个ListNode类型的变量，比如（图3-5）showList方法中的局部变量position。

```
public class StringLinkedList
{
    private ListNode head;                    本章稍后将给出这个类的更好的定义。

    public StringLinkedList()
    {
        head = null;
    }

    /**
     Returns the number of nodes in the list.
    */
    public int length()
    {
        int count = 0;
        ListNode position = head;
        while (position != null)
        {
            count++;
            position = position.getLink();
        }
```

图3-5 链表类

```
        return count;
    }

    /**
     Adds a node at the start of the list. The added node has addData
     as its data. The added node will be the first node in the list.
    */
    public void addANodeToStart(String addData)
    {
        head = new ListNode(addData, head);
    }

    public void deleteHeadNode()
    {
        if (head != null)
        {
            head = head.getLink();
        }
        else
        {
            System.out.println("Deleting from an empty list.");
            System.exit(0);
        }
    }

    public boolean onList(String target)
    {
        return (Find(target) != null);
    }

    /**
     Finds the first node containing the target data and
     returns a reference to that node.
     If target is not in the list, null is returned.
    */
    private ListNode Find(String target)
    {
        ListNode position;
        position = head;
        String dataAtPosition;
        while (position != null)
        {
            dataAtPosition = position.getData();
            if (dataAtPosition.equals(target))
                return position;
            position = position.getLink();
        }
        //target was not found
        return null;
    }

    public void showList()
```

图3-5　(续)

```
{
    ListNode position;
    position = head;
    while (position != null)
    {
        System.out.println(position.getData());
        position = position.getLink();
    }
}
```

图3-5 （续）

图3-5包含了一个链表类的定义，这个链表类使用了图3-4给出的节点类定义。注意，其中只有一个实例变量，名为head。这个head实例变量包含了对链表中第一个节点的引用，或者，如果链表为空（即，如果链表中不包含节点），head就为null。构造器将这个head实例变量设置为null，表示一个空表。

记住：用null表示空表

认为变量head中包含了对链表中第一个节点的引用。开始链表通常是空的。在添加第一个节点之前，给head赋什么值呢？要表示一个空表，可以给head赋值null。这是一种传统的技术，对很多用链表操纵的算法都适用。

在继续讨论如何从链表中添加、删除节点之前，假设链表中已经有一些节点，而想将所有节点的内容都写到屏幕上去。可以用（图3-5）中的方法showList来实现，这个方法的主体如下所示：

```
ListNode position;
position = head;
while(position != null)
{
System.out.println(position.getData());
position = position.getLink();
}
```

方法使用了一个名为position的局部实例变量，这个变量中包含了对一个节点的引用。变量position开始包含的引用与head实例变量一样。因此，开始它位于第一个节点。然后，通过赋值语句

```
position = position.getLink();
```

变量position将其位置从一个节点移到下一个节点。

图3-6对这个过程进行了说明。要弄清楚这条赋值语句是怎样将position变量“移动”到下一个节点的，就要注意到，position变量包含了对图3-6中position箭头指向的那个节点的引用。因此，position就是那个节点的名字，而position.link则是指向下一个节点的link的名字。link的值是由访问方法getLink生成的。因此，对链表中下一个节点的引用就是position.getLink()。通过给position变量赋值position.getLink()，就移动了这个变量。

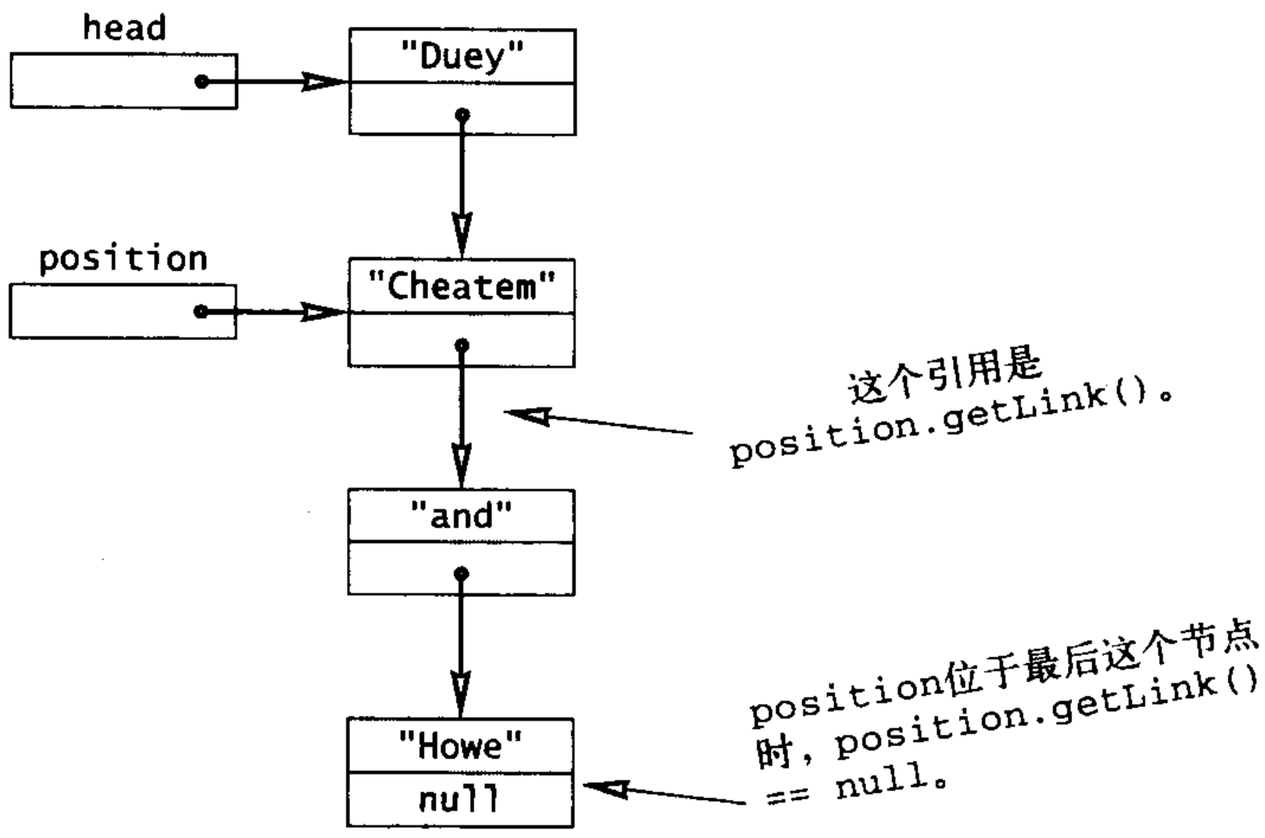


图3-6　沿着链表向下移动

方法showList继续将position变量向链表下端移动，并随着它的移动将每个节点中的数据输出。当position抵达最后一个节点时，输出那个节点中的数据，并再次执行

```
position = position.getLink();
```

研究一下图3-6，就会发现，当position离开最后一个节点时，它的值被设置成了null。此时，希望结束循环，因此，执行迭代循环while(position != null)。

接下来，考虑一下方法addANodeToStart是如何将节点添加到链表起始处，以便将新节点作为链表的第一个节点的。它是用一条语句来实现这项操作的：

```
head = new ListNode(addData, head);
```

新节点是用new ListNode(addData, head)创建的，这条语句会返回一个对新节点的引用。换句话说，变量head被设置为这个新节点的引用，使新节点成为链表中的第一个节点。要将这个新节点链接到列表中其他的节点上去，只需要将新节点的link实例变量设置为对原来第一个节点的引用就行了。我们已经这样做了：head指向原来的第一个节点，因此，如果在赋值操作符的右侧使用名字head，head表示的就是对原来第一个节点的引用。因此，new ListNode(addData,head)生成的新节点就指向原来的第一个节点。这样，所有事情都正常进行。图3-7对这个过程进行了说明。

注意，已经将一个节点添加到链表的起始处了。稍后，将讨论如何在链表的其他位置添加节点，其实，向链表的起始位置添加节点是最简单的。类似地，删除链表起始处的节点也是最简单的。

方法deleteHeadNode将第一个节点从链表中删除，并将变量head指向链表中原来的第二个节点（现在是第一个节点了）（即，使其包含对原来第二个节点的引用）。下面的赋值语句正确地完成了删除工作，对这个过程的说明留给读者考虑：

```
head = head.getLink();
```

很容易看出这项操作从链表中删除了第一个节点，而且链表比原来短了一个节点的长度，但你可能会奇怪，那个被删除的节点怎么样了。Java会在某个地方自动地将它以及所有其他不可访问的节点一起收集起来，回收它们占用的内存。这个过程被称为**自动垃圾回收**(automatic garbage collection)。

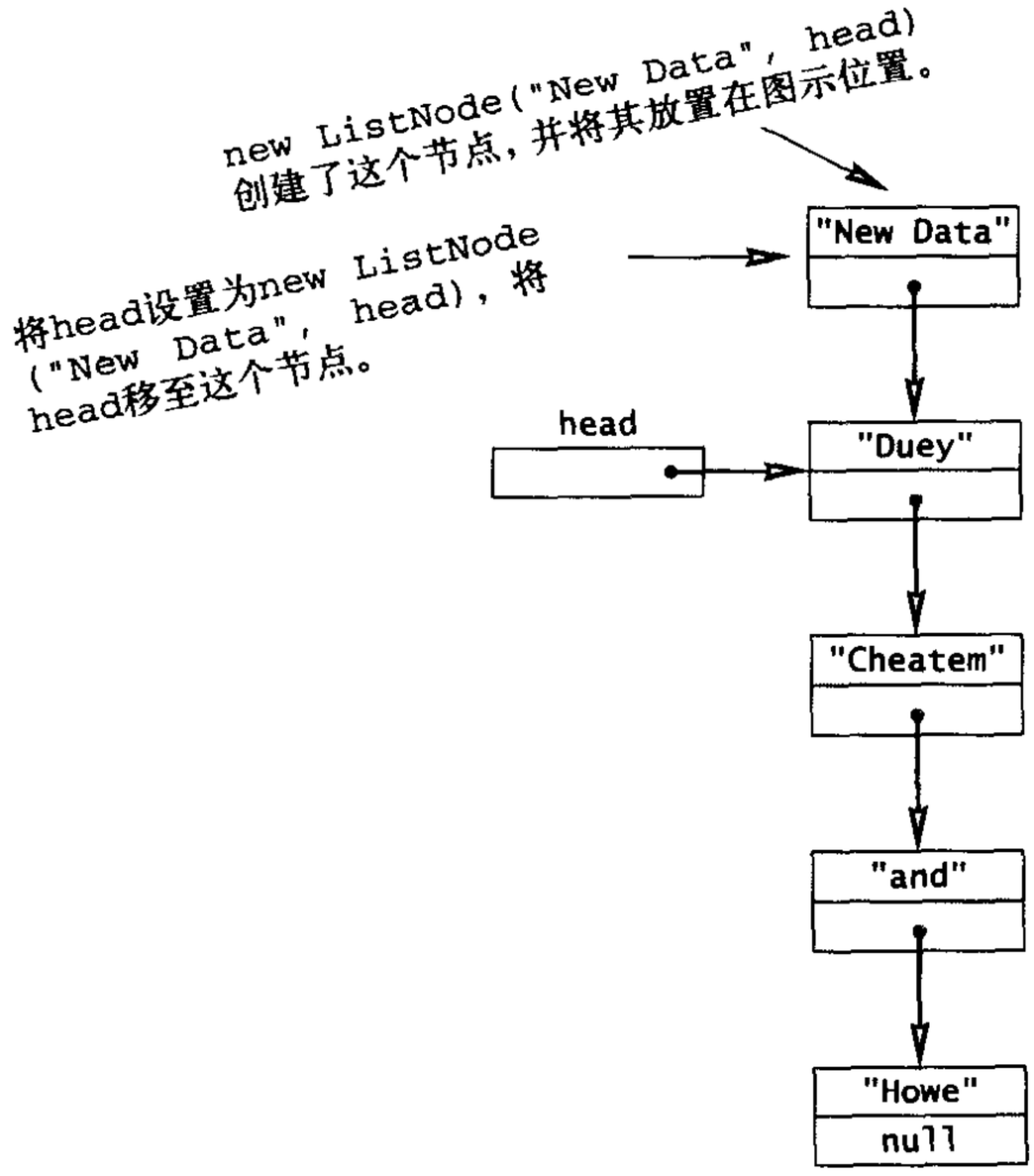


图3-7 在起始处添加一个节点

图3-8中包含了一个简单的程序，这个程序对StringLinkedList类中某些方法的行为进行了演示。

```
public class LinkedListDemo
{
    public static void main(String[] args)
    {
        StringLinkedList list = new StringLinkedList();
        list.addANodeToStart("One");
        list.addANodeToStart("Two");
        list.addANodeToStart("Three");
        System.out.println("List has " + list.length()
                           + " entries.");
        list.showList();
        if (list.onList("Three"))
            System.out.println("Three is on list.");
        else
            System.out.println("Three is NOT on list.");

        list.deleteHeadNode();

        if (list.onList("Three"))
            System.out.println("Three is on list.");
        else
            System.out.println("Three is NOT on list.");

        list.deleteHeadNode();
        list.deleteHeadNode();
```

图3-8 链表的演示

```
        System.out.println("Start of list:");
        list.showList();
        System.out.println("End of list.");
    }
}
```

屏幕输出

```
List has 3 entries.
Three
Two
One
Three is on list.
Three is NOT on list.
Start of list:
End of list.
```

图3-8　（续）

△ 易犯错误：空指针异常

运行程序的时候，你肯定收到过NullPointerException消息。（如果没收到过这条消息，那么，恭喜你！你是个超级细心的程序员。）消息NullPointerException说明代码试图访问类变量的某个成员（某些实例变量、方法或这样的东西），但那个类变量没有命名任何对象，也就是说，没有包含对任何对象的引用。现在这条消息对你来说可能有点儿意义了。在节点中，用null来说明一个链接实例变量中没有包含引用。因此，null值就说明没有对象引用，这就是将此异常称为NullPointer-Exception的原因。

NullPointerException是既不需要用catch块捕获，也不需要在throws子句中声明的异常之一。它说明你需要对代码进行修正，而不是添加一个catch块。　△

自测题

12. 如何标识链表的结尾？
13. 假设变量head中包含了对链表中第一个节点的引用，而且那个链表是空的。head包含的应该是什么值？
14. 定义一个可以添加到（图3-5的）StringLinkedList类中去的、名为empty的方法。方法empty返回一个boolean值。如果链表为空（没有节点）就返回true，如果链表中至少有一个节点，就返回false。
15. 下列代码会产生什么样的输出？

```
StringLinkedList list = new StringLinkedList();
list.addANodeToStart("A");
list.addANodeToStart("B");
list.addANodeToStart("C");
list.showList();
```

△ 易犯错误：隐私泄露

本节讨论的问题很重要，但有点儿微妙。先去复习一下《Java程序设计与问题解决：基础篇》第5章中与此节同名的易犯错误部分，会对理解本节的内容有所帮助。

考虑一下（图3-4）ListNode类中的getLink方法。它会返回一个ListNode类型的值。也就是说，它会返回一个对ListNode的引用。在《Java程序设计与问题解决：基础篇》的第5章中我们说过，如果一个（getLink这样的）方法返回对一个类类型实例变量的引用，那么（除了String这样特定的类之外），对实例变量的private限制是很容易被攻破的。获得对一个对象的引用，程序员就可以修改对象的私有实例变量，隐私泄露就是由此事实引发的。修正这个问题的方式有很多，如接下来两个小节“内部类”和“节点内部类”所讨论的那样，最直接的方式就是将ListNode类作为方法StringLinkedList的私有内部类。（另一种类似的解决方案是将ListNode类和StringLinkedList都放入一个包中，按照附录E中讨论的方法将私有实例变量限制改为包限制，并省略访问方法getLink。当然，采用这种方法需要阅读附录E。）

注意，在方法返回了对类类型私有实例变量的引用的情况下，都会出现这种隐私问题。ListNode类的方法getData()差点儿也存在这样的问题。在这个例子中，方法getData不会引起隐私泄露，但这仅仅是因为String类中没有允许用户（不改变引用，就）改变字符串值的方法。String类是个特例。几乎所有其他类类型的数据，都会遇到《Java程序设计与问题解决：基础篇》第5章关于隐私泄露的易犯错误小节中描述的那种问题。

假设链表中的数据类型是除String之外的其他类类型。getData()就会产生隐私泄露。假设ListNode类只是在StringLinkedList类（以及与之非常相似的类）的定义内使用，那么，在这种特定的情况下，隐私泄露不是问题。这是因为StringLinkedList类中没有哪个公有方法返回对节点的引用。注意，方法Find返回了对ListNode的引用，但是，它是一个私有方法。如果方法Find是公有的，就会产生隐私泄露。因此，要注意，Find方法定义中的private并不仅仅是一个小的风格标志。

尽管在StringLinkedList这样的类定义中使用ListNode类定义没什么问题，但是，除非做一些工作，比如将ListNode类设置为StringLinkedList类的私有内部类，否则就无法保证ListNode类仅仅会在这种方式下使用。（细节请参看接下来的两个小节。） △

3.2.2 内部类

内部类（Inner class）就是在其他类中定义的类。对内部类的完整描述超出了本书的范围。但是，内部类的一些简单的用法是很好理解也很有用的。本节对内部类进行了概要的描述，并说明了如何使用它。在下一小节中，给出了一个内部类的实例。具体来说，给出了一个链表类的例子，这个类的节点类是在该类中定义的一个内部类。这个节点内部类是对前一小节讨论的隐私泄露问题的一种解决方案。

定义一个内部类是很简单的。只要如下所示，将内部类的定义包含在另一个类中就行了：

```
public class OuterClass
{
    Declarations_of_OuterClass_Instance_Variables
    Definitions_of_OuterClass_Methods

  private class InnerClass
  {
    Declarations_of_InnerClass_Instance_Variables
```

```
        Definitions_of_InnerClass_Methods
    }
}
```

如上面的轮廓所示，包含内部类的类被称为**外部类**（outer class）。内部类的定义不一定要是外部类中的最后一项，但把它放在最后或开头会比较好，这样很容易找到。内部类不必一定是私有的，但在本书中只讨论这一种情况。

内部类定义是局部于外部类定义的。因此，在外部类定义之外，可以将内部类的名字重新用于其他目的。如果像我们通常所做的那样，内部类是私有的，在外部类定义之外就不能使用这个内部类了。

内部类定义可以在外部类定义的任意地方使用，为了方便这样使用，内部类还具有一种特殊的性质：内部类和外部类中的方法都可以访问对方的方法及实例变量，即使是私有的也可以。只要在内部类或外部类定义内部，（在内部及外部类中用到的）修饰符public和private就是等价的。尽管内部类用法的一些细节是很巧妙的，但我们只会考虑与下一小节的例子类似的简单情况。

我们将在本书第7章再次对内部类进行讨论。

3.2.3　节点内部类

可以通过将节点类设置为内部类，使StringLinkedList这样的数据结构成为自包含的。尤其是，可以按下列方式，将ListNode作为StringLinkedList类的内部类，使StringLinkedList类具有更高的自包含性：

```
public class StringLinkedList
{
    private ListNode head;
    <插入图3-5中的一些方法>
    private class ListNode
    {
        <ListNode定义的其余部分与图3-4中完全一样。>
    }
}
```

注意，将ListNode类设置成了一个私有内部类。如果不会在其他地方使用内部类，就应该将其设置为私有的。

将ListNode设置为私有内部类也更安全一些，因为这样可以将方法getLink隐藏起来，不为StringLinkedList定义之外的世界所知。正如在本章中名为“隐私泄露”的易犯错误部分中所说的那样，可以公开访问的方法返回对一个类实例变量的引用是很危险的。

如果将ListNode类设置为StringLinkedList定义中的一个私有内部类，就可以删除访问和设置方法（set和get方法），只允许对实例变量（data和link）的直接访问，以此来安全地简化ListNode的定义。在图3-9中用这种方式重新编写了StringLinkedList类。这个名为StringLinkedListSelfcontained的版本与图3-3中的StringLinkedList类拥有相同的方法，这些方法执行了相同的动作，从这个角度来说，它们是等价的。实际上，如果用StringLinkedListSelfcontained取代StringLinkedList来运行图3-8中的LinkedListDemo程序，示例对话框是不会改变的。（Web上提供的源代码中的StringLinkedListSelf-

containedDemo程序完成的就是这项工作。）

记住：节点内部类

可以通过将节点类设置为链表类的内部类，使链表（或其他链式数据结构）成为自包含的。

3.2.4 迭代器

拥有一个对象集合，比如链表中的那些节点时，通常需要遍历集合中所有的对象，并对每个对象执行一些动作，比如将其写到屏幕上去，或以某种方式编辑每个对象中的数据。**迭代器**（iterator）就是任意一个允许以这种方式对列表进行遍历的对象。

数组是对象（或基本类型值）的集合。数组的迭代器是一个int变量。如果数组名为a，int变量名为index，就可以按下列形式对数组中所有的对象进行遍历了：

```
for(index = 0; index < a.length; index++)
   process a[index];
```

int变量index就是迭代器。可以通过动作index++进行**迭代**（iterate）——即，进入下一个对象。

如果将链表中所有的数据对象都放入一个数组中去，就可以对这个数组进行迭代了。如果不想改变链表中的数据，而只是想看看这些数据的话，这个过程与迭代链表的过程是等价的。因此，链表类中通常会有一个方法，用来将链表中所有的数据放入一个数组中去。图3-10显示的就是这样一个方法。可以将这个名为arrayCopy的方法添加到图3-9的链表中去。（Web上源代码中包含的文件StringLinkedListSelfcontained.java中已经实现了这步功能。）

```
public class StringLinkedListSelfcontained
{
    private ListNode head;

    public StringLinkedListSelfcontained()
    {
        head = null;
    }

    /**
     Returns the number of nodes in the list.
    */
    public int length()
    {
        int count = 0;
        ListNode position = head;
        while (position != null)
        {
```

图3-9 将节点类作为内部类的链表

```
            count++;
            position = position.link;
        }
        return count;
    }

    /**
     Adds a node at the start of the list. The added node has addData as its data.
     The added node will be the first node in the list.
    */
    public void addANodeToStart(String addData)
    {
        head = new ListNode(addData, head);
    }

    public void deleteHeadNode()
    {
        if (head != null)
            head = head.link;
        else
        {
            System.out.println("Deleting from an empty list.");
            System.exit(0);
        }
    }

    public boolean onList(String target)
    {
        return (Find(target) != null);
    }

    /**
     Finds the first node containing the target data and returns a reference to that
     node. If target is not in the list, null is returned.
    */
    private ListNode Find(String target)
    {
        ListNode position;
        position = head;
        String dataAtPosition;
        while (position != null)
        {
            dataAtPosition = position.data;
            if (dataAtPosition.equals(target))
                return position;
```

注意，外部类可以直接访问内部类的实例变量，比如link。

图3-9　（续）

```
            position = position.link;
        }
        //target was not found
        return null;
    }

    public void showList()
    {
        ListNode position;
        position = head;
        while (position != null)
        {
            System.out.println(position.data);
            position = position.link;
        }
    }


    private class ListNode
    {
        private String data;
        private ListNode link;
        public ListNode()
        {
            link = null;
            data = null;
        }
        public ListNode(String newData, ListNode linkValue)
        {
            data = newData;
            link = linkValue;
        }
    }

} //end of StringLinkedListSelfcontained
```

内部类

外部类定义的结尾。

图3-9 （续）

快速参考：迭代器

假设有一个数据条目的集合，比如一个数组或一个链表。任何能够以合理的方式、一次一个条目地对集合进行遍历的对象都被称为**迭代器**（iterator）。“合理的方式”意味着在一次完整的迭代周期中，每个条目都正好被访问一次，可以将每个条目中的数据读出，如果数据条目允许的话，还可以对数据进行修改。

比如，一个装载了索引值的int变量就可以作为数组的迭代器使用。要进入集合（数组）的下一个条目，代码只需要将int变量的值加一就行了。

```
public string [] arrayCopy()
{
   String[] a = new String[length()];
   ListNode position;
   position = head;
   :i = 0
   while (position ! = null)
   {
      a[i] = position.data;
      i++,
      position = position.link;
   }
   return a;
}
```

可以将这个方法添加到图3-9的链表类中去。

图3-10　将链表数据放入一个数组中去

如果需要一个能够沿着链表移动，并执行一些操作的迭代器，比如修改节点数据，甚至插入或删除一个节点，那么，包含链表数据的数组就不够用了。但可以从数组迭代器的思想中得到一些启示。就像索引可以用来指定数组元素一样，对一个节点的引用也可以指定这个节点。因此，如果向图3-9中给出的StringLinkedListSelfcontained链表类中添加一个名为current的实例变量，就可以将这个实例变量作为迭代器使用了。图3-11就是这么做的，并将类重命名为StringLinkedListWithIterator。正如所看到的那样，我们添加了很多方法来操纵实例变量current。current是个迭代器，但它被标识为private，所以还需要一些方法来操纵它。我们还添加了一些方法，用来在链表中的任意位置添加或删除节点。由于迭代器提供了一种命名任意节点的方式，所以，对这些添加及删除节点的方法的表述就更简单了。来看看其中一些细节。

```
/**
 Linked list with a notion of "current node." The current node
 can be changed to the next node with the method goToNext. At any
 time after the iteration is initialized, one node is the current
 node, until the iteration has moved beyond the end of the list.
*/
public class StringLinkedListWithIterator
{
    private ListNode head;
    private ListNode current;
    private ListNode previous;

    public StringLinkedListWithIterator()
    {
        head = null;
        current = null;
```

图3-11　带有迭代器的链表

```
        previous = null;
    }

    /**
     Returns the number of nodes in the list.
    */
    public int length()
```

<定义的其余部分与图3-9相同>

```
    public void addANodeToStart(String addData)
    {
        head = new ListNode(addData, head);
        if (current == head.link && current != null)
        //if current is at old start node
            previous = head;
    }

    public boolean onList(String target)
```

<定义的其余部分与图3-9相同>

```
    /**
     Returns a reference to the first node containing the target
     data. If target is not in the list, null is returned.
    */
    private ListNode Find(String target)
```

<定义的其余部分与图3-9相同>

```
    public void showList()
```

<定义的其余部分与图3-9相同>

```
    public String[] arrayCopy()
```

<定义的其余部分与图3-10相同>

```
    public void resetIteration()
    {
        current = head;
        previous = null;
    }
    public void goToNext()
    {
        if (current != null)
        {
            previous = current;
            current = current.link;
        }
        else if (head != null)
        {
            System.out.println(
               "Iterated too many times or uninitialized iteration.");
            System.exit(0);
        }
        else
        {
            System.out.println("Iterating with an empty list.");
```

图3-11 (续)

```
            System.exit(0);
        }
    }

    public boolean moreToIterate()
    {
        return (current != null);
    }

    public String getDataAtCurrent()
    {
        if (current != null)
            return (current.data);
        else
        {
            System.out.println(
                    "Getting data when current is not at any node.");
            System.exit(0);
        }
        return null;//to keep the compiler happy
    }

    public void resetDataAtCurrent(String newData)
    {
        if (current != null)
        {
            current.data = newData;
        }
        else
        {
            System.out.println(
                 "Setting data when current is not at any node.");
            System.exit(0);
        }
    }

    /**
     Inserts node with newData after the current node. The current
     node is the same after invocation as it is before invocation.
     Should not be used with an empty list. Should not be
     used when the current node has iterated past the entire list.
    */
    public void insertNodeAfterCurrent(String newData)
    {
        ListNode newNode = new ListNode();
        newNode.data = newData;
        if (current != null)
        {
            newNode.link = current.link;
            current.link = newNode;
        }
        else if (head != null)
```

图3-11　(续)

```
{
    System.out.println(
                "Inserting when iterator is past all "
              + "nodes or uninitialized iterator.");
    System.exit(0);
}
else
{
    System.out.println(
          "Using insertNodeAfterCurrent with empty list.");
    System.exit(0);
  }
}

/**
 Deletes the current node. After the invocation,
 the current node is the node after the
 deleted node or null if there is no next node.
*/
public void deleteCurrentNode()
{
    if ((current != null) && (previous != null))
    {
        previous.link = current.link;
        current = current.link;
    }
    else if( (current != null) && (previous == null))
    {//At head node
        head = current.link;
        current = head;
    }
    else //current == null
    {
        System.out.println(
         "Deleting with uninitialized current or an empty list.");
        System.exit(0);
    }
}

private class ListNode
{
    private String data;
    private ListNode link;

    public ListNode()
    {
        link = null;
        data = null;
    }
```

有了deleteCurrentNode，就不再需要deleteHeadNode了，但要保留deleteHeadNode，就必须对其重新定义，并对current和previous的值进行说明。

图3-11 （续）

```
        public ListNode(String newData, ListNode linkValue)
        {
            data = newData;
            link = linkValue;
        }
    }
}
```

图3-11　(续)

除了实例变量head和current之外，还添加了一个名为previous的实例变量。这个变量的使用思路是，随着引用current顺着链表向下的移动，引用previous会以一个节点之差跟着它移动。这种设置提供了一种查阅current节点之前的那个节点的方式。由于链表中的链接都是顺着一个方向移动的，所以，需要节点previous以实现一些与回溯一个节点等价的工作。

方法resetIteration将第一个节点（头节点）的引用赋予current，将current的初始值设置在链表的开始，如下所示：

```
current = head;
```

由于实例变量previous没有前一节点可以引用，方法resetIteration就将null值赋给了它。

如下所示，方法goToNext将迭代器移到下一个节点上去了：

```
previous = current;
current = current.link;
```

图3-12对此过程进行了说明。在goToNext方法中，多分支if-else语句的最后两个子句只是在将方法goToNext用于一些毫无意义的场合时，产生一条错误语句。

只要current不等于null，也就是说，只要current中包含了对某个节点的引用，方法

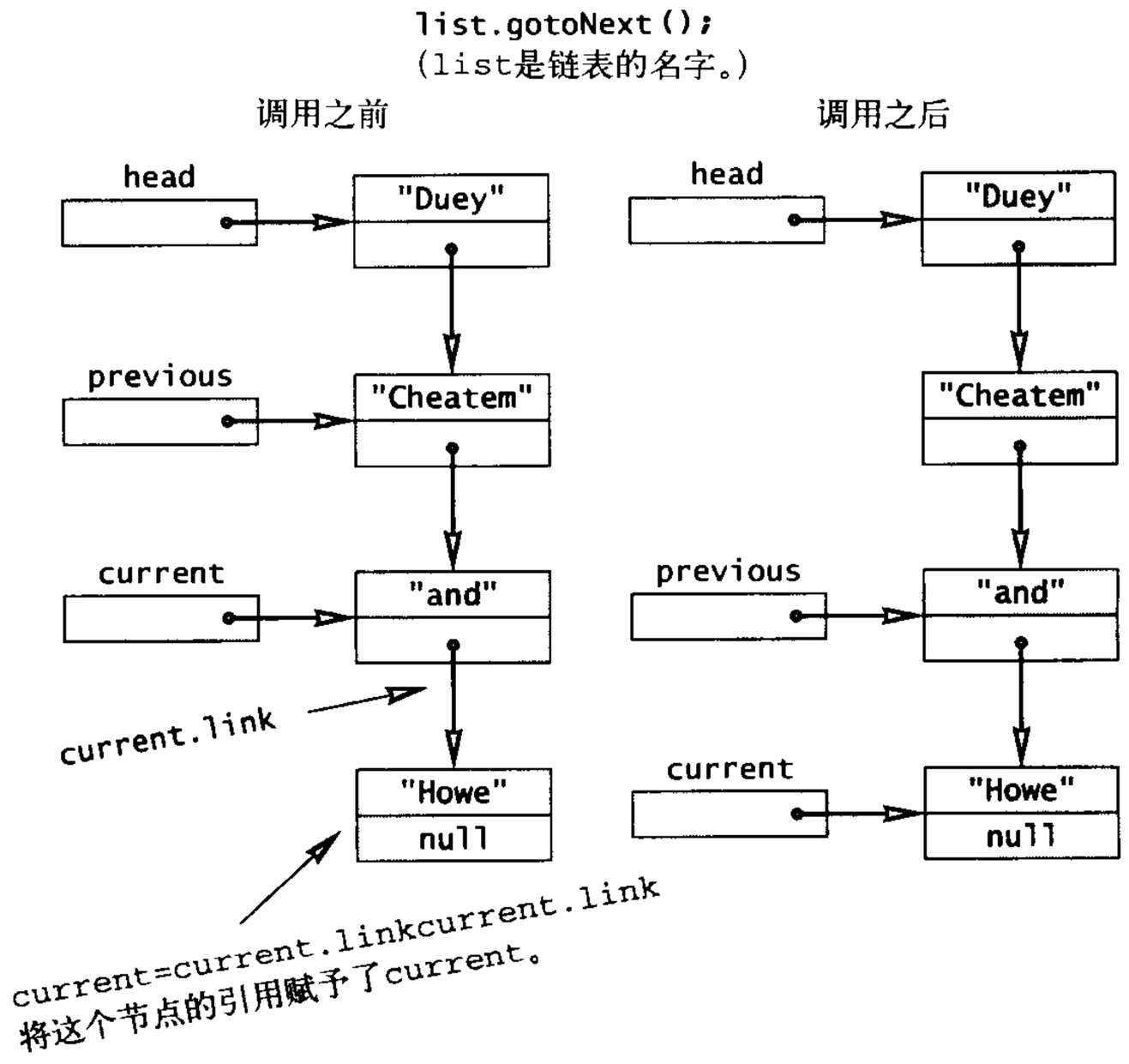


图3-12　goToNext

moreToIterate就返回true。在大多数情况下，这种方式显然都没有问题，但你可能会感到奇怪，current中包含了对最后一个节点的引用时，为什么方法还要返回ture。current包含了对最后一个节点的引用时，程序并不“知道”current在最后一个节点。在再一次调用goToNext之前，它无法辨别current是否在最后一个节点上。研究一下图3-12或者goToNext的定义，就会发现，如果在迭代器中包含了对最后一个节点的引用时，程序调用了goToNext，current就会被设置为null，表示整个链表都已经被迭代完了。因此，当current等于null时，moreToIterate就会返回false。

现在，链表有了一个迭代器，代码也就有了一种引用链表中任意节点的方式。实例变量current可以装载对任意一个节点的引用（一个指向任意节点的“箭头”），这个节点被称为**位于迭代器**（at the iterator）的节点。方法insertAfterIterator会在位于迭代器的节点（current）之后插入一个新节点。图3-13对这个过程进行了说明。方法deleteCurrentNode删除了位于迭代器的节点。图3-14对这个过程进行了说明。

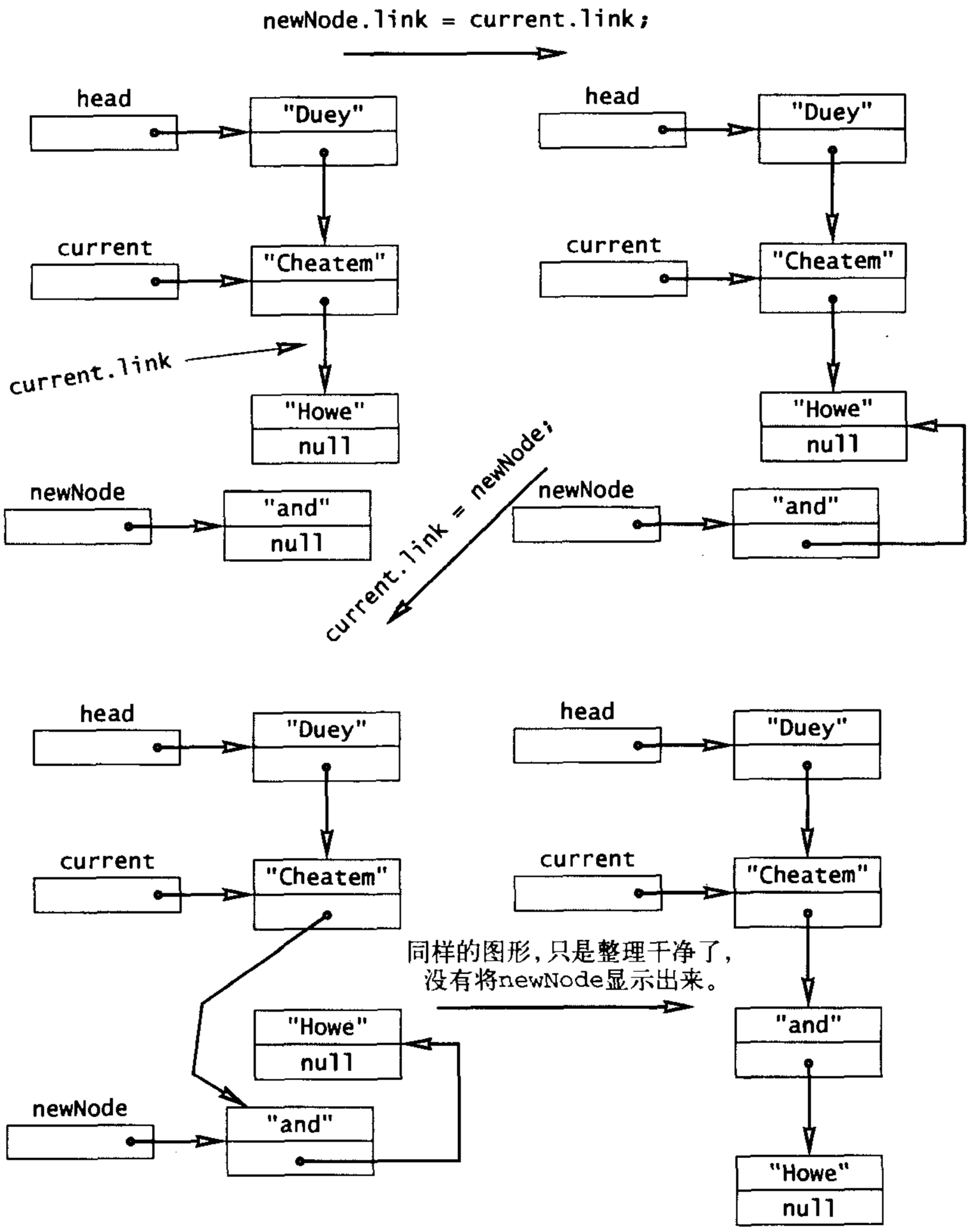


图3-13 添加一个节点

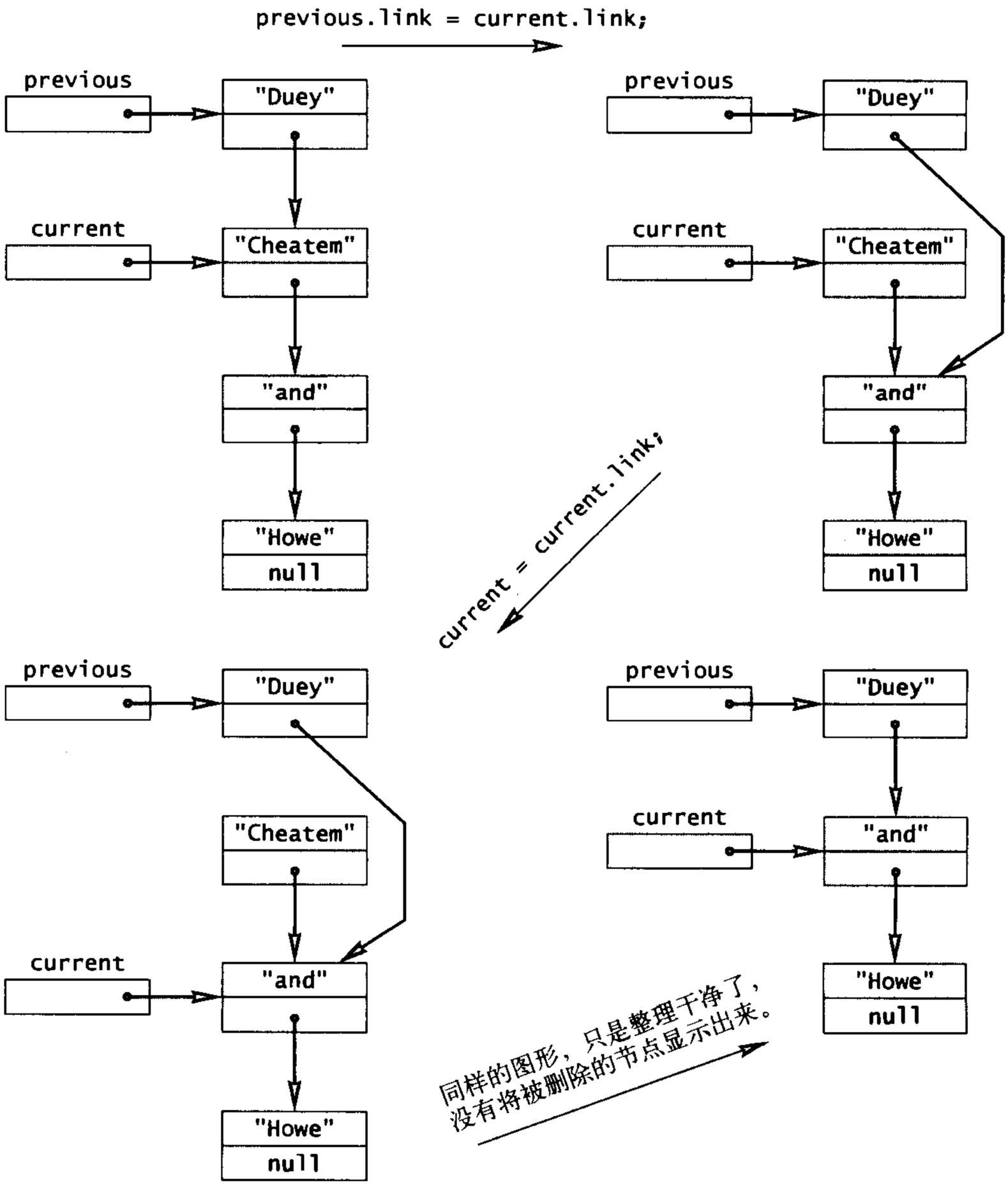


图3-14　删除一个节点

（图3-11中）StringLinkedListWithIterator类中的其他方法都相当简单，请自行阅读它们的定义，并弄清楚它们是怎样工作的。

常见问题：被删除的节点怎么样了？

代码从链表中删除一个节点（如图3-14所示）时，它删除的是链表中对那个节点的引用。对链表来说，那个节点不再位于链表中了。但是，没有任何命令来销毁这个节点，因此它一定还在计算机内存中的什么地方。如果没有其他对被删除节点的引用，它所占据的存储空间就应该可以用于其他用途。在很多编程语言中，程序员都必须记录被删节点这样的条目，并给出明确的命令将其内存回收以作它用。这个过程被称为**垃圾回收**（garbage collecting）。在Java中，这项工作是自动完成的，或者按照通常的表述方式来说，Java具有**自动垃圾回收**（automatic garbage collection）功能。

记住：链表可以作为其自身的迭代器

对图3-11中的链表类来说，我们说实例变量current是链表的迭代器。作为非正式的说法，这样表述没问题，但通常都希望链表的迭代器是某种类型的对象。如果希望迭代器是一个对象，就可以将同一个链表对象既作为链表，又作为迭代器使用。

编程提示：内部和外部迭代器

（图3-11中的）StringLinkedListWithIterator类使用了一个（名为current的）ListNode类型的实例变量作为迭代器，以便一个节点接一个节点地遍历链表。以这种方式在链表类内部定义的迭代器被称为**内部迭代器**（internal iterator）。

如果用方法copyToArray将链表中的值写到一个数组中去，就可以用一个int类型的变量作为数组的迭代器（如果链表不发生变化的话，也就可以作为链表的迭代器了）。int变量装载了数组的一个索引，就指定了数组的一个元素（也就指定了链表中的一个数据条目）。如果int变量名为position，数组名为a，那么迭代器position指定的就是元素a[position]。要移动到下一个条目上去，只要（比如，通过position++）将position的值加1就行了。在链表之外定义的迭代器，比如int变量position，就被称为（链表或数组的）**外部迭代器**（external iterator）。注意，重要的不是数组在链表之外，而是作为迭代器的int变量position，位于链表之外。为了更好地理解这一点，注意，int变量position也是数组的外部迭代器。可以定义直接对链表工作，而不是对装有链表数据的数组工作的外部迭代器。但是，这种技术要稍微复杂一些，本书不对其进行讨论。

自测题

16. 什么是内部类？
17. 为什么图3-9内部类ListNode的定义中没有图3-4的ListNode类定义中的访问和设置方法getLink、setLink、getData和setData？
18. 什么是一个条目集合（比如数组或链表）的迭代器？

3.2.5 对链表的异常处理

在阅读本小节之前，需要学习第1章关于异常处理的内容。

考虑一下图3-11中的StringLinkedListWithIterator类。我们为那个类定义了一些方法，这样，每当出什么问题的时候，方法都会向屏幕发送一条错误消息，并结束程序。但是，发生非正常情况时，不一定都要结束程序。如果希望程序员能在这些非正常情况下采取动作，那么，让程序抛出一个异常，并由程序员来决定如何对这个情况进行处理，这样做会更有意义。程序员仍然可以决定结束程序，但也可以选择作一些其他操作。例如，可以按下列形式重新编写方法goToNext：

```
public void goToNext() throws LinkedListException
{
   if(current != null)
   {
      previous = current;
      current = current.link;
   }
```

```
    else if (head != null)
       throw new LinkedListException("Iterated too many times" +
                          " or uninitialized iteration.");
    else
       throw new LinkedListException(
                          "Iterating with an empty list." )
 }
```

在这个版本中，用一个抛出异常的分支取代了每个会结束程序的分支。如图3-15所示，异常类LinkedListException可以是一个常规异常类。

```
public class LinkedListException extends Exception
{
    public LinkedListException( )
    {
        super("Linked List Exception");
    }

    public LinkedListException(String message)
    {
        super(message);
    }
}
```

图3-15　LinkedListException类

假设抛出异常的StringLinkedListWithIterator版本（就像goToNext所示的那样）被称为StringLinkedListWithIterator2。使用这个修正过的StringLinkedListWithIterator2类的程序员可以将抛出的异常用于很多不同的目的。一种可能是用这个异常来查看链表的结尾。例如，下列代码从链表中删除了所有包含指定BadString的节点：

```
StringLinkedListWithIterator2 list =
                new StringLinkedListWithIterator2() ;
String BadString;
<一些用来填充链表，以及设置BadString的代码>
list.resetIterati on() ;
try
{
     while(list.Iength () >= 0)
     {
            if (BadString.equals(list.getDataAtCurrent()))
                list.deleteCurrentNode() ;
            else
                list.goToNext() ;
     }
}
catch(LinkedListException e)
{
      if(e.getMessage(),equals ("Iterating with an empty list."))
      {//This should never happen, but the catch block is compulsory.
            System.out.println("Fatal Error. ") ;
            System.exit (0) ;
      }
```

```
}
System.out.println ("List cleaned of bad strings.") ;
```

这种用异常来测试链表是否结束的方法开始看起来可能有些奇怪，但Java在查看二进制文件结尾的时候，也需要类似的东西。所以在Java编程中确实有对异常的类似用法，当然，LinkedListException类还有很多其他用途。

后面的自测题请重新编写StringLinkedListWithIterator中的其他方法，使其在非正常或错误情况下抛出异常。

快速参考：Java的Iterator接口

Java有一个名为Iterator的接口，用来说明Java希望迭代器有怎样的行为。接口位于java.util包中（因此，需要导入这个包）。我们的迭代器并不满足这个接口，但是，要定义一些使用这些迭代器，且满足这个接口的类是很简单的。Iterator接口使用了异常处理，但并不难理解。在自测题23中，要求定义一个满足Iterator接口的链表类。（附录G给出了Iterator接口。）

自测题

19. 重新定义（图3-11）StringLinkedListWithIterator中的方法getDataAtCurrent，使其在发生一些不正常情况的时候（即，在图3-11中的定义会结束程序的时候），抛出一个异常而不是结束程序。
20. 重新定义（图3-11）StringLinkedListWithIterator中的方法resetDataAtCurrent，使其在发生一些不正常情况的时候（即，在图3-11中的定义会结束程序的时候），抛出一个异常而不是结束程序。
21. 重新定义（图3-11）StringLinkedListWithIterator中的方法insertNodeAfterCurrent，使其在发生一些不正常情况的时候（即，在图3-11中的定义会结束程序的时候），抛出一个异常而不是结束程序。
22. 重新定义（图3-11）StringLinkedListWithIterator中的方法deleteCurrentNode，使其在发生一些不正常情况的时候（即，在图3-11中的定义会结束程序的时候），抛出一个异常而不是结束程序。
23. （图3-11中的）StringLinkedListWithIterator类是其自身的迭代器，但它并没有完全实现Java的Iterator接口。（如果仔细地遵循这个问题中的指示，就不需要知道接口是什么东西了，尽管知道接口是什么可能会让人感觉更舒服一些。《Java程序设计与问题解决：基础篇》第7章的一个选读小节对接口进行了介绍。）如这个问题的第二段代码所述，Iterator接口需要next、remove和hasNext方法。重新定义StringLinkedListWithIterator类，使其实现Java的Iterator接口。要实现这一点，就得以下列代码作为类定义的开始：

```
import java.util.*;
public class StringLinkedListWithIterator2 implements Iterator
{
    private ListNode head;
    private ListNode current;
    private ListNode previous;    //follows current
    private ListNode towBack;    //follows previous
    private boolean removeSinceNext;    //true if removed has been
                    //called since the last invocation of next.
```

```
                    //Also true if next has not been called at all.
        public StringLinkedListWithIterator2()
        {
              head = null;
              current = null;
              previous = null;
              twoBack = null;
              removeSinceNext = true;
    }
```

除了添加了下列代码中给出的方法定义，并删除了冗余的方法deleteCurrentNode、deleteHeadNode、goToNext和moreToIterate之外，定义的其余部分与图3-11相同。对方法resetIterator也进行了一个小小的修改以重新设置twoBack。要添加的方法定义如下所示：

```
    /**
       Returns the next element (String) in the list.
       Throws a NoSuchElementException if there is
       no next element to return.
    */
    public Object next() throws NoSuchElementException
    /**
       Removes the last element that was returned by next.
       Throws an IllegalStateException if the next method has
       not yet been called or if the remove method has already
       been called after the last call to the next method.
    */
       public void remove() throws IllegalStateException

    /**
       Returns true if there is at least one more element
       for next to return. Otherwise, returns false.
       */
    public boolean hasNext()
```

提示：

(1) 尽管这个练习的细节听起来很复杂，但这个练习是相当简单的。用已有的方法来实现要添加的这三个方法定义是非常简单的。

(2) 注意，方法hasNext和图3-11中的方法moreToIterate不是完全相同的。

(3) 这里提到的异常类都是预定义的，不需要定义它们。这些所有的异常类都不需要捕捉、或在throws子句中声明。(Iterator接口说明，如果不支持remove方法，该方法就会抛出一个UnsupportedOperationException。但是，这里的方法remove永远也不需要抛出这个异常。)

3.2.6　链表的变体

有时，拥有对链表中最后一个节点的引用是很方便的。最后一个节点通常被称为列表的**尾巴**（tail），因此链表的定义可能会以下列形式开始。

```
public class StringLinkedListWithTail
{
      private ListNode head;
      private ListNode tail;
      private ListNode current;
```

```
        private ListNode previous;
```

必须对构造器和方法进行修改以容纳新的引用tail，但实现的细节是很普通的。

可以拥有包含任何类型数据的链表。只要用期望的数据类型取代节点类定义中（及其他相应位置上）的String类型就可以了。甚至可以用Object类型取代节点定义中（及其他相应位置上）的String类型，以拥有一个包含不同类型对象的链表，如下列所示。

```
private class ListNode
{
        private Object data;
        private ListNode link;
        public ListNode()
        {
                link = null;
                data = null;
        }
        public ListNode(Object newData, ListNode linkValue)
        {
                data = newData;
                link = linkValue;
        }
}
```

任意类类型的对象也都是Object类型的，所以，可以在具有此类型节点的链表中存储任意类型的对象。

普通的链表只允许沿着一个方向（顺着链表）向链表下方移动。**双向链表**（doubly linked list）中，一个链接拥有指向下一个节点的引用，一个链接拥有指向前一个节点的引用。用图形来表示的话，双向列表看起来如下所示。



双向链表的节点类可以以下列形式开始。

```
private class ListNode
{
        private Object data;
        private ListNode next;
        private ListNode previous;
```

为了适应附加的那个链接，需要（根据单向链表的定义）对双向链表类中的构造器和一些方法的定义进行修改。

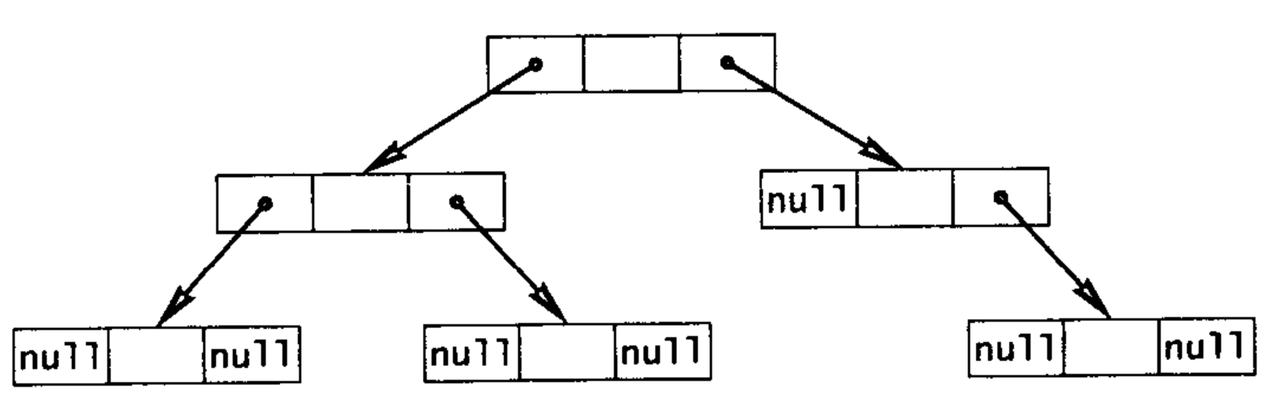
3.2.7 其他链式数据结构

作为链式数据结构的例子，我们对一些链表进行了讨论。对数据结构的研究是个很大的话题，关于这个话题有很多很好的书。要在本书中介绍编程的所有方面，无法对每个话题都进行非常详尽的介绍。因此，我们不会提供更多链式数据结构的细节介绍，只会对几个比较重要的链式数据结构进行一个非正式的介绍。

栈（stack）不一定是链式数据结构，但可以作为链表来实现。栈是一种按照与插入条目相反的顺序来移出条目的数据结构。因此，如果向一个栈中插入"one"，然后插入"two"，然后再插入"three"，再来移出它们，它们会按照"three"，然后"two"，最后"one"的顺序从

栈中弹出来。第4章对栈进行了更详细的论述。仅在列表头部进行插入和移出操作的链表（比如图3-5中的那个链表）实际上就是一个栈。

树（tree）是一种很常见也很有用的数据结构。在树中，每个节点都可以指向多个其他节点。最常见的树就是**二叉树**（binary tree），其中每个节点最多可以链接到两个其他节点上去。二叉树的节点类型与双向链表一样，但它们的用法很不相同。二叉树可以用下列图形来表示。



在二叉树图形中没有引用能沿着树形结构反向返回，这并不是个意外的失误。树中的节点一定不能构成循环，如这个图形所示。

二叉树顶端的节点（就是这个图形中所示的顶点）被称为**根节点**（root node），正如有一个对链表头（开始）节点的引用一样，通常也有一个引用指向这个根节点。沿着一系列适当的链接可以从根节点抵达每个节点。

3.3　泛型

generic 形容词。1.与整个组相关或对整个组的描述……

——《美国传统英语词典》，（第三版）

从版本5.0开始，Java允许在定义中包含类型参数。这种定义被称为**泛型**（generic）。本节对Java泛型进行了简要的介绍。用泛型编程可能是非常微妙、充满陷阱的。这里只是一个介绍，要想用泛型进行真正的编程工作，还需要参考更高级的教材。

泛型的基本概念

类和方法都可以有类型参数。然后就可以插入任意类类型来取代类型参数，生成一个特定的类类型或方法。例如，图3-16显示了一个带有**类型参数**（type parameter）T的非常简单的类定义。可以用任意非关键字标识符来表示类型参数，不一定非要用T来表示。但是，根据惯例，类型参数都是以大写字母开头的，而且好像还有用单个字母表示类型参数的传统。因为只能插入类类型来替换类型参数，所以，以大写字母开头是有道理的。使用单个字母的传统并不是强制性的。

带有类型参数的类定义是存储在一个文件中的，编译方法与所有其他类一样。例如，可以将图3-16中显示的参数化类存储在一个名为Sample.java的文件中。一旦将参数化类编译好了，就可以像所有其他类那样使用了，只是在代码中使用它的时候，必须指定一个类类型作为类型参数。例如，可以按下列方式使用图3-16中的Sample类：

```
Sample<String> o1 = new Sample<String>();
o1.setData("Hello");
```

```
Sample<Species> o2 = new Sample<Species>();
Species s = new Species();
<一些用来为对象s设置数据的代码>
o2.setData(s);
```

```
public class Sample<T>
{
    private T data;

    public void setData(T newValue)
    {
        data = newValue;
    }

    public T getData()
    {
        return data;
    }
}
```

图3-16　带有类型参数的类定义

可以像《Java程序设计与问题解决：基础篇》第4章那样定义Species类，但这些细节是无关紧要的，可以使用定义的任何类。

注意，尖括号既用来表示类型参数，也用来表示插入替换类型参数的类类型。

记住：不能插入基本类型作为类型参数

插入作为类型参数的类型必须是一个类类型。不能是int、double或char这样的基本类型。

□ Java提示：用选项-Xlint进行编译

使用类型参数时会遇到很多陷阱。如果用选项-Xlint进行编译，就能收到更多与代码中存在的任何问题或潜在问题有关的提示性诊断信息。例如，图3-16中的Sample类应该按下列形式编译：

```
javac -source 1.5 -Xlint Sample.java
```

如果在用IDE编译程序，查看一下文档，看看应该如何设置编译器选项。（对TextPad环境，可以在Configure菜单的Preferences方框中设置编译器选项。）

用-Xlink选项进行编译时，会比不用它时得到的警告多一些。警告并不是错误，如果编译器只给出警告而没有给出错误信息，那么类已经编译好，并可以使用了。但是，在大多数情况下，都要确定你理解了警告信息，并确信警告不是一个问题，否则，就要去修改代码以消除警告。对本书中的某些程序来说，可能会得到一条警告“no definition of serialVersionUID”（“没有serialVersionUID的定义”）。对这条警告的讨论超出了本书的范围，但可以安全地忽略这条警告。

快速参考：带有类型参数的类定义

可以定义带有类型参数的类。类型参数包含在类定义头部类名后面的尖括号中。可以用任意非关键字标识符来表示类型参数，但按照惯例，类型参数是以大写字母开头的。在类定义中，类型参数可以像任意其他类型一样使用。（对类型参数的使用范围有一些限制，名为“不是在任意可以使用类型名的地方都可以使用类型参数的”的易犯错误部分对此进行了讨论。）例子请参见图3-16。

参数化类的用法与任意其他类相似，只是需要插入一个指定的类类型作为类型参数。类名后面的尖括号中给出了这个类类型。

举例：

```
Sample<String> o1 = new Sample<String>();
o1.setData("Hello");
```

编程示例：泛型链表

图3-17重新编写了图3-9中的链表定义。重写版本用类型参数E取代String类型，作为链表中存储数据的类型。

```
import java.util.*;

public class LinkedList <E>
{
    private ListNode head;

    public LinkedList()
    {
        head = null;
    }

    public int length()
    {
        int count = 0;
        ListNode position = head;
        while (position != null)
        {
            count++;
            position = position.link;
        }
        return count;
    }

    /**
     The added node will be the first node in the list.
    */
    public void addANodeToStart(E addData)
    {
        head = new ListNode(addData, head);
    }

    public void deleteHeadNode()
    {
        if (head != null)
            head = head.link;
        else
        {
            System.out.println("Deleting from an empty list.");
            System.exit(0);
```

构造器头部没有在尖括号中包含类型参数。

图3-17　泛型链表类

```
    }
}

public boolean onList(E target)
{
    return (Find(target) != null);
}

/**
 Finds the first node containing the target data,
 and returns a reference to that node. If target is not
 in the list, null is returned.
*/
private ListNode Find(E target)
{
    ListNode position;
    position = head;
    E dataAtPosition;
    while (position != null)
    {
        dataAtPosition = position.data;
        if (dataAtPosition.equals(target))
            return position;
        position = position.link;
    }
    //target was not found.
    return null;
}

public void showList()
{
    ListNode position;
    position = head;
    ListNode dataAtPosition;
    while (position != null)
    {
        System.out.println(position.data);
        position = position.link;
    }
}

public Vector <E> vectorCopy()
{
    Vector <E> v = new Vector <E> (length());

    ListNode position;
    position = head;

    while (position != null)
    {
        v.addElement(position.data);
        position = position.link;
```

图3-17　(续)

```
        }

        return v;
    }


    private class ListNode                    注意，内部类头部没有类型参数，但
    {                                         可以在内部类定义中使用类型参数。
        private E data;
        private ListNode link;

        public ListNode()
        {
            link = null;
            data = null;
        }

        public ListNode(E newData, ListNode linkValue)
        {
            data = newData;
            link = linkValue;
        }
    }

}
```

图3-17　(续)

注意，构造器头部没有包含类型参数E。这与大多数人的直觉相反，但它就是这样实现的。构造器可以用E这样的类型参数作为构造器参数的类型，但构造器的头部不会在尖括号中包含<E>这样的类型参数。

图3-9的链表类中有一个名为arrayCopy的方法，这个方法可以生成一个数组，数组中包含了与链表相同的数据。在图3-17中，用返回向量的方法vectorCopy取代返回数组方法arrayCopy并不是一个意外。为了使arrayCopy适用于图3-17中的泛型链表，对其进行的直接修改可能如下所示：

```
public E[] arrayCopy()
{
   E[] a = new E[length()]; //Illegal
   ListNode position;
   position = head;
   int i = 0;
   while(position != null)
   {
      a[i] = position.data;
      i++;
      position = position.link;
   }
   return a;
}
```

如注释所述，这个定义是非法的。不允许使用表达式new E[length()]。在某些特定的情况下不能包含类型参数，不幸的是，这就是其中的一种情况。所以方法arrayCopy就不能这样实现了。

图3-18包含的是一个使用了参数化链表的简单实例。

```
import java.util.*;

public class GenericDemo
{
   public static void main(String[] args)
   {
      LinkedList<String> stringList = new LinkedList<String>();
      stringList.addANodeToStart("Hello");
      stringList.addANodeToStart("Good-bye");
      stringList.showList();

      LinkedList<Integer> numberList =
                                 new LinkedList<Integer>();

      int i;
      for (i = 0; i < 10; i++)
          numberList.addANodeToStart(i);
      Vector<Integer> v = numberList.vectorCopy();
      int position;
      int vectorSize = v.size();
      for (position = 0; position < vectorSize; position++)
          System.out.println(v.elementAt(position));
   }
}
```

只是因为有了自动装箱功能，这段代码才能工作。

屏幕对话示例

```
Good-bye
Hello
9
8
7
6
5
4
3
2
1
0
```

图3-18　使用泛型链表

△ 易犯错误：泛型构造器名中没有类型参数

参数化类定义的类名中有一个附加的类型参数——例如，图3-17中的LinkedList<E>。这可能会让人误以为需要在构造器定义的头部使用类型参数，但是，在构造器定义的头部是不需要重现类型参数说明<E>的。例如，应该用

```
public LinkedList()
```

而不用

```
public LinkedList<E>()
```

完整的例子请参见图3-17。

构造器可以用E这样的类型参数作为构造器的参数类型，但是，构造器头部不会在尖括号中包含类型参数，比如<E>。 △

△ 易犯错误：不是在任意可以使用类型名的地方都可以使用类型参数的

在参数化类定义的定义内部，有些地方是可以使用类型名，但不能使用类型参数的。在用new创建新对象的简单表达式中是不能使用类型参数的。例如，在带有类型参数T的参数化类定义的定义内部，下列代码都是非法的：

```
T object = new T();     //The first T is legal,
                        //the second one is illegal.
T[] a = new T[10];      //The first T is legal,
                        //the second one is illegal.
```

△

常见问题：一个类定义中可以有多个类型参数吗？

可以，可以像单个类型参数那样，在尖括号中列出多个类型参数，但要用逗号分隔。但是，很少需要使用多个类型参数，不在本书中对其进行讨论。

自测题

24. 重新编写图3-4中的ListNode类定义，使其成为一个带有类型参数，而不是String类型的参数化类。
25. 重新编写图3-5中的StringLinkedList类定义，使其成为一个带有类型参数，而不是String类型的参数化类。类名StringLinkedList不合适了，所以，将类名改成LinkedList2。

小　结

- 可以把向量当作长度可变的数组。
- 在Java的最新版本中，Vector类被定义为有一个基类型参数的向量。
- 链表是一种由名为节点的对象组成的数据结构，每个节点都可以包含数据，而且每个节点还包含对另一个节点的引用，这样，节点就可以链接在一起形成一个列表了。
- 可以将节点类作为链表类的内部类，使链表（或其他链式数据结构）成为自包含的。
- 可以用迭代器对一个集合中的元素进行遍历，比如一个链表中的元素。
- 可以定义一个带有类型参数的类。

✓自测题答案

1. `v.addElement("Hello");`
2. `v.setElement("Go", 5);`
3. 不行。setElementAt的索引必须大于或等于0，且小于向量的大小。因此，你可以替换任何现存的元素，但不能将元素放到任何更高的索引位置上去。这种情况与数组的情况不同。如果一个数组被部分填充到了索引10，那么只要数组有那么大，就可以在索引位置20添加元素。对向量来说，无法在

超出最后一个已用索引位置的地方添加元素。

4. 不行。insertElementAt的索引必须大于或等于0，且小于或等于向量的大小。因此，可以在当前装载了元素的任意索引位置上插入，但除了第一个可用的未填充位置之外，不能在任何更高的索引位置上插入元素。注意，如果向量被命名为v，就可以用insertElementAt在任意一个小于或等于v.size()的索引位置上插入元素，但是，对setElementAt来说，索引位置必须小于或等于v.size()-1。

5. 可以。向量中可以包含多于20个的元素。作为构造器实参使用的数字20只是说明了向量的初始内存分配。需要的时候会自动分配更多的内存。

6.
```
int index;
for(index = 0; index < v.size(); index++)
   System.out.println(v.elementAt(index));
```

7.
```
import java.util.*;

/**
Class for sorting a vector of Strings lexicographically
(approximately alphabetically).
*/
public class StringSelectionSort
{
   /**
   Sorts the vector a so that a.elementAt(0), a. elementAt(1),...,
    a.elementAt(a.size()-1) are in lexicographic order.
   */
   public static void sort(Vector<String>a)
   {
      int index, indexOfNextSmallest;
      for (index = 0; index < a.size()-1; index++)
      {//Place the correct value in position index:
          indexOfNextSmallest =
                           indexOfSmallest(index, a);
          interchange(index, indexOfNextSmallest, a);
          //a.elementAt(0), a.elementAt(1), ...,
          //a.elementAt(index) are sorted. The rest of
          //the elements are in the remaining positions.
      }
 }
/**
 Precondition: i and j are legal indices for the vector a.
 Postcondition: The values of a. elementAt(i) and
 a.elementAt(j) have been interchanged.
*/
Private static void interchange(
                           int i, int j, Vector<String> a)
{
    String temp;
    temp = a. elementAt(i);
    a. setElementAt(a. elementAt(j), i);
    a. setElementAt(temp, j);
 }

 /**
 Returns the index of the Texicographically first value among
 a. elementAt(startIndex), a. elementAt(startIndex+1), ...,
```

```
 a.elementAt(a. size()-1)
*/
private static int indexOfSmallest(
                                int startIndex, Vector<String> a)
{
   String min = a.elementAt (startIndex);
   int indexOfMin = startIndex;
   int index;
   for (index = startIndex + 1; index <a.size(); index++>
   if((a.elementAt(index)).compareTo(min)<0)
   {
      min = a.elementAt(index);
      indexOfMin = index;
   }
   return indexOfMin;
  }
}
```

8. Object。

9. 不行，不能使用基类型为int的向量。但是，可以使用基类型为Integer的向量，由于具有自动装箱和拆箱功能，可以把它当作基类型为int的向量使用。

10. 方法调用v.size()返回的是向量v中的元素个数。方法调用v.capacity()返回的是向量当前分配了内存的元素的个数。

11. 索引号从10到19的新元素的值为null。

12. 将最后一个节点的实例变量link设置为null，就可以标识链表的结束了。

13. head的值应该为null。

14.
```
public boolean empty()
{
     return (length() == 0);
}
```

15.
```
C
B
A
```

16. 内部类是在其他类定义中定义的类。

17. 不需要，因为链表类中的方法可以访问节点的实例变量data和link。这种特性是使用内部类的优点之一。

18. 任何一种能以合理的方式一次一个条目地对集合进行遍历的对象都被称为迭代器。我们用“合理的方式”来表示在一个完整的迭代周期中，每个条目都正好被访问一次，且每个条目都可以将其数据读出，如果数据条目允许的话，还可以对数据进行修改。例如，装载了索引值的int变量可以作为数组的迭代器使用。要进入集合（数组）的下一个条目，代码只需要将int变量的值加1就行了。迭代器可以是一个不同于集合对象的对象，或者，也可以将集合对象作为它自己的迭代器使用。

19.
```
public String getDataAtCurrent() throws LinkedListException
{
   if (current != null)
      return(current.data);
   else
       throw new LinkedListException(
            "Getting data when current is not at any node.");
```

```
    }
```

20.
```
public void resetDataAtCurrent(String newData) throws LinkedListException
    {
        if (current != null)
            current.data = newData;
        else
            throw new LinkedListException(
              "Setting data when current is not at any node.");
    }
```

21.
```
/ **
Inserts node with newData after the current
node. The current node is the same after invocation as it is before invocation.
Should not be used with an empty list. Should not
be used when the current node has iterated past the entire list.
* /
public void insertNodeAfterCurrent(String newData)
                                    throws LinkedListException
{
    ListNode newNode = new ListNode();
    newNode.data = newData;
    if (current != null)
    {
        newNode.link = current.link;
        current.link = newNode;
    }
    else if (head != null)
    {
        throw new LinkedListException(
            "Inserting when iterator is past all"
          + " nodes or uninitialized iterator.");
    }
    else
    {
        throw new LinkedListException(
            "Using insertNodeAfterCurrent with empty list.");
    }
}
```

22.
```
/ **
Deletes current node. After the invocation,
the current node is the node after the
deleted node, or null if there is no next node.
* /
public void deleteCurrentNode() throws LinkedListException
{
    if ((current != null) && (previous != null))
    {
        previous.link = current.link;
        current = current.link;
    }
    else 1f( (current != null) && (previous == null))
    {//At head node
        head = current.link;
        current = head;
```

```
        }
        else //current == null
        {
            throw new LinkedListException(
                "Deleting with uninitialized current"
              + " or an empty list.");
        }
    }
```

23. `import java.util.*;`

```
public class StringLinkedListWithIterator2 implements Iterator
{
    private ListNode head;
    private ListNode current;
    private ListNode previous; //follows current
    private ListNode twoBack; //follows previous
    private boolean removeSinceNext;//true if removed has been
                    //called since the last invocation of next.
                //Also true if next has not been called at all.

    public StringLinkedListWithIterator2()
    {
         head = null;
         current = null;
         previous = null;
         twoBack = null;
        removeSinceNext = true;
    }
/ **
  Returns the next element (String) m the list.
  Throws a NoSuchElementException if there is
  no next element to return. You should invoke
  resetIteration before the first invocation of next().
* /
    public Object next()
    {
        if (current != null)
        {
            twoBack = previous;
            previous = current;
            current = current.link;
            removeSinceNext = false;
            return (previous.data);
        }
        else
        {
            throw new NoSuchElementException();
        }
    }
/ **
  Removes the last element that was returned by next.
  Throws an IllegalStateException if the next method has
  not yet been called or if the remove method has already
  been called after the last call to the next method.
* /
    public void remove()
```

```
{
   if ((previous != null) && (twoBack != null) && (!removeSinceNext))
        {//remove node at previous
        twoBack.link = previous, link;
        previous = twoBack;
        removeSinceNext = true;
        //twoBack not updated until next invocation of next()
   }
   else if( (previous != null) && (twoBack == null) && (!removeSinceNext))
        {//previous at head node and current
        //at head node after deletion
            head = current;
        previous = null;
        twoBack = null;
        removeSinceNext = true;
        }
        else
        {
            throw new IllegalStateException();
        }
   }
/**
  Returns true if there is at least one more element
  for next to return. Otherwise, returns false.
*/
   public boolean hasNext()
   {
       return (current != null);
   }

   public void resetIteration()
   {
        current = head;
        previous = null;
        twoBack = null;
        removeSinceNext = true;
   }
```

<除了删除掉的方法之外，定义的其余部分与图3-11一样。Web上源代码中的文件StringLinkedListWithIterator2.java给出了完整的定义。那个类还包含了自测题19到自测题22中的方法。>

24. Web上源代码中的文件ListNode.java给出了完整的定义。

```
public class ListNode<E>
{
    private E data;
    private ListNode<E> link;

    public ListNode()
    {
        link = null;
        data = null;
    }
    public ListNode (E newData, ListNode<E> linkValue)
    {
```

```
        data = newData;
        link = linkValue;
    }
    public void setData(E newData)
    {
        data = newData;
    }

    public E getData()
    {
        return data;
    }
    public void setLink(ListNode<E> newLink)
    {
        link = newLink;
    }
    public ListNode<E> getLink()
    {
        return link;
    }
}
```

25. Web上源代码中的文件LinkedList2.java给出了这个定义。省略了大部分注释以节省空间。

```
public class LinkedList2<E>
{
    private ListNode<E> head;
    public LinkedList2()
    {
        head = null;
    }

    public int length()
    {
        int count =0;
        ListNode<E> position = head;
        while (position != null)
        {
            count++;
            position = position.getLink();
        }
        return count;
    }

    public void addANodeToStart(E addData)
    {
        head = new ListNode<E>(addData, head);
    }

    public void deleteHeadNode()
    {
        if (head != null)
        {
            head = head.getLink();
        }
        else
        {
            System. out. println ("Deleting from an empty list.");
```

```
            System. exit(0);
        }
    }

    public boolean onList(E target)
    {
        return (Find(target) != null);
    }

    private ListNode<E> Find(E target)
    {
        ListNode<E> position;
        position = head;
        E dataAtPosition;
        white (position != null)
        {
            dataAtPosition = position.getData();
            if (dataAtPosition.equals(target))
                return position;
            position = posifion.getLink();
        }
            //target was not found.
            return null;
        }
    public void showList()
    {
        ListNode<E> position;
        position = head;
        while (position != null)
        {
            System.out.println(position.getData());
            position = position.getLink();
        }
    }
}
```

○ 编程项目

1. 实现《Java程序设计与问题解决：基础篇》第6章的编程项目4，只是，在这个练习中，类应该对向量而不是数组进行排序。假设向量中的元素都是String值。(提示：查看自测题7。)
2. 实现《Java程序设计与问题解决：基础篇》第6章的编程项目5，只是，在这个练习中，类应该对向量而不是数组进行排序。假设向量中的元素都是String值。(提示：查看自测题7。)
3. 编写一个程序，将一系列PetRecord从键盘读入一个向量，根据宠物名对向量进行字母排序，最后将向量中的记录写到屏幕上去。确保屏幕输出不会出现的太快，以便用户能够看到这些记录。《Java程序设计与问题解决：基础篇》第5章的图5-20给出了PetRecord类。
4. 编写一个程序，将Species记录从一个文件读入一个向量中去，根据Species名对向量进行字母排序，然后将排好序的记录输出到屏幕和一个文件中去。所有的文件名都由用户提供。确保屏幕输出不会出现的太快，以便用户能够看到这些记录。《Java程序设计与问题解决：基础篇》第4章的图4-19给出了Species类。可以用本书图2-19中的程序创建一个Species记录的文件。(如果还没学过本书第2章关于二进制文件的小节，就从键盘读入Species记录，并只将其输出到屏幕上去。)
5. 编写一个程序，从键盘将一系列PetRecord读入一个向量中去。根据宠物重量对向量中的记录进行排

序。最后将向量中（根据宠物重量排过序）的记录写到屏幕上去，后面跟上轻于5磅的宠物数量及其所占百分比，5到10磅之间的宠物数量及其所占百分比，以及重于10磅的宠物数量及其所占百分比。确保屏幕输出不会出现的太快，以便用户能够看到这些记录。《Java程序设计与问题解决：基础篇》第5章的图5-20给出了PetRecord类。（如果学过本书第2章的部分内容，就从一个文件中获取输入，并将输出发送到另一个文件中去。如果学习过二进制文件，就使用二进制文件，否则，就使用文本文件。文件名是从用户那儿获得的。）

6. 定义图3-9中StringLinkedListSelfcontained的一个变体，以存储Species类型而不是String类型的对象。编写一个程序，用那个链表类创建一个Species对象的链表，请用户输入一个Species名，然后对链表进行搜索，并根据名字是否位于列表中，显示下面的某条消息：

Species *species_Name* is one of the
number_of_Species_names_on_list species on the list.
The data on *Species_Name* are as follows:
Output_of_Species_Name_Data

或者

There are *number_of_Species_names_on_list* species on
the list, but *Species_Name* is not one of them.

允许用户输入更多的Species名，直到用户表示想结束程序为止。当然，应该分别插入适当的Species名称及列表中名字的数量来取代*Species_Name*和*number_of_Species_names_on_list*。

《Java程序设计与问题解决：基础篇》中第4章的图4-19给出了Species类。

如果学过本书第2章的部分内容，就从一个文件中读入Species记录。如果没有学过第2章的任何内容，就从键盘读入Species记录。

7. 定义图3-10中StringLinkedListSelfcontained的一个变体，用来存储Employee类型的对象而不是String类型的对象。编写一个程序，用链表类来创建一个Employee链表，请用户输入一个Employee的社会安全号码，然后搜索链表，如果有这样一个记录的话，就显示具有那个社会安全号码的雇员记录，否则，就显示一条消息说明无法找到所请求的社会安全号码。

用户可以输入更多的社会安全号码，直到用户表示想结束程序为止。

Employee类是作为本书第1章编程项目7的一部分定义的。如果还没有完成那个项目，就需要按照那里的描述来定义Employee类。

如果学过本书第2章的部分内容，就从一个文件中读入Employee记录。如果没有学过第2章中的任何内容，就从键盘读入Employee记录。

8. 为一个双向链表编写一个参数化类的定义，这个双向链表有一个参数用来说明存储在节点中的数据的类型。将节点类作为一个内部类。选择要定义的方法也是这个任务的一部分。同样，编写一个方法对你的类定义进行完整的测试。

第4章

递　　归

世界上有两种人：将世界分成两种人的人和没有将世界分成两种人的人。

——佚名

很多人都相信，永远都不能用任何东西自己来定义自己。他们说，这会产生一种循环。但在有些情况下，用方法自己来定义自己不仅是可能的，而且是很有用的。（尽管需要对此进行一些解释，但是）如果做得正确的话，甚至都不会产生循环。

在某种意义上，Java允许用方法自己定义自己。更精确地来说，一个Java方法定义中可以包含对正在定义的这个方法的调用。当方法定义中包含了对其自身的调用时，就称这个方法是**递归的**（recursive），这些递归的方法被统称为**递归**（recursion）。本章涵盖了递归的内容。

目标

- 熟悉递归的概念。
- 学习将递归作为一种编程工具使用。
- 熟悉作为递归实例的二分搜索算法。
- 熟悉作为递归实例的归并排序算法。

预备知识

本章的大部分内容都只需要《Java程序设计与问题解决：基础篇》第1章~第5章的知识。只有本章最后两节要求掌握一些附加知识。这两节需要了解该书第6章中数组的内容。另外，自测题7中用到了异常处理，但如果你还没有学习过异常处理的内容（本书第1章），可以跳过那个问题。

4.1　递归基础

这条语句是错的。

——对一条古老悖论的意译

通常，设计一个算法时都会在一个或多个子任务中用到同样的算法。例如，下面是一个在电话簿中搜索名字的算法概述：打开电话簿翻到中间一页。如果名字在那一页上，任务就完成了。如果按照字母序，名字在那页之前，就去搜索电话簿的前半部分。如果按照字母序，名字在那页之后，就去搜索电话簿的后半部分。搜索半本电话簿的任务就是原来搜索整本电话簿任务的一个较小的版本。

正如将看到的那样，可以用一个递归的Java方法来实现这类算法。更一般地说，只要一

个算法中有一个子任务是整个算法任务的较小版本，就可以用Java中的递归方法来实现这个算法。当然，必须以正确的方式来实现，否则Java代码就会产生错误，而本章的目标就是展示正确的方法。以一个说明了Java中递归概念的简单实例作为开始。

快速参考：递归调用

如果一个方法定义中包含了对这个被定义方法的调用，那个调用就被称为一个**递归调用**（recursive call或recursive invocation）。

案例分析：把数字用单词来表示

在这个案例分析中，要编写一个方法定义，将单个整数作为实参，并用单词将这个整数中的数字写出。例如，如果实参为整数223，那么方法就应该输出

```
two two three
```

方法的头部应该为

```
/**
 Precondition: number >= 0
 Action: The digits in number are written out in words.
*/
public static void inWords(int number)
```

如果这个数中只包含单个数字，就可以用一个很长的switch语句来决定应该为这个给定的数字使用哪个单词。图4-1中的方法digitWord就使用了这样一条switch语句，因此，digitWord(0)会返回“zero”，digitWord(1)会返回“one”，以此类推。

```
import java.util.*;

public class RecursionDemo
{
    public static void main(String[] args)
    {
        System.out.println("Enter an integer:");
        Scanner keyboard = new Scanner(System.in);;
        int number = keyboard.nextInt();
        System.out.println("The digits in that number are:");
        inWords(number);
        System.out.println();

        System.out.println("If you add ten to that number, ");
        System.out.println("the digits in the new number are:");
        number = number + 10;
        inWords(number);
        System.out.println();
    }


    /**
     Precondition: number >= 0
     Action: The digits in number are written out in words.
    */
```

图4-1　递归演示

```
public static void inWords(int number)
{
    if (number < 10)
        System.out.print(digitWord(number) + " ");
    else //number has two or more digits
    {
        inWords(number/10);   <-- 递归调用
        System.out.print(digitWord(number%10) + " ");
    }
}

/**
 Precondition: 0 <= digit <= 9
 Returns the word for the argument digit.
*/
private static String digitWord(int digit)
{
    String result = null;
    switch (digit)
    {
        case 0:
            result = "zero";
            break;
        case 1:
            result = "one";
            break;
        case 2:
            result = "two";
            break;
        case 3:
            result = "three";
            break;
        case 4:
            result = "four";
            break;
        case 5:
            result = "five";
            break;
        case 6:
            result = "six";
            break;
        case 7:
            result = "seven";
            break;
        case 8:
            result = "eight";
            break;
        case 9:
            result = "nine";
            break;
        default:
            System.out.println("Fatal Error.");
```

图4-1 （续）

```
                System.exit(0);
                break;
            }
            return result;
        }
    }
```

屏幕对话示例

```
Enter an integer:
987
The digits in that number are:
nine eight seven
If you add ten to that number,
the digits in the new number are:
nine nine seven
```

图4-1　(续)

现在我们来考虑一个包含多个数字的数。有很多不同的方法可以将这项任务划分成若干子任务。有些人会采用递归的解决方法，有些人则不会。一种好的解决方法是将这项任务分解成两个子任务，这样就可以立即解决其中一个子任务，而对另一个子任务使用递归了，这种方法如下所示：

将除了最后一个数字之外的其他数字都作为单词输出。

输出对应于最后一个数字的单词。

可以通过对方法digitWord的调用来完成第二项任务。第一项子任务是原始问题的一个较小版本。实际上，它和我们开始提出的那个问题是完全一样的，只是问题中的数要小一些。这就意味着，可以通过对正在定义的这个方法的递归调用来实现第一个子任务。这样，就可以写出下列用于方法inWords的算法概述了。

inWords(number)的算法

```
inWords(删除了最后一个数字的number);
System.out.print(digitWord(number的最后一个数字) + " ");
```

现在来考虑一个带有多个数字的数，比如534。要将534划分成两个数53和4。可以通过被10除的整数除法来完成这项任务。例如534/10为53，而534%10为4。因此，可以将算法精炼成下列Java代码：

```
inWords(number/10);
System.out.print(digitWord(number%10) + " ");
```

将这段代码插入方法中，生成下列定义，看起来好像任务已经完成了：

```
public static void inWords(int number) //Not quite right
{
    inWords(number/10);
    System.out.print(digitWord(number%10) + " ");
}
```

但是，正如那条注释所指出的那样，这个方法不能很好地工作。它的基本概念是对的，但它有一个很大的问题：前面的定义假定实参number中的数字多于一个。你还需要专门为只有一位数字的数编写一段程序。正如将看到的那样，除非这种简单的情况能够正常工作，否则，其他的情况也都不能正常工作。这种情形促使你重新编写了下列方法定义：

```
public static void inWords(int number)
{
    if (number < 10)
        System.out.print(digitWord(number) + " ");
    else //number has two or more digits
    {
        inWords(number/10);
        System.out.print(digitWord(number%10) + " ");
    }
}
```

现在方法inWords的定义就完成了。图4-1显示了嵌套在一个演示程序中的方法。但是，在结束这个案例分析之前，再来对inWords方法进行一些更深入的讨论。

下面这个对inWords方法的递归调用出现在inWords方法的定义中：

```
inWords(number/10);
```

注意，这个递归调用中使用的实参number/10比整个方法定义中使用的参数number要小。递归调用解决的问题是原始问题的一个“较小的”版本（就是直觉概念上的“较小”，在本章结束之前我们会对其进行更清晰的解释），这是很重要的。

正如将在下一小节中看到的那样，用递归方式定义的方法，比如inWords，的成功执行要求对最简单的情况应该以不包含递归调用的方式进行处理。在inWords的定义中，对这种最简单情况是按下列方式处理的：

```
if (number < 10)
    System.out.print(digitWord(number) + " ");
```

注意，如果实参number只有一位数字，就不能使用递归调用了。

4.1.1 递归是如何工作的

计算机到底是如何处理递归调用的呢？要想明白具体的细节，就要查看一下图4-1中对inWords方法的下列调用。

```
inWords(987);
```

尽管inWords的定义中包含了一个递归调用，但计算机没有对这个或任何其他的inWords调用进行什么特殊的处理。计算机将实参987插入方法定义中，并执行了将其插入后得到的代码。插入987替换方法定义中的形参number，就得到了下列等价的代码：

```
{//Code for invocation of inWords(987)
    if(987 < 10)
       System.out.print(digitWord(987) + " ");
    else //987 has two or more digits
    {
       inWords(987/10);
       System.out.print(digitWord(987%10) + " ");
    }
}
```

执行这段代码时，计算机首先会查看if后面的布尔表达式。布尔表达式计算得false，因为987不小于10。因为布尔表达式计算得false，就会执行else后面的复合语句。这条复合语句是以下列递归调用作为起始的。

```
inWords(987/10);
```

在完成这条递归语句之前，计算的其余部分都无法继续。计算机必须停止它正在做的工作，去处理这个新的递归调用。因此，代码inWords(987);的执行被挂起，计算机会去执行新的

递归调用inWords(987/10);。计算机完成了对inWords(987/10);的递归调用之后，会转去完成被中断的inWords(987);计算。

对新的递归调用inWords(987/10);的处理与任何其他的方法调用一样：插入实参987/10，替换方法定义中的形参number，然后执行得到的代码。由于987/10计算得98，所以，计算机会插入98替换形参number，得到下列等价的代码：

```
{//Code for invocation of inWords(98)
    if(98 < 10)
       System.out.print(digitWord(98) + " ");
    else //98 has two or more digits
    {
       inWords(98/10);
       System.out.print(digitWord(98%10) + " ");
    }
}
```

执行这段新代码时，计算机再次遇到一个递归调用，即，递归调用inWords(98/10);。此时，前面的计算会被挂起，计算机会接着执行递归调用inWords(98/10);。由于98/10等于9，所以，9被插入替换inWords定义中的形参number，这样，执行的就是下列代码：

```
{//Code for invocation of inWords(9)
    if(9 < 10)
       System.out.print(digitWord(9) + " ");
    else //9 has two or more digits
    {
       inWords(9/10);
       System.out.print(digitWord(9%10) + " ");
    }
}
```

因为9确实小于10，所以会执行if-else语句的第一部分。因此，只有下列代码被执行。

```
System.out.print(digitWord(9) + " ");
```

这种情况被称为**停止条件**（stopping case）——即，一种不带有递归调用的条件。对digitWord方法定义的快速查看表明前面的System.out.print会将字符串"nine"写到屏幕上去。现在，对inWords(98/10)的调用结束了。此时，就可以继续执行下列代码中显示的、被挂起的计算了：

```
{//Code for invocation of inWords(98)
    if(98 < 10)
       System.out.print(digitWord(98) + " ");
    else //98 has two or more digits
    {
       inWords(98/10); <----------
       System.out.print(digitWord(98%10) + " ");
    }
}
```

计算会从箭头指示的位置之后开始恢复，因此，会执行下列代码。

```
System.out.print(digitWord(98%10) + " ");
```

这条语句会将字符串"eight"输出到屏幕上去，结束对递归调用inWords(98);的调用。

这个过程马上就要结束了。一旦调用inWords(98);完成了，就只剩一个挂起的计算需要完成了，下面的代码显示了这个计算：

```
{//Code for invocation of inWords(987)
      if(987 < 10)
         System.out.print(digitWord(987) + " ");
      else //987 has two or more digits
      {
            inWords(987/10);
            System.out.print(digitWord(987%10) + " ");
      }
}
```

计算会从箭头指示的位置之后开始恢复，这样，就会执行下列代码：

```
System.out.print(digitWord(987%10) + " ");
```

这条语句会将字符串"seven"写到屏幕上去，整个过程结束。图4-2对递归调用的顺序进行了说明。

inWords(987);等于是在执行：

```
{//Code for invocation of inWords(987)
    if (987 < 10)
        System.out.print(digitWord(987)+"");
    else//987 has two or more digits
    {
        inWords(987/10);
        System.out.print(digitWord(987%10)+"");
    }
}
```

计算过程停在这里，等待递归调用的结束。

inWords(987/10); 等价于inWords(98);，后者等于是在执行：

```
{//Code for invocation of inWords(98)
    if (98 < 10)
        System.out.print(digitWord(98)+"");
    else//98 has two or more digits
    {
        inWords(98/10);
        System.out.print(digitWord(98%10)+"");
    }
}
```

计算过程停在这里，等待递归调用的结束。

inWords(98/10); 等价于inWords(9);，后者等于是在执行：

```
{//Code for invocation of inWords(9)
    if (9 < 10)
        System.out.print(digitWord(9)+"");
    else//9 has two or more digits
    {
        inWords(9/10);
        System.out.print(digitWord(9%10)+"");
    }
}
```

这次调用没有引发另一次递归调用的执行。

图4-2 在递归调用过程中发生了什么

注意，当计算机遇到一个递归方法调用时，没有作任何特殊的事情。它只是插入实参替换形参，并执行方法定义中的代码，就像对任何一个方法调用所作的那样。

记住：成功递归的关键

除非遵循了一些特殊的设计规范，否则，一个包含了对方法自身的递归调用的方法定义是无法正常工作的。下列规则适用于大多数包含递归的场合。

- 方法定义的核心可以是一条if-else语句或一些其他的分支语句，这些语句会根据被定义方法的参数的一些特性而转向各种不同的情形。
- 应该在一个或多个分支中包含对方法的递归调用。在某种程度上，这些递归调用应该使用“较小”的实参，或者能够解决这个方法所执行任务的“较小”版本。
- 应该有一个或多个分支中不包含递归调用。这些分支为**停止条件**（stopping case）[也被称为**基准条件**（base case）]。

△ 易犯错误：无限递归

考虑一下图4-1中定义的inWords方法。假设无意中将其定义为下面这样的形式了：

```
public static void inWords(int number) //Not quite right
{
    inWords(number/10);
    System.out.print(digitWord(number%10) + " ");
}
```

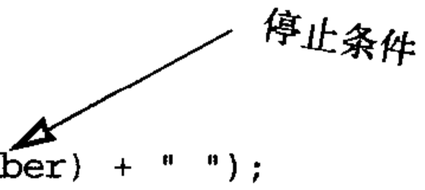
实际上，差点就要这样定义它了，直到我们注意到一种被忽略的情形为止。但是，假设我们没有注意到那种被忽略的情形，而使用了这种较短的定义。如果像在前一小节中那样，仔细查看递归调用inWords(987);，就会发现这个过程永远也不会停止。现在来快速地跟踪一下这个不正确的递归方法定义的计算过程。

除其他动作之外，方法调用inWords(987);还会产生对inWords(987/10);的递归调用，这个调用等价于inWords(98);。对inWords(98);的调用会产生对inWords(98/10);的递归调用，这个调用是等价于inWords(9);的。因为这个不正确的inWords版本中没有一种适用于一位数字的特殊情形，所以，对inWords(9);的调用会产生对inWords(9/10);的递归调用，这个调用等价于inWords(0);。现在问题就很清楚了。对inWords(0);的调用会产生对inWords(0/10);的调用，这个调用等价于inWords(0);。这样，对inWords(0);的调用就产生了另一个对inWords(0);的调用，它又会产生另一个对inWords(0);的调用，就这样永远进行下去（或者直到耗尽了计算机的资源为止）。这种情形被称为**无限递归**（infinite recursion）。

前面这个较短的、不正确的inWords定义，只是从它执行了错误的计算这个意义上来说是不正确的。它并不是非法的。Java编译器会接受这个inWords定义（以及任何与之类似的、没有停止条件可以停止这一系列递归调用的递归方法定义）。但是，除非递归方法的定义方式能够确保不会得到一个无尽的递归调用链，否则，调用方法时，就可能会得到一个无限的递归调用链，使程序永远运行下去，或非正常终止。

为了使一个递归方法定义能够正确工作，而不产生无限的递归调用链，就必须有一种或多种适用于某些参数值的情形能够在不产生任何递归调用的前提下终止。如前所述，这些情形被称为**基准条件**（base case）或**停止条件**（stopping case）。图4-1给出的正确inWords定义中有一个停止条件，在下列代码中将其突出显示出来了：

```
public static void inWords(int number)
{
    if (number < 10)
         System.out.print(digitWord(number) + " ");
    else //number has two or more digits
    {
         inWords(number/10);
         System.out.print(digitWord(number%10) + " ");
    }
}
```



设计停止条件时，必须使其能够终止每条递归调用链。一次方法调用会产生对同一方法的递归调用，而这次调用又会产生另一次递归调用，以此类推，会产生很多次的递归调用，但每个这样的递归调用链最终都必须通向一个不带递归调用的停止条件。否则，方法调用可能永远都不会终止（或者可能会到计算机资源耗尽的时候才会终止）。

一个典型的递归方法定义中会包含一条if-else语句或其他分支语句，这条分支语句会在一个或多个包含方法递归调用的情形，和一个或多个不包含任何递归调用就结束了方法调用的情形中进行选择。每个递归调用链最终都必须通向某个不包含任何递归调用的停止条件。

确保一定能抵达停止条件的最常用方法就是使方法中所有的递归调用都使用一个“较小”的实参（就是直觉概念上的“较小”）。例如，考虑一下图4-1中给出的、在前几段中重现的、正确的inWords定义。inWords的参数为number。inWords的递归调用的参数是一个较小的值number/10。按照这种方式，递归调用链上的每个递归调用都会有一个更小的实参。因为正确的inWords定义中有一个适用于所有“小”实参的停止条件，所以我们知道，它最终总会抵达一个停止条件的。 △

快速参考：栈溢出

如果一个方法调用引发了无限递归，程序就很可能会以一条表示“栈溢出”的错误消息作为结束。术语**栈**（stack）表示一种用来记录递归调用（以及其他内容）的数据结构。可以认为每个递归调用的记录都存储在像一张纸一样的东西上。将这些“纸张”“堆”起来，一张放在另一张的上面。当这个“栈”太大以至于计算机无法处理的时候，就发生了**栈溢出**（stack overflow）。

自测题

1. 下列程序会产生什么样的输出？

```
public class RecursionExercise
{
  public static void main(String[] args
  {
      methodA(3);
  }
  public static void methodA(int n)
  {
      if (n < 1)
          System.out.println('B')
      else
      {
```

```
            methodA(n - 1);
            System.out.println('R');
        }
    }
}
```

2. 下列程序会产生什么样的输出?

```
public class RecursionExercise2
{
  public static void main(String[] args)
  {
       methodB(3)
  }
  public static void methodB(int n)
  {
      if (n < 1)
          System.out.println('B')
      else
      {
          //The following two lines are the reverse of
          //What they are in Self-Test Question 1.
          System.out.println('R');
          methodB(n - 1);
      }
  }
}
```

3. 为下列方法编写一个递归方法定义:

```
/**
 Precondition: n >= 1
 Action: Writes out n of the symbol '#' on one line
 and advances to the next line.
*/
public static void sharp(int n)
```

注意，最后一个'#'之后的输出都会进入下一行。因此

```
sharp(3);
```

就等价于

```
System.out.println("###");
```

如果做这个练习有些困难，就先按照输出不进入下一行来实现。对这种较简单的情况来说，不需要考虑print和println之间的区别。在这种较简单的情况下，只用print就行了，不用使用println。完成了这种较简单的情况之后，试着按照练习的要求来完成。

4.1.2　递归及迭代定义

可以重新编写任意一个包含递归调用的方法定义，使其不使用递归来完成同样的任务。例如，图4-3包含的是图4-1程序的重写版本，但这个版本中的inWords定义没有使用递归。这两个inWords版本完成的是完全相同的动作——即，它们都向屏幕写出了同样的输出。如此例所示，方法定义的非递归版本中通常都包含一个循环来取代递归，因此被称为**迭代版本**(iterative version)。

方法定义的递归版本通常没有同一个方法的迭代定义效率高（也就是说，运行得更慢或使用的存储空间更多)，因为计算机在对递归调用以及被挂起的计算进行跟踪时需要一些开销。

因此，递归应当仅限于在它可以使代码更易理解的情况下使用。当然，在有些情况下，使用递归确实会对提高程序的清晰度有很大帮助。

```
import java.util.*;

public class IterativeDemo
{
    public static void main(String[] args)
    <main方法的其余部分与图4-1中相对应的部分相同>

    /**
     Precondition: number >= 0
     Action: The digits in number are written out in words.
    */
    public static void inWords(int number)
    {
        int divisor = powerOfTen(number);
        int next = number;
        while (divisor >= 10)
        {
            System.out.print(digitWord(next/divisor) + " ");
            next = next%divisor;
            divisor = divisor/10;
        }
        System.out.print(digitWord(next/divisor) + " ");
    }

    /**
     Precondition: n >= 0. Returns the number in the form "one
     followed by all zeros that is the same length as n."
    */
    private static int powerOfTen(int n)
    {
        int result = 1;
        while(n >= 10)
        {
            result = result*10;
            n = n/10;
        }
        return result;
    }

    private static String digitWord(int digit)
    <digitWord方法的其余部分与图4-1中相对应的部分相同>
}
```

对话与图4-1完全一样。

图4-3 inWords的迭代版本

4.1.3 返回值的递归方法

任何类型的方法中都可以包含递归。递归方法可以是一个void方法，也可以是一个带有返回值的方法。设计一个带有返回值的递归方法的方式与设计一个void递归方法的方式基本

相同。定义一个行为良好的、带有返回值的递归方法定义的基本技巧如下所示。

- 方法定义的核心可以是一条if-else语句，或其他一些会根据被定义方法参数的某些特性而转向各种不同情形的分支语句。
- 有一条或多条分支导向这样的情形：根据对同一个方法的调用（即，使用递归调用）计算出返回值。递归调用的实参应该随着每次调用变得“更小”。
- 有一条或多条分支导向这样的情形：不使用任何递归调用计算出返回值。这些情形被称为**基准条件**（base case），或**停止条件**（stopping case）。（每条递归调用链都应该以停止条件结束。）

图4-4定义的numberOfZeros方法对这种技术进行了说明。这个方法有一个int实参，并返回（以正常形式书写时）数字中零的个数。例如，numberOfZeros(2030)会返回2，因为2030中有两个数字零。来看看方法numberOfZeros是怎样工作的。

```
import java.util.*;

public class RecursionDemo2
{
    public static void main(String[] args)
    {
        System.out.println("Enter a nonnegative number:");
        Scanner keyboard = new Scanner(System.in);
        int number = keyboard.nextInt();
        System.out.println(number + " contains "
                           + numberOfZeros(number) + " zeros.");
    }

    /**
     Precondition: n >= 0
     Returns the number of zero digits in n.
    */
    public static int numberOfZeros(int n)
    {
        if (n == 0)
            return 1;
        else if (n < 10)//and not 0
            return 0;//0 for no zeros
        else if (n%10 == 0)
            return(numberOfZeros(n/10) + 1);
        else //n%10 != 0
            return(numberOfZeros(n/10));
    }
}
```

屏幕对话示例

```
Enter a nonnegative number:
2005
2005 contains 2 zeros.
```

图4-4　带有返回值的递归方法

方法numberOfZeros的定义用到了下列简单事实：

如果n是两位或多位数字长的，那么，如果最后一位数字为零，n中数字零的个数就（是去掉最后一位数字之后n中零的个数）加上一。

例如，20030中零的个数就是2003中零的个数加上一个1（表示最后一个零）。20031中零的个数就是2003中零的个数，不加任何数值了，因为最后那个数字不为零。头脑中有了这个定义之后，我们来看一个使用了numberOfZeros的简单计算。

首先，考虑下面这个简单的表达式

```
numberOfZeros(0)
```

这个表达式可能会出现在某个赋值语句的右边。调用这个方法时，参数n的值被置为0，并会执行方法定义主体中的代码。由于n的值等于0，适用于多路if-else语句中的第一种情况，所以，返回值为1。

接下来，考虑另一个简单的表达式：

```
numberOfZeros(5)
```

调用这个方法时，参数n的值被置为5，并会执行方法定义主体中的代码。由于n的值不等于0，所以，不适用于多路if-else语句中的第一种情况。但是，n的值小于10，因此，适用于多路if-else语句的第二条分支，返回值为0。正如看到的那样，这两种简单的情况都能够正常工作。

现在，我们来看一个包含了递归调用的例子。考虑一下表达式

```
numberOfZeros(50)
```

调用这个方法时，n的值被置为50，并会执行方法定义主体中的代码。由于n的这个值不等于0，也不小于10，多路if-else语句中的前两种情况都不适用。但是，n%10（即，50%10）的值为0，因此，适用于第三条分支。因此，返回的值为

```
numberOfZeros(n/10) + 1
```

在这种情况下，就等价于

```
numberOfZeros(50/10) + 1
```

接下来，这个表达式又等价于

```
numberOfZeros(5) + 1
```

已经确定了numberOfZeros(5)会返回0，因此，numberOfZeros(50)返回的值就是

```
0 + 1
```

(就是1，是个正确的值)。

更大的数会生成更长的递归调用链。例如，考虑一下表达式

```
numberOfZeros(2005)
```

这个表达式的值是按下列形式计算的：

```
numberOfZeros(2005)为numberOfZeros(200) + 0
  numberOfZeros(200)为numberOfZeros(20) + 1
    numberOfZeros(20)为numberOfZeros(2) + 1
      numberOfZeros(2)为0（停止条件）
```

计算机抵达停止条件numberOfZeros(2)时，有三个挂起的计算。计算出停止条件的返回值之后，会恢复最近挂起的那个计算，以确定numberOfZeros(20)的值。之后，计算机会完成挂起的其他计算，将计算得到的每个值插入到另一个挂起的计算，直到抵达并完成了对原

始调用numberOfZeros(2005)的计算为止。被挂起的计算是按下列形式完成的（这是一个基本上按照自底向上（bottom to top）的形式完成前面那个被挂起的计算链的过程）：

```
numberOfZeros(2)为0（停止条件）
numberOfZeros(20)为numberOfZeros(2) + 1，就是0 + 1 == 1
numberOfZeros(200)为numberOfZeros(20) + 1，就是1 + 1 == 2
numberOfZeros(2005)为numberOfZeros(200) + 0，就是2 + 0 == 2
```

因此，调用numberOfZeros(2005)最后返回的值就是2，这是正确的，因为2005中有两个数字零。

记住：不要将递归与重载混淆起来

不要将递归和重载混淆起来。重载一个方法名时，是为两个不同的方法起了同样的名字。如果这两个方法中某个方法的定义包含了对另一个方法的调用，这并不是递归。在递归方法的定义中，方法的定义包含了对同一个方法的调用，这两个方法具有完全相同的方法定义，包含相同数量和类型的参数。

自测题

4. 下列程序会产生什么样的输出？

```
public class RecursionExercise4
{
    public static void main(String[] args)
    (
        System.out.println(mysteryValue(3));
    }

    public static int mysteryValue(int n)
    (
        if(n <= 1)
          return 1;
        else
          return(mysteryValue(n-1) + n);
    }
}
```

5. 完成这个给定方法的定义。定义应该是递归的。（提示：如果n>1，则10^n就是$10^{n-1}*10$。）

```
/**
Precondition: n >= 0
Returns 10 to the power n.
前置条件：n >= 0
返回10的n次方
*/
public static int tenToThe(int n)
```

6. 完成这个给定方法的定义。定义应该是递归的。应该使用与自测题5相同的技术，但应该多一个适用于负指数的递归调用情形。（提示：对负值的n来说，10^n就是$1/10^{-n}$。而且，如果n为负，$-n$就为正。这个练习与自测题5的区别就在于它允许将负数作为实参使用。）

```
/**
Precondition: n can be any int.
Returns 10 to the power n.
```

```
*/
public static double tenToThe(int n)
```

4.2 用递归编程

所有与编程技巧有关的短语句都是错的。

——佚名

本节提供了一些程序，以说明使用递归的方式。

○ 编程提示：不断地提问直到用户搞清楚为止

图4-5中的程序只是请求了一个正数，然后从那个数字开始倒数到零为止。那个数字是用方法getCount输入的。

注意，如果用户输入了一个非正的数字，方法getCount会进行一次递归调用。这次调用使得输入过程再次从头开始。如果用户键入另一个不正确的输入，就会进行另一次递归调用，输入过程会再次从头开始。这个过程不断重复直到用户输入一个正数为止。当然，实际上，很少会进行递归调用，但只要需要，就应该进行递归调用。

```
import java.util.*
;
public class CountDown
{
    private int count;

    public static void main(String[] args)
    {
         CountDown countDowner = new CountDown();
         countDowner.getCount();
         countDowner.showCountDown();
    }

    public void getCount()
    {
        System.out.println("Enter a positive number:");
        Scanner keyboard = new Scanner(System.in);
        count = keyboard.nextInt();
        if (count <= 0)
        {
            System.out.println("Input must be positive.");
            System.out.println("Try again.");
            getCount();//start over
        }
    }

    public void showCountDown()
    {
        int left;
```

图4-5 从头开始的递归

```
        System.out.println("Counting down:");
        for (left = count; left >= 0; left--)
            System.out.print(left + ", ");
        System.out.println("Blast Off!");
    }
}
```

屏幕对话示例

```
Enter a positive number:
0
Input must be positive.
Try again.
Enter a positive number:
3
Counting down:
3, 2, 1, 0 Blast Off!
```

图4-5　(续)

案例分析：二分搜索

这个案例分析中，假定你已经掌握《Java程序设计与问题解决：基础篇》第6章中有关数组的基本知识了。

在这个案例分析中，要设计一个递归方法，用来说明给定的数字是否位于一个整数数组中。如果这个被搜索的数字位于数组中，方法还会告诉你这个数字在数组中所处位置的索引。例如，数组中可能包含了一组中奖的彩票，你想搜索这个列表看看你是否中奖了。在《Java程序设计与问题解决：基础篇》第6章中，讨论过一种通过简单地查看每个数组位置的值来搜索数组的方法。(请参见该书的6.3.2节。) 在这个案例分析中使用的方法比那个简单的串行搜索要快很多。但是，要使这种较快的方法工作起来，数组必须是排好序的。

假定数组是排过序且完全填充的。因此，如果数组名为a，就知道

```
a[0] <= a[1] <= a[2] <= ··· <= a[a.length - 1]
```

通常，你想知道的不仅仅是一个元素是否位于某个数组中。如果某个元素位于这个数组中，通常，你还希望知道它在数组中的什么地方。例如，如果你在搜索一个中奖彩票号码，就可以将数组索引作为记录号。另一个同样以这些索引作为索引的数组中装载的可能就是一些电话号码，你可以打过去声明你中奖了。因此，如果被搜索的值位于数组中，你还希望方法能够告诉你它位于数组中的什么位置。这样，就可以对方法进行设计，使其返回一个整数，给出被搜索数字的索引。如果那个数字不在数组中，方法就返回-1。在开始操心类和方法的具体建立过程以及如何将方法与数组联系起来之前，先要设计一些伪代码来解决搜索问题。

设计的算法要利用数组中的数字是排过序的这一事实。注意，因为数组是排过序的，有时就可以将数组中不可能包含所搜寻数字的一整块区域排除掉。例如，如果要搜寻数字7，而你知道a[5]等于9，那么，当然就可以知道7不等于a[5]，但还能知道的更多：对于任何大于或等于5的i值，7都不等于a[i]。因为数组是排过序的，所以，就可以知道，只要i大于或等于5，就有

```
7 < a[5] <= a[i]
```

因此，所有i大于或等于5的元素a[i]都无需搜索了。不需查看就可以知道所要搜寻的值7不在其中。

类似地，如果所要搜寻的数字7大于a[5] (例如，如果a[5]为3而不是9)，就可以将所有i小于

或等于5的元素a[i]都排除掉了。

用数组中间的任意一个索引来取代（前面这个例子中的）5，就可以得出第一份算法草稿：

```
mid = 大约是0和(a.length - 1)的中间点
if (target == a[mid])
    return mid;
else if(target < a[mid])
    返回a[0]到a[mid - 1]的搜索结果
else if(target > a[mid])
    返回a[mid + 1]到a[a.length - 1]的搜索结果
```

注意，在一个数组的子区间内进行搜索（就像那两个else if中所做的一样）就是所要设计的任务的较小版本。因此，可以用对算法的递归调用来搜索数组子区间。

对应于递归调用的两段伪代码为

```
返回a[0]到a[mid - 1]的搜索结果
```

以及

```
返回a[mid + 1]到a[a.length - 1]的搜索结果
```

但是，还有一个麻烦的问题。（不总是会存在这样的问题吗？）要实现这些递归调用，就需要更多的参数。这些递归调用说明要对数组的一个子区间进行搜索。在第一种情况下，就是索引号从0到mid-1的那些元素。在第二种情况下，就是索引号从mid+1到a.length-1的那些元素。因此，需要两个额外的参数来说明所要搜寻的数组子区间的第一个和最后一个索引。分别将这些额外的参数称为first和last。用这些参数来说明要搜索的子区间，可以用下列形式对伪代码进行更详细的说明：

对a[first]到a[last]进行搜索的算法

```
mid = 大约是first和last的中间点
if (target == a[mid])
    return mid;
else if(target < a[mid])
    返回a[first]到a[mid - 1]的搜索结果。
else if(target > a[mid])
    返回a[mid + 1]到a[last]的搜索结果。
```

如果想对整个数组进行搜索，就将first置为0，将last置为a.length-1。每次递归调用都会用其他值来取代first和last。例如，第一个递归调用会将first设置为0，将last设置为mid-1。

总是应该对编写的递归算法进行检查，以防出现无限递归。我们来看看是否每种可能的算法调用都会指向停止条件。考虑一下嵌套if-else语句中的三种情形。在第一种情形下，在列表中找到了被搜索的数字，没有递归调用，因此过程终止了。在其他两种情况下，递归调用搜索的都是数组的一个较小子区间。如果被搜索的数字在数组中，算法就会逐渐缩小区间，直到找到这个数字为止。但是，如果数组中没有这个数字会怎么样呢？如果这个数字不在数组中，得到的递归调用序列最终会指向一个停止条件吗？不幸的是，它不会，不过，这个问题并不难修正。

注意，在每次递归调用中，first的值都会增加，或者last的值会减少。如果这两个值打过照面，而且first变得比last大了，那么，就知道没有剩余的索引需要检查了，数字target不在数组中。如果将这项测试添加到伪代码中去，就可以得到下列更完整的伪代码：

对a[first]到a[last]进行搜索的算法

```
mid = 大约是first和last的中间点
if (first > last)
```

```
        return -1;
    else if (target == a[mid])
        return mid;
    else if(target < a[mid])
        返回a[first]到a[mid - 1]的搜索结果。
    else if(target > a[mid])
        返回a[mid + 1]到a[last]的搜索结果。
```

接下来，要将这个伪代码算法转换成Java代码。将方法称为search，并将其放在一个名为ArraySearcher的类中。类中会有一个实例变量来命名这个数组，并由构造器将那个名字赋给要搜索的数组。图4-6显示了最终的代码。图4-7给出的图示说明了这个方法是如何对一个示例数组进行操作的。

```
/**
 Class for searching an already sorted array of ints.
 To search the sorted and completely filled array b,
 use the following:
 ArraySearcher bSearcher = new ArraySearcher(b);
 int index = bSearcher.find(target);
 index will be given an index of where target is located.
 index will be set to -1 if target is not in the array.
*/
public class ArraySearcher
{
    private int[] a;

    /**
     Precondition: theArray is full and is sorted
     from lowest to highest.
    */
    public ArraySearcher(int[] theArray)
    {
        a = theArray;//a is now another name for theArray.
    }
    /**
     If target is in the array, returns the index of an occurrence
     of target. Returns -1 if target is not in the array.
    */
    public int find(int target)
    {
         return search(target, 0, a.length - 1);
    }

    //Uses binary search to search for target in a[first] through
    //a[last] inclusive. Returns the index of target if target
    //is found. Returns -1 if target is not found.
    private int search(int target, int first, int last)
    {
        int result = -1;//to keep the compiler happy.
        int mid;
        if (first > last)
            result = -1;
        else
        {
```

图4-6　二分搜索类

```
            mid = (first + last)/2;
            if (target == a[mid])
                result = mid;
            else if (target < a[mid])
                result = search(target, first, mid - 1);
            else //(target > a[mid])
                result = search(target, mid + 1, last);
        }
        return result;
    }
}
```

图4-6 （续）

target为33

排除了一半的数组元素：



数组中包括：

a[0]=5 a[1]=7 a[2]=9 a[3]=13 a[4]=32 a[5]=33 a[6]=42 a[7]=54 a[8]=56 a[9]=88

mid = (0 + 9) / 2 (为4)。
33 > a[mid] (也就是说，33 > a[4])。

因此，如果33在数组中，33就是下列元素之一
a[5]=33 a[6]=42 a[7]=54 a[8]=56 a[9]=88。

排除了剩余数组元素中的一半：



a[5]=33 a[6]=42 a[7]=54 a[8]=56 a[9]=88

mid = (5 + 9)/2 (为7)。
33 < a[mid] (也就是说，33 < a[7])。

因此，如果33在数组中，33就是下列元素之一
a[5]=33 a[6]=42

排除了剩余数组元素中的一半：

a[5]=33 a[6]=42

mid = (5 + 6)/2 (为5)。
33 = = a[mid],这样，我们发现33位于索引5上。

在a[5]上发现了33。

图4-7 二分搜索实例

你会注意到方法search中有一些额外的参数，用户总是要将这些参数设置为0和a.length-1，以说明要搜索的整个数组。你并不希望用户来操心这些细节，因此，添加了方法find，这个方法使用户只需指定target值，而无需再操心索引的问题了。方法find只是调用了方法search，但这一

步为用户省去了很多麻烦。方法search现在只是一个辅助方法，所以，可以将其设置为私有方法。

图4-8给出了一个简单的程序，以说明ArraySearcher类是如何工作的。

在这个案例分析中使用的二分搜索算法运行得非常快。在二分搜索算法中，一开始就排除了半个数组，然后又排除了数组的另外四分之一，然后是另外八分之一，以此类推。这种排除方法就意味着对大部分数组根本就不需要进行搜索，从而节省了大量的时间。例如，对一个带有1000个元素的数组来说，二分搜索只需要将大约10个数组元素与目标值进行比较。比较起来，一个简单的线性搜索可能要将1000个数组元素与目标值进行比较，平均来说，它也要将500个数组元素与目标值进行比较。

```
import java.util.*;

public class ArraySearcherDemo
{
    public static void main(String[] args)
    {
        int [] a = new int[10];
        System.out.println("Enter 10 integers in increasing order.");
        System.out.println("One per line.");
        Scanner keyboard = new Scanner(System.in);
        int i;
        for (i = 0; i < 10; i++)
            a[i] = keyboard.nextInt();
        System.out.println();
        for (i = 0; i < 10; i++)
           System.out.print("a[" + i + "]=" + a[i] + " ");
        System.out.println();
        System.out.println();
        ArraySearcher finder = new ArraySearcher(a);

        String ans;
        do
        {
            System.out.println("Enter a value to search for:");
            int target = keyboard.nextInt();
            int result = finder.find(target);

            if (result < 0)
                System.out.println(target + " is not in the array.");
            else
                System.out.println(target + " is at index " + result);

            System.out.println("Again?");
            ans = keyboard.next();
        }while (ans.equalsIgnoreCase("yes"));

        System.out.println("May you find what you're searching for.");
    }
}
```

图4-8　二分搜索的演示

屏幕对话示例

```
Enter 10 integers in increasing order.
One per line.
0
2
4
6
8
10
12
14
16
18
a[0]=0 a[1]=2 a[2]=4 a[3]=6 a[4]=8 a[5]=10 a[6]=12 a[7]=14 a[8]=16 a[9]=18
Enter a value to search for :
14
14 is at index 7
Again?
y
Enter a value to search for :
0
0 is at index 0
Again?
y
Enter a value to search for:
2
2 is at index 1
Again?
y
Enter a value to search for:
13
13 is not in the array.
Again?
n
May you find what you're searching for.
```

图4-8 （续）

○ 编程提示：将问题一般化

设计一个递归算法时，通常都需要解决一个比你想要解决的问题更普遍的问题。例如，考虑一下在前面那个案例分析中设计用来搜索整个数组的方法search。要将其设计为不仅能搜索整个数组，而且能搜索数组的任意一个子区间的方法。这种设计对那些递归子情况来说，是必要的。在设计一个递归算法时，必须使问题更普遍一些，这样才能方便地表示那些子情况，通常都是如此。

自测题

7. 如果数组未经排序，二分搜索算法还能工作吗？

8. ArraySearcher的构造器使用的数组中的所有值都必须不同吗？还是可以有重复的值？

9. 假设希望ArraySearcher类能够为那些值从大到小而不是从小到大排列的数组工作。需要对

ArraySearcher的定义作哪些修改?

编程示例：归并排序——一种递归排序方法

一些最有效的排序算法使用递归算法来表述都是最简单的。对数组进行排序的归并排序算法就是这样一个例子。在这个例子中，提供了一个方法，这个方法实现了将一个int值数组中的数据按从小到大的顺序进行排序的归并排序算法。经过少量修改，就可以得到一个能对条目为任意可排序类型的数组进行排序的方法。在自测题中对一些修改实例进行了讨论。

归并排序（merge sort）是一个分而治之的算法实例。要排序的数组被一分为二，并通过递归调用对这两半数组进行排序。这个过程会生成两个较小的已排序数组。然后这两个已排序数组会归并形成一个包含了两个较小数组中所有元素的已排序数组。算法的轮廓如下所示。

对数组a进行排序的归并排序算法

如果数组a中只有一个元素，就什么也不做（停止条件）。
否则，完成下列任务（递归条件）：
　将a中的前半部分元素拷贝到一个较小的名为front的数组中去。
　将数组a中其余的元素拷贝到另一个较小的名为tail的数组中去。
　用递归调用对数组front进行排序。
　用递归调用对数组tail进行排序。
　将数组front和tail中的元素归并到数组a中去。

用Java代码来实现这个算法的大部分细节都是很简单的。但是，将两个数组front和tail归并成一个已排序数组a确实需要进行一些解释。

归并算法的基本思想如下所示：数组front和tail都是从小到大进行排序的。因此，数组front中最小的元素为front[0]，而数组tail中最小的元素为tail[0]。因此，这两个数组中所有元素的最小值就是front[0]和tail[0]中较小的那个。将那个最小的元素移到a[0]中去。

例如，假设那个较小的元素为tail[0]，这样，就将那个元素移到a[0]中去。剩下的要移到a中去的最小元素就是front[0]和tail[1]中较小的那个。将那个元素移到a[1]中去，并继续执行这个过程。

归并过程的Java代码看起来会与下列代码类似：

```
int frontIndex = 0, tailIndex = 0, aIndex = 0;
while (some condition)
{
   if (front[frontIndex] < tail[tailIndex])
   {
      a[aIndex] = front[frontIndex];
      aIndex++;
      frontIndex++;
   }
   else
   {
      a[aIndex] = tail[tailIndex];
      aIndex++;
      tailIndex++;
   }
```

```
}
```

现在，while循环的布尔条件是什么呢？注意，只有数组front和tail中都有元素可移动，循环才是有意义的。因此，必须循环到数组front或tail中某个数组的所有元素都被完全移到a中去为止，而不是循环到数组a满了为止。因此，while循环的条件可以是这样的：

```
while((frontIndex < front.length) && (tailIndex < tail.length))
```

这个while循环结束时，front或tail数组中的一个可能会有（实际上，很可能会有）一些元素要移到数组a中去。这些元素是已排序的，而且都比已经移到数组a中去的元素要大。因此，我们所要做的就是将有剩余元素的数组（不论是front还是tail）中的所有剩余元素都移到数组a中去。完整的代码是在图4-9中作为方法merge给出的。

```
/**
 Class for sorting an array of ints from smallest to largest,
 using the merge sort algorithm.
*/
public class MergeSort
{
    /**
     Precondition: Every indexed variable of a has a value.
     Action: Sorts a so that a[0] <= a[1] <= ... <= a[a.length - 1].
    */
    public static void sort(int[] a)
    {
        if (a.length >= 2)
        {
            int halfLength = a.length/2;
            int[] front = new int[halfLength];
            int[] tail = new int[a.length - halfLength];

            divide(a, front, tail);
            sort(front);
            sort(tail);
            merge(a, front, tail);

        }
        //else do nothing. a.length == 1, so a is sorted.
}

/**
 Precondition: a.length = front.length + tail.length.
 Postcondition: All the elements of a are divided
 between the arrays front and tail.
*/
private static void divide(int[] a, int[] front, int[] tail)
{
    int i;
    for (i = 0; i < front.length; i++)
        front[i] = a[i];

    for (i = 0; i < tail.length; i++)
        tail[i] = a[front.length + i];
}

/**
 Precondition: Arrays front and tail are sorted from smallest
 to largest, and a.length = front.length + tail.length.
 Postcondition: a contains all the values from front and tail,
 and a is sorted from smallest to largest.
*/
```

图4-9 MergeSort类

```
private static void merge(int[] a, int[] front, int[] tail)
{
    int frontIndex = 0, tailIndex = 0, aIndex = 0;

    while ((frontIndex < front.length)
                        && (tailIndex < tail.length))
    {
        if (front[frontIndex] < tail[tailIndex])
        {
            a[aIndex] = front[frontIndex];
            aIndex++;
            frontIndex++;
        }
        else
        {
            a[aIndex] = tail[tailIndex];
            aIndex++;
            tailIndex++;
        }
    }
    //At least one of front and tail has been
    //completely copied to a.

    while (frontIndex < front.length)//Copy rest of front, if any.
    {
        a[aIndex] = front[frontIndex];
        aIndex++;
        frontIndex++;
    }

        while (tailIndex < tail.length)//Copy rest of tail, if any.
        {
            a[aIndex] = tail[tailIndex];
            aIndex++;
            tailIndex++;
        }
    }
}
```

图4-9　（续）

图4-9给出了归并排序算法完整的Java实现。图4-10显示了一个演示程序，这个程序使用了图4-9中的MergeSort类。

```
public class MergeSortDemo
{
    public static void main(String[] args)
    {
        int[] b = {7, 5, 11, 2, 16, 4, 18, 14, 12, 30};

        System.out.println("Array values before sorting:");
        int i;
        for (i = 0; i < b.length; i++)
            System.out.print(b[i] + " ");
        System.out.println( );

        MergeSort.sort(b);
```

图4-10　MergeSort类的演示

```
        System.out.println("Array values after sorting:");
        for (i = 0; i < b.length; i++)
            System.out.print(b[i] + " ");
        System.out.println( );
    }
}
```

屏幕输出

```
Array values before sorting:
7 5 11 2 16 4 18 14 12 30
Array values after sorting:
2 4 5 7 11 12 14 16 18 30
```

图4-10 （续）

归并排序算法比在《Java程序设计与问题解决：基础篇》第6章中展示的选择排序算法的效率高得多。实际上，没有哪个排序算法能比归并排序算法的效率高上“几个数量级”。“几个数量级”含义的细节超出了本书的范围，但我们可以对其含义进行一些提示。

实际上，有些算法比归并排序算法更有效。但是，对那些很大的数组来说，这些算法中的任意一个（包括归并排序算法在内）都比选择排序算法要快的多，以至于对选择排序算法的改进要比各种更快的算法之间的区别更加引人注目。

自测题

10. 怎样用MergeSort类对下列数组进行排序？

```
int[] myArray = {9, 22, 3, 2, 87, -17, 12, 14, 33, -2};
```

11. 对MergeSort类进行哪些修改，才能使其对double类型（而不是int类型）值的数组进行排序？
12. 对MergeSort类进行哪些修改，才能使其对int类型值的数组进行降序排序，而不是升序排序？
13. 如果一个基类型为int的数组中，有一个值出现了两次（比如b[0] == 7，而且b[5] == 7），而用方法MergeSort.sort对数组进行了排序，排过序之后，数组中会有一份还是两份那个重复值的副本呢？

小 结

- 如果一个方法定义中包含了对方法自身的调用，这个调用就被称为递归调用。在Java中，递归调用是合法的，有时还会使一个方法定义更加清晰。
- 只要算法有一个子任务，而且这个子任务是整个算法所要完成任务的一个较小版本，就可以用一个Java递归方法来实现这个算法。
- 要避免无限递归的出现，递归方法定义中就应该包含两类情形：一个或多个包含了递归调用的情形，以及一个或多个不包含任何递归调用的停止条件。
- 二分搜索算法和归并排序算法是两个很好的递归算法实例。

✓ 自测题答案

1. B
   R

```
R
R
```

注意，'B'是第一个输出，不是最后一个输出。

2.
```
R
R
R
B
```

注意，'B'是最后一个输出。

3.
```
/**
  Precondition: n >= 1.
  Action: Writes out n of the symbol '#' on one line
  and advances to the next line.
*/
public static void sharp(int n)
{
      if (n <= 1)
          System.out.println('#');
      else
      {
          System.out.print('#');
          sharp(n-1);
      }
}
```

4. 6

5.
```
/**
  Precondition: n >= 0
  Returns 10 to the power n.
*/
public static int tenToThe(int n)
{
     if (n <= 0)
         return 1;
     else
         return (tenToThe(n - 1)*10);
}
```

6.
```
/**
  Precondition: n can be any int.
  Returns 10 to the power n.
*/
public static double tenToThe(int n)
{
     if (n == 0)
         return 1;
     else if (n > 0)
         return (tenToThe(n-1)*10);
     else // n < 0
         return (1/tenToThe(-n));
}
```

7. 不能。

8. 只要数组排好了序，就可以有重复的值。

9. 需要对方法search中多路if-else语句的两个比较运算符进行修改，使其如下所示：

```
if(target == a[mid])
    result = mid;
else if (target > a[mid])//Changed from < to >
    result = search(target, first, mid-1);
else if (target < a[mid])//Changed from > to <
    result = search(target, mid+1, last);
```

不需要进行其他修改，但需要修改注释，以反映出数组从大到小进行排序的事实。

10. `MergeSort.sort(myArray);`

11. 只要将数组元素的类型改成double就行了。例如，应该用

    ```
    private static void divide(double[] a, double[] front, double[] tail)
    ```

    来取代

    ```
    private static void divide(int[] a, int[] front, int[] tail)
    ```

12. 要使代码实现按照降序进行排序的功能，所要做的就是在merge定义的下列行中用>取代<：

    ```
    if (front[frontIndex] < tail[tailIndex])
    ```

13. 如果一个基类型为int的数组中有一个值出现了两次，而用方法MergeSort.sort对数组进行了排序，排好序之后数组中会有这个重复值的两份拷贝。

○ 编程项目

1. 为一个方法编写静态递归方法定义，这个方法有一个int类型实参，并会将（用常用方式书写的）实参的长度返回。这个方法应该是一个静态方法。必须允许使用正的和负的实参。对负的实参来说，符号不算长度的一部分。因此，-123的长度就为3。将这个方法嵌入到一个程序中去，对其进行测试。

2. 为一个方法编写静态递归方法定义，这个方法有一个值为int的数组参数，并会返回数组中元素的总和（即，数组中那些整数之和）。可以假设数组中的每个索引变量都有一个值。将方法嵌入一个测试程序中进行测试。

3. 递归最常见的实例是一个用来计算整数**阶乘**（factorial）的算法。符号n!用于表示整数n的阶乘，定义如下所示：

   ```
   0!等于1
   1!等于1
   2!等于2*1 = 2
   3!等于3*2*1 = 6
   4!等于4*3*2*1 = 24
   .
   .
   .
   n!等于n*(n-1)*(n-2)*…*3*2*1
   ```

   另一种用来描述n!的方式是递归公式n*(n-1)!，加上一个定义为1的停止条件0!。编写一个静态方法，实现这个计算阶乘的递归公式。将方法放在一个允许用户（通过调用这个静态方法）计算n!的测试程序中，程序的n值由用户输入。程序应该允许用户输入另一个n值，并重复计算，直到用户希望结束这个程序为止。

4. 递归公式的一个常见实例是用来计算前n个整数之和的，1+2+3+…+n。可以将递归公式表示为

   $1+2+3+\cdots+n = n+(1+2+3+\cdots+(n-1))$

   编写一个实现这个递归公式的静态方法来计算前n个整数之和。将方法放在一个允许用户（通过对这个静态方法的调用）计算前n个整数之和的程序中进行测试，n值由用户输入。程序应该允许用户输入另一个n值，并重复进行计算，直到用户希望终止程序为止。（注意：方法定义不应该使用循环来把前n个整数加起来。这样的方法不是递归方法，或者至少不是一个好的递归方法。）

5. 为一个方法编写静态递归方法定义，这个方法有一个String类型的参数，并会返回一个boolean值。如果实参是回文，就返回true，否则就返回false。**回文**（palindrome）是一个从前往后和从后往前读都一样的字符串，比如"radar"。不考虑空格和标点符号，并且认为同一个字母的大写和小写版本是一样的。例如，方法应该将下列字符串当成一个回文：

   ```
   "Straw? No, too stupid a fad, I put soot on warts."
   ```

   方法不需要查看字符串是否是一个正确的英文短语或单词。方法会将字符串"xyzczyx"当成一个回文。将方法嵌入一个程序进行测试。

6. **几何级数**（geometric progression）和**调和级数**（harmonic progression）是两种常见的级数，几何级数定义为前n个整数的乘积，调和级数定义为前n个整数倒数的乘积。几何级数的数学表示法为：

   Geometric(n)等于 $\prod_{i=1}^{n} i$

   这种表示法意味着将从1到n的整数相乘。

   调和级数的数学表示法如下所示：

   Harmonic(n)等于 $\prod_{i=1}^{n} \frac{1}{i}$

   这两种类型的级数都有一个等价的递归定义：

   对几何级数：$\prod_{i=1}^{n} i = n \times \prod_{i=1}^{n-1} i$

   对调和级数：$\prod_{i=1}^{n} \frac{1}{i} = \frac{1}{n} \times \prod_{i=1}^{n-1} \frac{1}{i}$

   编写实现这些递归公式的静态方法来计算Geometric(n)和Harmonic(n)。不要忘记包含一个停止条件，刚才给出的公式中没有，但一定要确定一个。将方法放到一个允许用户（通过调用这些静态方法）为输入的整数n计算Geometric(n)和Harmonic(n)的测试程序中进行测试。程序应该允许用户输入另一个n值，并重复计算过程直到用户希望终止程序为止。（注意：定义的任何一个方法都不应该使用循环对n个数进行相乘。这样的方法不是递归方法，或者至少不是好的递归方法。）

7. 自然界中某种理想化动物种群的增长率通常会以裴波那契（Fibonacci）序列出现。这个序列以0和1开始，每个后继的裴波那契数都是前两个裴波那契数之和。因此，序列中第3个数就是0+1，为1；第四个数就是1+1，为2；第五个数就是1+2，为3，等等。以类似的方法继续算下去，前10个裴波那契数为0、1、1、2、3、5、8、13、21和34。除种群增长率之外，这个序列还会出现在自然界的很多环境中（例如，可以用这个序列来描述一个螺旋的形状），而且序列中最后一个数字和倒数第二个数字的比值会收敛于一个约等于1.618的常量，这个常量被称为“黄金分割”。人们发现这个比值是如此的美丽宜人，以至于人们经常会以此作为房间或明信片的长宽比。

   用一个递归公式定义一个静态方法，计算出以n作为实参时的第n个裴波那契数。（方法不应该用循环来逐个计算所有的裴波那契数，直到那个期望的裴波那契数为止，而是应该使用一个更好、更简单的递归方法。）将这个静态递归方法放入一个程序中，这个程序展示了裴波那契数的比值是如何收敛的。程序要请用户指定要计算的裴波那契数的数量，然后显示出那些裴波那契数，每行一个。在前两行之后，还要在每行上显示当前裴波那契数和前一个裴波那契数之间的比值。（对前两行上的裴波那契数来说，这个比值没有任何意义。）如果用户输入的是5，输出看起来就应该像下面这样。

   ```
   Fibonacci #1 = 0
   Fibonacci #2 = 1
   Fibonacci #3 = 1 and ratio = 1/1 = 1
   Fibonacci #4 = 2 and ratio = 2/1 = 2
   Fibonacci #5 = 3 and ratio = 3/2 = 1.5
   ```

第5章

使用Swing的视窗界面

“没有图片或对话，”爱丽丝想，
“一本书还能有什么用呢？”

——刘易斯·卡莱尔，《爱丽丝漫游奇境记》

到目前为止，几乎所有的程序使用的都是最简单的输入形式。用户从键盘输入简单的文本，并将简单的、未经修饰的文本传送到屏幕上去作为输出。现代的程序是不会使用这种简单的输入和输出的。

现代的程序使用的是视窗界面，这些界面上带有允许用户通过使用鼠标进行选择的菜单和按钮。在本章和第7章中，将学习如何用一种特殊的、名为**Swing**的类库编写一些Java程序，创建出用于输入和输出的这种现代视窗界面。Swing是一种随Java一同安装的标准Java库。关于Swing可以写出整本的书，而且确实也有这样的书，因此，在这两章中没有足够的空间对Swing进行完整的描述。但是，讲述的内容已足以编写一些简单的视窗界面了。

还有另一个较老的类库可以用来编写视窗界面。这个较老的类库被称为**抽象视窗工具集**（Abstract Windows Toolkit），或简称为**AWT**。可以将Swing看成AWT的一个改进版本。但是，Swing并没有取代AWT。相反，它向AWT中添加了一些内容，生成了一个更丰富的类集合，而AWT仍然是Swing库的一个必备补充。会用到来自Swing和AWT这两个库中的类。

目标

- 了解事件驱动编程的基础知识。
- 学会设计并用代码实现一个包含按钮和文本的简单GUI。（菜单是在本书第7章介绍的。）
- 了解Swing（或与Swing相关的）类BorderLayout、Color、Container、FlowLayout、GridLayout、JButton、JComponent、JFrame、JLabel、JPanel、JTextArea、JTextField、WindowAdapter及其他一些类。

预备知识

在学习本章（以及接下来关于applet和更多Swing知识的两章）内容之前，需要学习《Java程序设计与问题解决：基础篇》的第1章～第5章，以及该书中介绍了继承概念的第7章。除了本章末尾的一个小节之外，要想理解本章的内容，都不需要阅读本章之前的其他章节。

要理解5.5.3节的内容，需要先阅读第1章。如果还没有读过第1章，可以跳过那个小节。

5.1　背景知识

event 名词 1.发生的一件事情、事件或体验，尤其是重要的事件。

——《美国传统英语词典》，（第1版）

以一些基本的背景知识作为开始，这些基本知识涉及任意视窗界面中都要用到的一些元素，以及一种在编写视窗界面时会用到的、名为事件驱动编程的编程技术。

5.1.1　GUI——图形用户界面

与用户进行交互的视窗系统通常被称为GUI。**GUI**读作“gooey”，表示**图形用户界面**（graphical user interface）。这个词本身已经把它的意思解释的很清楚了。它被称为图形（graphical）是因为它使用了窗口、按钮和菜单这样的图形元素。它被称为用户界面（user interface）是因为它是与用户之间存在界面（与用户进行交互）的程序的一部分。GUI获取来自用户的信息，并将其送给程序处理。程序将信息处理好之后，GUI将结果返回给用户，通常会以某种视窗形式返回。

来简单列举一些术语，这些术语是用于那些组建GUI的基本元素的。尽管毫无疑问你以前曾用过所有这些元素，但你用的名字和我们所用的可能不一样。**窗口**（window）是用户屏幕上作为一个较小屏幕使用的区域。窗口通常都有一个用来定义其外围边缘的边界，以及一个用来为窗口起名字的标题。在窗口内部，可能会有一些较小的与窗口类似的对象。菜单就是一种较小的类似窗口的对象。**菜单**（menu）是提供给用户的一系列选项，通常是以名字列表的形式提供的。用户通常会在光标位于菜单条目之上时，点击鼠标按钮（这个过程通常被称为点击菜单条目），从这些选项中选择一个条目。**按钮**（button）与菜单中的条目类似。按钮看起来就像一个可以按压的按钮一样，而且通常会有一个标签。要“按”一个按钮，可以用鼠标在按钮上点击一下。在Swing中对这些元素都有更详细的定义。

快速参考：GUI

与用户进行交互的视窗系统通常被称为**GUI**。GUI读作“gooey”，表示**图形用户界面**（graphical user interface）。

5.1.2　事件驱动编程

Swing程序和大多数其他GUI程序都使用了事件和事件处理程序。图形用户界面中的**事件**（event）就是一个表示某些动作的对象，比如点击鼠标按钮、拖动鼠标、按下键盘上的一个按键、点击窗口上的关闭窗口按钮，或任何其他期望得到响应的动作。在很多上下文中，事件都不仅仅是图形用户界面中的一个动作。例如，一条从打印机传递给操作系统的、表明打印机已准备好打印另一份文档的消息，就可以被当成一个事件。但本章所关心的只是那些在图形用户界面中产生的事件。

一个对象产生一个事件时，就称其**激发**（fire）了那个事件。在Swing中，每个可以激发事件的对象，比如一个可以被点击的按钮，都可以有一个或多个**侦听器对象**（listener object）。

任何能够激发事件的对象的侦听器对象都是由程序员来指定的。例如，如果点击了一个会激发事件的按钮，而且那个按钮有一个与之相关的侦听器对象，那么，这个事件就会被自动地发送到那个侦听器对象中去。侦听器对象中有一些方法，用来说明各种不同类型的事件被送到侦听器中去时会发生什么事情。这些负责处理事件的方法被称为**事件处理程序**（event handler）。这些处理事件的方法是由程序员来定义（或重新定义）的。图5-1以图形方式显示了一个激发事件的对象，比如一个按钮，及其事件处理侦听器之间的关系。

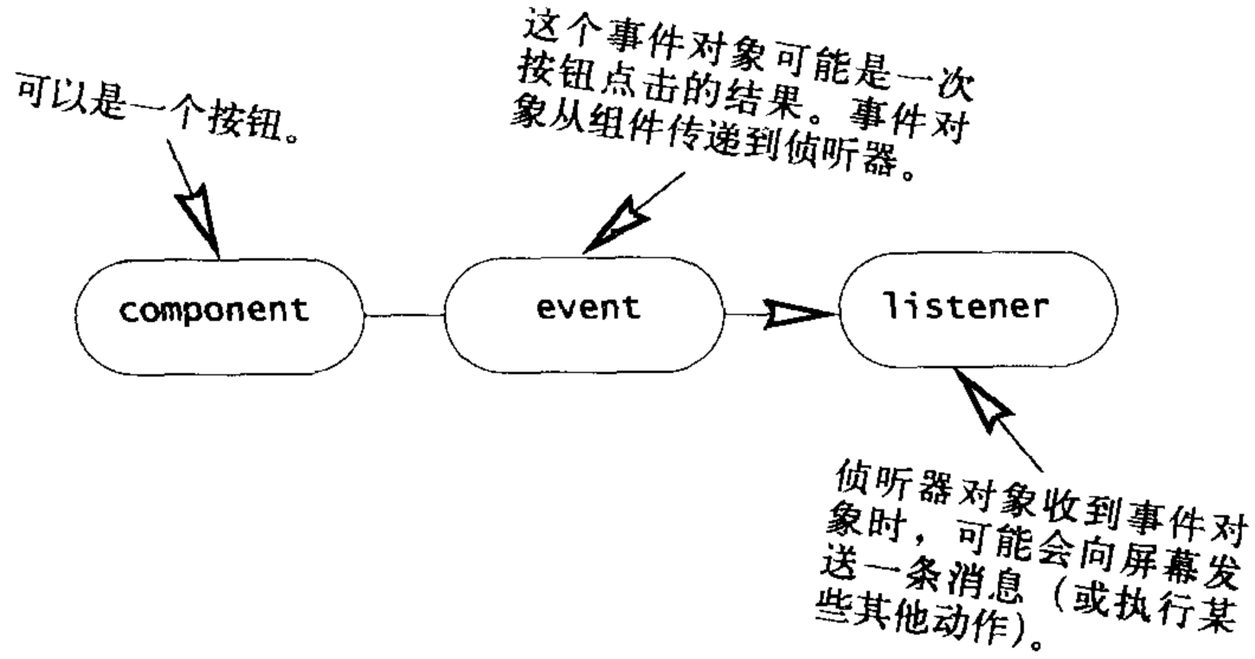


图5-1 事件的激发与事件侦听器

注意，事件驱动编程与到目前为止所见过的编程类型有很大的不同。前面所有的程序都是由一系列按照某种顺序执行的语句组成的。这里所说的执行一系列语句包含了各种不同的形式：循环会重复执行一些语句，分支会从一系列语句中选择将要执行的某条语句，方法调用会引发另一个不同语句列表的执行。但是，从某种程度来说，到目前为止见过的所有程序都设计为由一个代理（计算机）执行，这个代理会遵循一系列形式为“先做这个，再做那个，然后做其他一些事情”的简单指令集来执行。

事件驱动编程是一件完全与之不同的事情。在事件驱动编程中，要创建可以激发事件的对象，还要创建对事件进行响应的侦听器对象。在大多数情况下，程序都无法确定事情发生的顺序。顺序是由事件确定的。运行一个事件驱动程序时，接下来发生的事情取决于下一个事件。

侦听器对象就像一个坐在房间里等电话的人一样。每个人都有自己的电话，电话响时，拥有那部电话的人就会去应答，并完成电话里让他做的事情。可能电话里会说：“乔，我是妈妈。我希望你把自己房间的窗户关上。”然后乔就会回家关上他房间的窗户。在一个图形用户界面中，这类消息包括“关闭窗口”，“'A'键被按下去了”或者“鼠标被从一个地方拖到了另一个地方”。消息就是事件。事件被激活时，会被自动地发送到激发事件的那个特定对象的侦听器对象中去。然后，侦听器对象就会调用适当的事件处理方法来处理这个事件。

如果从来都没有进行过事件驱动编程，你可能会觉得它的一项特性很奇怪：*你要编写一个自己永远也不会在任何程序中调用的方法定义*。这看起来可能很奇怪，因为一个方法如果不被调用是毫无用处的。因此，除了程序员之外的某个人或某样东西必须要去调用这些方法。这正是实际发生的事情。当一个事件说明需要调用那个方法时，Swing系统会自动地调用那些

特定的方法。

在将要用Swing库来进行的事件驱动编程中大量使用了继承。定义的类将是Swing库中一些基本的预定义类的派生类。这些派生类将从它们的父类（基类）那里继承一些方法。其中的部分继承方法用为父类所编写的代码就能工作得很好。但是，通常这些类定义都需要被覆盖，以提供适用于派生类的新定义。

自测题

1. 事件驱动编程与在前面的章节中进行的其他类型的编程有什么不同？
2. “GUI”读作什么？这些字母表示什么意思？

5.2　基本的Swing细节

（如果不摇摆，就）毫无意义。

——歌名，艾灵顿公爵

如前所述，**窗口**是用户屏幕的一部分，是作为屏幕内的一个较小屏幕提供的。本节会讲述怎样用Swing创建简单的窗口。

设计一个GUI时，可以放入一个窗口的东西很多。将以一些简单、但很有用的元素作为开始，展示如何用这些元素来构建窗口。本节中要介绍的窗口元素提供了一种关闭（即，结束）窗口的途径，一种将文本放入窗口中去的途径、一种为窗口着色的途径、以及一种在窗口中放置标题的途径。这些特性听起来好像不怎么样，但它引入了一些用Swing进行各类编程时所需的基本方法。

本节本质上只有一个示例程序，尽管为这个程序提供了二个不同的版本。本节的主要目的是介绍Swing的一些细节，因此我们开始介绍程序时，有意把它简化了。在本节的最后，按照编写Swing程序时应该遵循的风格重写了这个程序。

△ 易犯错误：运行一个Swing程序之前将所有的工作保存起来

使用Swing的程序可以控制计算机的屏幕、鼠标和键盘。如果程序出错，与计算机进行通信的一般途径都可能会被关闭，而程序是经常会出错的。在一台PC机上，遇到这种情况可能要重启计算机。在任何系统中，都可能要求你在某种意义上“重新启动整个过程。”在运行任何还没有完全调试好的Swing程序时，一定要将所有已打开的文件关闭。如果Swing程序使计算机重启的话，任何打开的文件都可能会被破坏。△

编程示例：一个简单的窗口

图5-2中包含了一个用Swing生成简单窗口的Java程序。程序代码之后的图片是运行这个程序时屏幕的样子。这个窗口没做什么事情。它只是出现在屏幕上，并显示了文本"Please don't click that button!"它唯一可以做的另外一件事就是消失。如果点击关闭窗口按钮，程序会终止，窗口也会消失。显示的那张图片是这个程序生成的一个典型窗口示例。在你的系统上，这个窗口看起来可能会稍有不同。来看看这第一个GUI程序的代码。

```
import javax.swing.*;

/**
 A simple demonstration of a window constructed with Swing.
*/
public class FirstSwingDemo
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;

    public static void main(String[] args)
    {
        JFrame myWindow = new JFrame();
        myWindow.setSize(WIDTH, HEIGHT);
        JLabel myLabel =
                    new JLabel("Please don't click that button!");
        myWindow.getContentPane().add(myLabel);

        WindowDestroyer myListener = new WindowDestroyer();
        myWindow.addWindowListener(myListener);

        myWindow.setVisible(true);
    }
}
```

这只是一个简单的演示程序，不是通常会在Swing程序中使用的典型风格。

得到的GUI

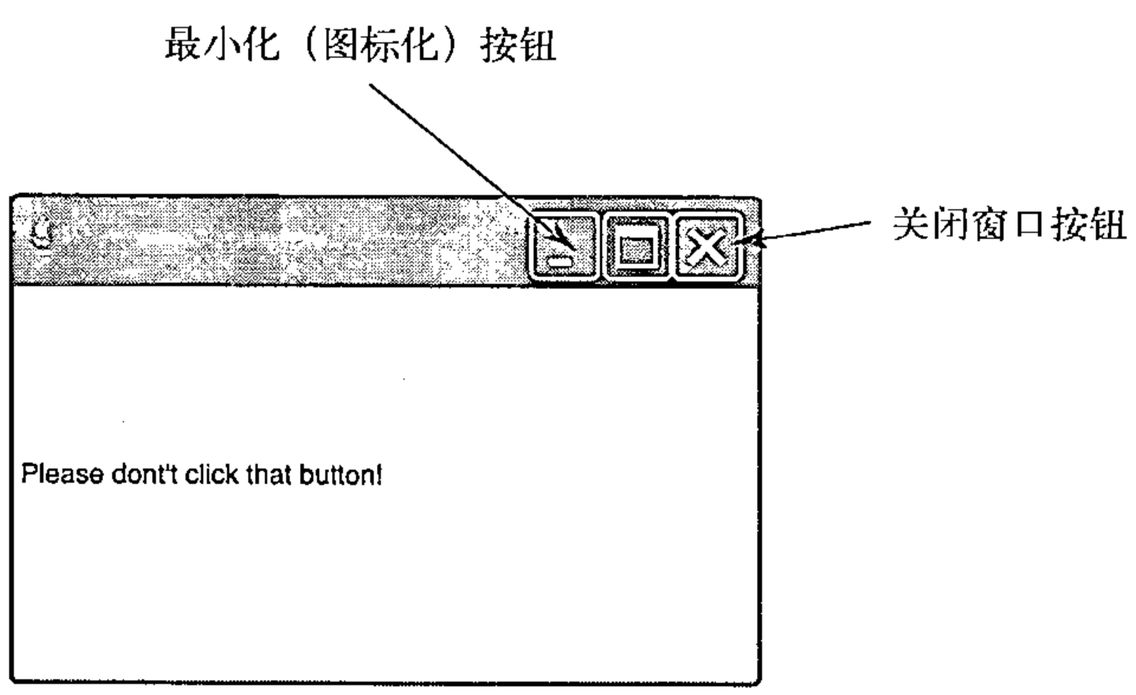


图5-2 一个非常简单的Swing演示程序

下面显示的第一行代码，说明程序使用了Swing库：

```
import javax.swing.*;
```

包含任何使用了Swing库的程序的文件都应该以此行作为开始（可能还有其他的import语句）。

程序的其余部分是个只带有一个main方法的简单类定义。来看看这个main方法中的代码。第一行代码创建了一个JFrame类的对象。这行代码如下所示：

```
JFrame myWindow = new JFrame();
```

myWindow这个名字是由程序员确定的。对象myWindow是JFrame类的对象。Swing中的JFrame就是可能认为是窗口的那个东西。JFrame是个很简单的窗口，但除了其他内容之外，它确实有一个边界、

一个放置标题的位置（这个程序中没有使用标题），以及任意一个窗口都应该有的关闭窗口按钮。很快就会看到，通常不使用简单的JFrame对象，而是定义一个JFrame类的派生类，并使用那个派生类的对象。但是，对这第一个演示程序我们会直接使用JFrame。

接下来的一行代码设置了JFrame窗口的尺寸，如下所示：

```
myWindow.setSize(WIDTH,HEIGHT);
```

方法setSize是JFrame类的一个方法，用来设置窗口的尺寸。

本章稍后的一个小节对Swing的尺寸单位进行了讨论。现在，只要注意到这行代码说明窗口是WIDTH单位宽、HEIGHT单位高就行了，还不需要关心那些单位是什么。

接下来的代码行创建了一个JLable类的对象，并将此对象命名为myLabel：

```
JLabel myLabel = new JLabel("Please don't click that button!");
```

myLabel这个名字是由程序员确定的。JLabel类的对象通常就被称为一个**标签**（label），是可以添加到JFrame（或很多其他类型对象）中去的一个特殊文本类型。用于标签的字符串是作为JLabel类构造器的实参给出的，因此，在这种情况下，标签的文本字符串为"Please don't click that button!"。

程序中的下一行代码将标签myLabel添加到名为myWindow的JFrame中去了，如下所示：

```
myWindow.getContentPane().add(myLabel);
```

需要对这行代码进行一些解释。我们把这个表达式分开来看。方法getContentPane是JFrame类的一个方法，会生成JFrame的**内容面板**（content pane）。每个JFrame都有一个内容面板。不能直接向JFrame添加任何东西。相反，得将内容添加到它的内容面板中去。可以认为内容面板是JFrame的“内部”。因此，myWindow.getContentPane()就是myWindow的内容面板（myWindow的“内部”）。标签myLabel是用方法add添加到myWindow.getContent Pane()（即，myWindow的内容面板）中去的。每个JFrame内容面板都有一个名为add的方法。方法add已经定义好了，你不需要去定义它。

快速参考：JLabel类

JLabel类的对象只不过是一行可以添加到JFrame（或某些其他类）中去的文本罢了。

举例：

```
JFrame myWindow = new JFrame();
JLabel myLabel = new JLabel("Please don't click that button!");
myWindow.getContentPane().add(myLabel);
```

注意，如前面的例子所示，JLabel是通过getContentPane().add添加到JFrame中去的。

可以通过点击关闭窗口按钮来关闭这个窗口。点击那个按钮时，窗口会激发一个事件，并将其发送到一个侦听器对象中去。侦听器对象将窗口关闭。在这个程序中，侦听器对象被命名为myListener，是WindowDestroyer类的一个成员。程序中的下列代码行创建了WindowDestroyer类的一个新对象，并将其命名为myListener：

```
WindowDestroyer myListener = new WindowDestroyer();
```

由程序员负责确定myListener的名字。下一行代码将对象myListener与对象（窗口）myWindow关联起来，这样myListener就会收到对象myWindow激发的任意事件了，代码如下所示：

```
myWindow.addWindowListener(myListener);
```

像myListener这样的、负责从另一个对象接收事件的对象被称为侦听器。负责侦听窗口事件的侦听器被称为**窗口侦听器**（window listener），比如点击关闭窗口按钮这样的事件。将侦听器与其侦听的对象进行关联的动作被称为**注册侦听器**（registering the listener）。

WindowDestroyer类的对象会在myWindow激发一个适当事件的时候，关闭myWindow窗口。需要定义WindowDestroyer类，但稍后再做这件事。现在，假设WindowDestroyer类已经定义好了，这样当用户点击myWindow的关闭窗口按钮时，对象myListener就会关闭myWindow，并结束这个程序。

main中的最后一条语句是对方法setVisible的调用：

```
myWindow.setVisible(true);
```

这个对setVisible的调用会使名为myWindow的窗口出现在屏幕上。方法setVisible是JFrame类的一个方法（也是很多其他Swing类中的方法）。如图5-2所示，如果实参为true，就将对象显示出来。如果使用实参false，对象就不会显示出来。

那是main代码的结束，但并不是程序的结束。窗口只是看起来很漂亮地出现在屏幕上，直到用户点击关闭窗口按钮为止。那时，对象myWindow会激发一个事件e。事件e会被发送到一个名为myListener的侦听器对象中去。myListener识别出e是关闭myWindow的事件信号，然后myListener关闭窗口并结束程序。图5-3以图形方式给出了这个事件的处理过程。

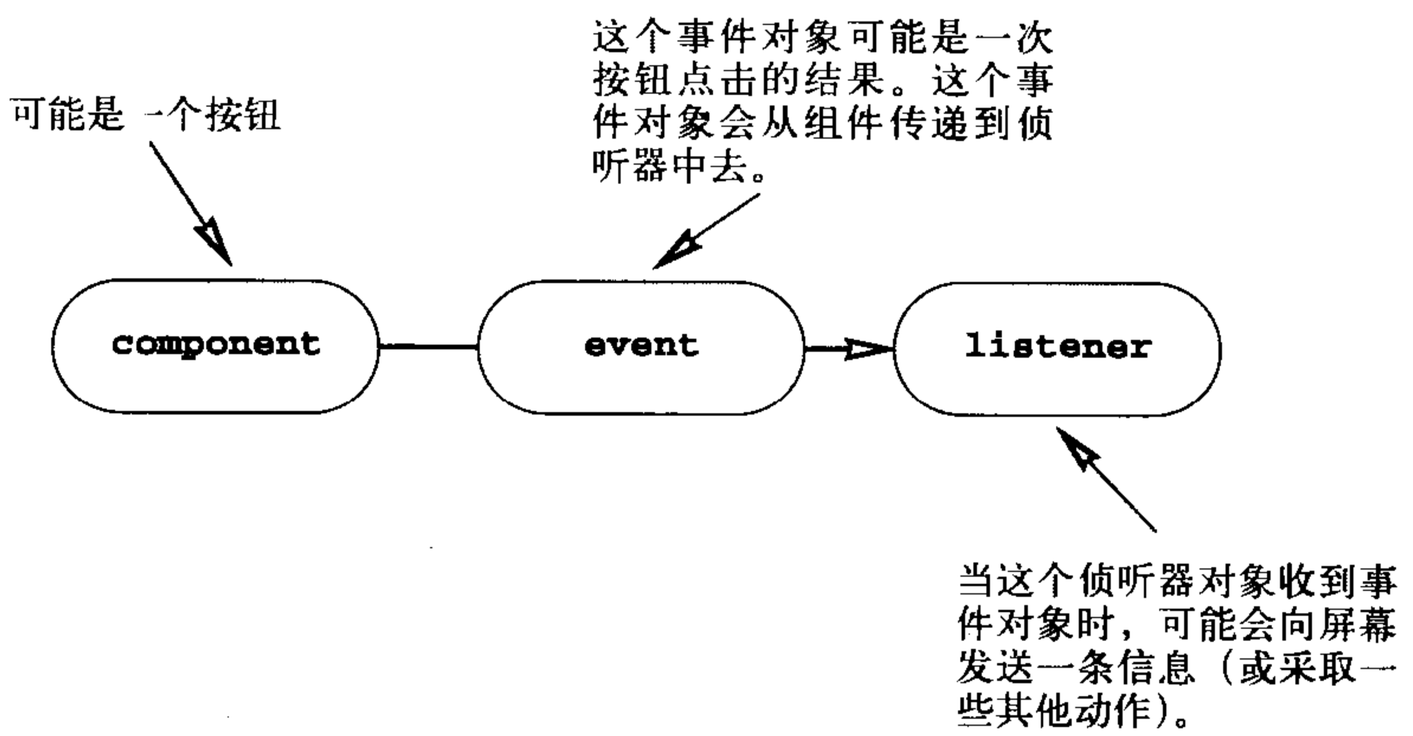


图5-3 点击关闭窗口按钮

对象myListener是WindowDestroyer类的一个成员。假定WindowDestroyer类是已经定义好了的，但在实际运行图5-2中的程序之前，需要对其进行定义和编译。现在开始进行这些操作。

图5-4定义了侦听器类WindowDestroyer。包含窗口（更精确地说，是包含了JFrame类对象）的GUI的窗口侦听器类通常都是WindowAdapter类的派生类。这种情况是由类定义第一行上的短语extends WindowAdapter说明的。WindowAdapter类的派生类，比如WindowDestroyer类，会从WindowAdapter继承它所有的方法。每个这样的方法都会自动地响应一种不同类型的事件。通常，不会添加新的方法，因为每种类型的事件都已经有一个方法了。但是，处理事件的方式是由你根据定义的窗口来决定的。通常，应该对类中的、从WindowAdapter中继承的一个或多个方法进行重新定义（重写）。

```
import java.awt.*;
import java.awt.event.*;

/**
 If you register an object of this class as a listener to any
 object of the class JFrame, then if the user clicks the
 close-window button in the JFrame, the object of this class
 will end the program and close the JFrame.
*/
public class WindowDestroyer extends WindowAdapter
{
    public void windowClosing(WindowEvent e)
    {
        System.exit(0);
    }
}
```

Java中没有定义WindowDestroyer类，这是一个必须由程序员定义的类。

图5-4　窗口事件的侦听器类

只需要定义（实际上，是重新定义）窗口会用到的那些方法。对这个应用，需要侦听器类响应的唯一一个事件就是示意窗口应该关闭的事件。处理这些事件的方法名为windowClosing。因此，只要重新定义方法windowClosing就行了。方法定义非常简单，它只执行了一条命令：

```
System.exit(0);
```

正如在后面的Java提示中解释的那样，这条命令终止了程序，也就关闭了窗口。

要运行图5-2中的程序，就必须定义并编译图5-2中的FirstSwingDemo类和图5-4中的WindowDestroyer类。注意，WindowDestroyer类不是一个Swing类。它是一个必须由程序员定义的类。在这个例子中，我们已经将其定义好了，但你应该把它当成一个需要自己定义的类。

注意，WindowDestroyer类所需的import语句如下所示：

```
import java.awt.*;
import java.awt.event.*;
```

这些导入语句只是告诉编译器WindowAdapter和事件处理程序的定义位于什么地方。（可以回想一下，前面说过AWT是Swing库的前身。WindowAdapter类和事件处理程序模型在AWT中，这就是import语句中有awt的原因。）

常见问题：其他内建的JFrame按钮是如何工作的？

JFrame中的关闭窗口按钮是由三个按钮组成的按钮组中的一个。对其他两个按钮来说，不用对它们编程，它们就基本上可以工作了。图5-2中显示的最小化（图标化）按钮将窗口缩小为一个图标，这个图标通常都位于屏幕的底端。另一个按钮允许将窗口（JFrame）的尺寸在全屏和一个较小窗口之间来回转换。

□ Java提示：结束一个Swing程序

GUI程序通常是基于某种无限循环的。它可能有，也可能没有Java循环语句，但无论如何，GUI程序总是不需要结束的。视窗程序通常都停留在屏幕上，直到用户表明（比如，通过点击关闭窗口按钮来表明）它应该退出为止。如果用户没有请视窗程序退出，它就永远都不会退出。因此，在（用Swing）编写一个GUI程序时，需要用某种方式说明“现在将程序结束。”只要一执行《Java程序设计与问题解决：基础篇》第3章介绍的exit方法，就会结束一个Java程序。图5-4中使用了下列对方法exit的调用：

```
System.exit(0);
```

正如在《Java程序设计与问题解决：基础篇》的第3章所说的那样，作为System.exit实参使用的数字0会被返回给操作系统。在很多情况下，都可以使用任意的数字，程序会有相同的表现。但大多数操作系统都用0来说明程序的正常终止，用1来说明程序的非正常终止。

△ 易犯错误：忘记对关闭窗口按钮编程

图5-2中的下列代码行确保了在用户点击关闭窗口按钮时，程序会终止，窗口会退出：

```
WindowDestroyer myListener = new WindowDestroyer();
myWindow.addWindowListener(myListener);
```

不需要完全按照这些代码行对关闭窗口按钮进行编程。比如，可以定义一个自己的类来取代WindowDestroyer。但是，确实需要进行编程，以确保用户点击关闭窗口按钮时，GUI会按照希望的那样做。如果没有对关闭窗口按钮进行编程，那么用户点击这个按钮时，窗口会消失，但程序不会终止。如果GUI只有一个窗口，这种情况就意味着没什么简便的方法来终止这个程序了。还有一点让人困惑的是，虽然程序还在运行，但窗口已经消失了，所以看起来好像程序已经结束了一样。 △

△ 易犯错误：忘了使用getContentPane

回想一下，在图5-2中，通过下列方式将标签myLabel添加到名为myWindow的JFrame中去了。

```
myWindow.getContentPane().add(myLabel);
```

因为是在“将myLabel添加到myWindow中去”，所以，可以尝试使用下列语句。

```
myWindow.add(myLabel);
```

但是，如果省略了getContentPane()，程序就无法正确地工作。而且，编译器很可能不会警告这个错误，因此，要想避免这个问题，就得非常小心。 △

5.2.1 更多关于窗口侦听器的内容

正如图5-4中名为WindowDestroyer的窗口侦听器类一样，任何窗口侦听器类通常都是WindowAdapter类的派生类。WindowAdapter类有很多不同的方法，侦听器对象收到一个与某方法相匹配的事件时，会自动地调用那个方法。图5-5中给出了这些方法及相应的事件。在定义WindowAdapter类的派生类时，定义的只是需要那些方法。WindowDestroyer类唯一的任务就是关闭窗口，因此，定义WindowDestroyer时，只定义了方法windowClosing。（除了windowClosing之外，不需要任何方法，但从完整性角度考虑，图5-5列出了所有的方法。）

```
public void windowOpened(WindowEvent e)
```
在窗口打开时调用。

```
public void windowClosing(WindowEvent e)
```
在窗口被关闭的过程中调用。点击关闭窗口按钮会引发对这个方法的调用。

```
public void windowClosed(WindowEvent e)
```
在窗口被关闭时调用。

```
public void windowIconified(WindowEvent e)
```
在窗口被图标化时调用。点击JFrame上的最小化按钮时，窗口就被图标化了。最小化（图标化）按钮的位置请参见图5-2。

```
public void windowDeiconified(WindowEvent e)
```
在窗口解除图标化时调用。激活一个最小化窗口时，它就被解除了图标化。

```
public void windowActivated(WindowEvent e)
```
在窗口被激活时调用。点击一个窗口，它就成为活动窗口了。其他动作也可以激活窗口。

```
public void windowDeactivated(WindowEvent e)
```
在窗口撤销激活时调用。任意一个窗口处于激活状态时，所有其他的窗口都是处于非激活状态的。其他动作也可以撤销对一个窗口的激活。

```
public void windowGainedFocus(WindowEvent e)
```
在窗口获得焦点时使用。(本书未讨论焦点的问题。)

```
public void windowLostFocus(WindowEvent e)
```
在窗口失去焦点时调用。(本书未讨论焦点的问题。)

```
public void windowStateChanged(WindowEvent e)
```
在窗口改变状态时调用。

图5-5　WindowAdapter类中的方法

在讨论图5-4中的WindowDestroyer类定义时，说重新定义（redefine）了而不是说定义（define）了方法windowClosing，可能要更恰当一些。这是因为是在修改windowClosing的定义。如果在一个从WindowAdapter类派生出来的类的定义中，没有给出图5-5中某个方法的定义，那么，这个类就会继承WindowAdapter类中的定义。但是，如果继承了一个方法，而没有重新定义它，它是不会完成希望的任务的。只有在某个派生类，比如在WindowDestroyer类中，对其进行了重写，它们才会成为有用的方法。

WindowAdapter类是一个抽象类，这就意味着无法用new WindowAdapter()来创建这个类的对象。可以在定义其他类时将WindowAdapter作为一个基类。抽象类是在《Java程序设计与问题解决：基础篇》的第7章中讨论的。

在本书第7章中有更多关于WindowAdapter的讨论及相关话题。

5.2.2　屏幕对象的尺寸单位

在Swing中，屏幕上对象的尺寸是以像素为单位测量的。像素（pixel）是屏幕上可以书写的最小空间单位。把一个像素当成一个小长方形，这个长方形可以是数量固定的几种颜色之一，可以把屏幕看成是由这些小像素铺满的。(尽管现在屏幕不仅仅提供了黑色和白色，但是，把屏幕当成一个像素不是黑色就是白色的简单黑白屏，是会有所帮助的。) 屏幕上的像素越多，

屏幕的分辨率就越高。也就是说，像素越多，看到的细节就越精细。

像素的尺寸取决于屏幕的尺寸和屏幕的分辨率。尽管Swing使用像素时好像把它当成了长度单位，但实际上它们并不代表任何固定的长度。像素的长度会随着屏幕的不同而有所变化。在一个高分辨率（有很多像素）的屏幕上，一个300乘200尺寸的对象看起来会很小。在一个分辨率较低（没有太多像素）的屏幕上，一个300乘200尺寸的对象看起来就很大了。例如，考虑下列来自图5-2中的语句：

```
myWindow.setSize(WIDTH, HEIGHT);
```

这条语句等价于

```
myWindow.setSize(300, 200);
```

这说明对象myWindow（碰巧是一种窗口）会有300像素宽，200像素高，但实际尺寸要取决于运行程序时所用屏幕的分辨率。

注意，尽管Java和Swing都是可移植的，用Swing编写的代码可以在任何支持Java的系统上运行，但在屏幕上生成的确切尺寸会随显示器的不同而有所不同。这是Java的一个不太理想的可移植特性。要获得期望的窗口尺寸，可能需要对窗口尺寸进行修改以适应特定的屏幕尺寸和屏幕分辨率。但是，只有绝对尺寸是会变化的。不管在哪儿运行Swing应用程序，至少物体的相对尺寸会是相同的。而且，对大多数系统中的大部分类似窗口的对象来说，窗口显示出来之后，用户都可以用鼠标来调整窗口的大小。

常见问题：什么是像素？

像素是在屏幕上可以书写的最小空间单位。对Swing来说，屏幕上对象的尺寸和位置都是以像素为单位来测量的。屏幕上的像素越多，屏幕的分辨率就越高。因此，对于一个屏幕对象来说，在高分辨率屏幕上看起来会比较小，而在低分辨率屏幕上看起来则会比较大。

记住：分辨率与尺寸的关系

分辨率和尺寸的关系开始看起来可能会有些混乱。高分辨率屏幕比低分辨率屏幕的质量要好，为什么在高分辨率屏幕上的对象看起来更小呢？不是越大越好吗？这不是那么简单的。如果需要完整的解释，就得去数数那些像素了，但有一种方法可以使其看起来更合理一些：如果屏幕的分辨率低，就看不到那些较小的东西了。因此，当屏幕分辨率较低时，显示一个小物体的唯一方法就是将其放大。

5.2.3 更多关于setVisible的内容

考虑一下图5-2中的程序调用的方法setVisible。图5-2中包含这个方法调用的特定代码行如下所示：

```
myWindow.setVisible(true);
```

Swing对象的很多类中都有setVisible方法。

方法setVisible有一个boolean类型的实参。换句话说，setVisible的实参或者为true，或者为false。如果w是一个对象，比如一个可以显示在屏幕上的窗口，那么调用

```
w.setVisible(true);
```

就会使w成为可见的，而调用

```
w.setVisible(false);
```

就会使w成为不可见的。

你可能会认为在屏幕上显示一个对象应该是自动发生的事情。毕竟，如果不想把视窗对象显示在屏幕上，又怎么会去定义它呢？答案是你可能不希望一直将其显示出来。毫无疑问，你肯定用过那些或者因为不再需要某些窗口（比如做过选择之后的下拉菜单），或者因为某些窗口被其他窗口盖住了，而出现窗口打开、关闭现象的视窗系统。Swing无法读入程序员的思想以决定什么时候应该显示窗口（或其他GUI对象），因此，程序员必须告诉系统什么时候显示窗口。程序员是通过插入对方法setVisible的调用来完成这项任务的。

再次运行图5-2中的程序，但略去对setVisible的调用，这样在屏幕上就什么都看不见了。窗口会被构建起来，但不会被显示出来。（警告：因为窗口没有被显示出来，那么关闭窗口的按钮也就不会显示出来，所以，就没办法终止程序了！）

快速参考：setVisible方法

Swing对象的很多类都有一个setVisible方法。这个setVisible方法有一个boolean类型的实参。如果w是一个可以显示在屏幕上的对象，比如一个窗口，那么调用：

```
w.setVisible(true);
```

就会使w成为可见的。调用

```
w.setVisible(false);
```

就会将w隐藏起来，也就是说，使w成为不可见的。

语法（调用setVisible方法）：

Object_For_Screen.setVisible(*Boolean_Expression*);

举例（来自图5-2）：

```
pubic static void main(String[] args)
{
      JFrame myWindow = new JFrame();
            .
            .
            .
      myWindow.setVisible(true);
}
```

自测题

3. 对图5-2中的程序进行怎样的修改，才能使窗口中的文本显示为"I love you!"而不是"Please don't click that button!"？
4. 图5-2程序的main方法中出现的下面这条对setSize的调用使用的是什么测量单位？

   ```
   myWindow.setSize(WIDTH, HEIGHT);
   ```

   在程序中，这行代码等价于

   ```
   myWindow.setSize(300, 200);
   ```

   换句话来说，300什么？英寸？英尺？厘米？类似的，200什么？
5. 通常会用哪个Swing类来定义一个窗口？定义的任意窗口类通常都是这个类的对象。
6. 给出一条执行时会立即结束程序的Java语句。

7. 给出一条能够将标签superLabel添加到名为myGUI的JFrame中去的Java语句。
8. 假设messageWindow是一个JFrame，而n是具有某个值的int变量。给出一条Java语句，在n为正时使messageWindow可见，否则就使其不可见。
9. 点击图5-2显示的JFrame上的最小化（图标化）按钮时，会发生什么事情？
10. 假设messageWindow是一个JFrame，而warningLabel是一个JLabel。下列语句有什么问题吗？

```
messageWindow.add(warningLabel);
```

编程示例：比第一个Swing程序更好的版本

图5-6和图5-7合在一起是对图5-2中演示程序的重写。这个新版本所做的事情和图5-2中的程序所做的事情本质上是一样的，只是新版本中显示了两个而不是一个窗口。但是，在编写自己的GUI时应该遵循新版本的书写风格。注意（图5-6中）窗口的定义自身就位于一个类中（不是故意使用的双关语）。然后，将窗口显示在一个使用那个类的程序（图5-7）中。图5-6中FirstWindow这样的窗口类通常都是可以在很多程序中使用的输入/输出接口类。现在更仔细地观察一下这个类。

```
import javax.swing.*;

/**
 A simple window class.
*/
public class FirstWindow extends JFrame
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;

    public FirstWindow()
    {
        super();

        setSize(WIDTH, HEIGHT);
        JLabel myLabel = new JLabel("Please don't click that button!");
        getContentPane().add(myLabel);

        WindowDestroyer listener = new WindowDestroyer();
        addWindowListener(listener);
    }
}
```

图5-6 一个Swing窗口类

观察发现，FirstWindow是JFrame类的派生类。这种方式是定义一个窗口界面的最常用方式。基类JFrame给出了一些基本的窗口工具，然后可以在派生类中添加任何希望在窗口界面中使用的附加特性。

注意，图5-6中的构造器是通过下列代码行调用父类JFrame的构造器作为开始的。

```
super();
```

正如在《Java程序设计与问题解决：基础篇》第7章中所述，这条语句确保了通常要对JFrame类型的所有对象进行的任何初始化动作实际上都会被完成。如果调用的基类构造器没有实参，那么无论是否包含super();，它都会被自动地调用，因此可以省略图5-6中对super()的调用。但是，如果基类构造器像

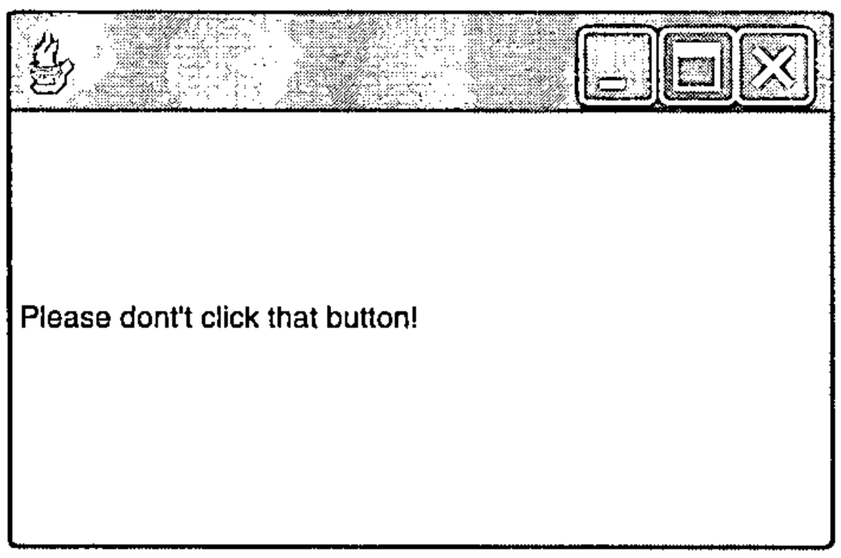
在其他一些情况下那样，需要一个实参，就必须包含对基类构造器super的调用。

```
import javax.swing.*;

/**
 A simple demonstration of the use of a window class. To see
 both windows you will probably have to move the top window.
*/
public class FirstWindowDemo
{
    public static void main(String[] args)
    {
        FirstWindow window1 = new FirstWindow();
        window1.setVisible(true);

        FirstWindow window2 = new FirstWindow();
        window2.setVisible(true);
    }
}
```

得到的GUI



如果运行这个程序时看起来好像只有一个窗口，就移动一下那个窗口。可能一个窗口覆盖在另一个窗口上面了。

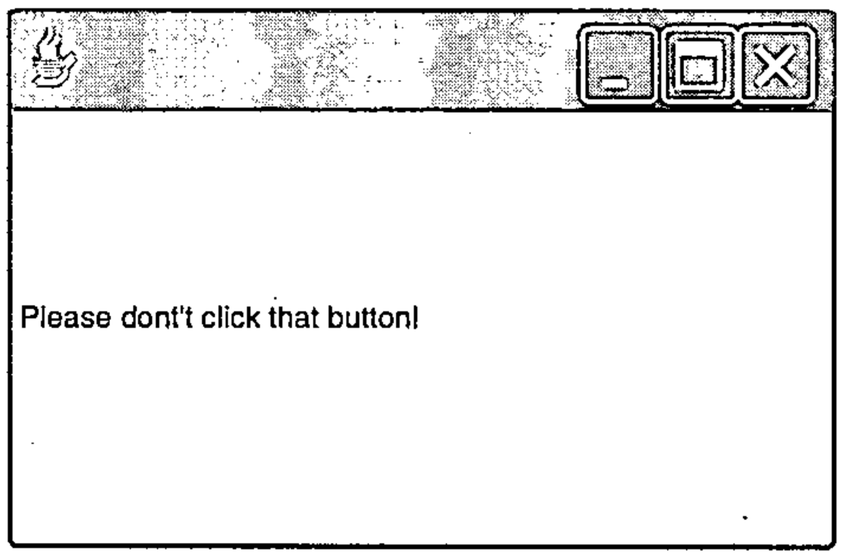


图5-7　一个使用FirstWindow类的程序

注意，几乎所有对图5-6中窗口FirstWindow的初始化动作都被放在那个类的构造器中。事情本来就应该是这样的。设置初始窗口尺寸这样的初始化动作应该是类定义的一部分，而不应该是类对象执行的动作（就像图5-2中那样）。所有的初始化方法，比如setSize、getContentPane和addWindowListener都是从JFrame类继承来的。因为它们是在窗口的构造器中调用的，窗口自身就是调用对象。换句话说，下面的这条方法调用

```
setSize(WIDTH, HEIGHT);
```

等价于

```
this.setSize(WIDTH, HEIGHT);
```

与之类似，方法调用

```
getContentPane().add(myLabel);
```

等价于

```
this.getContentPane().add(myLabel);
```

快速参考：向JFrame添加条目

可以按下列形式向JFrame中添加JLabel（稍后将看到，其他条目也可以用同样的方法添加）。

语法（构造器内部）：

```
getContentPane().add(JLabel);
```

举例（在一个名为FirstWindow的类的构造器内部）：

```
public FirstWindow()
{
        .
        .
        .
    JLabel myLabel =
            new JLabel("Please don't click that button!");
    getContentPane().add(myLabel);
        .
        .
        .
}
```

除了是在一个派生类的构造器中实现的之外，图5-6中FirstWindow定义的细节本质上与图5-2中的定义是一样的。

接下来考虑一下图5-7中那个使用了FirstWindow类的程序。这个程序做的所有工作就是显示了两个（相同的）FirstWindow类的对象。这两个窗口很可能一个正好显示在另一个的上面。因此在运行程序时，可能看起来好像只显示了一个窗口。但是，如果用鼠标移开上面的窗口，就会看到它下面的第二个窗口了。

注意，几乎所有对图5-7中窗口的初始化工作都被移到FirstWindow类的构造器中去了。但是，将对setVisible方法的调用放在了使用窗口类FirstWindow的应用程序中。也可以将对setVisible的调用放在FirstWindow的构造器中，并省略（图5-7的）应用程序中对setVisible的调用。如果这样做，运行这个应用程序时会得到同样的结果。但在通常情况下，应用程序“知道”应该在什么时候显示窗口，因此把setVisible方法调用放在应用程序中也很常见。编写FirstWindow类的程序员无法预测使用这个窗口的程序员会在什么时候使其成为可见（或不可见）的。

快速参考：JFrame类

说一个类是JFrame类，实际上是说那个类是JFrame类的一个子孙类。例如，图5-6中的FirstWindow类就是一个JFrame类。

编程示例：一个带颜色的窗口

图5-8中包含的是图5-6显示的那个类的变体。这个版本有两个构造器。默认构造器与图5-6中的相同，但添加了四个新元素。新元素包括标题"Second Window"，一个名为contentPane的、用来装载JFrame内容面板的局部变量，一个背景颜色（蓝色[1]）和一种添加窗口侦听器的新途径。我们一个一个地来看看这四个新元素。现在，只考虑这个默认构造器（就是不带实参的那个）。

```
import javax.swing.*;
import java.awt.*; //needed for the Color class

public class SecondWindow extends JFrame
{
    public static final int WIDTH = 200;
    public static final int HEIGHT = 200;

    public SecondWindow()
    {
        super();

        setSize(WIDTH, HEIGHT);

        Container contentPane = getContentPane();
        JLabel label = new JLabel("Now available in color!");
        contentPane.add(label);

        setTitle("Second Window");
        contentPane.setBackground(Color.BLUE);

        addWindowListener(new WindowDestroyer());
    }

    public SecondWindow(Color customColor)
    {
        super();

        setSize(WIDTH, HEIGHT);

        Container contentPane = getContentPane();
        JLabel label = new JLabel("Now available in color!");
        contentPane.add(label);

        setTitle("Second Window");
        contentPane.setBackground(customColor);

        addWindowListener(new WindowDestroyer());
    }
}
```

如果愿意，可以省略这个对super的调用。由于是默认构造器，所以无论是否调用它，它都会被自动调用的。

如果愿意，可以省略这个对super的调用。

图5-8　另一个用Swing构建的简单窗口

通过下列方法调用为窗口赋一个标题。

```
setTitle("Second Window");
```

方法setTitle是从JFrame类中继承来的。方法setTitle将一个字符串作为实参，并将那个字符串写到窗口的标题栏中去。这个调用没有调用对象，因为它位于构造器中，调用对象就是隐式的this。前

① 图5-10中以深灰表示蓝色。

面的那个调用等价于：

```
this.setTitle("Second Window");
```

构造器中的其他方法调用也有一个隐式的this。

注意图5-8的第一个构造器中的下列代码行：

```
Container contentPane = getContentPane();
```

这行代码为定义的JFrame视窗GUI的内容面板起了一个名字contentPane。这样，就可以用一种更简单的形式来调用方法add了。

```
contentPane.add(label);
```

而不使用这种更复杂（效率也更低）的表达方式。

```
getContentPane().add(label);
```

这里需要注意的一个重要问题就是方法getContentPane生成了一个Container类型的对象。本章稍后我们将对Container类进行一些更多的介绍。现在，关于这个类需要知道的所有内容就是：这是方法getContentPane返回的对象（也就是JFrame的内容面板）所用的类型。

常见问题：为什么JFrame有内容面板？

遗憾的是，为什么JFrame有内容面板这个问题没有一个简单的答案。这个问题与使用JFrame对象的方式有关，本书不对此进行深入的讨论。如果觉得好像没有必要使用内容面板，这种想法在目前也说得过去。在本书中所作的工作是不需要它的。实际上，JFrame类的祖先类（位于一个更老的类库中）并没有内容面板。但是，JFrame对象确实有内容面板，而且你必须与这个内容面板打交道，否则程序就无法正确地工作。

在图5-8中，还用下列方法调用给出了窗口的背景颜色：

```
contentPane.setBackground(Color.BLUE);
```

方法setBackground是从JFrame类继承的另一个方法。方法setBackground有一个颜色实参。Color类中包含了很多用于常见颜色的常量。Color.BLUE是个表示蓝颜色的预定义常量。图5-9列出了可用的颜色常量。要看看每种颜色是什么样的，可以用想看的颜色取代图5-8中的常量Color.BLUE，然后对修改过的程序进行编译，并运行。也可以用Color类定义自己的颜色，但本书不对此话题进行讨论。因为Color类位于AWT包（库）中，所以，当一个类或程序使用Color类时，需要使用下列导入语句：

```
import java.awt.*; //needed for the Color class
```

Color.BLACK	Color.MAGENTA
Color.BLUE	Color.ORANGE
Color.CYAN	Color.PINK
Color.DARK_GRAY	Color.RED
Color.GRAY	Color.WHITE
Color.GREEN	Color.YELLOW
Color.LIGHT_GRAY	

Color类位于AWT包（库）中。因此，使用这些颜色时，需要下列import语句。

```
import java.awt.*;
```

图5-9 颜色常量

常见问题：青色（cyan）是种什么颜色？

可能会奇怪，为什么Java库的设计者会选择为图5-2中的那些颜色定义常量。当然，黑色、白色、红色、绿色和大部分其他颜色都是常见的颜色，但是为什么选中了青色和品红色呢？答案是：将黑色、黄色、青色和品红色混合起来是生成颜色的一种常见方式。因此，从某种意义上来说，青色和品红确实是“基本颜色”。

青色是一种很淡的蓝。品红是紫色的一种。另一种混合颜色的常见方式是将红色、绿色和蓝色混合起来。电视机使用的就是红色、绿色和蓝色。印刷成彩色的书使用的是黑色、黄色、青色和品红色。

在第一个构造器定义的末尾，通过下列对方法addWindowListener的调用为窗口添加了一个侦听器。

```
addWindowListener(new WindowDestroyer());
```

某些方法，比如addWindowListener，需要将对象作为实参，但是，只要将给出的对象作为一个实参赋给了方法，就再也不需要在编程中引用它了。这就意味着，不需要为实参起名字了。像前面的addWindowListener方法调用中new WindowDestroyer()这样的实参，就是一种创建对象并将其作为实参传递给方法addWindowListener，而不需要为实参起名字的一种方法。

调用

```
addWindowListener(new WindowDestroyer());
```

等价于下列代码：

```
WindowDestroyer listener = new WindowDestroyer();
addWindowListener(listener);
```

前面两种添加窗口侦听器的方式之间唯一的区别就是：在第一种形式中，不需要为WindowDestroyer类型的对象起名字，而在第二种形式那个两行的版本中，给侦听器对象起了个名字listener。

记住：将new作为方法实参的一部分

可以创建一个对象，并将其作为实参传递给一个方法，而不用为那个对象起名字。用new和类名作为实参就可以实现这一点。在这些情况下，类名是作为类的构造器名使用的。

语法：

Method_Name(new *Class_Name_As_Constructor*(*Possibly_Parameters*));

举例：

```
addWindowListener(new WindowDestroyer());
```

因为实参new WindowDestroyer()没有名字，通常会被称为**匿名对象**（anonymous object）。

现在，来看看第二个构造器。除了处理背景颜色的方式之外，它与默认构造器基本一样。这第二个构造器有一个Color类型的参数，并将背景颜色设置为这个参数指定的颜色。看看图5-10中的演示程序，就可以很容易地看出这两个构造器之间的区别。除了一个窗口是蓝色背景，一个窗口是粉色背景之外，生成的两个窗口是一样的。（就像前面那个例子一样，运行图5-10中的程序时，一个窗口很可能会在另一个窗口之上，这样，看起来就好像只有一个窗口似的。用鼠标移开上面的窗口，就会看到另一个窗口了。）

```
import java.awt.*; //for the class Color used in an argument.

public class SecondWindowDemo
{
    /**
     Creates and displays two windows of the class SecondWindow.
    */
    public static void main(String[] args)
    {
        SecondWindow window1 = new SecondWindow();
        window1.setVisible(true);

        SecondWindow window2 = new SecondWindow(Color.PINK);
        window2.setVisible(true);
    }
}
```

得到的GUI

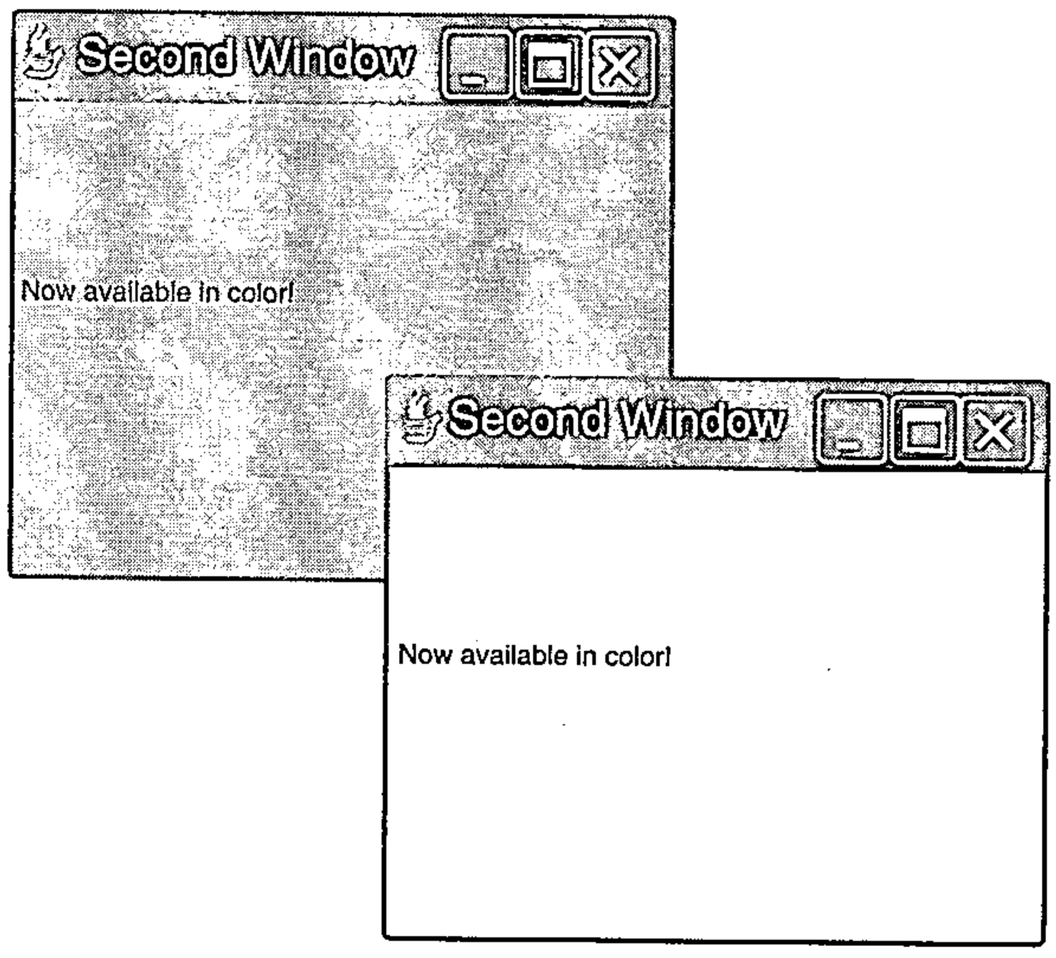


图5-10 SecondWindow的演示程序

5.2.4 JFrame类的一些方法

图5-11中包含了JFrame类的一些方法。回想一下，JFrame是构建窗口时的常用基类。窗口类通常是JFrame类的派生类，因此，窗口类会继承所有这些方法。

记住：使用哪些import语句

记录一个Swing类定义或Swing程序需要哪条import语句是相当困难的。下列import语句对大多数Swing窗口界面来说已经足够了：

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
```

包含一条额外的import语句造成的效率损失是很少的，因此，不需要费劲儿去使import语句数量保持为最少。其实本书的类定义中就只使用了那些必需的import语句。

方　法	描　述
JFrame()	用来创建一个新的JFrame的构造器
JFrame(String title)	用给定的title作为窗口标题创建一个新JFrame的构造器
add	JFrame有一个add方法，但是不应该使用它。(它继承自一个祖先类，现在已经没有用处了。)要向JFrame添加某些东西，可以使用getContentPane().add(*Item_Added*)
Void addWindowListener(WindowListener ear)	将ear注册为由JFrame激发的事件的侦听器。
Container getContentPane()	返回JFrame的内容面板对象。注意，返回的内容面板是Container类型的。
void setBackground(Color c)	将背景颜色设置为c。
void setForeground(Color c)	将前景颜色设置为c。
void setSize(int width, int height)	重新将窗口尺寸设置为指定的宽度和高度。
void setTitle(String title)	在窗口的标题栏中显示title。
void setVisible(boolean b)	如果实参为true，就使窗口可见。如果实参为false，就使其不可见。

图5-11　JFrame类中的一些方法

自测题

11. 对图5-8中的类定义进行怎样的修改，才能使默认构造器生成的窗口是黄色而不是蓝色的?
12. 如何对用Swing创建的窗口的标题进行设置?
13. 假设用下列代码声明并创建了一个MyWindowClass类的窗口对象：

```
MyWindowClass gui = new MyWindowClass();
```

窗口gui在屏幕上还不是可见的。给出一条程序语句使其成为可见的。

14. 重新编写下列代码，不要为对象listener起名字：

```
WindowDestroyer listener = new WindowDestroyer();
addWindowListener(listener);
```

15. 假设w是一个JFrame类的对象。给出一条Java语句，声明一个名为contentPane的变量，并对其进行设置，用它来命名w的内容面板。
16. 要想使用Container类型，需要哪条import语句?

5.2.5　布局管理器

已经看到，可以通过方法getContentPane获得JFrame的内容面板，然后用方法add将JLabel添加到内容面板中去，这样就可以将JLabel添加到JFrame中去了。例子请参看图5-8。如果只添加一个标签，标签放在哪儿好像都没什么问题，但是如果要添加两个或多个标签呢？如何排列这些标签呢？一个在另一个上面？一个挨着另一个？哪个是第一个，哪个是第二个呢？这种排列过程是由一种名为**布局管理器**（layout manager）的特殊对象实现的。布局管理器会根据一些特定的规则来排列添加的那些条目。不同的布局管理器遵循不同的规则。下一个例子向JFrame添加了三个标签，并用一个布局管理器来管理这三个标签。

图5-12包含了一个类的实例，这个类创建了一个带有三个标签的窗口。在这个例子中使用了一个布局管理器，它将这些标签放在三行上，一个在另一个的下面，而没有像其他的方案那样将所有的标签都放在一行上。来看看其中的一些细节。

```
import javax.swing.*;
import java.awt.*;

/**
 Simple demonstration of the use of a layout manager
 to arrange labels.
*/
public class BorderLayoutDemo extends JFrame
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;

    /**
     Creates and displays a window of the class BorderLayoutDemo.
    */
    public static void main(String[] args)
    {
        BorderLayoutDemo gui = new BorderLayoutDemo();
        gui.setVisible(true);
    }

    public BorderLayoutDemo()
    {
        setSize(WIDTH, HEIGHT);
        addWindowListener(new WindowDestroyer());
        setTitle("Layout Demonstration");
        Container content = getContentPane();

        content.setLayout(new BorderLayout());

        JLabel label1 = new JLabel("First label here.");
        content.add(label1, BorderLayout.NORTH);

        JLabel label2 = new JLabel("Second label there.");
        content.add(label2, BorderLayout.SOUTH);

        JLabel label3 = new JLabel("Third label anywhere.");
        content.add(label3, BorderLayout.CENTER);
    }
}
```

得到的GUI

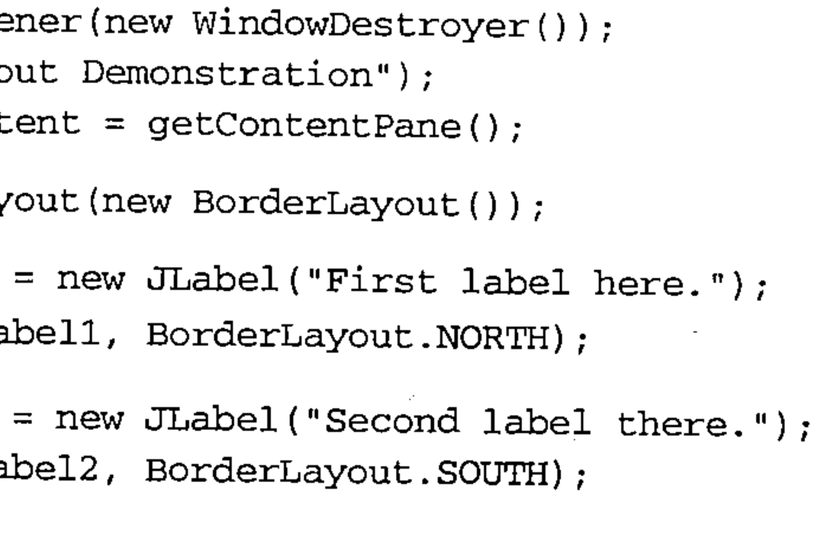


图5-12 使用BorderLayout管理器

首先，要注意到在对Swing的讨论中还未见过的（尽管曾在其他类类型中见过——比如，在《Java程序设计与问题解决：基础篇》第5章的图5-7中）、很小的一个问题。在类定义中放置了一个演示main方法。通常，Swing GUI类是用来在除Swing GUI类之外的其他类的main方法（或其他方法）中创建并显示一个GUI的。但是，将main方法放在GUI类定义中，以方便地显示这个GUI的示例，这是完全合法，而且有时是很方便的。注意，在类自身内部的main方法的编写与位于其他类中的main方法的编写方式是一样的。尤其是，需要像来自图5-12 main方法中的下列代码行一样，构建一个类对象：

```
BorderLayoutDemo gui = new BorderLayoutDemo();
```

现在，来讨论图5-12中一些全新的东西。

布局管理器是通过下列代码行添加到图5-12的GUI中去的：

```
content.setLayout(new BorderLayout());
```

BorderLayout是一个布局管理器类，因此new BorderLayout()就会生成一个新的BorderLayout类对象。这个BorderLayout对象的任务就是对添加到GUI中去的组件（在这个例子中，就是一些标签）进行排列。注意到前面对setLayout的调用与下面两行代码等价，是会有帮助的。

```
BorderLayout manager = new BorderLayout();
content.setLayout(manager);
```

注意，方法setLayout不是由JFrame自身，而是由JFrame的内容面板调用的，在这个例子中，JFrame的内容面板被命名为content。这是因为，我们实际上是将标签添加到内容面板而不是（直接）添加到JFrame中去的。总的来说，应该用调用add的对象来调用setLayout（到目前为止，那个对象总是JFrame的内容面板）。

BorderLayout管理器会将这些标签（或其他组件）放入五个区域BorderLayout.NORTH、BorderLayout.SOUTH、BorderLayout.EAST、BorderLayout.WEST和BorderLayout.CENTER中去。这5个区域是按下列形式排列的：

<table>
<tr><td colspan="3">BorderLayout.NORTH</td></tr>
<tr><td>BorderLayout.WEST</td><td>BorderLayout.CENTER</td><td>BorderLayout.EAST</td></tr>
<tr><td colspan="3">BorderLayout.SOUTH</td></tr>
</table>

在前面这个图中，外围的方框表示内容面板（或其他要向其中添加东西的容器）。这5个区域是由图形内部较细的线来划分的。除非做了一些工作使这些细线可见，否则它们是不可见的。我们把这些线画出来是为了展示每个区域是如何分布的。来看看图5-12中的例子。我们标签是按下列形式添加的：

```
JLabel label1 = new JLabel("First label here.");
content.add(label1, BorderLayout.NORTH);
JLabel label2 = new JLabel("Second label there.");
content.add(label2, BorderLayout.SOUTH);
JLabel label3 = new JLabel("Third label anywhere.");
content.add(label3, BorderLayout.CENTER);
```

尽管不需要将BorderLayout.NORTH、BorderLayout.SOUTH、BorderLayout.EAST、BorderLayout.WEST和BorderLayout.CENTER当作BorderLayout管理器的5个区域之外的任

何东西，但它们实际上是BorderLayout类中定义的5个常量。注意，使用BorderLayout管理器时，要将区域作为方法add的第二个实参给出，就像下面这条语句这样。

```
content.add(label1, BorderLayout.NORTH);
```

注意，并不需要以任何特定的顺序来添加这些标签（或要添加的其他组件），因为第二个实参清楚地说明了要将标签放在什么地方。

还剩一个问题就可以结束对图5-12的讨论了。BorderLayout.EAST和BorderLayout.WEST所表示的区域怎么样了呢？答案很简单：没用到它们。不需要使用所有的5个区域。如果某些区域没有用到，所有多余的空间都会被添加到BorderLayout.CENTER中去。

快速参考：布局管理器

添加到一个容器类中去的对象都是由一个名为**布局管理器**的对象排列的。可以用方法setLayout来添加布局管理器，每个容器类，比如JFrame的内容面板，或任何其他将在本章稍后介绍的容器类的对象，都有这个方法。如果没有添加布局管理器，Java会提供一个默认的布局管理器。

语法：

Container_Object.setLayout(new *Layout_Manager_Class*());

举例（在一个名为FLowLayoutDemo的类的构造器内）：

```
public FlowLayoutDemo()
{
    ...
Container contentPane = getContentPane();
contentPane.setLayout(new FlowLayout());
JLabel label1 = new JLabel("Labels are good.");
contentPane.add(label1);
    ...
}
```

快速参考：指定BorderLayout区域

用5个常量BorderLayout.NORTH、BorderLayout.SOUTH、BorderLayout.EAST、BorderLayout.WEST和BorderLayout.CENTER指定了放置按钮（或其他组件）的区域。如果愿意的话，可以用5个字符串"North"、"South"、"East"、"West"和"Center"作为替代。下面的例子就用这些字符串对图5-12中代码进行了重新编写：

举例：

```
JLabel label1 = new JLabel("First label here.");
content.add(label1, "North");

JLabel label2 = new JLabel("Second label there.");
content.add(label2, "South");

JLabel label3 = new JLabel("Third label anywhere.");
content.add(label3, "Center");
```

使用字符串"North"、"South"等节省了输入的时间。但是，区域并不是一个字符串，因此有些程序员会出于哲学及风格上的考虑而宁愿使用常量，比如BorderLayout.NORTH。每种方法都是合理的。

这里的讨论听起来好像每个区域只能放置一个条目似的，但在本章稍后就会看到有一种方法可以将一些条目组织起来，这样就可以在每个区域中放置多个条目了。

有一些标准的预定义布局管理器，也可以定义自己的布局管理器。但是，对大多数目标来说，预定义的布局管理器就可以满足所有需要了，我们不讨论如何创建自己的布局管理器类。本章描述了三种最常用的预定义布局管理器：你已经见过的BorderLayout管理器，以及两个接下来要讨论的布局管理器。（本书第7章会介绍一种额外的、名为BoxLayout管理器的布局管理器。）

最简单的布局管理器为**FlowLayout管理器**。FlowLayout类的对象是一个布局管理器，它以最显而易见的方式来排列添加到类中去的组件：组件是按照方法add向类中添加的顺序、一个接一个从左至右排列的。例如，如果图5-12中的类使用了FlowLayout管理器，它可能执行了下列代码。

```
content.setLayout(new FlowLayout());
JLabel label1 = new JLabel("First label here.");
content.add(label1);
JLabel label2 = new JLabel("Second label there.");
content.add(label2);
JLabel label3 = new JLabel("Third label anywhere.");
content.add(label3);
```

注意，如果像前面的代码那样使用了FlowLayout管理器，那么，add方法就只会有一个实参。还要注意，使用FlowLayout管理器时，条目是按照它们的添加顺序显示的，这样，前面的那些标签就会像下面这样全部显示在一行中：

```
First label here.Second label there.Third label anywhere.
```

（如果想看完整的程序，可以去看看Web上的FlowLayoutDemo.java程序。）在本章稍后，会看到很多使用FlowLayout管理器类的GUI实例。

GridLayout管理器是按照行和列来排列组件的，每个条目的尺寸都相同。例如，下列语句说明为aContainer（可以是一个内容面板或其他容器）使用的是一个GridLayout管理器。

```
aContainer.setLayout(new GridLayout(2, 3));
```

作为实参赋给构造器GridLayout的两个数字指定了行数和列数。前面的语句会生成下面这种类型的布局。

除非做了些使这些线可见的事情，否则它们是不可见的。它们只是用来表示区域边界的。

尽管在创建GridLayout管理器时指定了列数，但实际上列数是由添加到容器中的条目数决定的。如果添加了6个条目，栅格就如上所示。如果添加了七八个条目，就会自动添加第4列，以此类推。（如果添加的条目少于6个，布局会有2行，而列数则会减少。）

使用GridLayout类时，方法add只有一个实参。条目是从左至右放置在栅格中的，首先填充最上面一行，然后第二行，以此类推。（尽管稍后你会看到，可以添加一些不显示出来的东西，因此给出跳过了一个栅格位置的假象，但是，）不允许跳过任何栅格位置。

如果想看GridLayout类是如何工作的，可以去看看Web上的GridLayoutDemo1.java和

GridLayoutDemo2.java类。

图5-13总结了对这3种布局管理器的描述。更多有关布局管理器的信息将在本书第7章给出。

布局管理器	描 述
FlowLayout	像平常在纸上写东西那样，从左至右显示组件。
BorderLayout	将组件显示在5个区域中：北、南、东、西和中。通过add方法的第二个实参来说明应该将一个组件放到哪个区域中去。
GridLayout	将组件在一个栅格中展开，展开每个组件以填满栅格中的那个方框。

图5-13 一些布局管理器

快速参考：默认的布局管理器

如果没有添加布局管理器，Java会提供一个默认的布局管理器。例如，在图5-8中，没有指定任何布局管理器，但还是可以添加标签，因为自动提供了一个默认布局管理器。JFrame内容面板的默认布局管理器类为BorderLayout类。（使用BorderLayout类和没有第二个实参的add的效果，与将BorderLayout.CENTER作为第二个实参的效果一样。最开始的几个例子都是依赖于这个细节的，但现在我们了解布局管理器了，在用BorderLayout管理器向一个容器添加组件时，我们总是会使用第二个实参的。）

在了解布局管理器之前，我们只是把事情留给默认布局管理器去做。但指定一个布局管理器总是更可取的。这种方式会使代码更清晰，更容易适应新版本的Swing类库将来可能发生的变化。从现在开始，我们总是会使用显式的布局管理器。

自测题

17. 对图5-12进行怎样的修改才能使那三个标签显示为下列形式？

```
First label here.
Second label there.
Third label anywhere.
```

（每两行之间可能会有些空间。）

18. 对图5-12进行怎样的修改才能使那三个标签显示为下列形式？

```
First label here.
                                                Second label there.
Third label anywhere.
```

（每两行之间可能会有些空间。）

19. 下列行出现在图5-12的构造器定义中：

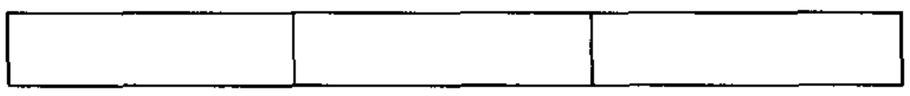
```
content.setLayout(new BorderLayout());
```

new是什么意思？使用的是什么类型的实参？

20. 假设以通常的方式将一个视窗GUI类定义为一个JFrame类的派生类，并假设构造器中包含了下面这样的内容面板。

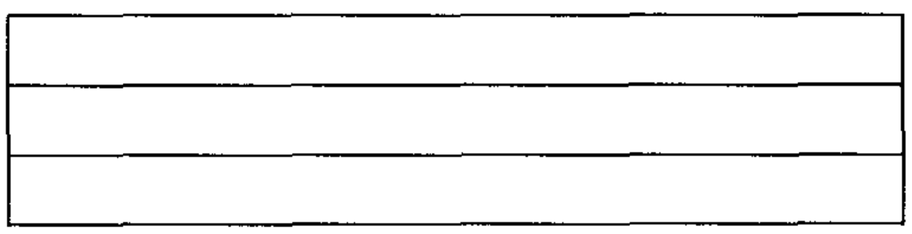
```
contentPane = getContentPane();
```

现在假设想为contentPane指定一个布局管理器，使其产生下列形式的布局。



setLayout的实参应该是什么样的?

21. 假设情况如自测题20所述，只是希望使用下列形式的布局：



setLayout的实参应该是什么样的?

5.3　按钮和动作侦听器

我声明我并没有控制事件，但是我坦白地承认
事件控制着我。

——亚伯拉罕·林肯，给A.G.豪吉斯的信（1864年4月4日）

到目前为止，我们用Swing生成的GUI中基本上没有动作。它们只是呆在那儿显示一些文本而已。它们采取的唯一动作就是在用户点击关闭窗口按钮时退出。本节开始展示如何设计出能做一些动作的GUI，比如改变颜色，修改文本，或执行一些更复杂的动作。在第一个例子中，会将动作作为对用户点击按钮的回应。

按钮只是GUI中一个看起来像按钮一样的组件，用鼠标点击它时，会做一些动作。创建按钮的方式与创建标签的方式很类似。将按钮添加到JFrame中去的方式与将标签添加到JFrame中去的方式类似，但按钮也有一些新的东西：可以将一个动作与按钮关联起来，这样当用户用鼠标点击按钮时，GUI就会执行某些动作。举一个例子会使其中的细节更清晰一些。

编程示例：添加按钮

图5-14中的程序创建了一个带有两个按钮的窗口，这两个按钮被标记为"Red"和"Green"。运行程序时，会显示图5-14中展示的那个窗口。用鼠标点击标记为"Red"的按钮，窗口的颜色就会从蓝色（或原来的任何一种颜色）变成红色。如果点击标记为"Green"的按钮，窗口的颜色就会变成绿色。这就是程序所作的所有工作（但正如所看到的那样，你在慢慢地学着构建更复杂的窗口界面）。要结束程序，使窗口消失，点击关闭窗口按钮就可以了。

图5-14中的大部分内容你都已经很熟悉了。ButtonDemo类是从JFrame类中派生而来的，因此它与你在本章见过的那些窗口界面类似。与前面的例子一样，WindowDestroyer类的窗口侦听器是通过方法addWindowListener添加进去的。窗口的尺寸是通过getContentPane设置的，内容面板也是通过它获得的。尽管这次使用的是FlowLayout管理器类，但布局管理器的使用方式与前面的例子（图5-12）一样。

图5-14中的新知识点就是JButton类按钮对象的使用，及一种新的侦听器类。在接下来的几个小节中，会对按钮及新的侦听器类进行讨论。

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

/**
 Simple demonstration of putting buttons in a JFrame.
*/
public class ButtonDemo extends JFrame implements ActionListener
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;
    /**
     Creates and displays a window of the class ButtonDemo.
    */
    public static void main(String[] args)
    {
        ButtonDemo buttonGui = new ButtonDemo();
        buttonGui.setVisible(true);
    }

    public ButtonDemo()
    {
        setSize(WIDTH, HEIGHT);
        addWindowListener(new WindowDestroyer());
        setTitle("Button Demo");
        Container contentPane = getContentPane();
        contentPane.setBackground(Color.BLUE);

        contentPane.setLayout(new FlowLayout());

        JButton stopButton = new JButton("Red");
        stopButton.addActionListener(this);
        contentPane.add(stopButton);

        JButton goButton = new JButton("Green");
        goButton.addActionListener(this);
        contentPane.add(goButton);
}
```

要完整地解释这个程序需要多个小节才能完成。解释过程要持续到5.3.2节末尾才结束。

```
public void actionPerformed(ActionEvent e)
{
   Container contentPane = getContentPane();
```

图5-14 添加了按钮的GUI

```
        if (e.getActionCommand().equals("Red"))
            contentPane.setBackground(Color.RED);
        else if (e.getActionCommand().equals("Green"))
            contentPane.setBackground(Color.GREEN);
        else
            System.out.println("Error in button interface.");
    }
}
```

得到的GUI

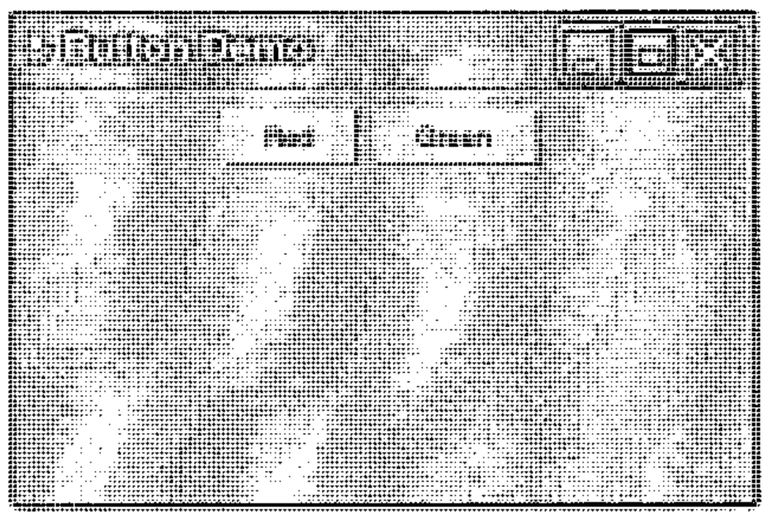


图5-14　（续）

5.3.1　按钮

本节会讲述如何向GUI添加按钮。在接下来的小节会告诉你如何说明按钮被按下来的时候会发生的事情。

创建按钮对象的方式与创建任何其他对象的方式相同，但要使用JButton类。例如，图5-14中的下列代码行就创建了一个按钮：

```
JButton stopButton = new JButton("Red");
```

构造器的实参，在这个例子中就是"Red"，是一个在显示按钮时会写在按钮上的字符串。看看图5-14中的GUI，就会发现这两个按钮被标记为了"Red"和"Green"。

按钮是通过下列代码行添加到内容面板中去的：

```
contentPane.add(stopButton);
```

因为使用的是FlowLayout管理器，所以方法add没有第二个实参。如果使用的是BorderLayout管理器类，就应该使用第二个实参，比如BorderLayout.NORTH。

在下一小节中，将对图5-14中包含方法addActionListener的代码行进行解释。

快速参考：JButton类

JButton类的对象是作为一个看起来像按钮一样的组件显示在GUI中的。用鼠标点击按钮来模拟按压它的动作。用new创建一个JButton类的对象时，可以向构造器给出一个字符串实参，这个字符串会显示在按钮上。

可以用方法add将JButton类的对象添加到JFrame的内容面板上去。稍后会看到还可以用类似的方法将按钮添加到其他的GUI对象中去。

举例：（在一个名为ButtonDemo的类的构造器内部）

```
public ButtonDemo()
```

```
{
        .
        .
        .
    Container contentPane = getContentPane();
        .
        .
        .
    contentPane.setLayout(new FlowLayout());
        .
        .
        .
    JButton stopButton = new JButton("Red");
        .
        .
        .
    contentPane.add(stopButton);
        .
        .
        .
}
```

记住：关闭窗口按钮不在JButton类中

添加到GUI中去的按钮都是JButton类的对象。（在JFrame类的任意派生类中都可以自动获得的）关闭窗口按钮不是JButton类的对象。它是JFrame对象的一部分。

常见问题：为什么有那么多类名都是以“J”开头的？

“J”表示“Java”。对吗？好吧，是的，但使用它不仅仅是由于这个原因。为什么不将类命名为Frame、Label、Button等，而要分别将其命名为JFrame、JLabel和JButton呢？答案是Swing的前身AWT，已经将这些类命名为Frame、Label、Button了。由于Swing并没有完全取代AWT库，就必须将AWT与Swing一同使用，这样，Swing类名就应该与AWT类名有所不同。因此，在Swing中，相应的类被命名为JFrame、JLabel、JButton等。

5.3.2 动作侦听器和动作事件

用鼠标点击一个按钮（或者激活GUI中其他一些条目）会创建一个名为事件的对象，并会将这个对象送给另一个（或多个）名为侦听器的对象。这个过程被称为激发事件。然后，侦听器会执行某些动作。当我们说事件被“发送”给侦听器对象时，实际上是说，会以事件对象为实参调用侦听器对象中的某个方法。这个调用是自动发生的。Swing GUI类定义通常不需要包含对这个方法的调用。但是，Swing GUI类定义确实需要做两件事：首先，对每个按钮来说，都要指定哪个（或哪些）对象是会对那个按钮激发的事件产生响应的侦听器，这一步被称为侦听器**注册**（register）；第二，必须定义一个（或多个）在事件发送到侦听器中去时会被激活的方法。注意，这些方法是自己定义的，但通常情况下，不需要编写对这些方法的调用。这些调用都会自动进行。

图5-14中的下列代码行将this注册为一个侦听器，以接收来自名为stopButton的按钮的事件。

```
stopButton.addActionListener(this);
```

还有一条类似的语句将this注册为一个侦听器，以接收来自名为goButton的按钮的事件。因

为实参为this，所以这条语句就意味着this（ButtonDemo类自身）就是侦听器类。回想一下，在类定义内部，那个类的对象就被称为this。因此，ButtonDemo类自身就是ButtonDemo内部那些按钮的侦听器类。（更精确一点来说，ButtonDemo类的每个对象都是那个对象中的按钮的侦听器。）接下来，解释一下怎样才能使ButtonDemo这样的类成为按钮激发事件的侦听器类。

不同类型的组件需要不同类型的侦听器类来处理它们激发的事件。按钮可以激发一些名为**动作事件**（action event）的事件，这些事件是由名为**动作侦听器**（action listener）的侦听器来处理的。

动作侦听器是一个ActionListener类型的对象。ActionListener不是一个类，而是一种特性，可以将其赋予定义的任何一个类。（ActionListener这样的特性被称为接口，本章稍后会对此进行更详细的讨论[①]。）要使一个类成为ActionListener，要做两件事：

（1）在类定义的起始处添加短语implements ActionListener，通常添加在第一行的末尾。

（2）定义一个名为actionPerformed的方法。

在图5-14中，就是通过这种方法使一个名为ButtonDemo的JFrame类成为ActionListener的。下面将ButtonDemo类定义的轮廓复制了过来（用三个点表示省略了的部分）。

```
public class ButtonDemo extends JFrame implements ActionListener
{
        .
        .
        .
    public void actionPerformed(ActionEvent e)
    {
        .
        .
        .
    }
        .
        .
        .
}
```

我们可以定义一个单独的类，这个类除了处理按钮事件之外什么都不干，但将窗口类ButtonDemo作为处理按钮事件的ActionListener会更方便一些。它之所以方便，是因为按钮事件可能会修改窗口，而使用窗口内部的方法则是修改窗口最简单的方式。

假设如下所示创建了一个ButtonDemo类的对象buttonGui。

```
ButtonDemo buttonGui = new ButtonDemo();
```

此时，ButtonDemo类定义中的参数this指的是buttonGui。因此buttonGui就是buttonGui内两个按钮的动作侦听器。点击一个按钮时，会将按钮激发的事件作为方法actionPerformed的实参并自动调用该方法。剩下需要解释的内容就是方法actionPerformed是如何工作的。图5-15以图形方式说明了按钮和侦听器间的交互。接下来，对ButtonDemo 类的buttonGui对象进行一些解释。

---

① 在《Java程序设计与问题解决：基础篇》第7章的一个选修小节中也对接口进行了讨论。要理解本章的内容不需要阅读那个选修小节。

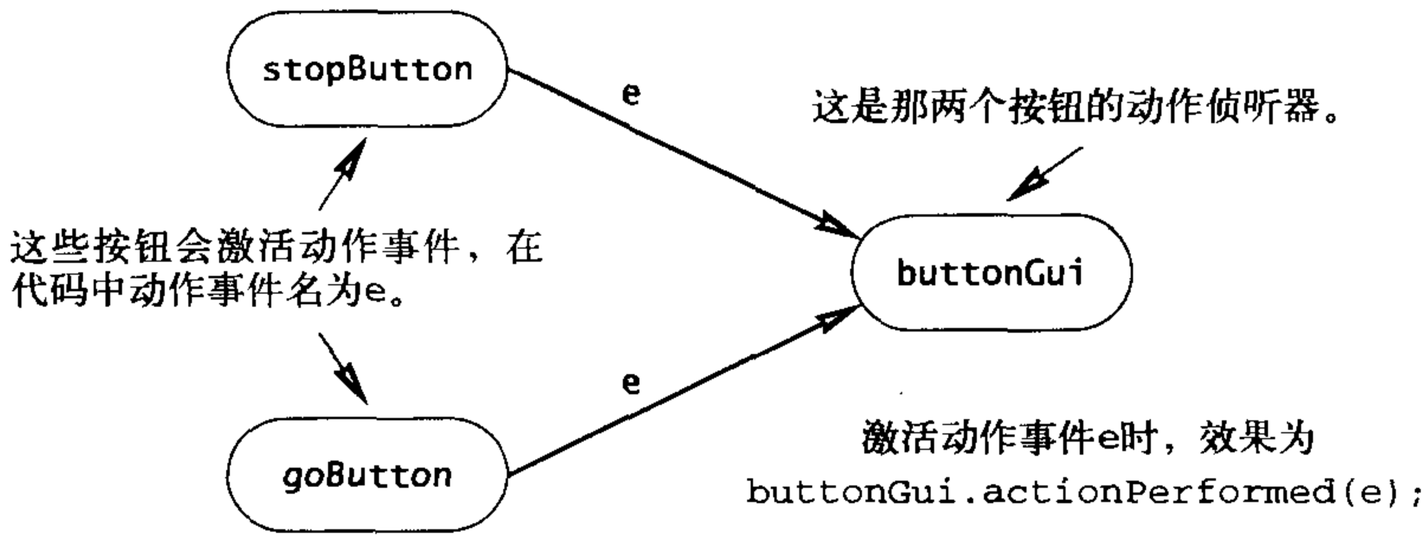


图5-15 按钮与动作侦听器

如果用鼠标点击buttonGui内的一个按钮，就会有一个动作事件传递给那个按钮的动作侦听器。但buttonGui就是buttonGui中那些按钮的动作侦听器，因此动作事件会传递到buttonGui中去。当动作侦听器收到一个动作事件时，会将事件自动地传递给方法actionPerformed。actionPerformed方法通常是一条分支语句，用来确定哪种动作事件被激活了，然后执行一些适当的动作。来看看图5-14的ButtonDemo类中actionPerformed方法的代码。方便起见，将定义复制在下面：

```
public void actionPerformed(ActionEvent e)
{
      Container contentPane = getContentPane();
      if(e.getActionCommand().equals("Red"))
          contentPane.setBackground(Color.RED);
      else if(e.getActionCommand().equals("Green"))
          contentPane.setBackground(Color.GREEN);
      else
          System.out.println("Error in button interface.");
}
```

在这种情况下，方法actionPerformed需要知道动作事件是来自标记为"Red"的按钮还是来自标记为"Green"的按钮。如果e是一个由按钮点击动作激发的动作事件，那么e.getActionCommand()就会返回写在按钮上的字符串。在这个例子中，它会返回"Red"或"Green"。因此，方法actionPerformed所要做的就是看看e.getActionCommand()是"Red"还是"Green"，然后为那个按钮执行适当的动作。注意，e.getActionCommand()是一个String类的对象。String类中有一个equals方法，可以用来查看e.getActionCommand()是等于"Red"还是"Green"（或任何其他字符串）。如果e.getActionCommand()等于*String_Argument*，方法调用

```
e.getActionCommand().equals(String_Argument)
```

就返回true，否则就返回false。因此，下列代码对e.getActionCommand()是等于"Red"还是"Green"进行了测试，并对GUI的颜色进行了相应的修改：

```
if(e.getActionCommand().equals("Red"))
    contentPane.setBackground(Color.RED);
else if(e.getActionCommand().equals("Green"))
    contentPane.setBackground(Color.GREEN);
else
    System.out.println("Error in button interface.");
```

最后一条else子句应该永远都不会执行。把它放在那儿，只是为了当代码中出现一些未被注意的错误时，能够得到一条警告消息。

快速参考：动作事件和动作侦听器

按钮及某些其他组件都会激发ActionEvent类中的事件。这些事件被称为**动作事件**，必须由动作侦听器来处理。任何类都可以作为动作侦听器类。细节的提纲如下所示。

(1) 通过在类定义的头部添加短语

```
implements ActionListener
```

并为一个名为actionPerformed的方法添加定义，就可以使某个类（可能是一个窗口类）成为一个动作侦听器。动作侦听器可以是任意类。特别是，这个类除了作为动作侦听器之外，还可以执行其他一些功能。

(2) 将对象ActionListener注册到按钮（或其他能够激发动作事件的组件）上去。要实现这一步，就要使用方法addActionListener。按钮或其他组件可以注册多于（或少于）一个的侦听器。

举例（完整的细节显示在图5-14中）：

```
public class ButtonDemo extends JFrame implements ActionListener
{
        .
        .
        .
 public ButtonDemo()
 {
        .
        .
        .
   Container contentPane = getContentPane();
        .
        .
        .
   JButton stopButton = new JButton("Red");
   stopButton.addActionListener(this);
   contentPane.add(stopButton);
        .
        .
        .
 }
 public void actionPerformed(ActionEvent e)
 {
        .
        .
        .
 }
}
```

快速参考：actionPerformed方法

要成为一个动作侦听器，除了其他事情之外，一个类必须有一个名为actionPerformed的方法，这个方法要有一个ActionEvent类型的参数。这是ActionListener接口需要的唯一一个方法。

语法：

```
public void actionPerformed(ActionEvent e)
{
    Code_for_Actions_Performed
}
```

*Code_for_Actions_Performed*通常是一条取决于e的某些特性的分支语句。通常，分支语句取决于e.getActionCommand()。如果e是一个通过点击按钮激发的事件，那么，e.getActionCommand()就是一个被称为**动作命令**（action command）的字符串。除非另外指定了一个动作命令，否则，动作命令就是写在按钮上的字符串。

（可以用方法setActionCommand指定一个不同的动作命令。细节请参见本章稍后名为“使用方法setActionCommand”的Java提示。）

举例：

```
public void actionPerformed(ActionEvent e)
{
    Container contentPane = getContentPane();
    if(e.getActionCommand().equals("Red"))
        contentPane.setBackground(Color.RED);
    else if(e.getActionCommand().equals("Green"))
        contentPane.setBackground(Color.GREEN);
    else
        System.out.println("Error in button interface.");
}
```

△ 易犯错误：修改actionPerformed的参数列表

使一个类成为一个动作侦听器时，方法actionPerformed的头部就已经确定了。它必须只有一个参数，那个参数必须是ActionEvent类型的，如下所示。

```
public void actionPerformed(ActionEvent e)
```

如果改变了参数类型，或者添加（或减少）了一个参数，就无法给出动作侦听器的正确定义了[①]。唯一能够改变的就是参数e的名字，因为它只是一个占位符。因此，下列修改是可以接受的。

```
public void actionPerformed(ActionEvent theEvent)
```

当然，如果进行了这种修改，那么在方法actionPerformed的主体内部，就要用标识符theEvent来取代标识符e了。 △

5.3.3 接口

再来看看图5-14。我们希望ButtonDemo类既是一个JFrame，又是一个ActionListener。但是ButtonDemo只能从一个基类中派生出来，所以将其作为JFrame的派生类。这就意味着ActionListener必须提供与基类JFrame类似、但又不同的功能。ActionListener不是一个类，而是一个接口。**接口**（interface）是类的一种特性，用来说明类必须具有哪些方法。称ButtonDemo这样满足了一个接口要求的类**实现了这个接口**（implement the interface）。

在《Java程序设计与问题解决：基础篇》第7章名为“接口”的选读小节中对接口进行了讨论。因为这部分内容现在不再是选读的了，所以，在此再次提供了部分内容。

尽管接口不是一个类，但它是一种类型。例如，一个方法可以有一个ActionListener类型的参数。任意一个实现了ActionListener接口的类都可以作为那个方法的实参。

① 尽管这是一种相当有问题的方式，但是，确实可以对方法名actionPerformed进行重载，这样就可以有多个版本的actionPerformed方法，每个版本都有不同的参数列表。但是，只有前面那个actionPerformed版本才能使类成为一个动作侦听器。

要实现一个接口，类必须要做两件事：

(1) 必须在类定义的起始处包含短语

```
implements Interface_Name
```

要实现多个接口，只要像

```
implements MyInterface, YourInterface
```

这样列出所有的接口名就行了，接口名之间以逗号分隔。

(2) 类必须实现一个（或多个）接口定义中列出的所有方法头部。

要实现ActionListener接口，类定义就必须在方法定义的起始处包含短语implements ActionListener——例如

```
public class ButtonDemo extends JFrame implements ActionListener
{
```

这个类还必须实现一个具有头部

```
public void actionPerformed(ActionEvent e)
```

的方法。实现一个接口的完整实例，请参见图5-14。

可以定义自己的接口。在本书中没必要这么做，但是如果想看看这是如何实现的，可以去阅读《Java程序设计与问题解决：基础篇》第7章关于“接口”的7.2.7节。

□ Java提示：对GUI的外观和动作分开编程

可以将一个为Swing GUI编写代码的工作分成两个主要的部分，以简化编程工作：对GUI屏幕上外观的编程，以及对GUI动作的编程。例如，考虑图5-14中的程序。这个程序的第一个版本可以使用下列actionPerformed方法定义。

```
public void actionPerformed(ActionEvent e)
{}
```

有了这个“什么都不做”的actionPerformed方法版本，程序就会运行，并会像图5-14那样在屏幕上显示一个窗口。如果点击任意一个改变颜色的按钮，什么事情都不会发生，但可以用这一段代码对一些细节进行调整，比如要先列出哪个按钮。(但是要注意，关闭窗口按钮确实能够工作。) 使GUI看起来如你所愿之后，就可以定义GUI的动作部分了，通常就是actionPerformed方法。正如前面所说的那样，这种方法将编写Swing GUI的任务分成了两个较小的任务：对其外观的编程和对其动作的编程。在这个例子中，这看起来好像不是什么大问题，但在一个复杂的Swing GUI中，这两个任务中的每一个可能都是很难，但又很合理的，而将整个GUI作为一个整体来编程可能会是非常困难的。在这种情况下，将工作分成两个部分来实现会有很大的帮助。

如果在JFrame定义的起始处包含了短语implements ActionListener，就必须包含对方法actionPerformed的定义。像

```
public void actionPerformed(ActionEvent e)
{}
```

这样的“什么都不做”或“只做很少事情”的方法定义，通常被称为**占位代码**（stub）。在很多上下文，而不仅仅是在Swing程序中，使用占位代码都是一种很好的编程技巧。

或者，在编写图5-14中这样的第一个版本的Swing GUI时，*如果省略了短语implements ActionListener，以及对addActionListener的调用*，就可以将方法actionPerformed的定义完全省略掉。

5.3.4 模型－视图－控制器模式

在前面的Java提示中提出的那种技术是一种通用的、名为**模型－视图－控制器**（Model-View-Controller）模式的技术的实例。图5-16给出了这个模式的图形表示。这个模式的模型部分执行了应用程序的核心功能。视图部分为输出部分，显示了模型的状态图形。控制器为输入部分，将命令从用户传递给模型。这三个互相交互的部分都是作为一个负责完成其自身功能的对象来实现的。对简单任务，可以使用一个带有不同方法的对象来实现每个模型、视图和控制器的功能，比如，实现图5-14中的JFrame的任务。

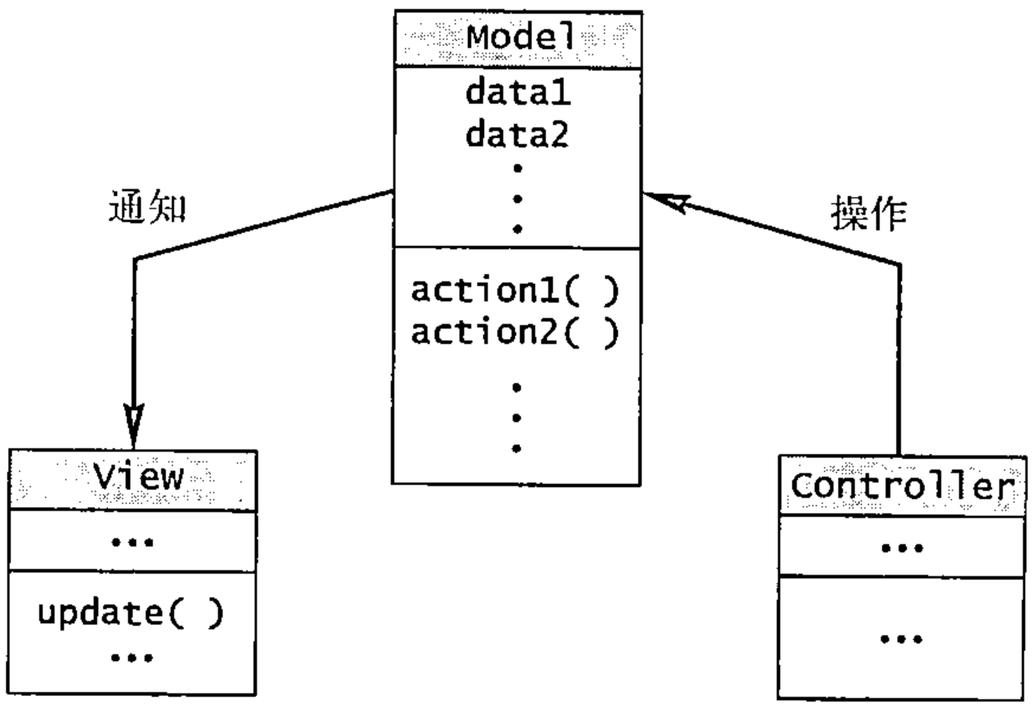


图5-16 模型－视图－控制器模式

对模型-视图-控制器模式的描述就好像用户要直接和控制器进行交互一样。这样做主要是为了简化讨论。控制器不一定要在用户的直接控制之下，可以由某些类型的软硬件组件来控制。在一个Swing GUI中，视图和控制器部分可以是独立的类，它们组合成为一个较大的类，显示单个窗口，允许这两部分与用户进行交互。

□ Java提示：使用方法setActionCommand

正如前面提到的那样，用鼠标点击按钮会激发一个名为动作事件的事件e。这个事件e通常会被传送到那个按钮的动作侦听器的actionPerformed方法中去。方法actionPerformed收到事件e时，要查明是哪个按钮被点击了。讨论图5-14中的ButtonDemo类时，我们说过方法调用e.getActionCommand()返回的是写在按钮上的字符串。这是一种比较简单的解释。实际上，调用e.getActionCommand()返回的是那个按钮的一个表示**动作命令**的字符串。默认的动作命令就是写在按钮上的那个字符串，但是，如果想为按钮指定另一个字符串作为动作命令的话，也是可以的。

例如，在图5-14中，按下列方式创建了stopButton。

```
JButton stopButton = new JButton("Red");
```

如果其他什么都不做，那么stopButton的动作命令就是"Red"。但是如果愿意的话，可以将动作命令改成其他字符串，比如"Stop"。可以这样做。

```
stopButton.setActionCommand("Stop");
```

然后，方法actionPerformed就会去查看字符串"Stop"，而不是字符串"Red"。Web上的文件SetActionCommandDemo.java中有一个ButtonDemo的重写版本，使用了setActionCommand。

最终你可能会发现自己处于这样一种境地：希望在两个不同的按钮上写上相同的字符串。在这种情况下，就可以用setActionCommand为它们给出不同的动作命令，来区分这两个按钮。

快速参考：setActionCommand和getActionCommand

每个按钮（以及每个能够激发事件的对象）都有一个与之相关的字符串，被称为那个按钮的**动作命令**。按钮被点击时，会激发一个动作事件e。下列调用会返回激发e的那个按钮的动作命令：

```
e.getActionCommand();
```

方法actionPerformed通常会用这个动作-命令字符串来确定哪个按钮被点击了。

一个按钮的默认动作命令就是写在按钮上的字符串，但是如果愿意的话，可以通过对方法setActionCommand的调用来修改动作方法。例如，下列代码会将字符串"Red"写在一个名为stopButton的JButton上，但是会将字符串"Stop"作为按钮stopButton的动作命令。

举例：

```
JButton stopButton = new JButton("Red");
stopButton.setActionCommand("Stop");
```

常见问题：动作命令存储在什么地方？

按钮的动作命令是存储在按钮的私有实例变量，以及由那个按钮激发的任意一个事件的私有实例变量中的。因此，方法getActionCommand和setActionCommand分别是访问方法和设置方法。

自测题

22. 一个按钮组件需要哪种类型的侦听器？如何创建这样一个侦听器？

23. 考虑下列来自图5-14的代码：

```
contentPane.setLayout(new FlowLayout());
JButton stopButton = new JButton("Red");
stopButton.addActionListener(this);
contentPane.add(stopButton);
JButton goButton = new JButton("Green");
goButton.addActionListener(this);
contentPane.add(goButton);
```

可以用下列代码取代它们而不改变其含义吗？

```
contentPane.setLayout(new FlowLayout());
JButton b = new JButton("Red");
b.addActionListener(this);
contentPane.add(b);
b = new JButton("Green");
b.addActionListener(this);
contentPane.add(b);
```

换句话来说，在有两个按钮的情况下，可以只使用一个名字b吗？

24. 图5-14的方法actionPerformed中包含了下列代码行：

```
Container contentPane = getContentPane();
```

一定要将内容面板命名为contentPane吗，还是也可以将其命名为别的标识符，如下所示。

```
Container insideOfJFrame = getContentPane();
```

25. 说一个事件被“发送”到侦听器对象中去，是什么意思？

26. 如果按下列形式，用标识符buttonEvent取代e，会对图5-14中的程序产生什么影响？

```
public void actionPerformed(ActionEvent buttonEvent)
{
```

```
    Container contentPane = getContentPane();
    if(buttonEvent.getActionCommand().equals("Red"))
       contentPane.setBackground(Color.RED);
    else if(buttonEvent.getActionCommand().equals("Green"))
       contentPane.setBackground(Color.GREEN);
    else
       System.out.println("Error in button interface.");
}
```

5.4 容器类

斟满所有的杯子，为什么
除了我之外所有的人都可以痛饮，
为什么，人哪，告诉我为什么？

——亚伯拉罕·考利，《阿那克里翁》

用Swing构建一个像我们定义的窗口那样的GUI时，可以根据已经存在的类为GUI构建新的类。从老的类中构建新的GUI类的方法主要有两种。一种方式是使用继承。例如，要构建一个窗口界面，通常可以使用Swing类JFrame，并使你的窗口成为JFrame的一个派生类。从老类中生成新类的第二种方式是将某个Swing类作为一个**容器**（Container），并将组件放入这个容器中。例如，在（图5-14的）ButtonDemo类中，就是按下列方式将一个按钮添加到窗口之中去的。

```
JButton stopButton = new JButton("Red");
   ...
ContentPane.add(stopButton);
```

不需要在这两种构建GUI的方式中进行选择，因为在大多数情况下，定义GUI类时，这两种技术都会被用到。

本节将学习JPanel类，通常这个类会被用来定义部分窗口。然后，继续对所有容器类（比如JPanel和JFrame的内容面板）的总体特性进行讨论。

5.4.1 JPanel类

GUI通常是按层次结构组织的，在一个类似窗口的容器中会包含其他的类似窗口的容器。本节将介绍一个对这种组织形式有帮助的新类。JPanel类是一个很简单的容器类，主要用来组织对象。它是最简单的容器类之一，但却是经常会用到的一个类。JPanel对象类似于将很多简单的Java语句组织成一条较大Java语句的圆括号。它将一些较小的对象，比如按钮和标签，组织成一个较大的组件（JPanel）。然后，可以将JPanel对象放入一个JFrame的内容面板中。因此，JPanel对象的主要功能之一就是将JFrame分成一些不同的区域。这些JPanel对象通常被简称为**面板**（panel）。

面板的正确使用对窗口中组件的排列产生的影响，与布局管理器的选择对其产生的影响不相上下。例如，使用BorderLayout管理器时，可以将组件放在这5个位置的任意一个中：BorderLayout.NORTH、BorderLayout.SOUTH、BorderLayout.EAST、BorderLayout.WEST和BorderLayout.CENTER。但是，如果想将两个组件都放在屏幕底端的BorderLay-

out.SOUTH位置上，该怎么办呢？要想实现这一功能，就得将这两个组件放在一个面板中，然后将这个面板放在BorderLayout.SOUTH位置上。

图5-17中的程序与图5-14有少许不同。在图5-17中，将按钮放在一个名为buttonPanel的面板中，这样，当窗口的其他部分改变颜色时，带有按钮的那部分窗口的颜色就不会发生变化。就像图5-14那样，点击图5-17的GUI中的"Red"按钮时，窗口的颜色变成红色，类似地，点击"Green"按钮时，窗口的颜色会变成绿色。但是，这里的按钮是在一个单独的白色面板中的，这个面板的颜色不会变化。正如所看到的那样，对JPanel对象使用布局管理器和方法add的方式，与对JFrame的内容面板使用这两者的方式是一样的。首先，要用add将按钮放到面板中去，如下所示。

```
buttonPanel.add(stopButton);
```

然后，用add将JPanel放到JFrame中去，如下所示。

```
contentPane.add(buttonPanel, BorderLayout.SOUTH);
```

注意，内容面板（contentPane）和面板（buttonPanel）都有它们自己的布局管理器。

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

/**
 Simple demonstration of putting buttons in a panel.
*/
public class PanelDemo extends JFrame implements ActionListener
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;

    public static void main(String[] args)
    {
        PanelDemo guiWithPanel = new PanelDemo();
        guiWithPanel.setVisible(true);
    }

    public PanelDemo()
    {
        setSize(WIDTH, HEIGHT);
        addWindowListener(new WindowDestroyer());
        setTitle("Panel Demonstration");
        Container contentPane = getContentPane();
        contentPane.setBackground(Color.BLUE);
        contentPane.setLayout(new BorderLayout());

        JPanel buttonPanel = new JPanel();
        buttonPanel.setBackground(Color.WHITE);

        buttonPanel.setLayout(new FlowLayout());

        JButton stopButton = new JButton("Red");
        stopButton.setBackground(Color.RED);
```

图5-17　将按钮放入一个面板中

```
        stopButton.addActionListener(this);
        buttonPanel.add(stopButton);

        JButton goButton = new JButton("Green");
        goButton.setBackground(Color.GREEN);
        goButton.addActionListener(this);
        buttonPanel.add(goButton);

        contentPane.add(buttonPanel, BorderLayout.SOUTH);
    }

    public void actionPerformed(ActionEvent e)
    {
        Container contentPane = getContentPane();
        if (e.getActionCommand().equals("Red"))
            contentPane.setBackground(Color.RED);
        else if (e.getActionCommand().equals("Green"))
            contentPane.setBackground(Color.GREEN);
        else
            System.out.println("Error in button interface.");
    }
}
```

得到的GUI

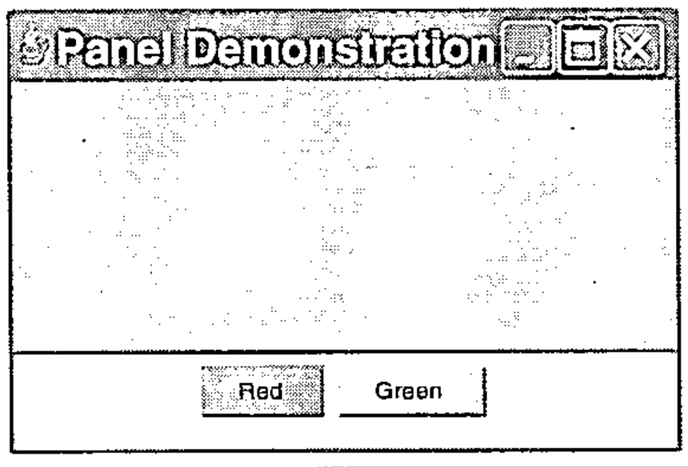


图5-17 （续）

一定要注意，向JFrame添加东西和向JPanel添加东西的方式略有不同。对JFrame来说，首先要用getContentPane获得内容面板，然后对内容面板使用方法add。对JPanel来说，是直接对JPanel对象使用add方法的。JPanel不需要考虑内容面板的问题。

注意一下动作侦听器是如何建立起来的。每个按钮都将参数this注册为侦听器，如下列代码行所示：

```
stopButton.addActionListener(this);
```

因为这行代码出现在PanelDemo类的构造器内部，所以，this参数指的就是PanelDemo，即整个窗口界面。因此，整个窗口容器就是侦听器，而不是JPanel。（记住，PanelDemo是带有JPanel的窗口。而不是JPanel自身。）因此，点击标记为"Red"的按钮时，变成红色的是较大窗口的背景。带有按钮的面板仍然是白色的。

注意，因为按钮点击事件的侦听器为PanelDemo，所以，实现ActionListener，并拥有方法actionPerformed的就是PanelDemo。

图5-17还引入了另一种较小，但全新的技术。除了为面板和内容面板着色之外，还为每个按钮分配了一个颜色。这是通过方法setBackground来实现的，所用的技术基本上与在前

一个例子中使用的技术相同。可以用setBackground为一个按钮或基本上所有其他条目赋一个颜色。

可以用方法add将一个JPanel添加到另一个JPanel中去，每个JPanel都可以有不同的布局管理器。可以通过这种方法构建出具有层次的面板，这样就可以在GUI内部创建出几乎所有的组件排序形式了。

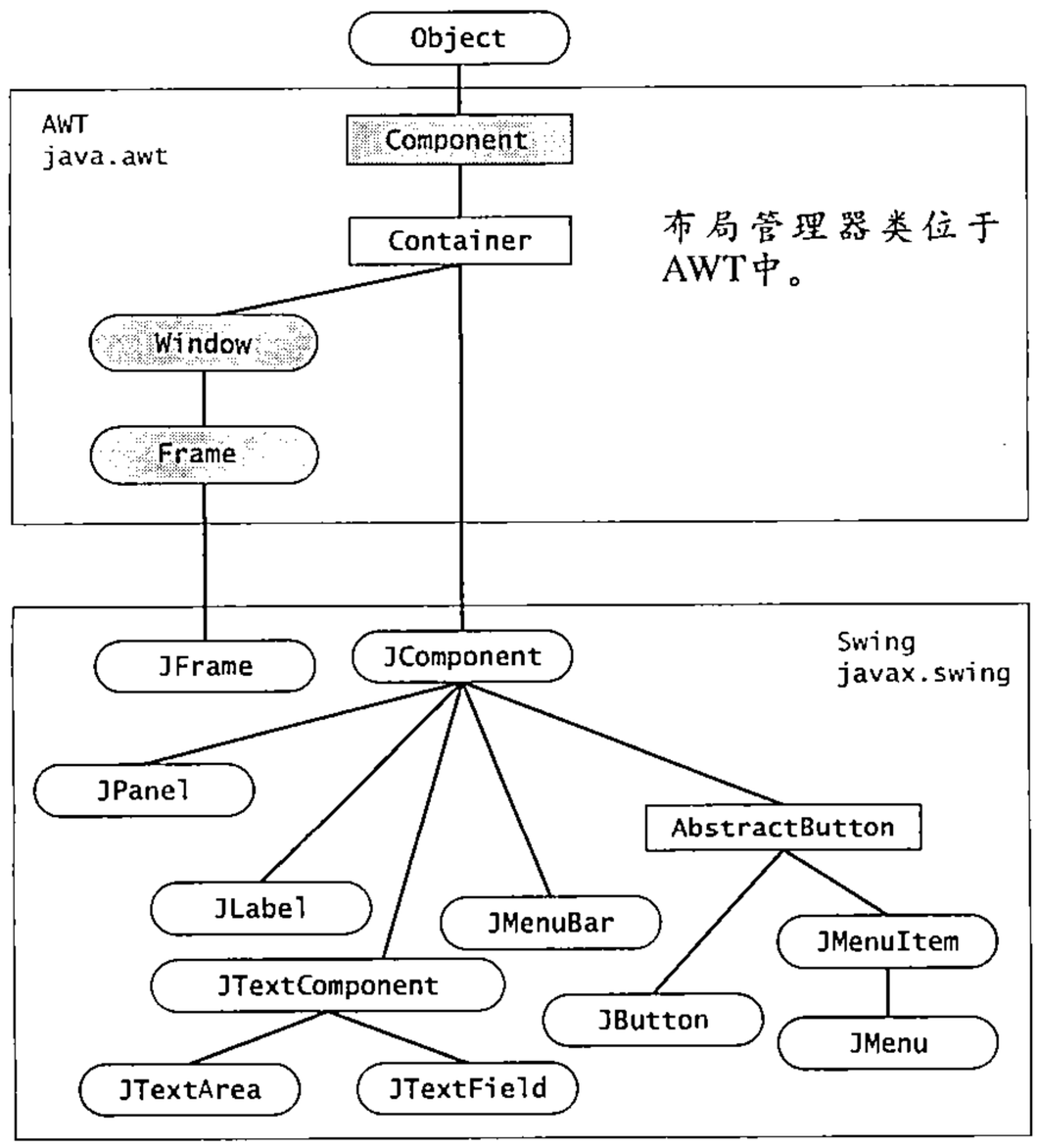
快速参考：JPanel类

JPanel对象可以将一些较小的对象，比如按钮和标签，组织成一个较大的组件（JPanel）。然后，通常会将JPanel对象添加到JFrame的内容面板（或另一个JPanel）中去。因此，JPanel对象的主要功能之一就是将JFrame分隔成一些不同的区域。

5.4.2 Container类

有一个名为Container的预定义类。可以向任何Container类的子孙类添加组件（或者，更精确地说，可以将组件添加到类的对象中去）。JFrame类是Container类的子孙类，因此JFrame类的任何子孙类都可以作为装载标签、按钮、面板或其他组件的容器。

类似地，JPanel类是Container类的子孙类，JPanel类的任何对象都可以作为装载标签、按钮、其他面板或其他组件的容器。图5-18显示了一部分Swing层次结构，以及AWT库中的一



如果两个类之间有一条线，那么较低的类就是较高类的派生类。

这种阴影颜色说明这是一个在本书中未用到的、放在这里作参考的类。如果没有听说过这些用阴影表示的类，也可以放心地忽略它们。

图5-18　Swing类的层次结构

些类。注意，Container类是在AWT库，而不是Swing库中的。这不是什么大问题，但这意味着在使用Container类时，需要导入AWT包。因此，任何引用了Container类的类定义中都应该包含下列import语句。

```
import java.awt.*;
```

Container类的所有子孙类都是**容器类**（container class）。JComponent类对组件扮演的角色与之类似。JComponent类的任何子孙类都被称为JComponent，或者有时会更简单地将其称为一个**组件**（component）。（请参见图5-18。）可以将任何JComponent对象添加到任何容器类对象中去。

注意，JComponent类是从Container类中派生出来的，因此，可以将一个JComponent添加到另一个JComponent中去。有时，这会是个好主意，有时则是需要避免的问题。随着讨论的进行会涉及各种不同的情况。Component类、Frame类和Window类都是AWT类，有些读者可能听说过。在图5-18中包含它们只是起参考作用，并不需要使用这些类。最终，我们会对图5-18中所有其他类进行讨论。

快速参考：容器类

Container类的任何子孙类都是**容器类**。尤其是，因为JFrame和JPanel类是Container类的子孙，所以这两个类的子孙类也都是容器类。可以向每个容器类中添加东西，比如标签、按钮、面板等。根据特定容器类的不同，可以用下列两种方式来添加条目：

（1）对一个JFrame对象，要用getContentPane方法来获取内容面板，然后再将内容面板作为调用对象来使用add添加条目。

（2）对本书中讨论的所有其他容器类，可以对容器类的对象直接使用add方法来添加条目。

快速参考：add方法

每个Swing容器类都有一个名为add的方法。可以用add方法向容器类添加任意的JComponent对象，无论是预定义的还是你自己定义的JComponent对象都可以。对JFrame来说，要先用getContentPane方法获取内容面板，然后再将内容面板作为调用对象来使用方法add。

记住：抽象类

看看图5-18，就会发现有些类是抽象类。**抽象类**（abstract class）纯粹是一种用来简化你的想法的类。不能——实际上，是无法——直接创建抽象类的对象。只能用抽象类作为基类来派生新类。《Java程序设计与问题解决：基础篇》的第7章对抽象类进行过讨论。

和一个Swing容器类打交道时，需要对付三类对象：容器类自身（很可能是某种类型的面板或类似窗口的对象）；添加到容器中去的组件（比如标签、按钮和面板）；一个在容器内对组件进行定位的布局管理器。在我们定义的几乎每个JFrame类中都可以见到这三种类型的对象。几乎所有完整的GUI界面和很多GUI子部分都是由这三种类型的对象组成的。

□ Java提示：简单窗口界面的创建指南

大多数简单的视窗GUI都遵循一种很易学的模式，可以通过这种模式开始对Swing的使用。这里

列出了到目前为止见过的一些要点：

(1) 典型的GUI是由一些从JFrame类中派生出来，并包含了很多组件（比如标签和按钮）的窗口对象组成的。

(2) 用户点击关闭窗口按钮时，窗口应该关闭，但除非程序注册了一个用来关闭窗口的窗口侦听器，否则，程序就不会正确地执行。完成这项任务的一种方法是将下列代码行添加到位于一个构造器定义内部的GUI类定义中去。

```
addWindowListener(new WindowDestroyer());
```

可以使用图5-4给出的WindowDestroyer定义。

(3) 可以将一些组件放入一个JPanel中，并将这个JPanel添加到GUI中去，以这种方式将组件组织起来。

(4) 应该用方法setLayout为GUI（即，JFrame），及GUI中的每个JPanel分配一个布局管理器。

(5) 如果任意一个组件，比如按钮，会产生一些动作事件，就要使GUI（或其他一些类）成为动作侦听器。每个会产生动作事件的组件都应该有一个注册了的动作侦听器。可以用方法addActionListener来注册一个动作侦听器。

(6) 要使视窗GUI（或其他类）成为一个动作侦听器，需要将下列代码行添加到类定义的起始处：

```
implements ActionListener
```

还需要向类中添加一个actionPerformed方法定义。

遵循这些指南并不是创建GUI窗口类的唯一方法，但这却是一种简单常用的实现方式，它基本上是到目前为止学过的唯一一种方法。

常见问题：什么时候需要使用getContentPane?

可以结合使用getContentPane和add来向JFrame添加条目，代码如下所示：

```
Container contentPane = getContentPane();
JLabel label = new JLabel("Please don't click the button!");
contentPane.add(label);
```

但是，对JPanel来说，不需要使用getContentPane，而是可以直接对JPanel对象使用add方法，如下列代码所示：

```
JPanel buttonPanel = new JPanel();
    ...
JButton stopButton = new JButton("Red");
    ...
buttonPanel.add(stopButton);
```

对JFrame对象（及某些本书中未讨论的对象）来说，需要使用getContentPane方法来获取JFrame对象的内容面板，然后将内容面板作为调用对象来使用方法add。对本书中讨论的所有其他容器类来说，都可以对容器类的对象直接使用add方法，而不需要使用内容面板。

自测题

27. 向JFrame添加组件时，需要使用getContentPane吗？向JPanel添加组件时，需要使用getContentPane吗？

28. JButton组件需要使用哪种类型的侦听器？如何创建这样一个侦听器？

29. （图5-4中的）WindowDestroyer类与（图5-14中的）ButtonDemo类有什么共同之处？
30. 用一个JFrame对象调用getContentPane时，对象返回的是什么类型，要使用这个类型名，需要使用哪条import语句？
31. 要使用布局管理器类，需要哪条import语句？
32. JPanel类的对象是容器类吗？是组件类吗？
33. FlowLayout管理器是怎样排列容器中的组件的？
34. BorderLayout管理器是怎样排列容器中的组件的？
35. 如果使用的是BorderLayout管理器，要将两个按钮放到BorderLayout.NORTH区域中去。该怎么做呢？
36. GridLayout管理器是怎样排列容器中的组件的？
37. 假设按下列形式为aContainer分配了一个布局管理器

```
aContainer.setLayout(new GridLayout(2,3));
```

可以向aContainer中添加多少组件？（添加一个JPanel只算添加了一个对象，即使JPanel中包含了多个组件也是如此。）

38. 使用GridLayout管理器时，不能使任何栅格元素为空，但可以通过某种手段使一个栅格元素看起来是空的。怎样才能做到这一点？
39. 已经习惯于定义Swing类JFrame的派生类了。其实，还可以定义其他Swing类的派生类。定义一个名为PinkJPanel的JPanel类的派生类。PinkJPanel类对象的使用方式与JPanel类对象的使用方式一样，只是PinkJPanel类对象是粉色的（除非显式地修改了它的颜色）。PinkJPanel类将只有一个构造器，即默认构造器。（提示：这个练习很简单。）

5.5 GUI的文本I/O

波洛涅斯：你读些什么，殿下？
哈姆雷特：词句，词句，都是词句。

——威廉 莎士比亚，《哈姆雷特》

本节将展示如何向Swing GUI中添加文本输入和文本输出。

5.5.1 文本区和文本域

图5-12中GUI的窗口中有一些文本，但那是静态文本。用户或程序都无法改变那些文本。图5-19中的程序生成了一个带有文本区的GUI，用户可以在文本区中输入任何想要的文本。然后，文本会被作为一个稍后可以取回的备注存储起来。在这个简单的例子中，用户只能使用两个备注，但这已足以说明文本区是如何创建及使用的了。中间的白色区域是JTextArea类的对象。用户可以向这个JTextArea输入任何类型的文本。如果用户点击了按钮"Save Memo 1"，文本就会被保存为备注1。如果用户点击了按钮"Save Memo 2"，文本就会被保存为备注2。通过点击按钮"Get Memo 1"或"Get Memo 2"，就可以分别将相应备注中的文本恢复到JTextArea中去。可以点击按钮"Clear"来清除JTextArea。我们来看看程序是如何实现这项功能的。

将图5-19中创建的按钮放入JPanel中，然后像图5-17那样，将这个JPanel放入JFrame中

去。JTextArea是用下列代码建立起来的，这段代码出现在图5-19的构造器定义中：

```
JPanel textPanel = new JPanel();
textPanel.setBackground(Color.BLUE);
theText = new JTextArea(LINES, CHAR_PER_LINE);
theText.setBackground(Color.WHITE);
textPanel.add(theText);
contentPane.add(textPanel, BorderLayout.CENTER);
```

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class MemoSaver extends JFrame implements ActionListener
{
    public static final int WIDTH = 600;
    public static final int HEIGHT = 300;
    public static final int LINES = 10;
    public static final int CHAR_PER_LINE = 40;

    private JTextArea theText;
    private String memo1 = "No Memo 1.";
    private String memo2 = "No Memo 2.";

    public MemoSaver()
    {
        setSize(WIDTH, HEIGHT);
        addWindowListener(new WindowDestroyer());
        setTitle("Memo Saver");
        Container contentPane = getContentPane();
        contentPane.setLayout(new BorderLayout());

        JPanel buttonPanel = new JPanel();
        buttonPanel.setBackground(Color.WHITE);
        buttonPanel.setLayout(new FlowLayout());
        JButton memo1Button = new JButton("Save Memo 1");
        memo1Button.addActionListener(this);
        buttonPanel.add(memo1Button);
        JButton memo2Button = new JButton("Save Memo 2");
        memo2Button.addActionListener(this);
        buttonPanel.add(memo2Button);
        JButton clearButton = new JButton("Clear");
        clearButton.addActionListener(this);
        buttonPanel.add(clearButton);
        JButton get1Button = new JButton("Get Memo 1");
        get1Button.addActionListener(this);
        buttonPanel.add(get1Button);
        JButton get2Button = new JButton("Get Memo 2");
        get2Button.addActionListener(this);
        buttonPanel.add(get2Button);
        contentPane.add(buttonPanel, BorderLayout.SOUTH);

        JPanel textPanel = new JPanel();
        textPanel.setBackground(Color.BLUE);
        theText = new JTextArea(LINES, CHAR_PER_LINE);
```

在这个图中有一个用来演示的main。

如果在设置memo 1之前去获取memo 1，会得到消息"No Memo 1"。

图5-19　带有文本区的GUI

```
        theText.setBackground(Color.WHITE);
        textPanel.add(theText);
        contentPane.add(textPanel, BorderLayout.CENTER);
    }

    public void actionPerformed(ActionEvent e)
    {
        String actionCommand = e.getActionCommand();
        if (actionCommand.equals("Save Memo 1"))
            memo1 = theText.getText();
        else if (actionCommand.equals("Save Memo 2"))
            memo2 = theText.getText();
        else if (actionCommand.equals("Clear"))
            theText.setText("");
        else if (actionCommand.equals("Get Memo 1"))
            theText.setText(memo1);
        else if (actionCommand.equals("Get Memo 2"))
            theText.setText(memo2);
        else
            theText.setText("Error in memo interface");
     }

    public static void main(String[] args)
    {
        MemoSaver guiMemo = new MemoSaver();
        guiMemo.setVisible(true);
    }
```

theText是一个实例变量。

得到的GUI

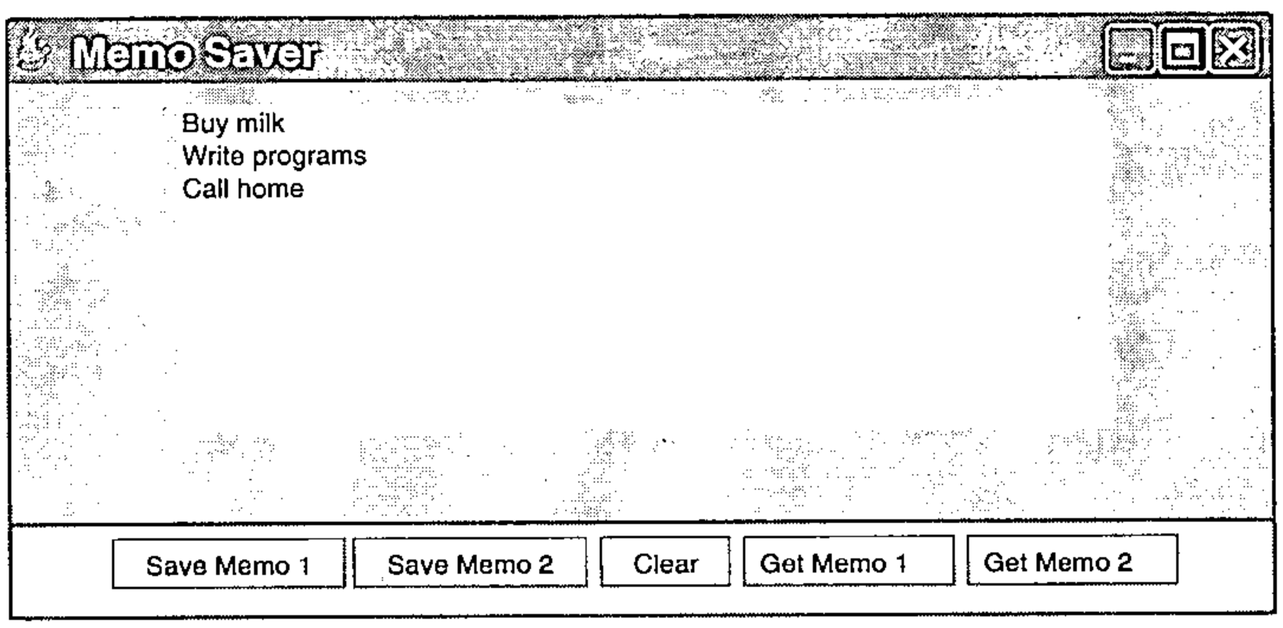


图5-19 （续）

对象theText是JTextArea的一个成员。注意JTextArea构造器中的实参。实参LINES是一个设置为10的定义常量，CHAR_PER_LINE是一个设置为40的定义常量，这两个实参说明这个文本区从顶端到底端有10行，且每行至少可以放置40个字符。如果输入的文本超出了构造器的这两个实参指定的文本区尺寸，结果是无法预料的。因此，最好将这些数字设置的足够大，以容纳最大预期的文本数量。（可以用本节末尾方框中描述的方法setLineWrap，或者本书第7章描述的滚动条来处理文本过多的问题。）

使用的是BorderLayout管理器。textPanel被添加到JFrame的BorderLayout.CENTER位置中去。BorderLayout.CENTER通常会占据JFrame空间中所有剩余的区域。这些程序设置了GUI的外观，但还需要讨论一下GUI是如何处理文本区中的内容的。

两个备注被存储在两个实例变量memo1和memo2中，它们都是String类型的。用户点击按钮"Save memo 1"时，就将文本区中的文本作为实例变量memo1的值保存起来了。这项任务是由下列代码实现的，这些代码是actionPerformed方法定义中的前三行：

```
String actionCommand = e.getActionCommand();
if(actionCommand.equals("Save Memo 1"))
   memo1 = theText.getText();
```

回想一下，getActionCommand方法返回的是被点击按钮的标签。因此，这段代码是说如果点击的是标记为"Save Memo 1"的按钮，就会执行下列代码：

```
memo1 = theText.getText();
```

方法getText()会返回写在JTextArea类的theText对象上的文本。换句话说，它会返回用户输入文本区的内容。如果接着往下读方法actionPerformed的定义，就会发现第二个备注也是用类似的方式存储起来的。

备注是在文本区中显示的，而文本区则是用方法actionPerformed定义中的下列子句清除的：

```
else if (actionCommand.equals("Clear"))
   theText.setText("");
else if (actionCommand.equals("Get Memo 1"))
   theText.setText(memo1);
else if (actionCommand.equals("Get Memo 2"))
   theText.setText(memo2);
```

JTextArea类的方法setText将文本区中的文本改成了作为实参传递给setText的内容。在前面代码的第二行中，中间什么也没有的那两个引号是作为setText的实参使用的。这两个引号表示的是一个空字符串，因此会产生一个空的文本区。

JTextField类（在图5-19中没有用到）与JTextArea十分相似，只是前面那个类只能显示一行文本。在用户只需给出几个文本字符，比如单个数字、文件名或人名的界面中，它是很有用的。JTextField的构造器有一个实参，就是文本域中可见的字符数。用户可以输入更多的字符，但只能显示出指定数量的字符。因此，务必要使文本域比期望用户输入的最长字符串长一到两个字符。

用new来创建一个对象时，JTextArea和JTextField类都可以指定一些初始文本内容。初始文本就是作为第一个实参赋给构造器的字符串——例如：

```
JTextField inputOutputField = new JTextField("Hello User.",20);
```

文本域inputOutputField可以容纳20个可见字符，且开始时会包含文本"Hello User."。

JTextArea和JTextField类都有一个不带实参的构造器，这个构造器会将各种参数都设置为默认值。

快速参考：JTextArea和JTextField类

可以用JTextArea和JTextField类向GUI中添加可变文本区。JTextArea类对象的尺寸由指定的行数和每行上指定的字符数组成。JTextField类对象只有一行，其中包含了指定数量的字符。可以向

JTextArea或JTextField输入比指定尺寸更多的文本，但多余的文本是不可见的。

举例（使用JTextArea）：

```
JPanel aPanel = new JPanel();
JTextArea someText = new JTextArea(10, 30);
aPanel.add(someText);
```

举例（使用JTextField）：

```
JPanel anotherPanel = new JPanel();
JTextField name = new JTextField(30);
anotherPanel.add(name);
```

快速参考：JTextField和JTextArea的尺寸

创建JTextArea或JTextField类对象时，要像下面这个例子这样，指定每行显示的字符数，它们指定的都是每行40个字符。

```
JTextField someText = new JTextField(40);
JTextArea someMoreText = new JTextArea(10, 40);
```

前面这两行代码中的数字40不是指任意40个字符的尺寸。在这里40表示的是40个双倍间距。一个双倍间距（em space）是指放置一个字母“m”所需的空间，字母“m”是字母表中最宽的字母。因此，像前面这个例子，宽为40的行总是至少可以装载40个字符的，而且根据特定字符的不同，放置的字符可能会多于40个。（前面例子中的10表示10行。对这个数字没有什么可解释的。）

快速参考：getText与setText

JTextArea和JTextField类中都包含名为getText和setText的方法。方法getText可以将写入文本区或文本域中的文本恢复出来。方法setText可以用来修改写入文本区或文本域中的文本。

举例（theText可以是JTextArea类的对象或JTextField类的对象）：

```
memo1 = theText.getText();
theText.setText("Memo 1 saved.");
```

快速参考：文本区中的自动换行

可以用方法setLineWrap为JTextArea设置自动换行策略。这个方法有一个boolean类型的参数。如果实参为true，那么在每行末尾，那行上额外的字符都会出现在文本区的下一行中。如果实参为false，额外的字符会在同一行上，且不可见。

举例：

如果将下列对setLineWrap的调用添加到图5-19的构造器中去，那么，当用户输入的字符在一行中放不下时，额外的字符会出现在下一行中：

```
public MemoSaver()
{
    ...
    theText.setLineWrap(true);
```

快速参考：只读文本组件

可以指定一个JTextField或JTextArea不能被用户写入。要实现这一点，可以使用JTextField

和JTextArea类中都有的方法setEditable。如果theText命名的是一个JTextField或JTextArea类的对象，那么

```
theText.setEditable(false);
```

会将theText设置为只能由GUI程序，而不能由用户修改的文本。执行了这条setEditable调用之后，如果用户点击文本组件theText，然后在键盘上输入的话，theText中的文本是不会改变的。

要改变这种状况，对theText进行设置，使用户可以编辑文本组件中的文本，可以按下列形式用true取代false。

```
theText.setEditable(true);
```

如果没有调用setEditable，那么用户就可以修改文本组件中的文本。

编程示例：标记一个文本域

有时需要为文本域使用一个标签。例如，假设GUI需要一个名字和一个识别号，并希望用户将这些信息输入两个文本域中。在这种情况下，GUI需要标记这两个文本域，这样用户就会知道在哪个域中输入名字，在哪个域中输入识别号了。可以使用JLabel类对象来标记一个文本域或Swing GUI中的其他组件。

图5-20中的程序说明了如何将标签附加到一个文本域（或任意其他组件）中。可以将文本域和标签都放到一个面板中。然后将整个面板添加到容器中去。图5-20中的高亮代码完成的就是这件事情。

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

/**
 Class to demonstrate placing a label on a text field.
*/
public class LabelDemo extends JFrame implements ActionListener
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;

    private JTextField name;

    public LabelDemo()
    {
        setTitle("Name Tester");
        setSize(WIDTH, HEIGHT);
        addWindowListener(new WindowDestroyer());
        Container content = getContentPane();
        content.setLayout(new GridLayout(2, 1));

        JPanel namePanel = new JPanel();
        namePanel.setLayout(new BorderLayout( ));
        namePanel.setBackground(Color.LIGHT_GRAY);

        name = new JTextField(20);
        namePanel.add(name, BorderLayout.SOUTH);
        JLabel nameLabel = new JLabel("Enter your name here:");
        namePanel.add(nameLabel, BorderLayout.CENTER);
```

图5-20　标记一个文本域

```
        content.add(namePanel);

        JPanel buttonPanel = new JPanel();
        buttonPanel.setLayout(new FlowLayout());
        JButton b = new JButton("Test");
        b.addActionListener(this);
        buttonPanel.add(b);
        b = new JButton("Clear");
        b.addActionListener(this);
        buttonPanel.add(b);

        content.add(buttonPanel);
    }

    public void actionPerformed(ActionEvent e)
    {
        if (e.getActionCommand().equals("Test"))
            name.setText("A very good name!");
        else if (e.getActionCommand().equals("Clear"))
            name.setText("");
        else
            name.setText("Error in window interface.");
    }

    public static void main(String[] args)
    {
        LabelDemo w = new LabelDemo();
        w.setVisible(true);
    }


}
```

得到的GUI

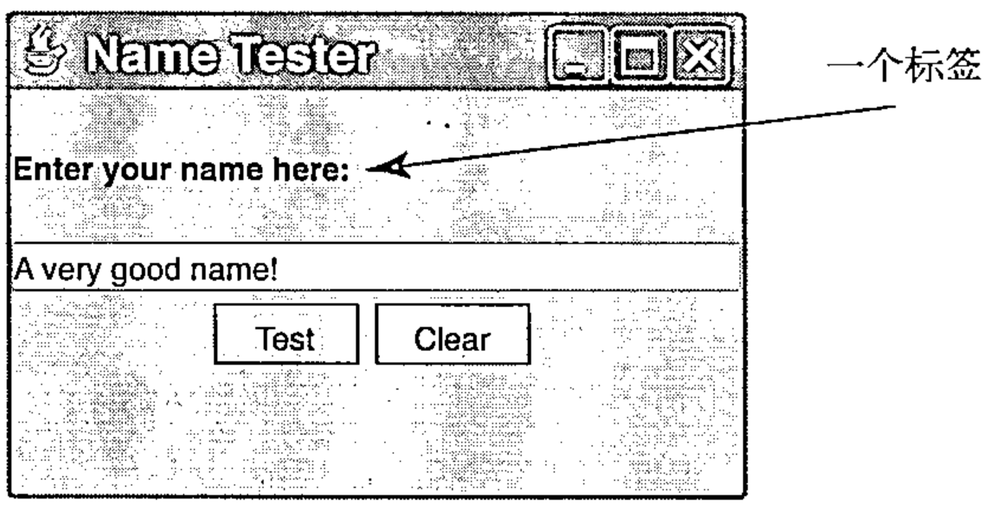


图5-20 （续）

图5-20中的程序只是一个演示程序，因此没做太多事情。如果用户在文本域中输入一个名字，并点击按钮"Test"，GUI会给出对这个名字的评价。但是，所有的名字都会收到同样的评价，即"A very good name!"。因此如果用户输入一个名字并点击按钮"Test"，得到的图看起来就会像图5-20显示的GUI那样。

自测题

40. JTextArea类对象和JTextField类对象之间有什么区别?
41. 对图5-19中的程序进行怎样的修改，才能使按钮出现在文本的上面而不是下面?
42. 编写一条语句，创建一个名为name的、可以容纳30个可见字符，且以文本"Your name here"为初始值的文本域。
43. 修改图5-19中方法actionPerformed的定义，使得用户将文本保存为备注1时，文本区会变成"Memo 1 saved."，当备注2被保存下来时，文本区会改为显示"Memo 2 saved."。
44. 在图5-19中，实例变量memo1和memo2都是String类型的。一个String类型的对象就是单个字符串。但是用户可以输入由很多行组成的文本。什么地方出问题了吗？在一个String对象中怎么能获得多行文本呢?

5.5.2　输入和输出数字

要想通过一个以Swing构建的GUI输入数字，这个GUI就必须将输入文本转换成数字。比如，在JTextArea或JTextField中输入数字42时，程序收到的会是字符串"42"，而不是数字42。程序必须将输入字符串值"42"转换成整数值42。想通过一个以Swing构建的GUI输出数字，就必须将数字转换成一个字符串，并将那个字符串输出。例如，如果想输出数字43，程序要将整数值43转换成字符串值"43"。对Swing来说，用户键入的所有输入都是字符串输入，显示的所有输出都是字符串输出。

考虑一下数字的输入。程序需要将一个字符串，比如"42"，转换成一个数字。Integer类中的静态方法parseInt可以实现这个转换。例如

```
Integer.parseInt("42")
```

会返回整数42。如果不明白这些细节的含意，应该去复习一下《Java程序设计与问题解决：基础篇》的5.2.4节。

GUI程序可以用方法getText将输入字符串恢复出来。因此，如果数字被写入一个名为inputOutputField的JTextField中，inputOutputField.getText()就会生成输入字符串。要将输入字符串改成一个数字，可以使用下列表达式：

```
Integer.parseInt(inputOutputField.getText())
```

如果用户可能会在输入之前或之后添加额外的空格，就应该为字符串对象inputOutputField.getText()添加一个对方法trim的调用。因此，一种更强壮的获取输入数字的方式如下所示（添加了对方法trim的调用）：

```
Integer.parseInt(inputOutputField.getText().trim())
```

一旦程序得到了这个数字，就可以像使用任何其他数字那样使用它了。例如，要将这个数字存储在一个名为n的int变量中，就可以使用下列赋值语句：

```
int n = Integer.parseInt(inputOutputField.getText().trim())
```

如果想输入double类型的数字，只要用Double类取代Integer类，用方法parseDouble取代parseInteger就行了。例如，要将inputOutputField中的数字存储在一个名为x的double类型变量中，就可以使用下列赋值语句：

```
double x = Double.parseDouble(inputOutputField.getText().trim());
```

还可以对Long和Float类进行类似的转换。

你应该能够理解，甚至编写出像下面这样的长表达式。

```
Integer.parseInt(stringObject.trim())
```

但是，如果定义一个方法将这条语句表示为一个简单的方法调用，代码会更易读易编。下面就是这样一个方法：

```
private static int stringToInt(String stringObject)
{
    return Integer.parseInt(stringObject.trim());
}
```

然后，一个

```
n = Integer.parseInt(inputOutputField.getText().trim())
```

这样的表达式就可以更清晰的表示为

```
n = stringToInt(inputOutputField.getText());
```

如果这个方法只是某些GUI类中的一个工具，在类之外没什么用处，就应该将其声明为私有的。或者，可以将其作为一个具有很多不同实用功能的工具类的一部分。在那种情况下，使其成为公有的可能会更有意义一些。

要向GUI的JTextField或JTextArea发送一个要输出的数字，就要使用静态方法toString。例如，假设数字43存储在int型变量sum中。可以按下列形式将这个43转换成字符串"43"：

```
Integer.toString(sum)
```

由于处理的是整数，所以在调用toString时使用的是Integer类。另一方面，如果有一个double类型的变量total，而你想将total中的数字转换成一个字符串，就应该使用

```
Double.toString(total)
```

如果希望int变量sum中的数字出现在名为inputOutputField的JTextField中，可以按下列方式使用setText：

```
inputOutputField.setText(Integer.toString(sum));
```

在下一个编程实例中对输入和输出数字的技巧进行了说明。

快速参考：用GUI输入数字

可以用Swing来设计一个GUI，使输入中包含从文本域（或文本区）中输入的数字。下列代码会返回在文本域中输入的int类型的数字（假设在文本域中除了数字和空格之外没有输入任何其他内容）。

语法：

```
Integer.parseInt(Name_Of_Text_Field.getText().trim())
```

举例（将一个int值存储在一个变量n中）：

```
int n =Integer.parseInt(inputOutputField.getText().trim());
```

可以对double、float和long类型的数字做同样的事情。只要分别用Double、Float或Long类取代Integer类，用方法parseDouble、parseFloat或parseLong取代方法parseInt就行了。

举例（将一个double值存储在变量x中）：

```
double x =Double.parseDouble(inputOutputField.getText().trim());
```

举例：

还可以将这些长公式包装到一个方法中去，比如

```
private static double stringToDouble(String stringObject)
```

```
{
    return Double.parseDouble(stringObject.trim());
}
```

(根据这个方法定义及使用位置的不同，它可以是public的，也可以是private的。)

可以用这个方法将前面的例子简化为

```
double x =stringToDouble(inputOutputField.getText());
```

快速参考：用GUI输出数字

可以用Swing设计一个GUI，使输出中包含显示在文本域（或文本区）中的一个（或多个）数字。下列语法会将一个变量中的数字显示在文本域中，用于JTextArea，而不是JTextField时，细节也是一样的：

语法：

Name_Of_Text_Field.setText(*Wrapper_Class*.toString(*Variable*));

举例：

```
inputOutputField.setText(Integer.toString(sum));
inputOutputField.setText(Double.toString(area));
```

可以对float和long类型的数字做同样的事情。只要用Float或Long类取代Integer或Double类就行了。

自测题

45. 编写一个表达式，将double值7.77转换成字符串"7.77"。用一条赋值语句将那个字符串存储在一个String类型的变量s中。(使用前面的小节中提供的技巧。)
46. 编写一个表达式，将字符串"3.14159"转换成double值3.14159。用一条赋值语句将那个值存储在一个double类型的变量x中。(使用前面的小节中提供的技巧。)
47. 假设String对象s的值为"3.14159"。编写一个表达式，以String对象s开始，并返回double值3.14159。用一条赋值语句将那个值存储到double类型的变量x中去。(使用前面的小节中提供的技巧。)

编程示例：GUI加法机

图5-21显示了一个GUI加法机程序。得到的GUI显示在画面的末端。起初，白色文本域中包含了文本"Numbers go here."。用户要输入数字时，把光标拖曳到白色文本域之上，然后键入数字，这样，输入的数字就成为文本域的内容了。当用户点击按钮"Add"时，会将这个数字添加到计算的总和中去，新的总和会显示在文本域中。只要用户愿意，就可以不断地向这个总和中添加更多的数字。要想从头开始，用户点击按钮"Reset"就可以将计算的总和清零。

注意，窗口自身（名为Adder的JFrame）就是响应按钮点击的侦听器。这种配置方式需要类定义实现ActionListener。因此，类定义要以下面这行代码开始。

```
public class Adder extends JFrame implements ActionListener
```

希望将类自身作为"Add"这样的按钮的侦听器，因此，将this注册为每个按钮的侦听器——例如：

```
JButton addButton = new JButton("Add");
addButton.addActionListener(this);
```

由于GUI类Adder是ActionListener，它就必须有一个actionPerformed方法来描述GUI是如何

响应按钮点击的。方法actionPerformed位于图5-21的后半部分。与以往一样，这个定义的核心是一条多路if-else分支，会根据被点击按钮的不同执行不同的动作。表达式e.getActionCommand()会返回写在按钮上的字符串，由那个字符串来确定执行if-else语句的哪一条分支。多路if-else语句的核心部分在图5-21被高亮显示出来了。

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

/**
 GUI for totaling a series of numbers.
*/
public class Adder extends JFrame implements ActionListener
{
    public static final int WIDTH = 400;
    public static final int HEIGHT = 200;

    private JTextField inputOutputField;
    private double sum = 0;

    public static void main(String[] args)
    {
        Adder guiAdder = new Adder();
        guiAdder.setVisible(true);
    }

    public Adder()
    {
        setTitle("Adding Machine");
        addWindowListener(new WindowDestroyer());
        setSize(WIDTH, HEIGHT);
        Container contentPane = getContentPane();
        contentPane.setLayout(new BorderLayout());

        JPanel buttonPanel = new JPanel();
        buttonPanel.setBackground(Color.GRAY);
        buttonPanel.setLayout(new FlowLayout());
        JButton addButton = new JButton("Add");
        addButton.addActionListener(this);
        buttonPanel.add(addButton);
        JButton resetButton = new JButton("Reset");
        resetButton.addActionListener(this);
        buttonPanel.add(resetButton);
        contentPane.add(buttonPanel, BorderLayout.SOUTH);

        JPanel textPanel = new JPanel();
        textPanel.setBackground(Color.BLUE);
        textPanel.setLayout(new FlowLayout());

        inputOutputField = new JTextField("Numbers go here.", 30);
        inputOutputField.setBackground(Color.WHITE);
        textPanel.add(inputOutputField);
```

图5-21 一个加法GUI

```
        contentPane.add(textPanel, BorderLayout.CENTER);
    }

    public void actionPerformed(ActionEvent e)
     {
        if (e.getActionCommand().equals("Add"))
        {
            sum = sum +
                stringToDouble(inputOutputField.getText());
            inputOutputField.setText(Double.toString(sum));
        }
        else if (e.getActionCommand().equals("Reset"))
        {
            sum = 0;
            inputOutputField.setText("0.0");
        }
        else
            inputOutputField.setText("Error in adder code.");
     }

    private static double stringToDouble(String stringObject)
    {
        return Double.parseDouble(stringObject.trim());
    }
}
```

得到的GUI

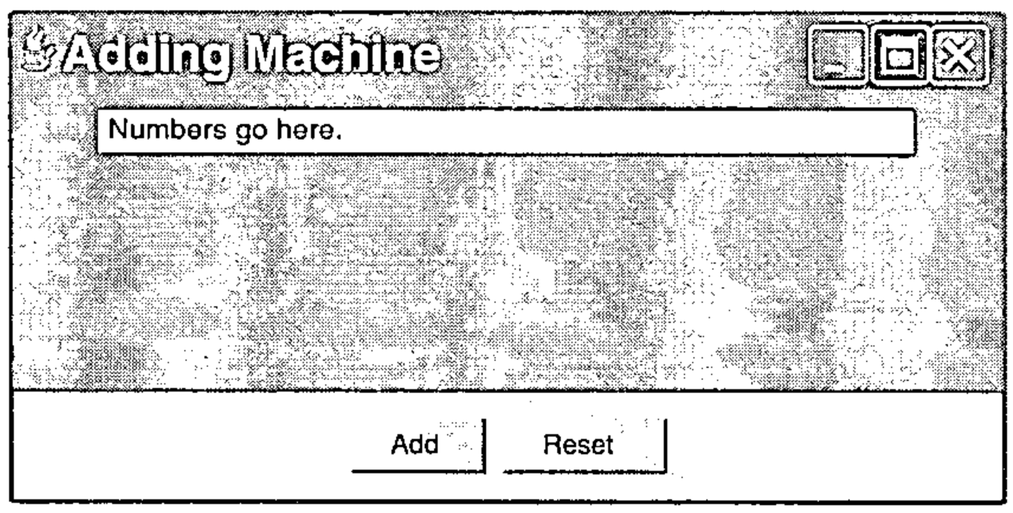


图5-21　（续）

将组件放入容器，并对组件、颜色等特性进行初始化的工作都与前面所见过的其他例子类似。

自测题

48. 如果运行图5-21中GUI的用户在文本域中输入数字10，并三次点击按钮"Add"，会发生什么情况？对你的答案进行解释。
49. 在图5-21的GUI中，为什么要将文本域inputOutputField作为一个实例变量，而没有将任意一个按钮（addButton和resetButton）作为实例变量？
50. 假设将图5-21中的main方法改成了下列形式：

```
public static void main(String[] args)
{
    Adder guiAdder1 = new Adder();
```

```
    guiAdder1.setVisible(true);
    Adder guiAdder2 = new Adder();
    guiAdder2.setVisible(true);
}
```

这种改变会显示出两个加法器窗口。(如果一个窗口在另一个窗口之上，可以用鼠标移开上面那个。)如果在一个加法器中添加数字，会对另一个加法器产生什么影响吗？最好的办法就是去试一试，但如果试过之后还是不确定的话，在本章末尾有问题的答案。

51. 假设按照自测题50所述，对图5-21中的main方法进行了修改。这种修改会显示出两个加法器窗口。如果点击一个窗口的关闭窗口按钮，是只有一个窗口消失还是两个窗口都会消失呢?
52. 假设，要在图5-21的GUI中，添加第三个按钮，这个按钮会清除文本域，但不会重置计算总和。这个按钮上应该写上"Clear"。例如，如果计算总和为10，而点击了按钮"Clear"，文本域会变空。然后，如果输入数字5，并点击"Add"按钮，那么计算总和就会变成15，而且会在文本域中显示"15"。要对Adder类的定义进行怎样的修改才能完成这项任务呢?

5.5.3 捕获一个NumberFormatException

本节是构建在第1章——异常处理的内容之上的。如果还没有学过异常处理的内容，可以跳过本小节直到学过本书第1章为止。如果学过本书第1章的部分内容，本节就不是特别难，也就不需要跳过它了。

图5-21中的GUI是作为一个加法机使用的，用户输入数字，它就对数字相加，并维持一个计算总和。但是，那个GUI有一个问题。如果用户以不正确的格式输入一个数字，比如在数字中加入一个逗号，就会有一个方法抛出一个NumberFormatException。在一个Swing程序中，抛出一个未被捕获的异常并不会终止GUI，但会将GUI置于一种不可预测的状态。在某些系统中，用户可能可以重新输入数字，但是不能指望它一定会这样，而且即使可以这样做，用户可能也不知道是否应该，以及是否能够重新输入最后输入的那个数字。图5-22对图5-21中的GUI类进行了少量修改。用户输入一个格式不正确的数字时，会抛出一个NumberFormatException，在这个版本中，这个异常被捕获后，会请求用户重新输入那个数字（如图5-22的GUI画面所示)。

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

/**
 GUI for totaling a series of numbers. If the user
 enters a number in an incorrect format, such as
 2,000 with a comma, then an error message is generated
 and the user can restart the computation.
*/
public class ImprovedAdder extends JFrame
                           implements ActionListener
{
    public static final int WIDTH = 400;
    public static final int HEIGHT = 200;

    private JTextField inputOutputField;
```

图5-22 带有异常处理的GUI

```
    private double sum = 0;

    public static void main(String[] args)
    {
        ImprovedAdder guiAdder = new ImprovedAdder();
        guiAdder.setVisible(true);
    }

    public ImprovedAdder()
        <定义的其余部分与图5-21中Adder的构造器一样。>
    public void actionPerformed(ActionEvent e)
    {
        try
        {
            tryingCorrectNumberFormats(e);
        }
        catch (NumberFormatException e2)
        {
            inputOutputField.setText("Error: Reenter Number.");
        }
    }
```

除了类名和方法actionPerformed有所变化之外，这个类与图5-21中的Adder类完全相同。

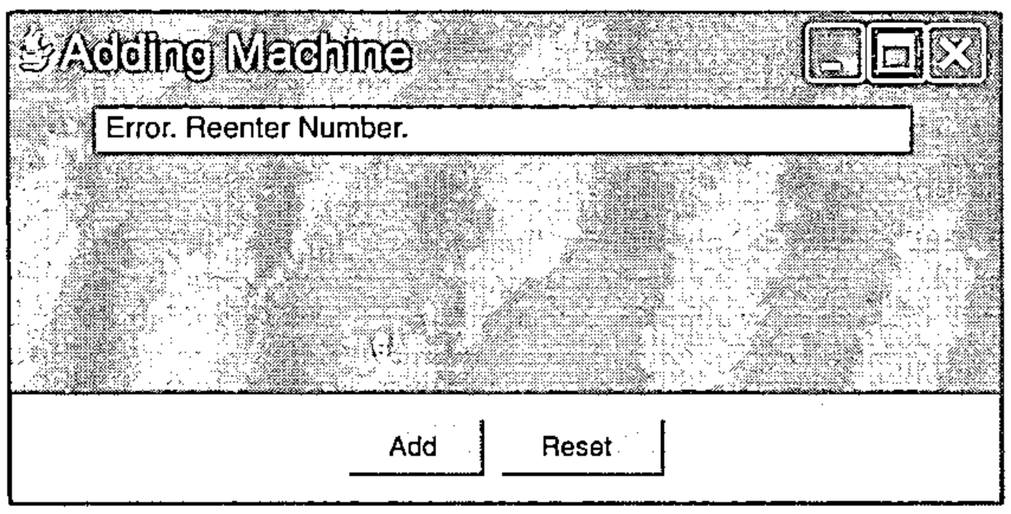
不需要在throws子句中声明这些NumberFormatException，但对它们的捕获方式与对其他异常的捕获方式相同。

```
    //This method can throw NumberFormatExceptions.
    public void tryingCorrectNumberFormats(ActionEvent e)
    {
        if (e.getActionCommand().equals("Add"))
        {
            sum = sum + stringToDouble(inputOutputField.getText());
            inputOutputField.setText(Double.toString(sum));
        }
        else if (e.getActionCommand().equals("Reset"))
        {
            sum = 0;
            inputOutputField.setText("0.0");
        }
        else
            inputOutputField.setText("Error in adder code.");
         }

        //This method can throw a NumberFormatException.
        private static double stringToDouble(String stringObject)
        {
            return Double.parseDouble(stringObject.trim());
        }
    }
```

图5-22　(续)

得到的GUI



只要用户输入的是格式正确的数字，这个GUI的表现就与图5-21中的那个GUI完全相同。

如果用户输入了一个格式不正确的数字，比如一个带有逗号的数字，GUI看起来就会是这样的。

图5-22 （续）

用户点击图5-22 GUI中的某个按钮时，会激发一个动作事件e。然后，会自动地以这个动作事件e为实参来调用方法actionPerformed。方法actionPerformed会将动作事件e作为实参来调用方法tryingCorrectNumberFormats。然后，方法tryingCorrectNumberFormats会像前面的程序版本（图5-21）那样处理事件。如果用户输入的所有数字都具有正确的格式，那么所有的表现都会与前面的那个版本（图5-21）一样。但是如果用户输入了一个格式错误的数字，就会发生一些不同的情况。

例如，当用户输入带有逗号的2,000，而不是2000时，方法tryingCorrectNumber-Formats就会以（被称为数字字符串的）"2,000"为实参来调用方法stringToDouble。然后stringToDouble会调用Double.parseDouble，但Double.parseDouble无法将"2,000"转换成一个数字，因为任何Java数字字符串中都不能包含逗号。因此，方法Double.parse-Double会抛出一个NumberFormatException。由于这种情况是在对stringToDouble的调用中发生的，因此，stringToDouble也会接着抛出一个NumberFormatException。stringToDouble调用是在tryingCorrectNumberFormats调用中发生的，因此trying CorrectNumberFormats会将它从stringToDouble调用中接收到的NumberFormatException抛出。但是，对tryingCorrectNumberFormats的调用是位于一个try-catch块中的（图5-22中的高亮部分）。这个异常会在catch块中被捕获。那时，会将（名为inputOutputField的）JTextField设置为错误消息"Error: Reenter Number."，这样，GUI看起来就像图5-22中显示的那样了。

无论什么时候，只要Swing GUI中有数字输入，代码就必须将"2000"这样的字符串输入转换成2000这样的数字。如果用户输入了一个字符串，而这个字符串不具有相应类型数字的正确格式，就会抛出一个NumberFormatException。捕获所有这样的NumberFormatException是个很好的主意，这样GUI就可以适度地做一些事情，比如给出一条错误消息，而不会进入一种尴尬的状态。

注意，如果抛出了NumberFormatException，实例变量sum的值是未被改变的。NumberFormatException可能是方法tryingCorrectNumberFormats的下列代码行中的stringToDouble调用抛出的：

```
sum = sum + stringToDouble(inputOutputField.getText());
```

如果抛出了异常，方法的执行会立即结束。因为异常是由方法stringToDouble抛出的，那么

异常就会在执行前面的加法操作之前抛出。因此，如果抛出了异常，那么，会改变sum值的那个加法操作就还没有执行。因此，sum未被改变，用户可以重新输入最后一个数字，就像从来没有输入过那个不正确数字一样继续运行这个GUI。

注意，图5-22中没有哪个方法具有下列形式的throws子句

```
throws NumberFormatException
```

这是因为NumberFormatException是RuntimeException类的子孙，而Java不要求在throws子句中声明RuntimeException。但是，仍然可以捕获NumberFormatException，就像捕获所见过的所有其他异常一样。

小　结

- GUI（图形用户界面）是用事件驱动编程方式进行编程的。在事件驱动编程中，像用鼠标点击按钮这样的用户动作会生成一个事件，而那个事件会被自动地传送到一个会执行适当动作的事件处理方法中去。
- 用Swing构建GUI的方法主要有两种。可以通过继承来创建某个预定义Swing类的派生类，或者可以通过向一个容器类中添加组件来构建GUI。在定义一个类时，这两种方法通常都会被用到。
- JFrame类是一个用来创建视窗GUI的Swing类。视窗GUI是作为JFrame类的派生类定义的。
- 标签是JLabel类的对象。可以用标签向GUI添加一个文本字符串。
- 按钮是JButton类的对象。点击按钮会激发一个动作事件，这个事件由一个动作侦听器负责处理。任意一个实现了ActionListener接口的类都可以作为动作侦听器。
- 可以用方法add向一个容器类对象中添加组件，比如向一个窗口添加按钮。容器中的组件是由一个名为布局管理器的对象排列的。
- 不能对JFrame类的对象使用方法add。相反，要用方法getContentPane得到对象的内容面板，然后，对那个内容面板使用add方法。
- 面板是一个容器对象，用来组织较大容器内部的多个组件。面板是JPanel类中的对象。
- 在用Swing构建的GUI中，用文本域（JTextField类的对象）和文本区（JTextArea类的对象）进行文本输入和输出。

✓ 自测题答案

1. 在本章之前，见过的所有程序都设计为由一个代理（计算机）执行，这个代理会遵循一系列形式为“先做这个，再做那个，然后做其他一些事情”的简单指令集来执行。在事件驱动编程中，要创建一些对象，并使它们通过事件进行交互。每个对象只是等待一些事件的发生，如果适用的话，就对事件进行响应。因此，接下来会发生的事是取决于下一个事件的。
2. GUI发“gooey”的音，表示图形用户界面。
3. 将创建标签的代码行从

```
JLabel myLabel = new JLabel("Please don't click that button!");
```

改成

```
JLabel myLabel = new JLabel("I love you");
```

4. Swing中的尺寸是以像素来衡量的。
5. JFrame类。
6. System.exit(0);

7. myGUI.getContentPane().add(superLabel);

8. messageWindow.setVisible(n > 0);

   下列代码也能工作，但风格不太好：

```
if (n > 0)
    messageWindow.setVisible(true);
else
    messageWindow.setVisible(false);
```

9. 点击最小化（图标化）按钮时，JFrame被缩小为一个图标，通常位于监视器屏幕的底端。

10. 需要按照下列形式调用getContentPane：

```
messageWindow.getContentPane().add(warningLabel);
```

11. 将图5-18构造器中的代码行

```
setBackground(Color.BLUE);
```

    改成

```
setBackground(Color.YELLOW);
```

12. 使用从JFrame类中继承的setTitle方法。例如，图5-8构造器中的下列代码行就将窗口标题设置为"Second Window"了：

```
setTitle("Second Window");
```

13. 使用从JFrame类中继承的setVisible方法。下列代码行会将窗口显示出来：

```
gui.setVisible(true);
```

    要使窗口不可见，可以使用

```
gui.setVisible(false);
```

14. addWindowListener(new WindowDestroyer());

15. Container contentPane = w.getContentPane();

16. Container类型需要下列import语句：

```
import java.awt.*;
```

17. 需要按下列重写代码段中的形式修改add语句：

```
JLabel label1 = new JLabel("First label here.");
content.add(label1, BorderLayout.NORTH);

JLabel label2 = new JLabel("Second label there.");
content.add(label2, BorderLayout.CENTER);

JLabel label3 = new JLabel("Third label anywhere.");
content.add(label3, BorderLayout.SOUTH);
```

18. 需要按下列重写代码段中的形式修改add语句：

```
JLabel label1 = new JLabel("First label here.");
content.add(label1, BorderLayout.NORTH);

JLabel label2 = new JLabel("Second label there.");
content.add(label2, BorderLayout.EAST);

JLabel label3 = new JLabel("Third label anywhere.");
content.add(label3, BorderLayout.SOUTH);
```

19. 实参new BorderLayout()创建了一个BorderLayout类的新对象，并将这个实参传递给了方法setLayout。这样使用new可以在创建一个实参时不为其起名。

20. setLayout(new GridLayout(1, 3));

    用BorderLayout或FlowLayout也可以完成类似的任务，不过在这里GridLayout管理器会工作的

更好一些。

21. `setLayout(new GridLayout(3, 1));`

用BorderLayout也可以完成类似的任务，不过在这里GridLayout管理器会工作的更好一些。

22. 需要一个动作侦听器。在类头部的末尾（如果类是一个窗口对象，就在`extends JFrame`的后面）添加`implements ActionListener`，就可以使一个类成为一个动作侦听器。还必须向类中添加方法actionPerformed的定义。每个会产生动作事件的组件通常都应该有一个或多个注册到这个组件上的动作侦听器。可以用方法addActionListener来注册动作侦听器。

23. 是的。这两段代码是等价的。有些人认为每个按钮都必须有它自己的名字，但在这个例子中不需要。在替换代码段中，为一个按钮使用了名字b，然后，在这个按钮建立起来并添加到GUI中去之后，再次将名字b用于第二个按钮。实际上，第二种编写代码的方式是大多数有经验的程序员会使用的方式，但使用有两个名字的第一种方式以避免混淆，也没问题。

24. 可以用（除了关键字之外）喜欢的任意标识符来命名内容面板。因此下列代码是完全合法的：

```
Container insideOfJFrame = getContentPane();
```

（当然，如果进行了这种修改，就要用insideOfJFrame来替代在方法actionPerformed中其他地方出现的所有contentPane。）

25. 我们说一个事件被“发送”到侦听器对象中去，实际上是说以事件对象作为实参调用了侦听器对象中的某些方法。这种调用是自动发生的。通常，Swing GUI类定义中不需要包含对这个方法的调用。

26. 对程序没有任何影响。e是方法actionPerformed的一个参数，可以以任意（非保留字）标识符作为参数。

27. 向一个JFrame中添加组件时，需要用getContentPane来获取内容面板，然后，将组件添加到内容面板中去。将组件添加到JPanel中去时，没有内容面板，也不需要使用getContentPane。

28. 来自JButton的事件会被传送到动作侦听器中去。按钮及其他一些组件会激发ActionEvent类中的事件。这些事件被称为动作事件，必须由动作侦听器处理。任何类都可以作为动作侦听器类。细节的提纲如下所示。

（1）通过向类定义头部添加短语

```
implements ActionListener
```

以及添加一个名为actionPerformed的方法定义，使某个类（可能是一个窗口类）成为一个动作侦听器。动作侦听器可以是任何类。尤其是，除了作为动作侦听器之外，这个类还可以完成其他的功能。

（2）为会激发动作事件的按钮（或其他组件）注册ActionListener对象。要完成这项任务，就要使用方法addActionListener。按钮或其他组件可以注册多于（或少于）一个的侦听器。

29. 它们在很多细节上都有共性，但在这个问题中所要寻找的共性是：它们都是侦听器类。WindowDestroyer是一个窗口侦听器类，而ButtonDemo是一个动作侦听器类。

30. 类型为Container。要使用Container，需要下列代码行：

```
import java.awt.*;
```

（很可能还需要其他的import语句，但这条语句是使用Container类型（类）所需的。）

31. `import java.awt.*;`

（很可能还需要其他的import语句，但这条语句是使用布局管理器类所需的。）

32. 两者都是。JPanel类的对象既是一个容器类又是一个组件类。

33. FlowLayout管理器将添加到容器中去的组件一个接一个、从左至右排列。组件是按照其添加到容器中去的顺序排列的。

34. BorderLayout管理器会将组件放入5个区域BorderLayout.NORTH、BorderLayout.SOUTH、BorderLayout.EAST、BorderLayout.WEST和BorderLayout.CENTER中去。这5个区域是按下列形式排列的。

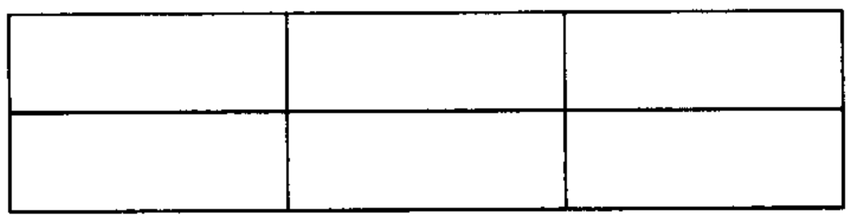
	BorderLayout.NORTH	
BorderLayout.WEST	BorderLayout.CENTER	BorderLayout.EAST
BorderLayout.SOUTH		

35. 将两个按钮放入一个JPanel对象中，并将JPanel对象添加到区域BorderLayout.NORTH中去。

36. GridLayout管理器将组件按照行和列排列，每个条目都有相同的尺寸。例如，下列代码行说明GridLayout管理器应该与aContainer（可以是内容面板或者其他容器）一同使用：

```
aContainer.setLayout(new GridLayout(2, 3));
```

作为实参赋予构造器GridLayout的两个数字分别说明了行数和列数。此代码会生成下列形式的布局。



使用GridLayout类时，方法add只有一个实参。条目在栅格中是从左至右放置的，首先填入最上面一行，然后第二行，以此类推。不能跳过任何栅格位置。

37. 这行代码说明了一个两行三列的布局管理器。因此，看起来它最多可以装载6个组件。但是，你可以添加任意数量的组件。如果添加的组件多于6个，就会有更多列被添加上去。

38. 要想让它看起来好像有一个空的栅格元素，可以向栅格元素中添加一个空面板。

39.
```
import javax.swing.*;
import java.awt.*;
public class PinkJPanel extends JPanel
{
   public PinkJPanel()
   {
      setBackground(Color.PINK);
   }
}
```

读一下Web上的PinkJPanelDemo.java程序，就会发现使用PinkJPanel的方式与使用普通JPanel的方式是一样的。

40. 文本域（JTextField）只显示一行文本。文本区（JTextArea）可以显示多行文本。

41. 将构造器中的代码行

```
contentPane.add(buttonPanel, BorderLayout.SOUTH);
```

改成

```
contentPane.add(buttonPanel, BorderLayout.NORTH);
```

42.
```
JTextField name = new JTextField("Your name here.", 30);
```

43. 下列代码中用颜色标出的部分是对图5-19进行的修改：

```
public void actionPerformed (ActionEvent e)
{
      String actionCommand = e. getActionCommand();
      if (actionCommand .equals("Save Memo 1")
      {
```

```
            memo1 = the Text.getText();
            theText.setText("Memo 1 saved.");
        }
        else if (actionCommand.equals("Save Memo 2")
        {
            memo 2 = theText.getText();
            theText. setText("Memo 2 saved.");
        }
else if (actionCommand.equals("Clear")
        theText.setText("");
else if (actionCommand.equals("Get Memo 1")
        theText.setText(memo1);
else if (actionCommand.equals("Get Memo 2"))
        theText.setText(memo2);
else
        theText.setText("Error in memo interface");
}
```

44. 要在一个String对象中获得多行，只要在断行的地方插入字符'\n'就行了。getText返回JTextArea中的内容时，这个过程是自动进行的。

45. `String s = Double.toString(7.77);`

46. `double x = Double.parseDouble("3.14159");`

    或者，可以将方法stringToDouble（请参见名为“用GUI输入数字”的快速参考方框）添加到类中去，并使用

    `double x = stringToDouble("3.14159");`

47. `double x = Double.parseDouble(s.trim());`

    或者，可以将方法stringToDouble（请参见名为“用GUI输入数字”的快速参考方框）添加到类中去，并使用

    `double x = stringToDouble(s);`

    （trim位于方法stringToDouble中。）

48. 每次用户点击按钮"Add"时，都会执行方法actionPerformed中的下列子句：

```
if(e.getActionCommand().equals("Add"))
{
   sum = sum + stringToDouble(inputOutputField.getText());
   inputOutputField.setText(Double.toString(sum));
}
```

    每次产生"Add"按钮事件时，都会对sum赋值。因此，用户点击多少次"Add"按钮，就会在总和上加上多少次文本域中的数字。而且，文本域中的值是计算总和，因此会将计算总和与它自己相加。如果用户三次点击按钮"Add"，会将文本域中的数字三次加到总和中去。

    假设用户启动GUI，输入10，然后点击按钮"Add"。这会向sum加10，因此sum的值为10，并将10显示出来。现在用户再次点击按钮"Add"。这样会再次向sum加10，因此，sum的值为20，20被显示出来。现在用户第三次点击"Add"。这一次，文本域中为20，因此会将它加到sum中去，sum为20。因此，现在sum的值为40，40被显示出来。注意，添加到sum中去的总是文本域中的数字。

49. 将文本域作为实例变量是因为需要在方法actionPerformed的定义中引用它，如下所示：

    `sum = sum + stringToDouble(inputOutputField.getText());`

    另一方面，对按钮唯一的直接引用出现在构造器中。因此，只需要在构造器定义中为按钮起名。（不需要为了成为Swing对象或任何其他类型对象的一部分，将一个对象命名为实例变量。只要在使用，

对象就不会消失，而按钮是由Adder类的对象使用的，因此无论是否为它们取一个永久的名字，它们都会存在的。）

50. 从数字角度来说，这两个加法器是完全独立的。一个加法器中的总和以及添加到这个加法器中的数字都对另一个加法器没有影响。

51. 点击任意一个加法器窗口中的关闭窗口按钮，就会执行对System.exit(0);的调用，整个程序就会结束，因此，这两个窗口都会消失。

52. 需要通过向构造器添加下列代码来添加"Clear"按钮：

```
JButton clearButton = new JButton("Clear");
clearButton.addActionListener(this);
buttonPanel.add(clearButton);
```

还需要将方法actionPerformed改成下列形式（附加的部分以加粗的形式显示出来）：

```
public void actionPerformed (ActionEvent e)
{
      if (e. getActionCommand().equals("Add"))
      {
         sum = sum +
            stringToDouble(inputOutputField.getText());
         inputOutputField.setText(Double.toString(sum));
      }
      else if (e.getActionCommand().equals("Reset"))
      {
         sum = 0
         inputOutputField.setText("0.0");
      }
      else if (e.getActionCommand().equals("Clear"))
         inputOutputField.setText("")
         //Note that sum is not changed.
      else
         inputOutputField.setText("Error in adder code.")
}
```

试着进行这些修改，然后运行修改后的程序是个不错的主意。

○ 编程项目

1. 重新编写图5-17中的程序，使得较大面板变成红色时，那个带有按钮的面板变成粉色，较大的面板变成绿色时，那个带有按钮的面板变成蓝色。向较大的面板中添加一个写有"Watch this panel!"的标签，向按钮面板中添加一个标记为"Change"的按钮。点击按钮"Change"时，颜色会发生变化（分别从粉色和红色变成蓝色和绿色，或相反。）按钮"Change"对初始配置没有影响，在初始配置中大面板为蓝色，按钮面板为灰色。

2. 重新编写图5-19中的程序，对其进行下面几项修改。

   i. 类名为MemoSaver2。

   ii. 有6个而不是5个按钮，按下列形式排列：

Save Memo 1	Save Memo 2	Clear
Get Memo 1	Get Memo 1	Exit

   按钮仍然位于GUI的底部，文本区在按钮的上面。（提示：在按钮面板上使用GridLayout管理器）

   iii. 用户将文本保存为备注1时，修改文本区，使其显示"Memo 1 saved."，保存备注2时，修改文

本区，使其显示"Memo 2 saved."（提示请参见自测题43。）

iv. 点击"Exit"按钮时，程序结束，窗口消失。关闭窗口按钮也会结束程序。因此，"Exit"按钮和关闭窗口按钮执行的是同样的动作。

v. 除了默认构造器，还有一个下列形式的构造器：

```
public MemoSaver2(int lineCount, int charCount)
```

使用这个构造器时，除了文本区的尺寸为lineCount行，每行charCount个字符之外，生成的画面是一样的。

vi. 文本区有自动换行，因此如果输入的字符在那一行中放不下，多余的字符会自动地进入下一行。

vii. 方法main构建了两个窗口，一个使用了默认构造器，一个使用了条目v.中的构造器，所用的参数按顺序为5和60。

3.（应该在实现这个项目之前完成编程项目2。）用Swing编写一个GUI，这个GUI的行为如下所示：运行程序时，会出现一个窗口，询问用户希望一个备注保存器有多少行，每行有多少个字符。如果用户点击关闭窗口按钮，程序就结束。更一般地来说，用户将这两个数字输入两个文本域中。GUI中有一个写着字符串"Continue"的按钮。如果用户点击按钮"Continue"，这个窗口就会消失，出现另一个窗口。第二个窗口和编程项目2中的那个备注保存器一样，只是文本区中的行数和每行的字符数是由用户在前面的窗口中指定的。

4.（这个项目的Swing部分非常简单，但是，需要了解一些从一种进制转换成另一种进制的知识。）编写一个程序，将数字从10进制（普通的十进位）记法转换成2进制记法。程序用Swing通过一个视窗界面来执行输入和输出。用户在一个文本域中输入一个10进制数字，并点击一个写有"Convert"的按钮。然后，等价的2进制数字会出现在另一个文本域中。要确保两个文本域都被标记了。要包含一个在点击时会将两个文本域都清空的"Clear"按钮。同样，要确保关闭窗口按钮能够正确地工作。

5.（在实现这个项目之前完成编程项目4会是很有帮助的。）编写一个程序将数字从2进制记法转换成10进制记法（普通的十进位）。程序用Swing通过一个窗口界面来执行输入和输出。用户在一个文本域中输入一个2进制数字，并点击写着"Convert"的按钮。然后，等价的10进制数就会出现在另一个文本域中。要确保两个文本域都被标记了。要包含一个在点击时会将两个文本域都清空的"Clear"按钮。同样，要确保关闭窗口按钮能够正确地工作。（提示：包含一个私有方法，它将2进制数字的字符串转换成一个等价的int值。）

6.（在实现这个项目之前完成编程项目4和5会是很有帮助的。）编写一个程序，实现数字从2进制记法到10进制记法（普通的十进位），及反方向的转换。程序用Swing通过一个窗口界面来执行输入和输出。有两个文本域，一个用于2进制数字，一个用于10进制数字。有三个分别标识着字符串"To Base 10"、"To Base 2"和"Clear"的按钮。如果用户在2进制文本域中输入一个2进制数字，并点击上面写有"To Base 10"的按钮，等价的10进制数字就会出现在10进制文本域中。类似地，如果用户在10进制文本域中输入一个10进制数，并点击上面写有"To Base 2"的按钮，等价的2进制数字就会出现在2进制文本域中。要确保两个文本域都被标记了。要包含一个在点击时会将两个文本域都清空的"Clear"按钮。同样，要确保关闭窗口按钮能够正确地工作。

7. 编写一个程序，产生一个功能和外观都很像手持计算器的GUI。计算器应该可以进行加法、减法、乘法和除法运算。应该可以保存两个不同的值，并在稍后将其恢复出来。以图5-22中的程序作为模版。如果没有学过图5-22的内容，就将图5-21作为模版。

8. 完成本书第2章的编程项目4，但将程序用Swing GUI编写。

9. 完成本书第2章的编程项目7，但将程序用Swing GUI编写。

第6章

applet与HTML

访问我们的网站。

——常见的广告用语

本章之前的所有内容，对于学习如何编程，以及Java语言的学习来说，都是很重要的。但是，还没有涉及到最初Java是怎么成名的。Java的出名很大程度上要归功于它与因特网的联系。

本章描述了一个可以在因特网上运行的Java程序版本。万维网（Word Wide Web，简写成Web）是可以用Web浏览器查看的、因特网站点的集合。applet只是从Web文档（页面）中运行的Java程序。HTML是一种用来创建Web文档的语言。尽管Java applet是设计为从HTML文档内部运行的，但也可以将applet作为一个独立的程序运行而不涉及HTML或因特网。因此，先来讨论applet，然后再讨论HTML。

目标

- 弄清楚如何编写applet。
- 学会编写简单的HTML文档。
- 学习如何将一个applet嵌入到HTML文档中。

预备知识

节　　号	预 备 知 识
6.1节	第5章（及其预备知识）
6.2节	不需要预备知识，甚至也不需要6.1节的知识
6.3节	6.1节和6.2节

如果想跳过本章，或推迟本章的阅读，也是可以的。第7章没有使用任何本章中的内容。

如果只想学习如何编写applet，而且不会从一个HTML文档（Web页面）中运行这些applet，那么，6.1节就可以给出需要的所有信息了。

如果已经了解了HTML，但不了解applet，可以跳过6.2节，从6.1节进入6.3节。

要想从6.2节和6.3节中获益，就应该知道如何使用Netscape Navigator或微软的Internet Explorer这样的Web浏览器。本章讨论的所有结构生成的内容都可以用Web浏览器浏览。你有可能用Web浏览器读过Web上的内容，但有可能不知道如何创建可以在Web上浏览的内容。大多数读者只是通过使用Web浏览器，而不需要任何指导或阅读就可以获得足够的经验了。要想完全地从本章获益，还应该了解在所用的操作系统中，路径名是如何使用的，这样就可以命名一个位于不同目录（不同文件夹）中的文件了。

6.1 applet

一天一个applet（小苹果），医生远离我。

——佚名

单词applet听起来好像指的是一个小苹果，但我们期望它听起来像个小应用程序。因此，从小（little）这个字的某种意义上来说，applet只是“小Java程序”。但是，applet的特性并不是来自于它的大小，而是来自于它们是如何运行以及从哪里运行的。applet是可以在Web站点上显示，并通过因特网查看的Java程序。也可以在不连接因特网的情况下，在本地计算机上运行。本节将对如何编写applet，以及如何在没有因特网连接的情况下查看applet进行讨论。6.3节将描述一种通过因特网查看applet的简便方式。

applet与Swing GUI非常相似，如果理解了一些与Swing有关的细节，就会发现编写applet是非常容易的。在本节中，假定你已经非常熟悉第5章中讲述的有关Swing的内容了，将向你展示如何编写一些简单的applet，来完成与第5章介绍的视窗系统相同的工作。如果继续学习更多有关Swing的知识的话，几乎可以将所有新的Swing技巧都用在applet中。

6.1.1 applet基础

applet是JApplet类的一个派生类（实际上，可以是JApplet子孙类的任意派生类，但通常会是直接派生类。）JApplet类是Swing库中的一个类，因此在使用JApplet类时，需要下列import语句。

```
import javax.swing.*;
```

编写一个applet时，可能还需要AWT库，因此，完整的import语句列表可能如下所示：

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
```

图6-1显示了部分类层次图，以帮助将JApplet类与其上下文联系起来。注意，JApplet是一个Container。因此，可以向applet中添加一些内容。向applet添加组件的方式与向JFrame添加组件的方式相同。

在图6-1中，可能注意到了Applet类。这个类是JApplet类的较老版本。Applet类是在引入JApplet之前用来创建applet的。

设计一个applet类的方式，与第5章中用Swing设计一个视窗程序的方式基本相同。主要的区别在于applet类是从JApplet类而不是JFrame类中派生出来的。一个applet类和一个普通的Swing视窗类之间的其他区别主要都是由applet类定义中省略的内容造成的。

applet没有使用setVisible方法。applet是自动显示的。因此，applet通常也没有main方法。图6-2显示了一个非常简单的applet。查看这个applet时，它只是显示了文本

```
Hello out there!
```

注意，与JFrame一样，JApplet有一个内容面板，要将组件添加到JApplet的内容面板，而不是直接添加到JApplet中去。如图6-2所示，向一个JApplet添加JLabel（或其他组件）的方式与向JFrame添加组件的方式相同。

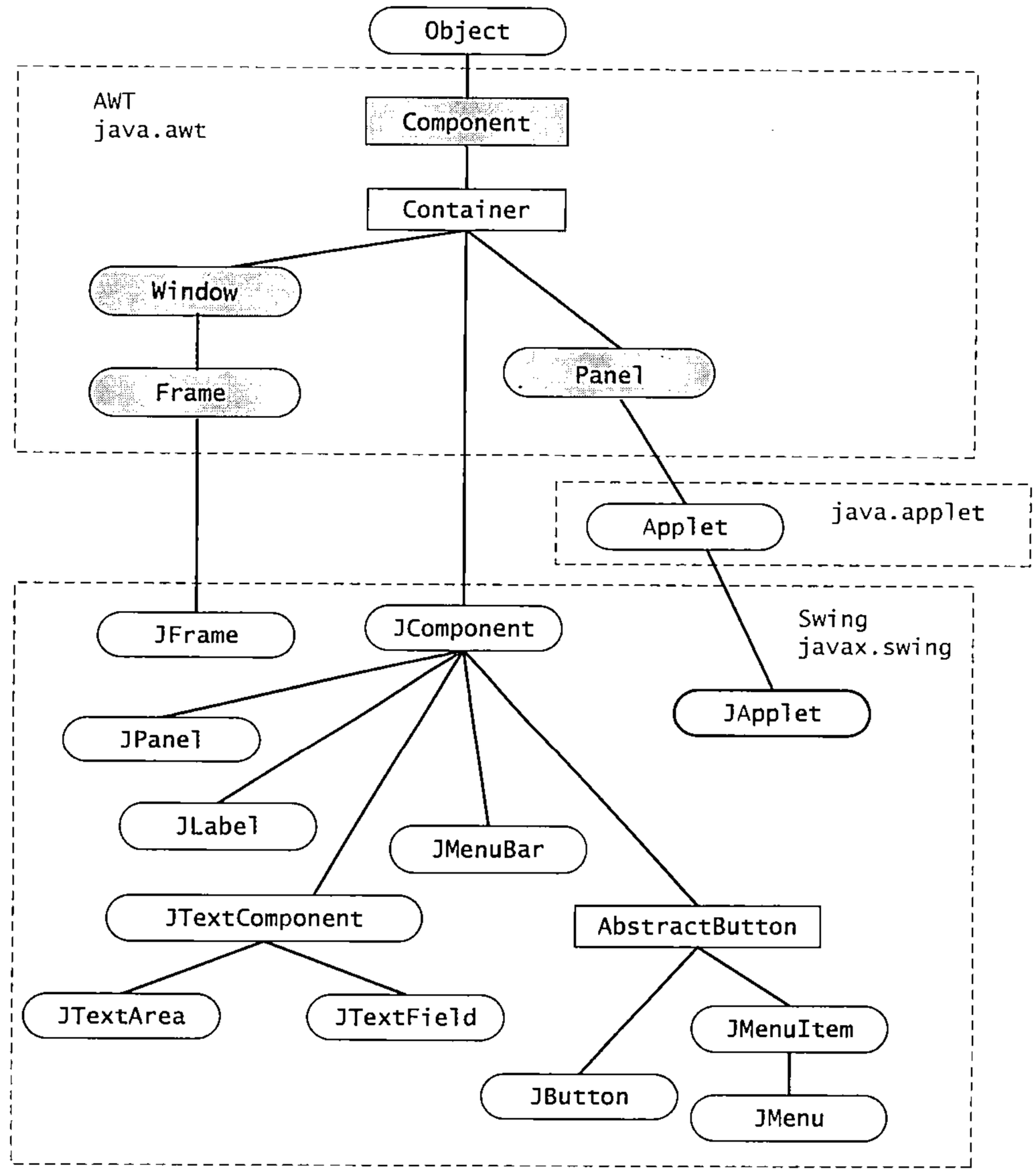


图6-1 将applet放入类层次结构中

applet没有标题，因此，没必要在applet中使用setTitle方法。这是因为applet通常是一个Web站点画面的一部分，而那个画面是可以添加任意标题的。正如在图6-3中可以看到的那样，Web文档还负责调整applet的大小，因此，不需要用任何指令来设置applet的尺寸。

applet通常不使用构造器，但它们会使用一个用于类似目的的、名为init的方法。定义一个applet时，将所有的初始化动作——比如设置颜色、添加按钮以及添加文本域——都放在名为init的方法中。init方法没有参数。

applet不需要与侦听器一同关闭，所以，在它们的init方法中不包含对addWindowListener的调用。关闭Web文档时，applet也会被自动关闭。

```
import javax.swing.*;
import java.awt.*;//For Container class

public class HelloApplet extends JApplet
{
    public void init( )
    {
        Container contentPane = getContentPane();
        contentPane.setLayout(new FlowLayout());
        JLabel friendlyLabel = new JLabel("Hello out there!");
        contentPane.add(friendlyLabel);
    }
}
```

得到的GUI（使用Applet查看程序）

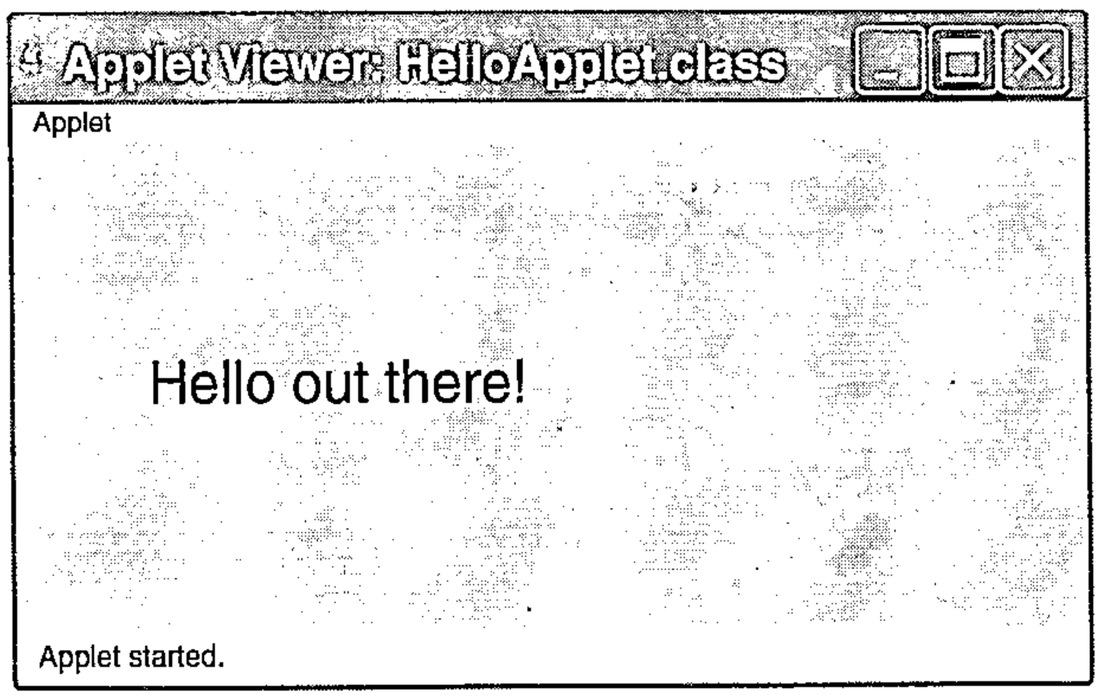


图6-2　一个小applet

快速参考：JApplet类

JApplet类是通常用来创建applet的类。这个类位于Swing库中，因此，使用这个类时，需要下列import语句：

```
import javax.swing.*;
```

6.1.2　运行一个applet

编译applet的方式与编译所见过的所有其他Java类的方式相同。但是，运行applet的方式不同于其他Java程序。运行applet的常见方式是将其作为一个Web站点文档的一部分。然后，通过一个Web浏览器来查看applet。将在6.3节讨论这种查看applet的方式。但是，还可以用applet查看程序（applet viewer）来查看applet，applet查看程序是将applet作为一个独立程序来运行的。如果使用的是一个集成环境，其中包含一条名为运行applet（Run Applet）、运行（Run）、执行（Execute）或其他类似名称的菜单命令，很可能就可以像运行普通的Java应用程序一样，用这样一条命令来运行一个applet了。（在TextPad环境中，命令为工具（Tools）菜单上的运行Java Applet（Run Java Applet）。这条环境命令会自动地调用一个applet查看程序。如果弹出一个窗

口请你选择一个文件，回答“No”。）如果这种方法不行，就要咨询本地专家，或者去阅读6.3节，以获取更多与查看applet有关的内容。

如果在applet查看程序中运行图6-3中显示的applet，结果看起来会和该图中显示的GUI很类似。

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class AdderApplet extends JApplet implements ActionListener
{
    private JTextField inputOutputField;
    private double sum = 0;


    public void init()
    {
        Container contentPane = getContentPane();
        contentPane.setLayout(new BorderLayout());

        JPanel buttonPanel = new JPanel();
        buttonPanel.setBackground(Color.GRAY);
        buttonPanel.setLayout(new FlowLayout());
        JButton addButton = new JButton("Add");
        addButton.addActionListener(this);
        buttonPanel.add(addButton);
        JButton resetButton = new JButton("Reset");
        resetButton.addActionListener(this);
        buttonPanel.add(resetButton);
        contentPane.add(buttonPanel, BorderLayout.SOUTH);
        JPanel textPanel = new JPanel();
        textPanel.setBackground(Color.BLUE);
        textPanel.setLayout(new FlowLayout());
        inputOutputField = new JTextField("Numbers go here.", 30);
        inputOutputField.setBackground(Color.WHITE);
        textPanel.add(inputOutputField);
        contentPane.add(textPanel, BorderLayout.CENTER);
    }

    public void actionPerformed(ActionEvent e)
    {
        if (e.getActionCommand().equals("Add"))
        {
            sum = sum + stringToDouble(inputOutputField.getText());
            inputOutputField.setText(Double.toString(sum));
        }
        else if (e.getActionCommand().equals("Reset"))
        {
            sum = 0;
            inputOutputField.setText("0.0");
        }
```

图6-3 一个applet加法机

```
            else
                inputOutputField.setText("Error in adder code.");
        }

        private static double stringToDouble(String stringObject)
        {
            return Double.parseDouble(stringObject.trim());
        }
    }
```

得到的GUI（使用applet查看程序）

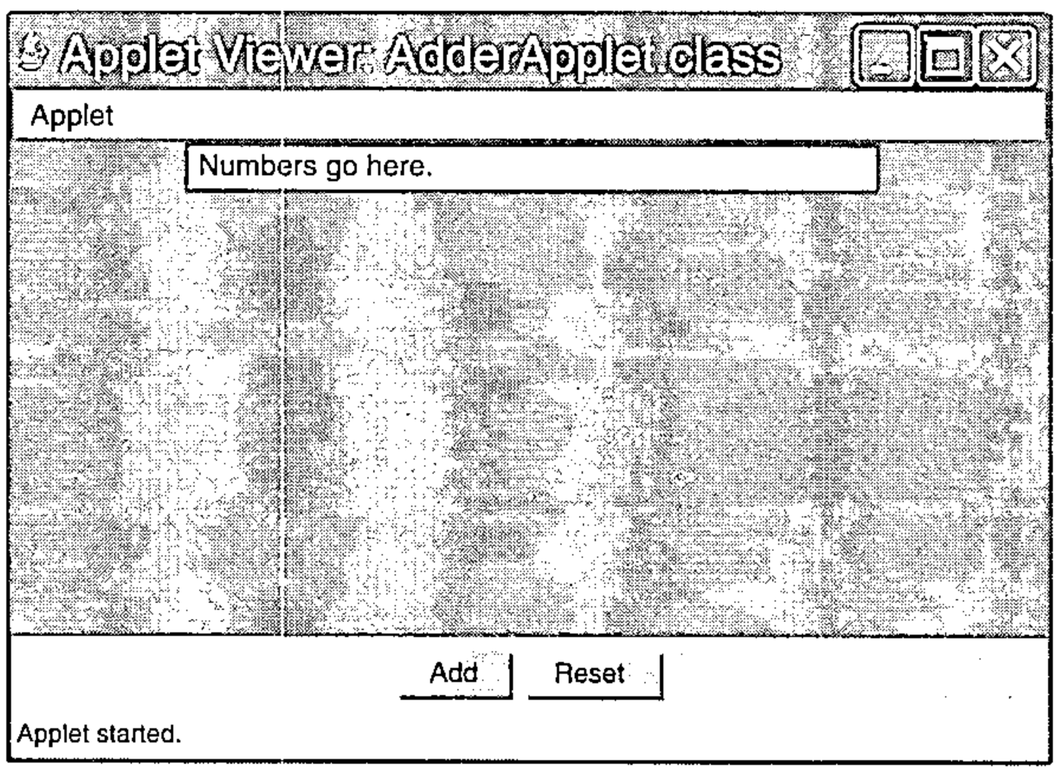


图6-3　（续）

编程示例：一个加法器applet

图6-3中包含了一个applet，这个applet会生成一个本质上与第5章生成的那个加法器一样的加法机窗口。细节基本上与在图5-21中定义的Swing类的细节相同。要从那个Swing类中得到这个加法器applet，只需做下列这些事情：

- 用extends JApplet取代extends JFrame；
- 删除main方法；
- 用init方法头部取代构造器头部；
- 删除一些applet不需要的行；

□ Java提示：将一个Swing应用程序转换成一个applet

将一个Swing应用程序转换成一个applet是很简单的。在大多数情况下，只要遵循这些指令就可以了。

(1) 从JApplet类而不是JFrame类中派生类。也就是说，在类定义的第一行中用extends JApplet取代extends JFrame。

(2) 删除main方法。applet不需要那些通常被放在main中的内容。applet会被自动设置为可见的，而且其尺寸是由applet查看程序，或它所嵌入的Web页面来决定的。

(3) 用一个名为init的方法来取代构造器。init方法的主体可以与被删除的构造器主体相同，

但要删除一些条目，如第4-6步所述。

(4) 删除所有对addWindowListener的调用。(applet查看程序或包含applet的Web页面会负责终止applet。因此，不需要任何像第5章的WindowDestroyer类对象那样的窗口侦听器对象。)

(5) 删除所有对setTitle的调用。(applet没有标题。)

(6) 删除所有对setSize的调用。(尺寸是由applet查看程序或嵌入了applet的Web页面来调整的。)

例如，就是根据这些原则，从图5-21中的Swing应用程序中得到了图6-3中的applet的。

自测题

1. 通常会在applet类定义中包含构造器吗?
2. 通常applet类中会包含main方法吗?
3. 在applet中会使用下列哪个方法? addWindowListener、getContentPane、setTitle、setSize。

6.1.3 向applet添加图标

图6-4说明了一种向applet添加图片的方法。《Java程序设计与问题解决：基础篇》的第1章给出了那个applet的简化版本，但现在了解了更多applet的知识，就可以提供一个更完整的描述了。图标就是一张图片。它通常，但不总是一张小图片。在applet中显示一个图标最简单的方式就是将图标放在一个JLabel中。在图6-4的applet中，文件duke_waving.gif中的图片是作为一个图标显示的，那个图标是名为niceLabel的JLabel的一部分。添加图标的那两行代码被复制在下面。

```
ImageIcon dukeIcon = new ImageIcon("duke_waving.gif");
niceLabel.setIcon(dukeIcon);
```

ImageIcon是Swing库中的一个类。标签niceLabel是用第5章中描述的那种常见方式创建起来的。

```
JLabel niceLabel = new JLabel("Java is fun!");
```

图标是标准格式的数码图片。必须在将图片——在这里就是duke_waving.gif——添加到一个标签中去之前，将其转换成一个ImageIcon。任务是这样完成的：new ImageIcon("duke_waving.gif")。因此，下列代码就创建了一个基于图片duke_waving.gif的ImageIcon，并将对这个图标的引用存储在变量dukeIcon中：

```
ImageIcon dukeIcon = new ImageIcon("duke_waving.gif");
```

如下列语句所示，方法setIcon将一个图标添加到标签中去：

```
niceLabel.setIcon(dukeIcon);
```

如果希望标签中只有图标没有文本，那么在创建JLabel时只要使用默认的构造器就行了。例如，如果在图6-4中使用了

```
JLabel niceLabel = new JLabel();
```

而不是

```
JLabel niceLabel = new JLabel("Java is fun!");
```

那么，字符串"Java is fun!"就不会出现在applet中，只有那个图标会被显示出来。

第7章中包含了更多有关图标的内容。

```
import javax.swing.*;
import java.awt.*;

public class DukeApplet extends JApplet
{
    public void init()
    {
        Container contentPane = getContentPane();
        contentPane.setLayout(new BorderLayout());
        JLabel spacer = new JLabel("                ");
        contentPane.add(spacer, "West");
        JLabel niceLabel = new JLabel("Java is fun!");
        ImageIcon dukeIcon = new ImageIcon("duke_waving.gif");
        niceLabel.setIcon(dukeIcon);
        contentPane.add(niceLabel, BorderLayout.CENTER);
    }
}
```

得到的GUI[①]

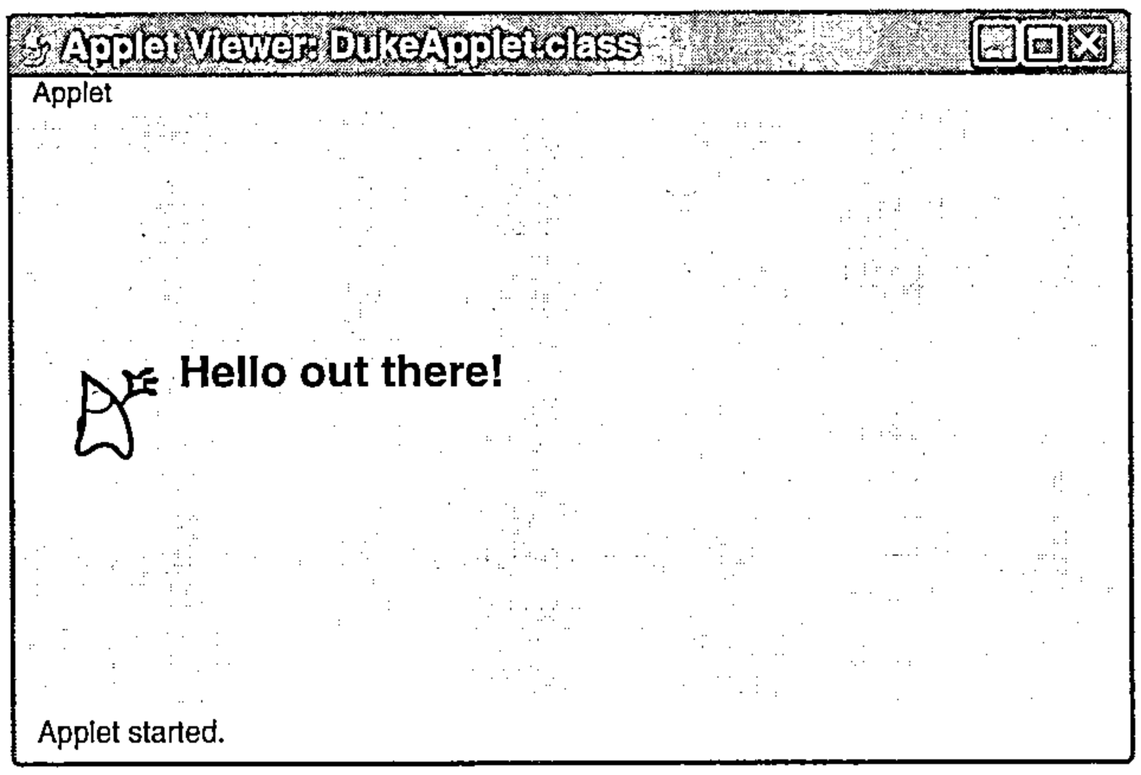


图6-4　带有图标的applet

快速参考：图标与ImageIcon类

图标（Icon）只是一个小图片，尽管它并不一定非得很小。ImageIcon类是用来将一个图片文件转换成Swing图标的。

语法：

```
ImageIcon Name_Of_ImageIcon = new ImageIcon(Picture_File_Name);
```

*Picture_File_Name*是一个字符串，它给出了图片文件的相对或绝对路径名。（因此，如果图片文件与applet位于同一个目录中，就只需要给出图片文件的名字。）

举例：

```
ImageIcon SmileyFaceIcon = new ImageIcon("smiley.gif");
```

① Java、Duke和所有基于Java的商标和徽标都是Sun微公司在美国及其他国家的商标或注册商标。

常见问题：Duke是谁？

Duke是图6-4的applet中显示的一个形象。他是Java语言的吉祥物。Duke图标在这里的使用得到了Sun微系统公司的许可。

自测题

4. 如何将一个基于数码图片文件smiley.gif的图标添加到一个applet中去？
5. 指定一个数码图片文件，比如smiley.gif，作为ImageIcon构造器实参时，可以使用路径名吗？

6.2 HTML简介

可以在一张漂亮的四开纸上看到它们，在那张纸上，一条清澈的文字小溪沿着草地边缘蜿蜒流过。

——里查得·布林斯里·谢立丹，《造谣学校》

在Web上，或者无论在哪里都可以通过一个Web浏览器阅读的文档，通常都是用一种名为**HTML**的语言编写的。HTML表示**超文本标记语言**（HyperText Markup Language）。**超文本**（HyperText）是包含了这样一些条目的文本：用鼠标点击这些条目时会进入到另一个文档中去。这些从文档到文档的连接被称为**链接**（link），或**超链接**（hyperlink）。文档自身通常被称为**页面**（page），这就是将个人或公司在Web上的主要站点位置称为**主页**（home page）的原因。术语**HTML文档**（HTML document）和**HTML页面**（HTML page）表示的是同一个意思，指的是用HTML语言创建的超文本文档。

HTML不是像Java这样完全成熟的编程语言。它只是为了将一个文本页面转换成可以用Web浏览器查看的内容而插入其中的一些简单命令的集合。可以通过这些命令向页面插入图片和超链接。还可以编写一些编辑命令来指定主标题、副标题、段落起始等。简而言之，大部分HTML只是用来格式化手稿，使其可以在Web上查看的一种语言。

这并不是一本关于HTML编程的书，因此只对这种语言进行简单的介绍。通过这里的介绍，就可以为Web（或仅仅为你的浏览器）设计一些很简单的文档了。如果想成为HTML专家，还是得去阅读专门讨论这个主题的书。

6.2.1 HTML基础

大多数HTML命令都具有下列形式。

<Command>

一些文本

</Command>

例如，下列命令会使短语“My home page”成为1级标题，即最大的标准标题。

```
<h1>
My home page
</h1>
```

注意，记法*</Command>*——在这个例子中，就是</h1>——标识了应用这条命令的文本的

结尾。

还可以设置被称为2级标题（命令h2）的较小标题，甚至被称为3级标题（命令h3）的更小的标题，以此类推。

记住：HTML是不区分大小写的

我们会用小写字母书写HTML命令。但是，与Java不同的是，HTML不区分命令中的大写和小写字母，因此如果用大写字母来表示这些命令，它们的表现也是一样的。（当然，任何要以文本形式显示给查看文档的人阅读的文本，都会将大写字母显示为大写，将小写字母显示为小写。）

有些命令不需要以*</Command>*形式的命令结束。

```
<br>
```

就是这样一条命令。这条命令会开始一个新行。另一条这样的命令为

```
<p>
```

这条命令会开始一个新的段落。

HTML中的命令不是会确定一段文本确切尺寸，甚至具体换行符的绝对命令。给出一条1级标题的命令时，可以合理地认为它会比2级标题大，但文本的确切尺寸是由浏览器决定的。可以通过**break命令**（break command）强制插入一个换行符：

```
<br>
```

如果输入一大段文本（或者甚至有时只是一小段文本），浏览器会通过一些规则来帮助其确定在什么地方插入换行符“看起来比较好”，然后在需要的地方插入一些换行符，以使文本适应于屏幕，浏览器会忽略换行符，除非是用
命令插入的。

还可以对布局进行一些说明。例如，在命令 和 之间的任何内容都会被显示在页面中间。下列命令会将前面讨论的1级标题显示在中间：

```
<h1>
<center>
My Home Page
</center>
</h1>
```

或者，如果愿意的话，也可以将代码写成下列形式：

```
<center>
<h1>
My Home Page
</h1>
</center>
```

记住：HTML文件名

HTML文件是一个普通的文本文件，可以像编写Java程序那样，用文本编辑器创建及编辑。HTML文件应该以.html结尾，除此之外，可以用与命名系统中其他文件相同的规则来命名它。

○ 编程提示：一个简单的HTML文档轮廓

图6-5包含了一个简单HTML文档的轮廓。那个图还说明了如何在HTML中编写注释。例如：

```
<!--Beginning of HTML document-->
```

就是一个注释。注释是以<!--开头，以-->结尾的。我们用注释对新的HTML命令进行了解释，但实际的文档中是不会有这么多注释的，也不会像我们这样对基本的HTML命令进行解释。

整个文档应该包含在<html>和</html>对之间，第一个标签（tag）放在开头，第二个标签放在末尾。文档的头部是包含在<head>和</head>之间的。查看文档时不显示头部，但它确实记录了浏览器会用到的一些信息。在我们文档中，它仅包含了一个（包含在<title>和</title>之间的）标题。标题是作为文档的名字使用的。例如，浏览器允许用户为文档设置一个书签，这样以后用户就可以回来看这个文档了。书签就会使用这个标题中给出的名字。（有些浏览器将书签称为“最喜爱的文档”。）

显示在屏幕上的那部分文档被分成了两部分。（包含在<body>和</body>之间的）主体（body）是文档的实际内容。显示出来的其他部分被包含在<address>和</address>之间，是可选的。这部分会给出一个可以用来联系文档主人的email地址，通常还会包含文档的最后修改日期。

图6-6显示了一个非常简单的HTML文档，图6-7说明了在浏览器中查看时，这个文档会是什么样子的。记住，具体的换行、字母尺寸和其他布局细节都是由特定的浏览器决定的，因此在你的浏览器上这个文档看起来可能会有些不同。讨论Sun微系统公司Web站点的那部分内容将在下一小节中解释。

6.2.2 插入超链接

如果HTML文档中不包含一些活动元素的话，这些努力好像都不太值得了。关键的活动元素是一个链接，查看文档的人可以点击这个链接去查看另一个HTML文档。另一个文档可能在同一台计算机上，也可能在数千英里之外的一台计算机上。这些链接被称为**超链接**，或简称为**链接**。

超链接的语法如下所示：

```
<a href="Path_To_Document">
Displayed_Text_To_Click
</a>
```

例如，下列代码就创建了一个到作者主页的链接：

```
<a href = "http://www-cse.ucsd.edu/users/savitch">
Walter Savitch
</a>
```

一定要按显示的那样包含引号。浏览器会将*Displayed_Text_To_Click*显示出来，并加上下划线（或者高亮显示出来）。在这个例子中，如果查看这个文档的人点击了文本Walter Savitch，浏览器就会显示作者的主页。可以将这段代码插入世界任何一个地方的任何一个文档中，用鼠标点击一下，它都会将你带到加利福尼亚的La Jolla去。

```
<html><!--Beginning of HTML document-->
<head><!--Begin the document head-->
<title><!--Begin document title. Used for browser "bookmarks"-->

Title of document.

</title><!--End document title-->
</head><!--End the document haed-->
<body><!--Stuff to appear on screen begins here-->
```

图6-5 一个简单HTML文档的轮廓

```
<h1>
First main heading.
</h1>
Maybe some text.
<h2>
First subheading.
</h2>
Probably some text.
<h2>
Second subheading.
</h2>
Probably some text.
<h1>
Second main heading.
</h1>
And then more of the same.
...
</body><!--Regular stuff ends here, but address is displayed-->
<address><!--Optional, but normally used-->
The e-mail address of the person maintaining the page.
Also, the date of the last time the page was changed.
(You can actually put in whatever you want here, but the
e-mail address and date are what people expect.)
</address>
</html><!--End of HTML document-->
```

真实的HTML文档中不会有这么多注释。

图6-5　（续）

图6-6中的HTML文档包含了一个到Sun微系统公司Java Web站点的链接。如果用一个浏览器来查看这个HTML文档，它看起来会和图6-7差不多。如果用户点击有下划线的文本，浏览器就会显示出Sun微系统公司的Java Web站点。（也就是说，它会显示Sun公司计算机上包含有关Java信息的那个HTML文档。）

```
<html>
<head>
<title>
Java Club Home Page
</title>
</head>
<body>
<h1>
<center>
Java Club
</center>
</h1>

<h2>
```

显示文档时，这些空行会被忽略，但它们可以使HTML代码更易读。

图6-6　一个非常简单的HTML文档

```
Club Purpose
</h2>
<p>
A major goal of the club is to encourage
its members to become good programmers.
<p>
The club provides a setting where people who
like to program in the Java language can meet
and talk with other like-minded programmers.

<h2>
Meeting Times
</h2>
The first Wednesday of each month at 7 PM.
<h2>
Sun Microsystems Java Website
</h2>

<a href="http://java.sun.com">
Click here for the website
</a>

<p>
</body>

<address>
javaclub.somemachine@someschool.edu
<br>
January 1, 2004
</address>
</html>
```

在浏览器中显示时，文本可能会在不同的地方换行。

一个新的段落总会产生一个换行符和一些间距。

在6.2.2节中对这段代码进行了解释。

图6-6 （续）

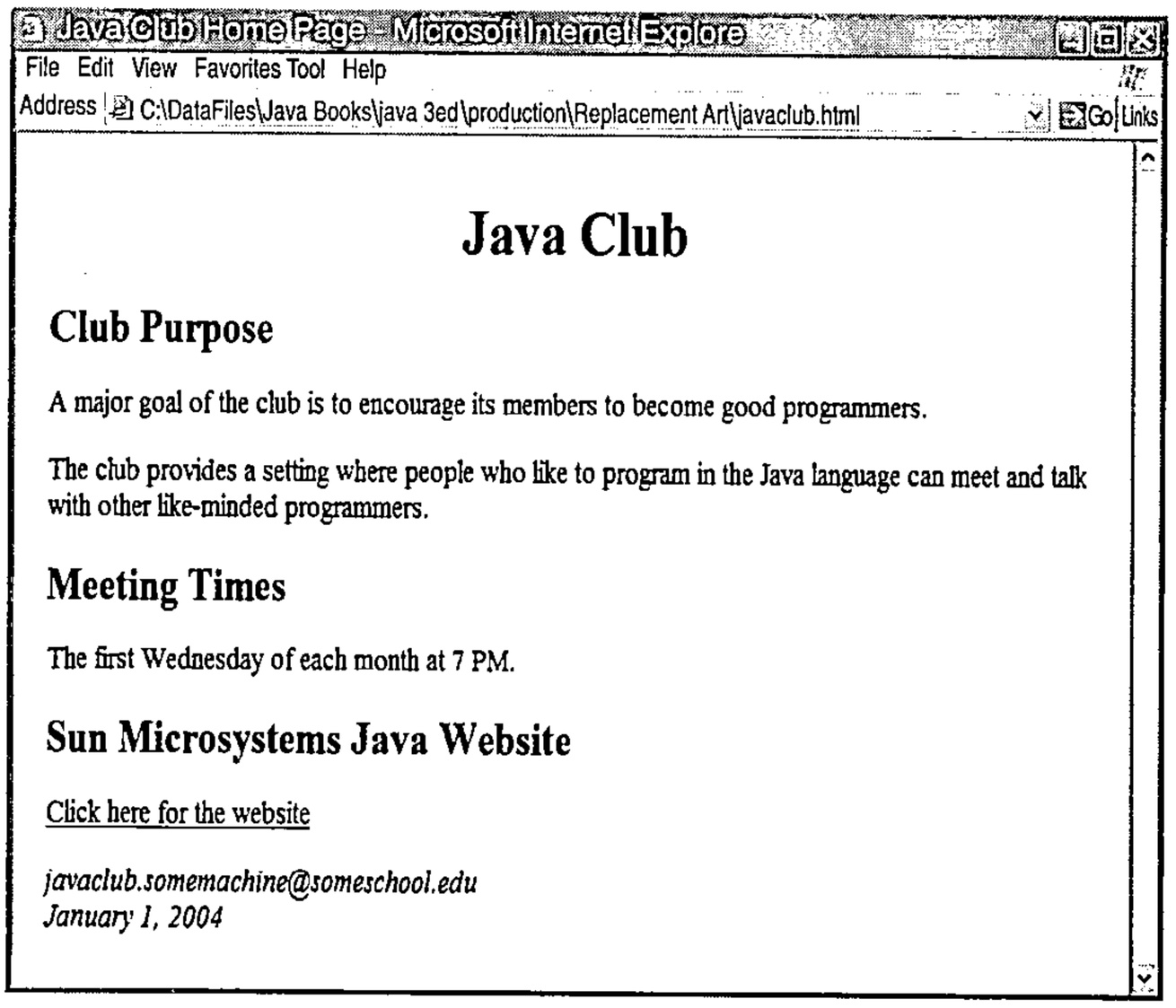


图6-7 图6-6的浏览器显示

快速参考：插入一个超链接

插入一个超链接的命令如下所示。

语法：

```
<a href = "Path_To_Document">
Displayed_Text_To_Click
</a>
```

它可能出现在任何文本中。

举例：

```
<a href = "http://java.sun.com">
Sun Microsystems Java website
</a>
```

*Path_To_Document*可以是Web上任何一个地方的HTML文件或URL的全称或相对路径名。如果查看文档的人点击了*Displayed_Text_To_Click*，就会将*Path_To_Document*指示的文本显示出来。

常见问题：URL是什么？

Web上HTML文档的名字被称为一个URL，是**统一资源定位符**（Uniform Resource Locator）的缩写。这个名字是一种万维网路径名，万维网是一个覆盖全球的系统。在本节描述的所有超链接都是URL，比如

http://java.sun.com

URL是文件的绝对路径名，这些文件可以位于世界的任何地方。对那些到自己计算机上的HTML文档的链接也可以使用相对路径名。

URL通常以http开头，这是用来传输及解释HTML文档的协议的名字（但这已经超出本书的范围了）。大多数浏览器都允许省略http://，并会自动把它加上。

△ 易犯错误：不使用重载（刷新）按钮

大多数浏览器中都保存了大部分近期使用过的HTML页面的副本。这样，如果想重新回到其中某个页面的话，浏览器就可以非常快速地将页面恢复出来。通常，这是件好事，因为这样可以使浏览器运行的更快。但是，在设计及调试一个HTML页面时，这可能是个问题。

假设用浏览器去测试一个HTML页面，并发现有些需要修正的地方。如果修改了HTML页面以修正那个问题，然后再用浏览器去查看那个页面，很可能看不到任何变化。即使退出浏览器，用其他页面重新打开浏览器，然后跳转到那个被修正的页面中去，还是会发生这种情况。正如前面所说的，问题在于大多数浏览器中都保存了大部分最近显示过的页面的副本。浏览器会重新使用这些副本，而不会装入这些页面的新版本。

要确保浏览器显示的是页面的最新版本，可以点击标记为重载（Reload）、刷新（Refresh）或其他类似内容的按钮。这个动作会使浏览器重新下载页面，给出最新的版本。　△

记住：通过javadoc实现自动文档化

Java语言带有一个名为javadoc的程序，可以自动为Java类生成文档。javadoc生成的是可以用浏览器阅读的HTML文档，就像任意其他HTML文档一样。附录I对javadoc程序进行了描述。

6.2.3 显示一张图片

本章中，用HTML完成的工作是不需要图片的，不过可能希望在HTML文档中放置一张图片。插入一张图片的命令如下所示：

```
<img src="File_With_Picture">
```

例如，假设在子目录images中有一张数码图片。具体来说，假设图片文件mypicture.gif位于目录images中，而images是HTML页面所处目录的子目录。就可以通过插入下列代码将图片添加到HTML文档中去。

```
<img src="images/mypicture.gif">
```

图片可以位于任意一个目录中，但必须给出导向图片文件的路径。可以使用全称路径名或相对路径名（也就是，相对于包含HTML文档的目录的路径名）来指明图片文件的位置。大多数常用的图片编码格式都是可用的。

自测题

6. 命令<h2>和<H2>有什么区别？
7. 如何插入一个指向下列主页的链接？

   http://www.fool.com/

8. 浏览器会怎样显示下列代码？

```
<p>
A major goal of the club is to encourage
its members to become good programmers.
<p>
The club provides a setting where people who
like to program in the Java language can meet
and talk with other like-minded programmers.
```

（会在什么地方换行？浏览器会在什么地方添加空白？）

○ 编程提示：HTML是一种低级语言

HTML是一种低级语言。Web浏览器的HTML与计算机的汇编语言类似。大多数Web页面设计者现在都使用一种可以翻译成HTML的高级设计语言。如果计划做大量的Web页面设计，应该考虑学习其中的一种语言。Dreamweaver（Macromedia公司）、FrontPage（微软公司）以及GoLive（Adobe系统公司）就是三种这样的语言。

6.3 HTML中的applet

在这里编写；在那里运行！

——听起来像对什么人的广告的复制[①]

本节，将学习如何将一个applet嵌入HTML文档中，以便离你有半个世界远的浏览器都可以查看这个文档和applet。

① 但这是我自己编的。

6.3.1　将applet放入HTML文档中

如果将下列命令放入一个HTML文档中，文档就会显示图6-3的applet创建的加法器窗口。

```
<applet code="AdderApplet.class" width=400 height=200>
</applet>
```

这条命令假设HTML文件和文件AdderApplet.class位于同一个目录（同一个文件夹）中。如果它们不在同一个目录中，可以为文件AdderApplet.class使用绝对或相对路径名。前面显示的这种表达式通常被称为一个**applet标签**（applet tag）。

例如，图6-8包含了一个示例HTML文档，这个文档中包含了图6-3给出的applet。用浏览器显示这个HTML文档时，看起来和图6-9显示的差不多。

注意，将applet放在HTML文档中时，给出的是以.class结尾的字节码文件名，而不是.java文件名或其他一些文件名。还要注意，applet的宽度和高度是在这条命令中，而不是applet的类定义中指定的。宽度和高度是以像素为单位给出的。

快速参考：HTML文档中的applet

可以按指定代码所示将applet放入HTML文档中去。使用的是applet的字节码（.class）文件。如果文件与HTML文档不在同一个目录（同一个文件夹）中，可以为那个applet文件使用全称或相对路径名。宽度和高度是以像素为单位给出的。

语法：

```
<applet code = "Name_Of_.class_File" width=Integer height = Integer>
</applet>
```

举例：

```
<applet code = "AdderApplet.class" width=400 height = 200>
</applet>
```

```
<html>
<head>
<title>
Budget Help
</title>
</head>

<body>
<h1>
The Budget Help Home Page
<br>
Helpful Hints for a Balanced Budget
</h1>

<h2>
Pay off your credit cards every month.
</h2>

<h2>
Do not spend more than you earn.
</h2>

<h2>
```

图6-8　带有applet的HTML文档

```
Here is an adder to help you plan your budget:
</h2>
<applet code="AdderApplet.class" width=400 height=200>
</applet>
<p>
</body>

<p>
<address>
budgethelp@fleeceyou.com
<br>
December 31, 2004
</address>
</html>
```

图6-8 （续）

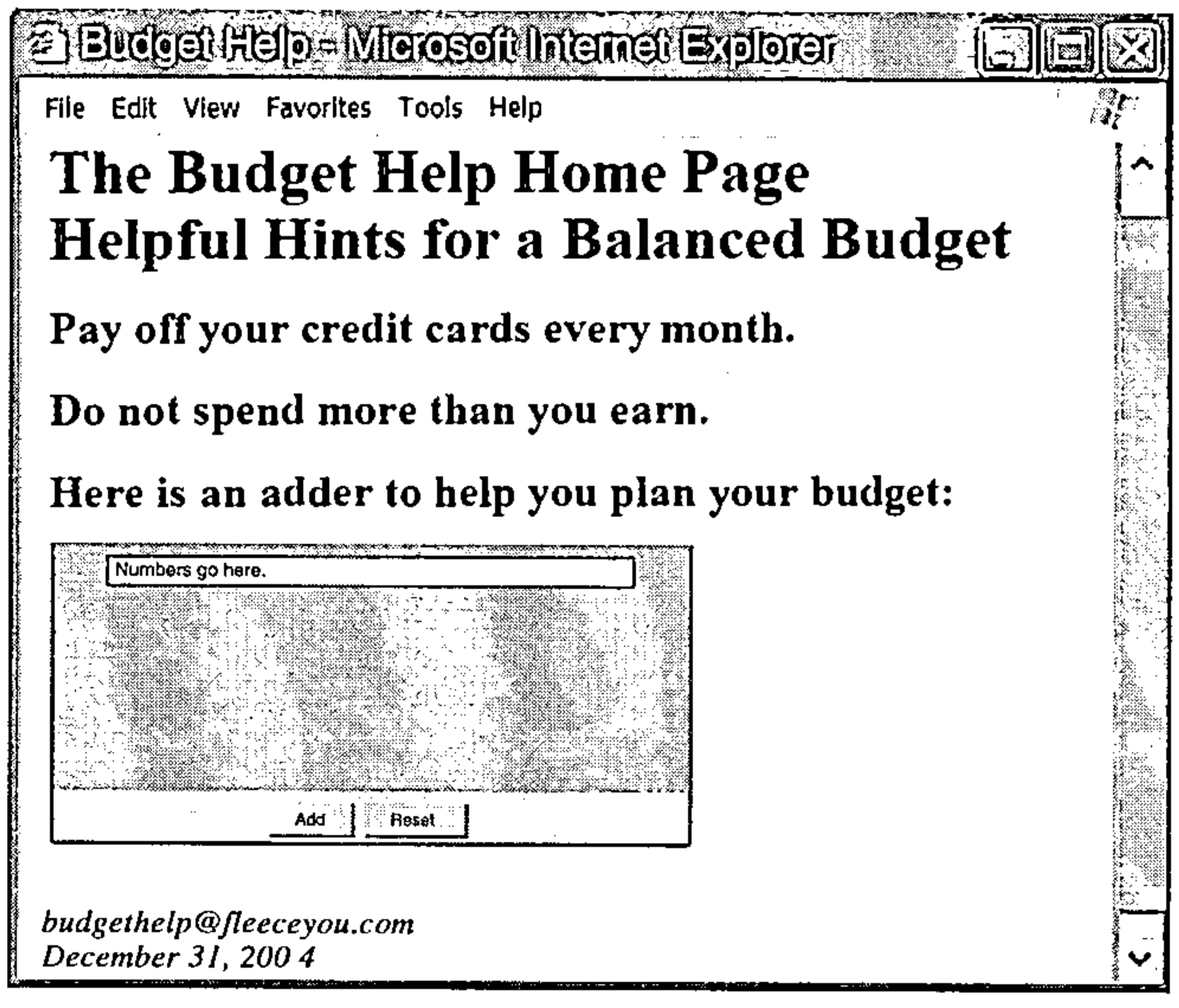


图6-9 图6-8的浏览器显示

□ Java提示：对applet查看程序的重新讨论

即使无法从一个环境中运行applet查看程序，肯定也可以用一条单行命令运行applet查看程序。例如，可以按下列形式运行图6-3中的applet：

```
appletviewer AdderApplet.html
```

但是，如果以这种方式用单行命令运行一个applet，就需要自己创建一个HTML文档（在这个例子中名为AdderApplet.html），并将applet标签放入HTML文档中去。

自测题

9. 在一个HTML文档中列出applet时，列出的是.java文件还是.class文件？

10. 给出将一个名为HelloApplet的applet插入HTML文档中去的HTML代码。

△ 易犯错误：使用老的Web浏览器

必须设置Web浏览器使其能够运行applet。Web浏览器使用的Java解释程序与运行普通的Java应用程序时使用的Java解释程序不同。如果使用的是老的Web浏览器，没有为applet进行设置，或者只为用早期Java版本创建的applet进行了设置，可能就无法从HTML文档中运行applet了。即使Java应用程序在系统中运行良好，也可能出现这种情况。在这种情况下，你所能做的就是获取一个新的浏览器。

但是，即使获取一个新的浏览器也不总是有所帮助的，因为Java对浏览器的更新通常都比对核心Java语言的更新慢很多。而且，如果将一个applet放在HTML文档中，并希望其他人通过Web查看这个applet，那么applet的.class文件会被发送到其他人的计算机上去，并在他们的浏览器中运行。当然，不能期望所有其他人都使用最新的浏览器。

如果希望一个applet能够在Web上被广泛浏览，就要在一些不同的浏览器上对其进行测试。在Netscape Navigator和微软的Internet Explorer上进行测试就足够了，但应该在这些浏览器的较老及最新版本上对其进行测试。如果让你的applet工作在较老的浏览器上比较困难，可以考虑使用6.3.2节中描述的类。

尽管在浏览器上运行applet会存在一些问题，但只要有最近的Java版本，那么，从applet查看程序中运行这些applet就应该不会存在这些问题了。因此，即使无法从一个HTML页面中运行applet，也应该可以运行applet，并对其进行测试。但是，如果最终不将它们放入HTML文档的话，也可以使用从JFrame派生出来的普通Swing GUI。　△

6.3.2　较老的applet类（选读）

如果发现applet在很多浏览器中都无法工作，可以试着使用较老的Applet类来取代JApplet类。通常只要简单地删掉所有的J，用Applet取代JApplet，用Button取代JButton，用Label取代JLabel，等等就可以实现这一点。这种方式需要使用下列import语句。

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
```

最好还要删掉下列import语句：

```
import javax.swing.*;
```

还需要进行另一项，稍微复杂一些的替换。与JApplet不同，Applet没有内容面板。因此，要对applet自身而不是内容面板使用add，一般来说，对JApplet的内容面板做的所有工作，都要直接对Applet操作。例如，如果JApplet中的方法init包含了行

```
getContentPane().add(friendlyLabel);
```

那么，要将其转换成一个Applet，就需要用下列代码来取代这条语句。

```
add(friendlyLabel);
```

对大多数简单的applet来说，这些改动会生成一个能够在较老的浏览器上运行也能在较新浏览器上运行的applet。但是，有些事情是可以对JApplet做，而不能对较老的Applet类做的。例如，Applet类不能方便地使用图标，因此使用Applet类时，不应该包含图标。在第7章中，讨论了菜单。可以对JApplet使用菜单，但不能对Applet使用，因此必须用按钮来替代。

遵循这些规则为图6-2和6-3中的applet生成了基于Applet类，而不是JApplet类的applet。结果存储在Web上的OlderHelloApplet.java和OlderAdderApplet.java中。

快速参考：applet类（选读）

Applet类是在JApplet出现之前用来生成applet的一个较老的applet类。用Applet类生成的applet的特性没有用JApplet类生成applet的特性那么多。但是，它们可以在更多的浏览器上工作。因此，如果希望applet能够被大范围的用户浏览，使用Applet类而不是JApplet类会是比较明智的。对非常简单的applet来说，将一个JApplet转换成一个Applet是很容易的。在本小节中给出了一些细节。Applet类需要使用下列import语句：

```
import java.applet.*;
```

6.3.3 applet与安全

假设Web上的某个人浏览了你的HTML页面，而那个HTML页面中包含了一个applet。那个applet的字节码是在读者机器中的浏览器上运行的。因此，你的applet会是一个在其他人的计算机上运行的程序。更可怕的是，其他人的applet也可以在你的计算机上运行。而且，在将一个HTML页面装载到浏览器中之前，你是不知道其中包含了一个applet的，这时候想拒绝applet就太晚了。那时，它已经存储在你的计算机中了。

无论什么时候，别人的程序在你的机器上运行就会存在严重的安全问题。程序会在你的计算机上留下病毒吗？它会从你的文件中读走一些机密信息吗？它会修改并破坏你的操作系统吗？applet的设计使其无法（或者至少无法方便地）做所有这些事情。applet无法运行你的任何程序，无法对你计算机上的文件进行读或写（除非这个applet起源于你的计算机）。但是要注意：除了applet之外，有一些程序可以通过浏览器运行，并获取对你计算机上某些东西的访问权——那些你可能认为是私密的东西。一些商业的反病毒和其他保护软件可以帮助计算机抵御这种攻击程序的入侵。

自测题

11. 6.3.2节：JApplet有一个init方法。Applet有init方法吗？

12. 6.3.2节：JApplet有内容面板。Applet有内容面板吗？

小　结

- applet是放置在Web站点文档内，并从其中运行的Java程序。
- applet与从JFrame类中派生的Swing GUI类似。
- applet通常是JApplet类的派生类。
- applet中通常没有main方法，也没有构造器。但是，对applet来说，方法init的作用与构造器相同。
- applet的init方法中不包含任何对addWindowListener、setTitle或setSize的调用。
- 设计用来通过万维网或通过Web浏览器浏览的文档通常都是用一种名为HTML的语言编写的。HTML表示超文本标记语言。applet可以嵌入HTML文档中，并从其中运行。

✓自测题答案

1. applet类通常不使用构造器。但是，它们确实有一个用于类似目的的init方法。

2. applet类通常没有main方法。
3. 定义一个applet时很可能会用到getContentPane。通常不会使用addWindowListener、setTitle和setSize，因为applet没有标题，而且其尺寸及其关闭都是由applet查看程序或applet所嵌入的Web文档来处理的。（在一些会被添加到applet中去的组件里可能会用到它们，但这不是这个问题讨论的重点。）
4. 用JLabel类的构造器创建一个（带或不带文本的）标签。用ImageIcon类的构造器根据smiley.gif创建一个图标。用方法setIcon将图标添加到标签中去。例如，可能会在applet的init方法中用到下列代码：

```
JLabel niceLabel = new JLabel();
ImageIcon smileyFaceIcon = new ImageIcon("smiley.gif");
niceLabel.setIcon(smileyFaceIcon);
getContentPane().add(niceLabel);
```

5. 指定一张数码图片文件时，可以使用相对或绝对路径名。
6. 没有区别。HTML是不区分大小写的。
7.

```
<a href="http://www.fool.com/">
这里可以放任何想说的话。
</a>
```

8. 在每个<p>的位置都会添加一个换行符及一些额外的空格。文本中的换行符会被忽略，浏览器会在需要的地方插入换行符，以使文本适应于浏览器窗口。
9. .class文件。
10.

```
<applet code = "HelloApplet.class" width=400 height=200>
</applet>
```

当然，可以将宽度和高度分别设置为400和200之外的其他值。

11. applet像JApplet一样有init方法。
12. applet没有内容面板。

○ 编程项目

1. 为自己设计一个HTML主页。发挥想象，在主页上放上所有你希望人们了解的，或者你认为他们应该了解的关于你的内容。一些可能的内容包括你的名字、你的职业或课程表，你在哪儿上学和工作，你的爱好、你最喜欢的语录，如何通过email找到你，或者任何其他你想说的内容。注意，你的主页和任何其他主页一样，要包含几个文件，其中每个文件都包含一些HTML文档。超链接使得人们在阅读文档时，可以通过点击鼠标按钮从一个HTML文档转移到另一个文档中去。变通：如果在进行一门课程的学习，可以询问指导老师是否需要在主页中包含一个applet。记住，放在主页中的任何内容都可以被世界上的任何人看到。在主页中只包含那些不介意公开的内容。
2. 将图5-19中的Swing应用程序转换成一个applet，并将其放在一个HTML文档中。
3. 每个一年级的电气工程专业的学生都知道，两个电阻（电阻是一种常见的电气元件）可以通过串联或并联这两种配置方式之一进行连接，有一些简单的公式可以用来计算这两种配置下的等价电阻（可以用来取代这两个电阻的单个等价电阻的值）。如果$R1$和$R2$是两个电阻值，那么

串联阻抗 = $R1 + R2$，且

并联阻抗 = $(R1*R2)/(R1+R2)$。

编写一个applet，为用户提供一个视窗界面来输入这两个电阻值，并选择对哪种配置进行计算。用两个文本域（将它们标记为"Resistor 1"和"Resistor 2"）来读入这两个值，用两个按钮（将

其标记为"Series"和"Parallel"）来选择配置方式，用另一个文本域（将其标记为"Equivalent Resistance"）来显示计算得到的值，并说明选择的是哪种配置。例如，如果用户输入100表示*R*1，50表示*R*2，并点击"Series"按钮，那么显示的信息就是"Series Equivalent = 150"。如果用户输入同样的值，并点击"Parallel"按钮，显示的信息就是"Parallel Equivalent = 33.3"。将这个applet放入一个对此计算过程进行了解释的Web页面中。

4. 修改第5章编程项目7中的GUI计算器程序，将其作为一个applet运行。如果还未将此项目作为一个GUI实现，就现在来实现，但要将其作为一个applet来实现。
5. （这个练习的Swing部分是很简单的，但需要了解一些有关如何将数字从一种进制转换成另一种进制的知识。）编写一个applet，将数字从10进制（普通的十进位）记法转换成16进制（基于16的）记法。程序用Swing通过一个applet界面来实现输入和输出。用户在一个文本域中输入一个十进制整数，并点击一个标记为"Convert"的按钮。然后，等价的16进制数字就会出现在另一个文本域中。要确保这两个字段都被标记了。要包含一个"Clear"按钮来清除这两个文本域中的内容。
6. 重新编写图5-19中的程序，使其成为一个applet，这样就需要进行下面所有这些修改。
   i. 类名为MemoApplet。
   ii. 有六个而不是五个按钮，按下列形式排列：

Save Memo 1	Save Memo 2	Clear
Get Memo 1	Get Memo 1	Exit

   按钮仍然位于GUI（applet）的底部，文本区在按钮的上面。（提示：在按钮面板上使用GridLayout管理器）
   iii. 用户将文本保存为备注1时，修改文本区，使其显示"Memo 1 saved."，保存备注2时，修改文本区，使其显示"Memo 2 saved."（提示请参见第5章自测题43。）
   iv. 文本区有自动换行，这样如果输入的字符在那一行放不下的话，多余的字符会自动进入下一行。
7. 完成本书第2章的编程项目4，但将其作为一个applet编写。
8. 完成本书第2章的编程项目7，但将其作为一个applet编写。

第7章

更多有关Swing的内容

多多益善。

——约翰·海伍德，《格言》(1546)

本章，给出了一些关于Swing的额外信息，这样，就可以创建出更多看起来更专业的GUI了。在其他方面，还会介绍菜单、滚动条、图标和内部类。

在这个过程中，会了解到下列Swing（或者与Swing相关的）类：AbstractButton、Box、BoxLayout、BevelBorder、Dimension、EmptyBorder、EtchedBorder、ImageIcon、Inset、JMenu、JMenuBar、JMenuItem、JScrollPane、LineBorder、MatteBorder及其他一些类。

目标

- 学会向GUI中添加菜单、图标、边界和滚动条。
- 理解BoxLayout管理器和Box类。
- 理解内部类的一些用法及其优势。
- 了解WindowListener接口。
- 弄清楚如何创建GUI，才能使其组件从可见变成不可见，或从不可见变成可见的。

预备知识

在开始阅读本章之前，需要学习第5章，那一章对Swing进行了介绍。不需要阅读过第6章。

7.2节用到了7.1节的内容。除了这个依存关系之外，7.1节、7.2节、7.3节和7.4节都是彼此独立的。只要在学习7.2节之前学习了7.1节的内容，就可以以任意顺序来学习这些小节了。

7.5节用到了7.4节的内容，7.6节用到了7.1节和7.2节中的内容。

7.1 菜单

先付钱，再取你所需。

——*Punch*(1846)

Swing GUI中可以有菜单，而且，大部分需要了解的、与Swing菜单有关的内容都已经学过了。这些内容是在第5章学习按钮的时候学的。菜单条目的行为与按钮一样。举个例子就清楚了。

编程示例：带有菜单的GUI

图7-1中的程序构建了一个带有菜单的GUI。这个GUI与图5-19中的GUI做的事情是一样的。用户在文本区中输入备注，当重新取回备注时，它们会像图5-19那样，出现在文本区中。但是，图7-1的GUI中没有任何按钮。相反，在窗口顶端有一个菜单条。菜单条中列出了所有下拉菜单的名字。这个GUI只有一个下拉菜单，被称为"Memos"。但是，一个菜单条中是可以有多个下拉菜单的。

用户可以通过点击菜单条中某个菜单的名字将该菜单拉下来。图7-1中包含了GUI的两幅图片。第一幅是GUI第一次出现时显示的图片。在那幅图片上，可以在菜单条上看到菜单名"Memos"，但无法看到菜单。如果用鼠标点击单词"Memos"，菜单会像GUI的第二幅图片所示那样拉下来。如果点击"Save Memo 1"，就会将文本区中的文本保存起来。其他菜单选项的行为与图5-19中按钮的行为类似。

在这个菜单中有一个新的选项，即Exit。用户点击条目Exit时，程序退出，窗口消失。如果用户点击关闭窗口按钮，也会发生同样的事情。因此，用户有两种方式可以终止这个GUI。

在接下来的几个小节中，会对图7-1中程序的一些细节进行研究。

7.1.1 菜单条、菜单和菜单条目

在图7-1中添加菜单时，用到了三个Swing类：JMenuBar、JMenu和JMenuItem。菜单中的条目是JMenuItem类的对象。这些JMenuItem被放在JMenu中，然后通常会将JMenu放在一个JMenuBar中。现在来看看其中的一些细节。

JMenuItem类的对象就是菜单中的一个选项。它是通过标记它的字符串来识别的，比如"Save Memo 1"。JMenu类的对象是一个菜单，就像图7-1的GUI中显示的那个菜单一样。可以向一个菜单中添加任意多的JMenuItem。菜单会按照它们的添加顺序来排列条目。下列来自图7-1构造器的代码，创建了一个新的、名为memoMenu的JMenu对象，并添加了一个标记为"Save Memo 1"的JMenuItem。

```
JMenu memoMenu = new JMenu("Memos");
JMenuItem m;
m = new JMenuItem("Save Memo 1");
m.addActionListener(this);
memoMenu.add(m);
```

其他菜单条目也是用类似的方法添加的。

注意，正如对按钮所作的那样，将参数this注册为一个动作侦听器。定义动作侦听器以及为菜单条目注册侦听器的方式，与为按钮做这些工作的方式完全相同。实际上，甚至语法都是一样的。对图5-19和图7-1进行比较，就会发现在这两个例子中，方法actionPerformed是以同样的方式定义的。唯一的区别在于，在图7-1中，方法有一个用于新的"Exit"条目的额外例子。

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class MemoGUI extends JFrame implements ActionListener
{
    public static final int WIDTH = 600;
```

图7-1 带有菜单的GUI

```
public static final int HEIGHT = 300;
public static final int LINES = 10;
public static final int CHAR_PER_LINE = 40;

private JTextArea theText;
private String memo1 = "No Memo 1.";
private String memo2 = "No Memo 2.";

public MemoGUI()
{
    setSize(WIDTH, HEIGHT);
    addWindowListener(new WindowDestroyer());
    setTitle("Memo Saver");
    Container contentPane = getContentPane();
    contentPane.setLayout(new BorderLayout());

    JMenu memoMenu = new JMenu("Memos");
    JMenuItem m;

    m = new JMenuItem("Save Memo 1");
    m.addActionListener(this);
    memoMenu.add(m);

    m = new JMenuItem("Save Memo 2");
    m.addActionListener(this);
    memoMenu.add(m);

    m = new JMenuItem("Get Memo 1");
    m.addActionListener(this);
    memoMenu.add(m);

    m = new JMenuItem("Get Memo 2");
    m.addActionListener(this);
    memoMenu.add(m);

    m = new JMenuItem("Clear");
    m.addActionListener(this);
    memoMenu.add(m);


    m = new JMenuItem("Exit");
    m.addActionListener(this);
    memoMenu.add(m);

    JMenuBar mBar = new JMenuBar();
    mBar.add(memoMenu);
    setJMenuBar(mBar);

    JPanel textPanel = new JPanel();
    textPanel.setBackground(Color.BLUE);
    theText = new JTextArea(LINES, CHAR_PER_LINE);
    theText.setBackground(Color.WHITE);
```

WindowDestroyer类是在第5章定义并讨论的。

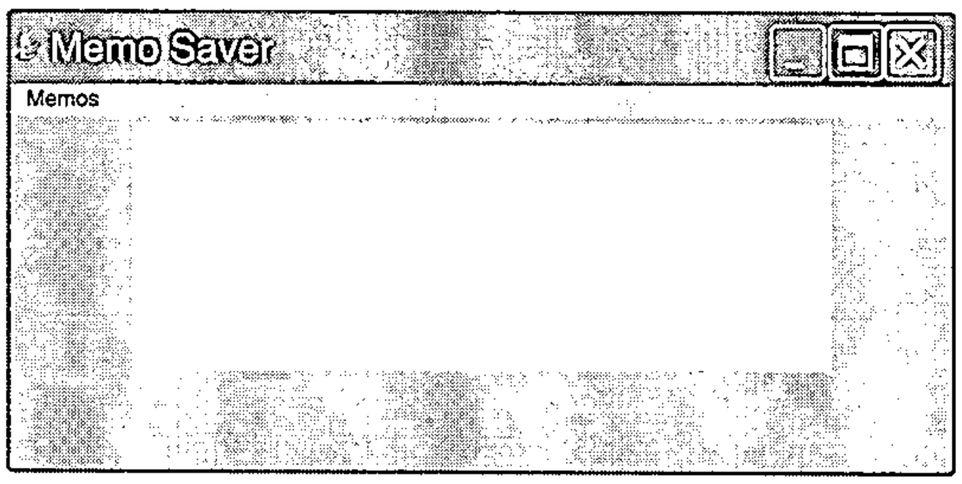
图7-1　(续)

```
        textPanel.add(theText);
        contentPane.add(textPanel, BorderLayout.CENTER);
    }

    public void actionPerformed(ActionEvent e)
    {
        String actionCommand = e.getActionCommand();
        if (actionCommand.equals("Save Memo 1"))
            memo1 = theText.getText();
        else if (actionCommand.equals("Save Memo 2"))
            memo2 = theText.getText();
        else if (actionCommand.equals("Clear"))
            theText.setText("");
        else if (actionCommand.equals("Get Memo 1"))
            theText.setText(memo1);
        else if (actionCommand.equals("Get Memo 2"))
            theText.setText(memo2);
        else if (actionCommand.equals("Exit"))
            System.exit(0);
        else
            theText.setText("Error in memo interface");
    }

    public static void main(String[] args)
    {
        MemoGUI gui = new MemoGUI();
        gui.setVisible(true);
    }
}
```

得到的GUI



（点击了菜单条中的Memos之后）得到的GUI

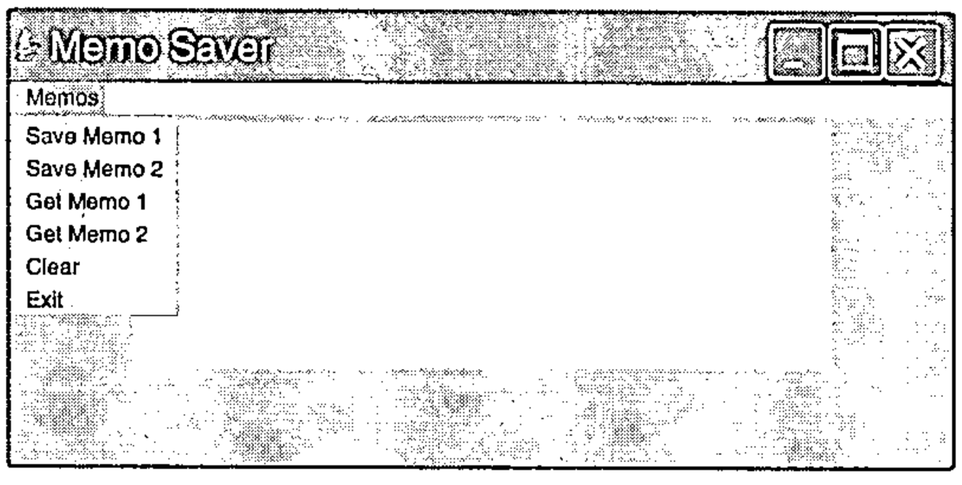


图7-1　（续）

用方法add向JMenu类对象中添加JMenuItem的方式，与向容器对象中添加按钮这样的组件的方式完全相同。而且，看看前面的代码，就会发现，为JMenuItem指定字符串的方式，与为按钮指定字符串标签的方式也是相同的。因此，向一个菜单中添加菜单条目的语法确实没什么新东西。

菜单条（menu bar）是菜单的容器，通常被放置在靠近视窗界面顶端的位置上。用方法add向一个菜单条添加菜单的方式，与向菜单添加菜单条目的方式相同。下列来自图7-1构造器的代码，创建了一个名为mBar的新菜单条，然后将名为memoMenu的菜单添加到这个菜单条中去：

```
JMenuBar mBar = new JMenuBar();
mBar.add(memoMenu);
```

有两种不同的方式可以将菜单条添加到JFrame中去。首先，可以使用方法setJMenuBar，如图7-1中的下列代码所示。

```
setJMenuBar(mBar);
```

这段代码设置了一个JMenuBar类型的实例变量，将菜单条命名为mBar。用句不太正式的话来说，这段代码将菜单条mBar添加到JFrame中去，并将菜单条放在了JFrame的顶端。

或者，可以将菜单条添加到JFrame的内容面板（或者任何其他容器）中去。添加的方式与添加任何其他组件，比如标签或按钮，的方式相同。Web上的文件MenuAdd.java中给出了一个用add将JMenuBar添加到JFrame的内容面板中去的例子。

快速参考：菜单和菜单条

菜单条目是JMenuItem类的对象，菜单是JMenu类的对象，而菜单条是JMenuBar类的对象。可以用方法add将一个菜单条目添加到菜单中去。也可以用方法add将一个菜单添加到一个菜单条中去。有两种不同的方式可以将菜单条添加到JFrame中去。可以用方法add以及想用的任意布局管理器，将JMenuBar添加到JFrame的内容面板（就此而言，还可以添加到任意其他容器中去）中去。或者，更具代表性地，可以用方法setMenuBar将一个菜单条添加到JFrame中去。

对菜单条目的事件和侦听器的处理与对按钮的处理是完全一样的。

快速参考：为菜单条目设置动作命令

与按钮的文本一样，JMenuItem的文本就是那个菜单条目的默认动作命令。而且，就像可以对按钮所作的那样，可以用方法setActionCommand将一个菜单条目的动作命令设置为其他字符串。

语法：

Menu_Item_Object.setActionCommand(*Action_Command_String*);

举例：

```
JMenuItem item = new JMenuItem("Click me!");
item.setActionCommand("From the click me item with love.");
```

7.1.2　嵌套式菜单

JMenu类是JMenuItem类的子孙。因此每个JMenu对象也都是JMenuItem对象。因此，一个JMenu也可以是另一个菜单中的菜单条目。这就意味着可以对菜单进行嵌套。例如，外层

的菜单会给出一系列菜单。可以通过点击希望显示的菜单名字显示出那个列表中的某个菜单。然后可以再次用鼠标从那个菜单中选择一个条目。要创建这些嵌套式菜单并不需要学习什么新的东西。只要像添加其他菜单条目那样，将菜单添加到菜单中去就行了。在Web上的文件NestedMenus.java中有一个嵌套式菜单实例。

快速参考：向JFrame添加菜单

在这个方框中，假设所有的添加工作都是在一个JFrame（的派生类）的构造器内部发生的。否则的话，大部分方法调用的前面都需要一个对象名和一个点。要想看看将这些例子组合在一起生成的完整的GUI，请参见图7-1中的构造器。

创建菜单条目

菜单条目是JMenuItem类的对象。如下例所示，可以用通常的方式来创建一个新的菜单条目：

```
JMenuItem m;
m = new JMenuItem("Save Memo 1");
```

实参位置上的字符串是菜单条目的标签。

添加菜单条目侦听器

处理菜单条目事件和侦听器的方式与对按钮的处理方式是一样的：菜单条目会激活由ActionListener类型的对象接收的动作事件。

语法：

JMenu_Item_Name.addActionListener(*Action_Listener*);

举例：

```
m.addActionListener(this);
```

向菜单中添加菜单条目

菜单是JMenu类的对象。可以使用方法add向菜单中添加菜单条目。

语法：

JMenu_Name.add(*JMenu_Item*);

举例（memoMenu是JMenu类的对象）：

```
memoMenu.add(m);
```

向菜单条添加菜单

菜单条是JMenuBar类的对象。可以按下列形式向菜单条添加菜单：

语法：

JMenu_Bar_Name.add(*JMenu_Name*);

举例（mBar是JMenuBar类的对象）：

```
mBar.add(memoMenu);
```

向Frame添加菜单条

有两种不同的方法可以向JFrame添加菜单条。首先，可以使用方法add将菜单条添加到JFrame的内容面板（或者任何其他容器）中去。或者，可以按下列方式使用已经定义好了的方法setJMenuBar。

语法：

```
setJMenuBar(JMenu_Bar_Name);
```

举例：

```
setJMenuBar(mBar);
```

自测题

1. 点击JMenuItem时会激发哪类事件？这类事件与点击JButton时激发的事件类型有何不同？
2. 要想修改JButton的动作命令，可以使用方法setActionCommand。要修改JMenuItem的动作命令可以使用哪个方法？
3. 下列代码在Java中是合法的吗？

```
JMenu myMenu = new JMenu();
 ...
JMenu mySubMenu = new JMenu();
 ...
myMenu.add(mySubMenu);
```

4. 在一个JFrame中可以有多少个JMenuBar？
5. JFrame有一个JMenuBar类型的私有实例变量。负责修改（“设置”）这个实例变量的设置方法叫什么名字？
6. 编写代码创建一个被标记为"Choose Me!"的、名为mItem的新菜单条目。
7. 假设用Swing构建了一个GUI界面。如果用户点击一个菜单条目，就会激发一个事件。接收事件的是什么类型的侦听器？
8. 假设定义了一个名为MenusGalore的类，这个类是JFrame类的派生类。编写代码将菜单条目mItem添加到菜单m中去。然后将m添加到菜单条mBar中，再将菜单条添加到JFrame MenusGalore中去。假设所有这些都发生在MenusGalore的构造器内部。还要假设所有的对象都已经用new构建好了，所有必须的侦听器都已经注册好了。只需要完成菜单特性的添加就行了。

7.2 使GUI更漂亮（功能也更强）

在至关重要的事情上，
最重要的是风格，而不是真诚。

——奥斯卡王尔德，《不可儿戏》

本节，要描述一些可以使GUI看起来更有吸引力、更专业的Swing工具。很多增强功能都既具有实用功能，又具有美化功能。本节讨论的话题包括图标、滚动条和边界。

7.2.1 添加图标

使用Swing时，标签、按钮和菜单条目中都可以有图标。**图标**只是一个小图片，尽管并不一定要很小。可以使用任何内容的图片。图片是以多种能够显示在计算机屏幕上的格式（比如GIF和JPEG）生成的。基本上所有标准格式的图片都可以作为图标的基础。Swing会将图片转换成图标，然后就可以将这个图标添加到标签、按钮或其他组件上去了。标签或按钮上可能只显示字符串，或者只显示图标，或者两者都显示（当然，上面也可能什么都没有）。

ImageIcon类用来将图片文件转换成Swing图标。例如，如果有一个图片文件名为duke_waving.gif，下列代码会为图片duke_waving.gif生成一个名为dukeWavingIcon的图标：

```
ImageIcon dukeWavingIcon = new ImageIcon("duke_waving.gif");
```

文件duke_waving.gif应该与包含这段代码的类位于同一个目录中。否则，可以用一个完整或相对路径名来指定图片文件。图片文件名是作为ImageIcon构造器的实参给出的。因为图片文件是作为字符串给出的，所以，应该在名字两端使用引号，或使用一个变量，或String类型的表达式。在Web上的源代码中提供了文件duke_waving.gif和本章用到的所有其他图片文件。

可以按下列形式将这个图标作为JLabel构造器的实参，将其添加到一个标签中去：

```
JLabel dukePicture = new JLabel(dukeWavingIcon);
```

然后，可以将标签dukePicture添加到JFrame、JPanel或其他容器中去，这样，标签就会显示为一张图片而不是一串文本。

还可以生成一个（只）带有图标的按钮。可以用类似的方式来完成这项任务：

```
JButton dukeButton = new JButton(dukeWavingIcon);
```

因为这个按钮上没有字符串，所以，如果用这种方式创建了一个按钮，就应该用setActionCommand显式地为按钮设置一条动作命令。（如果对这项操作不太熟悉的话，可以参见第5章名为“setActionCommand与getActionCommand”的快速参考框。）

快速参考：图标和ImageIcon类

图标只是一张小图片，尽管实际上并不要求它很小。ImageIcon类用来将图片文件转换成Swing图标。

语法：

```
ImageIcon Name_Of_ImageIcon = new ImageIcon(Picture_File_Name);
```

*Picture_File_Name*是一个字符串，给出了图片文件的相对或绝对路径名。（因此，如果图片文件与程序位于同一个目录中，就只需要给出图片文件的名字。）

举例：

```
ImageIcon smileyIcon = new ImageIcon("smiley.gif");
```

在本章中到目前为止，已经将图标放置在标签和按钮上了，而在第5章中，已经将字符串放在了标签和按钮上。如果想要，其实也经常需要，可以将图标和字符串都放在一个标签或按钮上。一种实现方法是将字符串作为标签或按钮的构造器实参，然后用方法setIcon来添加图标。下列代码说明了这个过程：

```
JButton helloButton = new JButton("Hello");
ImageIcon dukeWavingIcon = new ImageIcon("duke_waving.gif");
helloButton.setIcon(dukeWavingIcon);
```

可以用同样的方式将一个字符串和一个图标都添加到JLabel中去。

图7-2中的程序说明了如何对标签和按钮使用图标。文本域最初是空白的。如图7-2所示，当用户点击"Hello"按钮时，短语

```
"Glad to meet you!"
```

会出现在文本域中。用户点击"Good bye"按钮时，短语

```
"OK, click the upper right button. I'll miss you."
```

会出现在文本域中。用户点击关闭窗口按钮时，程序结束。

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

/**
 Simple demonstration of putting icons in buttons and labels.
*/
public class IconDemo extends JFrame implements ActionListener
{
    public static final int WIDTH = 400;
    public static final int HEIGHT = 200;

    private JTextField message;

    public IconDemo()
    {
        setSize(WIDTH, HEIGHT);
        addWindowListener(new WindowDestroyer());
        setTitle("Icon Demonstration");
        Container content = getContentPane();
        content.setBackground(Color.WHITE);
        content.setLayout(new BorderLayout());

        JLabel niceLabel = new JLabel("Nice day!");
        ImageIcon smileyIcon = new ImageIcon("smiley.gif");
        niceLabel.setIcon(smileyIcon);
        content.add(niceLabel, BorderLayout.NORTH);

        JPanel buttonPanel = new JPanel();
        buttonPanel.setLayout(new FlowLayout());
        JButton helloButton = new JButton("Hello");
        ImageIcon dukeWavingIcon = new ImageIcon("duke_waving.gif");
        helloButton.setIcon(dukeWavingIcon);
        helloButton.addActionListener(this);
        buttonPanel.add(helloButton);

        JButton byeButton = new JButton("Good bye");
        ImageIcon dukeStandingIcon =
                new ImageIcon("duke_standing.gif");
        byeButton.setIcon(dukeStandingIcon);
        byeButton.addActionListener(this);
        buttonPanel.add(byeButton);
        content.add(buttonPanel, BorderLayout.SOUTH);

        message = new JTextField(30);
        content.add(message, BorderLayout.CENTER);
    }

    public void actionPerformed(ActionEvent e)
    {
       if (e.getActionCommand().equals("Hello"))
           message.setText("Glad to meet you!");
```

图7-2　使用图标

```
    else if (e.getActionCommand().equals("Good bye"))
        message.setText(
                "OK, click the upper right button. I'll miss you.");
    else
        System.out.println("Error in button interface.");
}

/**
 Creates and displays a window of the class IconDemo.
*/
    public static void main(String[] args)
    {
        IconDemo iconGui = new IconDemo();
        iconGui.setVisible(true);
    }
}
```

得到的GUI[①]

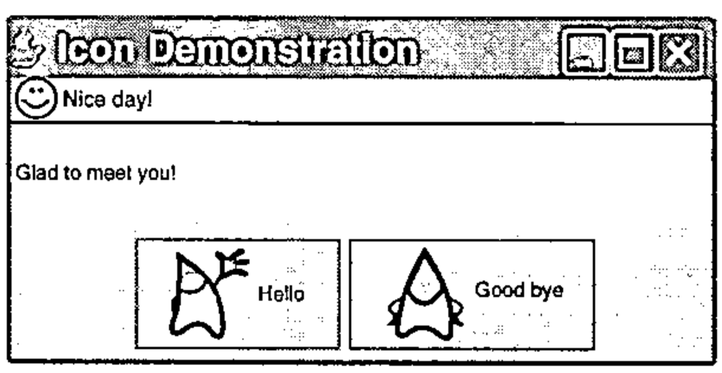


图7-2 （续）

注意，用

```
JButton helloButton = new JButton("Hello");
```

这样的代码创建一个按钮时，如图7-2的代码所示，即使还向按钮上添加了一个图标，动作命令也还是按钮上的字符串——在这个例子中，就是"Hello"：

```
helloButton.setIcon(dukeWavingIcon);
```

除了添加图标之外，图7-2中没有什么新东西。

JButton和JLabel类都有可以指定按钮或标签上的文本和图标的构造器。构造器可以不指定文本和图标、只指定文本、只指定图标，或者既指定文本又指定图标。既指定文本又指定图标时，文本是第一个实参，图标是第二个实参；如图7-3所示，JLabel的构造器还需要第三个实参。如果从构造器中省略了文本或者图标（或者两者都省略了），稍后可以分别用方法setText和setIcon来添加它们。图7-3给出了JButton和JLabel类中的一些方法。

```
public JButton()
public JLabel()
```

创建一个上面不带文本也不带图标的按钮或标签。（通常，稍后都可以对按钮或菜单条目使用setText或setIcon。）

```
public JButton(String text)
public JLabel(String text)
```

创建一个带有文本text的按钮或标签。

图7-3 JButton和JLabel类中的一些方法

① Java、Duke和所有基于Java的商标和徽标都是sun公司在美国及其他国家的商标或注册商标。

public JButton(ImageIcon picture)
public JLabel(ImageIcon picture)
创建一个带有图标picture的按钮或标签。

public JButton(String Text, ImageIcon picture)
public JLabel(String text, ImageIcon picture, int horizontalAlignment)
创建一个既带有文本text又带有图标picture的按钮或标签。horizontalAlignment是常量SwingConstants.LEFT、SwingConstants.CENTER、SwingConstants.RIGHT、SwingConstants.LEADING或SwingConstants.TRAILING之一。

public void setText(String text)
使text成为按钮或标签上唯一的文本。

public void setIcon(ImageIcon picture)
使picture成为按钮或标签上唯一的图标。

public void setMargin(Insets margin)
JButton中有方法setMargin，但JLabel中没有。
方法setMargin设置了按钮上文本和图标周围的边缘尺寸。在大多数简单的情况下，可以使用下面的特殊例子，其中的int值给出了从按钮边缘到文本或图标的像素数。
public void setMargin(new Insets(int top, int left, int bottom, int right))

public void setPreferredSize(Dimension preferredSize)
设置按钮或标签的优选尺寸。注意，对布局管理器来说，这个尺寸只是一个建议。不要求布局管理器使用这个优选尺寸。下面的特殊例子适用于大多数简单的情况，其中的int值给出了宽度和高度的像素数。
public void setPreferredSize(new Dimension(int width, int height))

public void setMaximumSize(Dimension maximumSize)
设置按钮或标签的最大尺寸。注意，对布局管理器来说，这个尺寸只是一个建议。不要求布局管理器遵循这个最大尺寸。下面的特殊例子适用于大多数简单的情况，其中的int值给出了宽度和高度的像素数。
public void setMaximumSize(new Dimension(int width, int height))

public void setMinimumSize(Dimension minimumSize)
设置按钮或标签的最小尺寸。注意，对布局管理器来说，这个尺寸只是一个建议。不要求布局管理器遵循这个最小尺寸。下面的特殊例子适用于大多数简单的情况，其中的int值给出了宽度和高度的像素数：
public void setMinimumSize(new Dimension(int width, int height))

public void setVerticalTextPosition(int textPosition)
设置文本相对于图标的垂直位置。textPosition应该是常量SwingConstants.TOP、SwingConstants.CENTER（默认位置），或SwingConstants.BOTTOM之一。

public void setHorizontalTextPosition(int textPosition)
设置文本相对于图标的水平位置。textPosition应该是常量SwingConstants.RIGHT、SwingConstants.LEFT、SwingConstants.CENTER、SwingConstants.LEADING或SwingConstants.TRAILING之一。

图7-3　（续）

快速参考：setIcon和setText

可以用方法setIcon向JButton、JLabel或其他一些组件添加图标。setIcon的实参必须是个ImageIcon对象。

语法：

Component.setIcon(*ImageIcon_Object*);

*Component*可以是JButton、JLabel或其他一些组件。

举例：

```
JLabel helloLabel = new JLabel("Hello");
ImageIcon dukeWavingIcon = new ImageIcon("duke_waving.gif");
helloLabel.setIcon(dukeWavingIcon);
```

可以用方法setText向JButton、JLabel或其他一些组件添加文本。

语法：

Component.setText(*Text_String*);

*Component*可以是JButton、JLabel或其他一些组件。

举例：

```
ImageIcon dukeWavingIcon = new ImageIcon("duke_waving.gif");
JLabel helloLabel = new JLabel(dukeWavingIcon);
helloLabel.setText("Hello");
```

这两个例子是等价的。

△ 易犯错误：重新设置按钮的尺寸

看看图7-3中描述的按钮方法，就会发现按钮的尺寸是可以改变的。设置优选、最大和最小尺寸的方法有点儿麻烦。它们只是对布局管理器的一些建议。不能保证尺寸指令会被遵守。

修改按钮的尺寸会改变按钮中的其他内容，比如按钮上文本的显示。有时，可以通过设置按钮内容与按钮边界的边缘尺寸来修正这个问题。例如，如果希望按钮b没有边缘，就可以使用下列代码：

```
b.setMargin(new Insets(0,0,0,0));
```

另一个与按钮尺寸有关的问题是，如果图标太大，在按钮上放不下的话，图片就会被裁切掉。重新设置按钮尺寸时，要做大量调整才能使GUI看起来好看一些。 △

7.2.2 向菜单条目添加图标

用方法setIcon向JMenuItem添加图标的方式，与向JButton添加图标的方式完全相同。Web上的文件MemoIconDemo.java中给出了一个例子。

快速参考：Dimension类和Insets类

Dimension类和Inset类的对象可以为按钮、标签及其他对象指定尺寸。下列构造器中的参数是以像素为单位的：

构造器：

```
Insets(int top, int left, int bottom, int right)
Dimension(int width, int height)
```

举例：

```
aButton.setMargin(new Insets(10, 20, 10, 20));
aLabel.setPreferredSize(new Dimension(20, 50));
```

自测题

9. 如果想创建一个上面既有文本"Push Me"，又有图片文件alice.gif的按钮，该怎么做呢？
10. 如何将文件alice.gif中的图片添加到名为picturePanel的JPanel中去呢？假设picturePanel有一个FlowLayout管理器。

11. 假设要创建一个带有名为alice.gif的图片文件，但没有文本的按钮。再进一步假设希望按钮的动作命令为"Curiouser and couriouser!"。该如何创建按钮并设置动作命令呢？
12. 假设b是一个JButton。下列代码行能保证按钮的尺寸至少是10像素乘10像素的吗？
```
b.setMinimumSize(new Dimension(10,10));
```

7.2.3　滚动条的JScrollPane类

如下例所示，创建一个文本区时，要指定可见的行数和每行的字符数：
```
JTextArea theText = new JTextArea(10, 40);
```
文本区theText的空间将有10行文本长，每行至少可以容纳40个字符。但是，为了用户能够方便地输入和浏览，最好不要对文本区的行数和每行的字符数有固定的限制。

在用过的专业生成的GUI中，肯定曾经见过可以输入任意数量文本，并可以在一个一次只显示部分文本的“窗口”或**视见区**（view port）中查看文本的文本区。可以通过使用沿着视见区边缘放置的滚动条来查看不同部分的文本。在这些专业生成的GUI中，文本好像写在一张无边界的纸上一样，但这张纸被另一张纸盖住了，上面这张纸被裁切掉了一个矩形区域，使能够看到部分文本。被裁切掉的区域就是视见区。图7-4对这个概念进行了说明。然后，可以用滚动条来移动视见区，这样，就可以通过这个裁切视见区看到不同的文本区域了。（可能有人希望将视见区当作固定的，而将文本当作移动的。这两种考虑方式是等价的。）Swing允许向文本区中添加滚动条。可以通过使用JScrollPane类来提供滚动条。

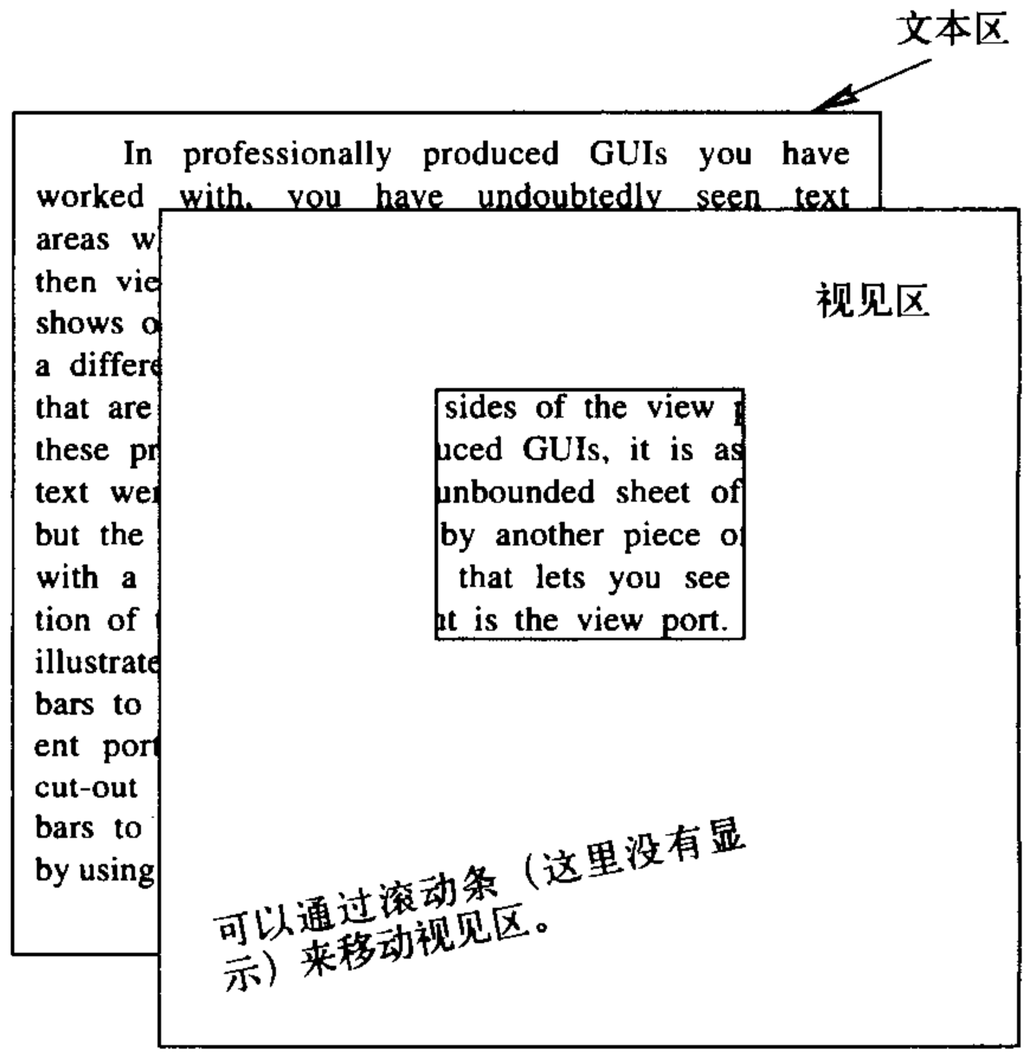


图7-4　一个文本区的视见区

JScrollPane类的对象本质上是一个带有滚动条的视见区。创建一个JScrollPane时，将文本区作为一个实参赋给了JScrollPane的构造器。例如，如果（如本节开始处的代码行中创建的那样）theText是JTextArea类的一个对象，就可以按下列形式将theText放入一个JScrollPane中去：

```
JScrollPane scrolledText = new JScrollPane(theText);
```

然后可以将JScrollPane添加到一个容器中去，比如JPanel或JFrame，如下所示：

```
textPanel.add(scrolledText);
```

图7-5的程序对这个过程进行了说明，除了有滚动条之外，图7-5的版本与图7-1中的那个版本是相同的。

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class ScrollBarDemo extends JFrame implements ActionListener
{
    public static final int WIDTH = 600;
    public static final int HEIGHT = 300;
    public static final int LINES = 10;
    public static final int CHAR_PER_LINE = 40;

    private JTextArea theText;
    private String memo1 = "No Memo 1.";
    private String memo2 = "No Memo 2.";

    public ScrollBarDemo()
    {
        setSize(WIDTH, HEIGHT);
        addWindowListener(new WindowDestroyer());
        setTitle("Scrolling Memo Saver");
        Container contentPane = getContentPane();
            .
            .
            .
        JPanel textPanel = new JPanel();
        textPanel.setBackground(Color.BLUE);
        theText = new JTextArea(LINES, CHAR_PER_LINE);
        theText.setBackground(Color.WHITE);
        JScrollPane scrolledText = new JScrollPane(theText);
        scrolledText.setHorizontalScrollBarPolicy(
                    JScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS);
        scrolledText.setVerticalScrollBarPolicy(
                    JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);
        textPanel.add(scrolledText);
        contentPane.add(textPanel, BorderLayout.CENTER);
    }

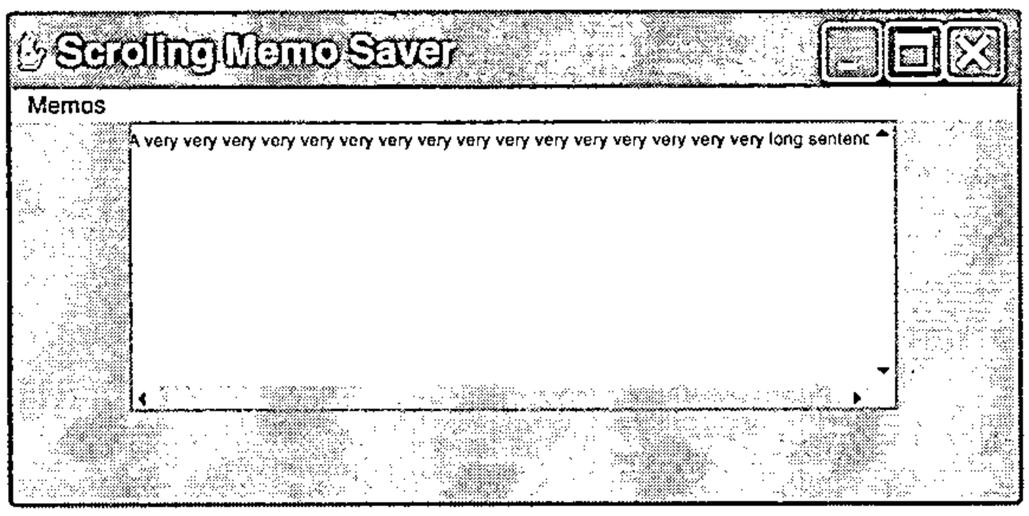
    省略的那些语句与图7-1中的语句相同。实际上，除了改变了类名和JFrame标题，并用前面那四条高亮语句取代了图7-1中的一条语句之外，这个构造器与图7-1中的那个是一样的。所有其他方法都与图7-1中的方法相同。

    public void actionPerformed(ActionEvent e)
    {
```

图7-5 带有滚动条的文本区

```
        <这个方法与图7-1中相应的方法相同>
    }
    public static void main(String[] args)
    {
        ScrollBarDemo guiMemo = new ScrollBarDemo( );
        guiMemo.setVisible(true);
    }
}
```

得到的GUI


Scroling Memo Saver
Memos
A very very very

```
public void setVerticalScrollBarPolicy(int policy)
```

设置显示垂直滚动条的策略。policy应该是下列常量之一

```
JScrollPane.VERTICAL_SCROLLBAR_ALWAYS
JScrollPane.VERTICAL_SCROLLBAR_NEVER
JScrollPane.VERTICAL_SCROLLBAR_AS_NEEDED
```

（如上所示，这些常量都是在JScrollPane中定义的。甚至都不需要知道它们拥有int值。将它们当成一些策略，而不是一些int值。）

```
JScrollPane.HORIZONTAL_SCROLLBAR_AS_NEEDED
JScrollPane.VERTICAL_SCROLLBAR_AS_NEEDED
```

JScrollPane类中的常量。短语"AS_NEEDED"意味着只有在需要的时候才显示滚动条。在正文中对这个概念进行了更全面的解释。其他策略常量的含义从它们的名字中就可以明显地看出来了。

图7-6 （续）

快速参考：JScrollPane

JScrollPane类可以用来向JTextArea（及其他一些组件）添加滚动条。JTextArea对象是作为创建JScrollPane的构造器的实参给出的。

语法：

```
JScrollPane Identifier = new JScrollPane(Text_Area_Object);
```

举例：

```
JTextArea theText = new JTextArea(LINES, CHAR_PER_LINE);
JScrollPane scrolledText = new JScrollPane(theText);
textPanel.add(scrolledText);
contentPane.add(textPanel, BorderLayout.CENTER);
```

常见问题：为什么Java要使用这么长的名字？

程序员使用长的、清晰的标识符，而不为标识符名使用缩略语是Java语言文化的一部分。但是，有些人可能会认为下面这样的名字有点儿太长了：

```
JScrollPane.HORIZONTAL_SCROLLBAR_AS_NEEDED
```

不管是否喜欢这些长名字，都必须承认虽然敲起来有点儿烦，但这些名字很好理解，也很好记。大多数程序员最终都习惯了这些长的、清晰的名字，甚至开始愿意使用这样的名字了。

自测题

13. 建立一个JScrollPane时，是否必须调用setHorizontalScrollBarPolicy和setVerticalScrollBarPolicy这两个方法呢？

14. 在图7-6中，按下列形式列出了JScrollPane的构造器。

```
public JScrollPane(Component objectToBeScrolled)
```

这行代码说明构造器的实参必须是Component类型的。但我们为构造器使用的是JTextArea类型的实参。这难道不是某种类型违规吗？

15. 想向JTextArea添加一个JScrollPane时，是将JScrollPane“添加”到JTextArea中去呢？还是将JTextArea“添加”到JScrollPane中去呢？

7.2.4　添加边界

可以向任何JComponent添加边界。**边界**（border）只是在组件四周围成组件的一个区域。正如你将看到的那样，边界可以有很多不同的形式。边界有两个作用。首先，它可以使组件更吸引人。第二，可以提供一种将某个组件与其他组件分隔开的方式。换句话说，它可以在组件周围添加一些空间。

如果程序使用了边界类，就需要包含下列import语句：

```
import javax.swing.border.*;
```

图7-7中的程序除了显示一些边界之外，没做什么事情。就内容和功能来说，生成的GUI与图5-20中程序生成的GUI是一样的。但是，从审美角度看，这两个GUI有很大的不同。图7-7中程序混合了各种不同的边界风格，因此生成的GUI看起来可能不太好看。但它确实给出了一些能够生成的边界类型实例。可以更严格地、带些艺术眼光地使用这些边界，以改善Swing GUI的外观。

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.border.*;

/**
 Class to demonstrate adding borders to components.
*/
public class BorderDemo extends JFrame implements ActionListener
{
    public static final int WIDTH = 400;
    public static final int HEIGHT = 300;

    private JTextField name;

    public BorderDemo()
    {
        setTitle("Name Tester with Borders");
        setSize(WIDTH, HEIGHT);
        addWindowListener(new WindowDestroyer());
        Container content = getContentPane();
        content.setLayout(new GridLayout(2, 1));

        JPanel namePanel = new JPanel();
        namePanel.setLayout(new BorderLayout());
        namePanel.setBackground(Color.WHITE);

        name = new JTextField(20);
        //The following border is not as dramatic as others,
        //but look closely and you will see it.
        name.setBorder(new EtchedBorder(Color.GREEN, Color.BLUE));
        namePanel.add(name, BorderLayout.SOUTH);
        JLabel nameLabel = new JLabel("Enter your name here:");
        //The following border insert space around the label.
        //To see the difference, comment out the following line:
        nameLabel.setBorder(new EmptyBorder(20, 10, 0, 0));
        namePanel.add(nameLabel, BorderLayout.CENTER);
```

图7-7　不同边界的演示

```
        namePanel.setBorder(new LineBorder(Color.BLACK, 10));
        content.add(namePanel);

        JPanel buttonPanel = new JPanel();
        buttonPanel.setLayout(new FlowLayout());
        JButton testButton = new JButton("Test");
        testButton.addActionListener(this);
        testButton.setBorder(new BevelBorder(BevelBorder.LOWERED));
        buttonPanel.add(testButton);

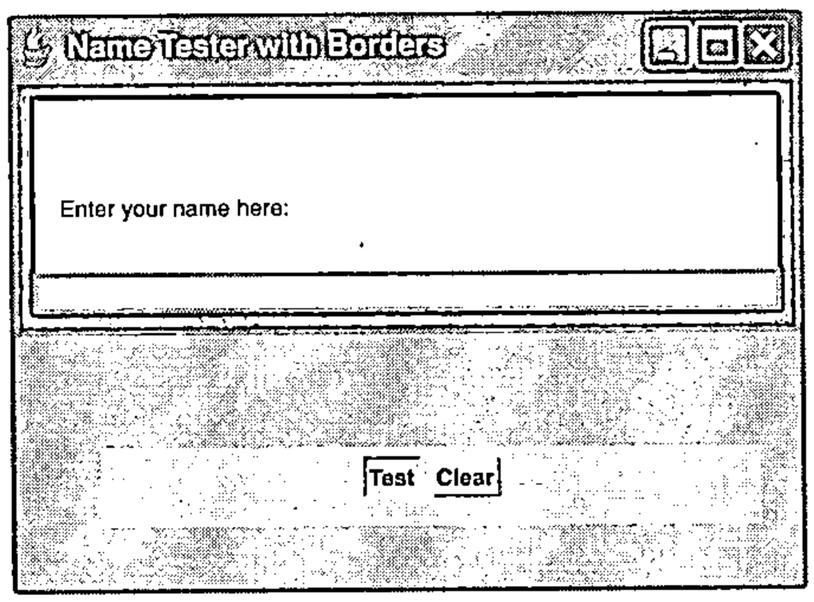
        JButton clearButton = new JButton("Clear");
        clearButton.addActionListener(this);
        clearButton.setBorder(new BevelBorder(BevelBorder.RAISED));
        buttonPanel.add(clearButton);

        buttonPanel.setBorder(new MatteBorder(60, 40, 30, 20, Color.PINK));
        content.add(buttonPanel);
    }

    public void actionPerformed(ActionEvent e)
    {
        if (e.getActionCommand().equals("Test"))
            name.setText("A very good name!");
        else if (e.getActionCommand().equals("Clear"))
            name.setText("");
        else
            name.setText("Error in window interface.");
    }

    public static void main(String[] args)
    {
        BorderDemo w = new BorderDemo();
        w.setVisible(true);
    }
}
```

得到的GUI



要想清楚地看看这些边界的样子，就运行一下程序。

图7-7 （续）

可以用方法setBorder向组件添加一个边界。例如，考虑下列来自图7-7的代码。

```
JButton testButton = new JButton("Test");
testButton.addActionListener(this);
```

```
testButton.setBorder(new BevelBorder(BevelBorder.LOWERED));
```

最后一行为按钮testButton给出了一个BevelBorder。通用的语法为：

```
JComponent.setBorder(Border_Object);
```

可以将边界放在任意JComponent周围，比如JButton、JLabel、JPanel或者JTextField。边界的确切样式取决于作为方法setBorder实参使用的*Border_Object*。

每个不同类型的边界类都会生成一个不同样式的边界。同样，每个边界类构造器都会用一些实参对其外观进行少量的调整。例如，BevelBorder类会生成一个边界，使组件看起来好像从GUI平面中凸出来，或者凹陷到GUI平面中去。实参BevelBorder.RAISED使组件看起来好像凸出来了，实参BevelBorder.LOWERED使组件看起来好像凹进去了。用于按钮时，BevelBorder类的效果会明显。但是，任意边界类都可用于任意JComponent。因此，在JPanel或任何其他的JComponent周围都可以使用BevelBorder。

为边界对象使用匿名实参是很常见的现象，就像在下面这行代码中一样。

```
testButton.setBorder(new BevelBorder(BevelBorder.LOWERED));
```

但是，如果愿意，可以为边界对象起个名字。前面的代码行与下列代码行是等价的。

```
BevelBorder myBorder = new BevelBorder(BevelBorder.LOWERED);
testButton.setBorder(myBorder);
```

图7-8对一些边界类及其可能的实参进行了总结。了解这些边界的最好方式就是尝试一下每种边界类，看看它会生成哪种类型的边界。

EmptyBorder对象只是在组件周围插入了一些空间。可以如图7-7示例程序中的下列代码所示，为四条边界中的每一条指定（以像素为单位的）空间大小：

```
JLabel nameLabel = new JLabel("Enter your name here:");
//The following border inserts space around the label.
//To see the difference, comment out the following line:
nameLabel.setBorder(new EmptyBorder(20, 10, 0, 0));
```

前面代码中设置的边界在标签上方添加了20个像素的空间，在标签左侧添加了10个像素的空间，在标签下方添加了0个像素的空间，在标签的后面添加了0个像素的空间。当然，GUI自身也会有些额外的空间，因此，即使没有添加任何空间，标签后面也会有些空间的。类似的评注也适用于其他的边界类。

LineBorder类在组件周围插入了一个指定尺寸的有色边界，在下列来自图7-7的例子中，它在一个名为namePanel的JPanel四周添加了一个10像素宽的黑色边界：

```
namePanel.setBorder(new LineBorder(Color.BLACK, 10));
```

MatteBorder与LineBorder类似，但可以为4条边界分别指定一个以像素为单位的尺寸。图7-7中的下列实例在buttonPanel周围放置了一个粉色边界：

```
buttonPanel.setBorder(new MatteBorder(60, 40, 30, 20, Color.PINK));
```

顶端的边界会有60像素宽，左侧会有40像素宽，底端会有30像素宽，右侧会有20像素宽。

EtchedBorder与BevelBorder类似，但它总是很窄，并会显示一些有色阴影。无法设置EtchedBorder的宽度。了解EtchedBorder会对类做些什么的最好方法就是，将其用于这些类，生成各种各样的实例，并对它们进行比较。

快速参考：setBorder方法

可以用setBorder方法向任意JComponent添加一个边界。

语法：

JComponent.setBorder(*Border_Object*);

*JComponent*可以是任意JComponent对象，比如JButton、JLabel、JPanel或者其他JComponent。*Border_Object*可以是图7-8中给出的任意边界类的对象。

举例：

```
namePanel.setBorder(new LineBorder(Color.BLACK, 10));
```

```
public BevelBorder(int bevelType);
```

创建一个新的BevelBorder对象的构造器。实参bevelType应该是常量BevelBorder.RAISED或BevelBorder.LOWERED之一。

```
public EtchedBorder(int etchType, Color highlight, Color shadow)
```

创建一个新的EtchedBorder对象的构造器。除了无法设置其尺寸，且具有采用指定高亮和阴影颜色的有色阴影特性之外，蚀刻（etched）边界与BevelBorder类似。实参etchType应该是常量EtchedBorder.RAISED或EtchedBorder.LOWERED之一。

```
public EtchedBorder(Color highlight, Color shadow)
```

用指定的高亮和阴影颜色创建一个新的凹陷的EtchedBorder对象的构造器。

```
public EmptyBorder(int top, int left, int bottom, int right)
```

创建一个新的EmptyBorder对象的构造器。空边界本质上指的就是组件周围的空间。这些实参给出了四个边界区域中以像素为单位的尺寸。

```
public LineBorder(Color theColor, int thickness)
```

创建一个新的LineBorder对象的构造器。线边界是一种具有指定宽度的有色边界。宽度是以像素为单位给出的。

```
public LineBorder(Color color, int thickness,boolean roundedCorners)
```

创建一个带有指定颜色、宽度和边角形状的线边界。宽度是以像素为单位给出的。如果roundedCorners为true，生成的就是圆角。如果roundedCorners为false，生成的就是直角。如果省略第三个实参，生成的就是直角。

```
public MatteBorder(int top, int left, int bottom, int right, Color theColor)
```

创建一个新的MatteBorder对象的构造器。MatteBorder与LineBorder类似，但可以为四个边界指定（以像素为单位的）尺寸。（还应该去读一下名为“MatteBorder中的图标”的快速参考框。）

图7-8 一些边界类

快速参考：MatteBorder中的图标

MatteBorder类有一个构造器可以用来指定一个图标而不是一种颜色。这个构造器如下所示：

```
public MatteBorder(int top, int left, int bottom, int right, ImageIcon theIcon)
```

这个构造器会创建一个新的、（像墙纸模式一样）铺满了theIcon副本的MatteBorder对象。Web上的文件BorderDemoWithIcon.java中有一个使用了这个构造器的程序实例。

△ 易犯错误：忘记导入javax.swing.border

如果编译器说它无法找到正在使用的边界类，一定就是忘记与边界打交道时所需的下列import语句了：

```
import javax.swing.border.*;
```

△

常见问题：还有更多的边界类吗？

是的，有更多的边界类。可以去查看Sun的Web站点上描述这些类的Java文档。但是，在大多数情况下，这里提供的边界类就可以很好地为你服务了。

常见问题：能对Swing GUI的观感进行修改吗？

GUI的观感（look and feel）指的是它的整体外观。包括形状、按钮的确切位置、默认的颜色，以及几乎所有会影响一个GUI的外观而不会影响它所完成的功能的东西。Swing GUI的观感是可以改变的，但这超出了本书的范围。现在，就只能接受系统提供的默认观感了。

自测题

16. 可以仅用边界在组件周围添加一些空间吗？
17. 下列代码行能使你的程序得以访问边界类吗？

```
import javax.swing.*;
```

7.3　更多布局管理器

box 名词 1.通常以四面垂直于底边的形式构建的一个容器，通常会有个盖子或封面。……

——《美国传统英语词典》，（第三版）

GUI通常需要一个面板用于管理一个水平或垂直的条目阵列，比如按钮阵列。如果需要的是水平阵列，使用带有FlowLayout管理器的JPanel就行了。如果需要的是垂直阵列，使用带有单列GridLayout管理器的JPanel就行了。但是，BoxLayout管理器类和Box容器类可以通过一种可能更简单、并能提供一些方便的附加功能的方式，生成这样的面板及布局管理器。这些附加功能包括一些创建不可见组件以分隔其他组件的方式，这样就可以在按钮之间增加空间了。Box容器本质上是一个类似于面板的类，它以一种很方便的形式使用BoxLayout管理器。首先讨论一个对JPanel使用了BoxLayout管理器的程序，然后讨论第二个程序，这个程序做的事情与第一个程序相同，但使用了Box容器类以简化并自动实现部分编程工作。这两个程序会生成同样的GUI，图7-9显示了这个GUI。

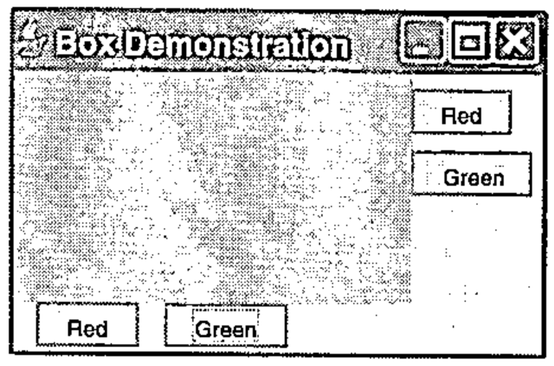


图7-9　用BoxLayout管理器构建的GUI

图7-9中的GUI沿着底边有两个标记为"Red"和"Green"的按钮，沿着GUI的右边有另外两个标记为"Red"和"Green"的按钮。最初中间面板是蓝色的。点击任意一个"Red"按钮，中间的面板都会变成红色。同样，点击任意一个"Green"按钮，中间面板都会变成绿色。这里GUI

所做的事情没什么新鲜的。新鲜的是BoxLayout管理器的使用。

7.3.1 BoxLayout管理器类

运行图7-10中的程序时，会生成图7-9中的GUI。它使用了BoxLayout管理器类。从很多方面来看，程序中的代码都只是以前已经见过的内容。JFrame的内容面板被分成了3个区域，每个区域中包含了一个面板。按钮是按照以前见过的方式创建并添加到面板中去的。方法actionPerformed与在本章前面和第5章中见过的actionPerformed方法基本相同。实际上，所有没有高亮显示的代码使用的都是以前见过的那些技术。

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

/**
 Simple demonstration of BoxLayout manager class and the use of struts
 to separate components (in this case buttons). For an alternative
 implementation see BoxClassDemo in Display 7.11.
*/
public class BoxLayoutDemo extends JFrame implements ActionListener
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;
    public static final int HORIZONTAL_STRUT_SIZE = 15;
    public static final int VERTICAL_STRUT_SIZE = 10;

    private JPanel colorPanel;

    public BoxLayoutDemo()
    {
        setSize(WIDTH, HEIGHT);
        addWindowListener(new WindowDestroyer());
        setTitle("Box Demonstration");
        Container content = getContentPane();
        content.setLayout(new BorderLayout());

        colorPanel = new JPanel();
        colorPanel.setBackground(Color.BLUE);
        content.add(colorPanel, BorderLayout.CENTER);

        //Horizontal buttons at bottom of frame:
        Jpanel horizontalPanel = new JPanel();
        horizontalPanel.setLayout(
            new BoxLayout(horizontalPanel, BoxLayout.X_AXIS));
        Component horizontalStrut =
            Box.createHorizontalStrut(HORIZONTAL_STRUT_SIZE);
        horizontalPanel.add(horizontalStrut);

        JButton hStopButton = new JButton("Red");
```

得到的GUI如图7-9所示。

图7-10 BoxLayout管理器

```
        hStopButton.addActionListener(this);
        horizontalPanel.add(hStopButton);

        Component horizontalStrut2 = Box.createHorizontalStrut(HORIZONTAL_STRUT_SIZE);
        horizontalPanel.add(horizontalStrut2);

        JButton hGoButton = new JButton("Green");
        hGoButton.addActionListener(this);
        horizontalPanel.add(hGoButton);

        content.add(horizontalPanel, BorderLayout.SOUTH);

        //Vertical buttons on right side of frame:
        JPanel verticalPanel = new JPanel();
        verticalPanel.setLayout(
            new BoxLayout(verticalPanel, BoxLayout.Y_AXIS));

        Componet verticalStrut =
            Box.createVerticalStrut(VERTICAL_STRUT_SIZE);
        verticalPanel.add(verticalStrut);

        JButton vStopButton = new JButton("Red");
        vStopButton.addActionListener(this);
        verticalPanel.add(vGoButton);

        Component verticalStrut2 =
            Box.createVerticalStrut(VERTICAL_STRUT_SIZE);
        verticalPanel.add(verticalStrut2);

        JButton vGoButton = new JButton("Green");
        vGoButton.addActionListener(this);
        verticalPanel.add(vGoButton);

        content.add(verticalPanel, BorderLayout.EAST);
    }

    public void actionPerformed(ActionEvent e)
    {
        if (e.getActionCommand().equals("Red"))
            colorPanel.setBackground(Color.RED);
        else if (e.getActionCommand().equals("Green"))
            colorPanel.setBackground(Color.GREEN);
        else
            System.out.println("Error in button interface.");
    }

    public static void main(String[] args)
    {
        BoxLayoutDemo gui = new BoxLayoutDemo();
        gui.setVisible(true);
    }
}
```

图7-10　（续）

有个次要细节以前没见过：有两个按钮上都写着"Red"，而且actionPerformed方法与哪个按钮被点击无关，因为它们都有着同样的动作命令，即，字符串"Red"。两个"Green"按钮的情况与之类似。

图7-10中真正的新内容是那些高亮代码。名为horizontalPanel的面板中装载了屏幕底端的那些按钮。这个面板使用的是BoxLayout管理器。创建一个BoxLayout管理器的语法与创

建其他布局管理器的语法略有不同，如图7-10中的下列语句所示：

```
horizontalPanel.setLayout(new BoxLayout(horizontalPanel, BoxLayout.X_AXIS));
```

注意，与在第5章看到的布局管理器类不同，BoxLayout管理器的构造器有实参，实际上，它有两个实参。第一个实参是将其作为布局管理器的那个容器。这条信息很明显是冗余的，因为用这种方式使用时它与调用对象是相同的。但不管是否冗余，都需要第一个实参。第二个实参必须是两个常量BoxLayout.X_AXIS或BoxLayout.Y_AXIS之一。常量用来说明布局是垂直的（X_AXIS）还是水平的（Y_AXIS）。

BoxLayout构造器的第二个实参为BoxLayout.X_AXIS时，布局是一个水平面板，组件从左至右添加。因此，前面显示的布局管理器基本等价于下列布局管理器：

```
horizontalPanel.setLayout(new FlowLayout());
```

开始，可能没注意到这两种布局管理器（水平的BoxLayout和FlowLayout）之间有什么区别。对水平BoxLayout管理器所做的工作进行初步估计时，把水平BoxLayout看作与FlowLayout一样是很有帮助的。但是，在这两个布局管理器类之间存在一些微小，但有时却很重要的区别。正如将看到的那样，水平BoxLayout管理器会严格按照告诉它的那样去做。如果告诉它在面板的起始处留出15个像素的空隙，它总是会正好留出15个像素的空隙。另一方面，FlowLayout管理器会“认为”它“知道的更多”，从而会添加一些额外的像素使图“看起来更好看”。

注意，对BoxLayout管理器来说，使用add方法添加组件的方式与为FlowLayout管理器添加组件的方式相同。如果像原来所讨论的那样，布局是水平的，就按照组件的添加顺序，从左至右放置。如果像我们接下来要讨论的那样，布局是垂直的，就按照组件的添加顺序从顶端至底端放置。

描述超出FlowLayout管理器功能的那部分内容需要进行大量的工作和解释！但马上就会有回报了。图7-10中其余的部分显露了BoxLayout管理器类的某些真实能力。

刚才讨论了如何才能通过使用BoxLayout管理器类获得组件的水平布局。要通过BoxLayout管理器获得一个垂直布局，只要用BoxLayout.Y_AXIS取代BoxLayout.X_AXIS作为BoxLayout构造器的第二个实参就行了。沿着图7-9中GUI的右侧边界显示按钮的面板对这种方法进行了说明。图7-10中的程序通过下列代码说明了面板是一个垂直面板：

```
verticalPanel.setLayout(new BoxLayout(verticalPanel, BoxLayout.Y_AXIS));
```

这个布局管理器会将按钮垂直地从顶到底，而不是水平地从左至右排列。

注意，FlowLayout管理器中没有等价于垂直布局的部分。可以通过只有一列的GridLayout管理器获得对垂直BoxLayout管理器的粗略近似，但间隔会有所不同。BoxLayout管理器对间隔可以有更多的控制。对其他布局管理器来说，通常只能对布局管理器提些建议，但不能指望这些建议一定会被遵守。例如，对GridLayout管理器来说，组件通常会被伸展开，使其大小相同，通常这些组件也会被一个挨一个地排列起来。

曾经说过BoxLayout管理器可以更好地控制组件间的间隔，但要利用这种控制功能，需要使用一些不可见的、名为strut和glue的组件。在接下来的小节中将对这些类型的组件进行讨论。

7.3.2 strut与glue

BoxLayout管理器类的重要性中有一部分来自于包含在相关的Box类中的静态方法。有些

静态方法能够生成一些可以添加到容器中去的不可见组件。可以用这些不可见组件在可见组件之间增加间隔。例如，考虑下列来自图7-10中的代码：

```
Component horizontalStrut = Box.createHorizontalStrut(HORIZONTAL_STRUT_SIZE);
horizontalBox.add(horizontalStrut);
```

方法createHorizontalStrut是Box类中的一个静态方法。它创建了一个被称为**strut**的Component类型的对象。strut是一个具有固定水平尺寸的不可见组件。例如，方法调用

```
Box.createHorizontalStrut(HORIZONTAL_STRUT_SIZE);
```

会生成一个水平尺寸为HORIZONTAL_STRUT_SIZE像素的strut，在图7-10的程序中这个值恰好是15个像素。布局管理器可以改变这个strut的垂直尺寸，但无法改变其水平尺寸（即，无法改变组件的宽度）。strut总是精确的15像素宽，因此，如果用BoxLayout管理器将这个strut添加到一个容器中去，它总会在组件之间产生一个15像素的间隔。

有两种类型的strut：**水平strut**（horizontal strut）和**垂直strut**（vertical strut）。我们刚刚讨论了水平strut。水平strut是用方法createHorizontalStrut创建的，有着无法改变的固定宽度。布局管理器可以改变strut的高度，但不能改变其宽度。垂直strut与之类似，只是它的高度是固定的。垂直strut是用方法createVerticalStrut创建的。布局管理器可以改变垂直strut的宽度，但不能改变其高度。可以把strut当作一个用来分隔可见组件的定长小棍。实际上，这就是*strut*这个词的根本含义，也就是Swing的设计者们用*strut*这个单词来表示这些组件的原因。

glue组件（glue components）是与strut类似的不可见组件，只是glue组件没有那么坚硬。可以用它们来分隔可见组件。实际上，使用单词*glue*会有些误导，用它作这些组件的名字并不是很好的选择。Glue组件并没有将任何东西与其他的东西粘合在一起。单词*glue*只是想传达这样一种思想：这些组件不像strut那么坚硬，而是像胶水那样湿软。（可能*goo*是个更好的名字，但我们得和*glue*粘在一起了，这可不是双关语。）

布局管理器可以使glue组件变大或变小。有**水平**（horizontal）和**垂直**（vertical）glue组件。它们是分别用静态方法createHorizontalGlue和createVerticalGlue创建的。这些方法都在Box类中。例如，下列代码行就创建了一个水平和一个垂直glue组件：

```
Component horizontalGlue = Box.createHorizontalGlue();
Component verticalGlue = Box.createVerticalGlue();
```

没有为glue组件指定尺寸。

用方法add将glue组件添加到容器中去的方式与添加strut的方式相同。Web上的GlueDemo.java文件提供了一个使用glue组件的程序实例。

自测题

18. 假设myPanel是个JPanel。怎样为myPanel分配一个垂直BoxLayout管理器？
19. 下列两种设置布局管理器的方式有什么区别？

```
horizontalPanel.setLayout(new BoxLayout(horizontalPanel, BoxLayout.X_AXIS);
```

和

```
horizontalPanel.setLayout(new FlowLayout());
```

20. 给出一条能够创建一个宽度为20个像素、名为horizontalSpace的水平strut的语句。
21. strut组件和glue组件有什么区别？

△ 易犯错误：对其他布局管理器使用strut和glue

strut和glue组件是设计成与BoxLayout管理器一起使用的。可以将它们与其他布局管理器一起使用，但这样使用时通常无法达到希望实现的目标。如果用其他布局管理器，比如FlowLayout管理器，添加一个15像素宽的水平strut，从技术上讲，它的宽度还是15像素，但布局管理器可能会在其周围添加额外的空间。因为strut是不可见的，所以添加额外空间与修改strut的尺寸几乎是等价的。将glue组件用于除BoxLayout管理器之外的其他布局管理器时，也存在同样的问题。

在接下来的小节中，讨论了Box容器。Box类的容器对象是随BoxLayout管理器自动提供的。因此，向Box容器中添加strut和glue组件是完全没有问题的。 △

7.3.3 设置组件间的间隔

BoxLayout管理器和下一节讨论的Box容器类用strut和glue设置组件间的间隔。在本书中讨论的所有其他布局管理器都使用了下列两种方式在组件间设置水平和垂直间隔：

```
/**
Sets the horizontal gap between components.
The argument is the gap expressed in pixels.
*/
public void setHgap(int hgap)
/**
Sets the vertical gap between components.
The argument is the gap expressed in pixels.
*/
public void setVgap(int vgap)
```

这两种方法不能用于BoxLayout管理器或Box类。

还可以对每个组件使用EmptyBorder来分隔组件。因为间隔是这种组件的一部分，所以这种技术对任何种类的布局管理器都适用。

7.3.4 Box容器类

我们已经讨论过Box类中的静态函数createHorizontalStrut、createVerticalStrut、createHorizontalGlue和createVerticalGlue了。在本小节中，来讨论Box类的对象。Box类对象的表现与一个使用BoxLayout管理器的面板的表现差不多。图7-11的程序对其中的细节进行了说明。那个程序会生成图7-9中的GUI。因此，图7-10中的程序和图7-11中的那个程序是等价的。但是，图7-11的程序是用Box容器——即，Box类的对象——而不是JPanel来装载按钮的。本节的其余部分对那些包含新内容的细节进行了讨论。

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

/**
 Simple demonstration of Box container class and the use of struts
 to separate components (in this case buttons). For an alternative
 implementation see BoxLayoutDemo in Display 14.10.
```

图7-11 Box容器类

```
*/
public class BoxClassDemo extends JFrame implements ActionListener
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;
    public static final int HORIZONTAL_STRUT_SIZE = 15;
    public static final int VERTICAL_STRUT_SIZE = 10;

    private JPanel colorPanel;

    public BoxClassDemo()
    {
        setSize(WIDTH, HEIGHT);
        addWindowListener(new WindowDestroyer());
        setTitle("Box Demonstration");
        Container content = getContentPane();
        content.setLayout(new BorderLayout());

        colorPanel = new JPanel();
        colorPanel.setBackground(Color.BLUE);
        content.add(colorPanel, BorderLayout.CENTER);

        //Horizontal buttons at bottom of frame:
        Box horizontalBox = Box.createHorizontalBox();

        Component horizontalStrut = Box.createHorizontalStrut(HORIZONTAL_STRUT_SIZE);
        horizontalBox.add(horizontalStrut);

        JButton hStopButton = new JButton("Red");
        hStopButton.addActionListener(this);
        horizontalBox.add(hStopButton);
        Component horizontalStrut2 =Box.createHorizontalStrut(HORIZONTAL_STRUT_SIZE);
        horizontalBox.add(horizontalStrut2);

        JButton hGoButton = new JButton("Green");
        hGoButton.addActionListener(this);
        horizontalBox.add(hGoButton);

        content.add(horizontalBox, BorderLayout.SOUTH);

        //Vertical buttons on right side of frame:
        Box verticalBox = Box.createVerticalBox();

        Component verticalStrut = Box.createVerticalStrut(VERTICAL_STRUT_SIZE);
        verticalBox.add(verticalStrut);

        JButton vStopButton = new JButton("Red");
        vStopButton.addActionListener(this);
        verticalBox.add(vStopButton);

        Component verticalStrut2 = Box.createVerticalStrut(VERTICAL_STRUT_SIZE);
        verticalBox.add(verticalStrut2);
```

图7-11　(续)

```
        JButton vGoButton = new JButton("Green");
        vGoButton.addActionListener(this);
        verticalBox.add(vGoButton);

        content.add(verticalBox, BorderLayout.EAST);
    }

    public void actionPerformed(ActionEvent e)
    {
        if (e.getActionCommand().equals("Red"))
            colorPanel.setBackground(Color.RED);
        else if (e.getActionCommand().equals("Green"))
            colorPanel.setBackground(Color.GREEN);
        else
            System.out.println("Error in button interface.");
    }

    public static void main(String[] args)
    {
        BoxClassDemo gui = new BoxClassDemo();
        gui.setVisible(true);
    }
}
```

图7-11 (续)

看看图7-11和7-10中的程序，就会发现它们之间唯一的区别就是图7-10中的horizontalPanel面板被图7-11中的Box对象horizontalBox取代了，而图7-10中的verticalPanel面板被图7-11中的Box对象verticalBox取代了。

对象horizontalBox是由图7-11中的下列语句创建的：

```
Box horizontalBox = Box.createHorizontalBox();
```

类似地，对象verticalBox是由图7-11中的下列语句创建的：

```
Box verticalBox = Box.createVerticalBox();
```

注意，创建box的方式与期望的方式可能有所不同。为了创建一个Box类对象，使用了静态方法Box.createHorizontalBox或Box.createVerticalBox。当然，这两个方法之间的区别在于第一个方法创建了一个水平box，第二个方法创建了一个垂直box。

不需要为了给Box对象分配一个布局管理器而使用方法setLayout。实际上，不应该对Box对象使用方法setLayout。创建Box类对象时，会自动为其分配一个BoxLayout管理器。

如果愿意的话，可以用new调用Box类的构造器，以常见的方式创建一个Box对象。例如，下面的两条语句是等价的：

```
Box horizontalBox = new Box(BoxLayout.X_AXIS);
Box horizontalBox = Box.createHorizontalBox();
```

类似地，下列两条语句是等价的：

```
Box verticalBox = new Box(BoxLayout.Y_AXIS);
Box verticalBox = Box.createVerticalBox();
```

注意，对Box类的构造器使用new时，需要给构造器一个实参。如果实参是常量BoxLayout.X_AXIS，创建的就是一个水平Box容器。如果实参是常量BoxLayout.Y_AXIS，

创建的就是一个垂直Box容器。

自测题

22. 给出创建名为vBox的垂直Box容器的两种方式。（提示：一种方式使用的是方法，一种方式使用的是构造器。）
23. 为一个使用了Box容器的Swing GUI编写代码时，要用setLayout方法为Box容器指定布局管理器吗？
24. strut和glue可以与本书中讨论的哪些布局管理器一起工作？
25. 方法setHgap和setVgap可以与本书中讨论的哪些布局管理器一起工作？

7.3.5 CardLayout管理器

CardLayout是一个可以向Swing GUI添加动态元素的布局管理器类。CardLayout管理器使用户可以拥有一组可变的视窗，就像翻一副牌一样，每次看一张。使用CardLayout管理器，就可以向一个容器中添加任意数量的视图，但一次只有一个是可见的。可以按顺序浏览这些视图，也可以从一个视图跳到另一个任意的视图中去。因此，这个布局管理器所做的不仅仅是在容器中排列组件。下面来看一个例子。

图7-12显示了一个使用了CardLayout管理器的GUI。GUI的中间部位有三个可能的视图。用户点击其中一个按钮，就可以从一个视图移到另一个视图中去。

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class CardLayoutDemo extends JFrame implements ActionListener
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;

    private CardLayout dealer;
    private JPanel deckPanel;

    public CardLayoutDemo()
    {
        setSize(WIDTH, HEIGHT);
        addWindowListener(new WindowDestroyer());
        setTitle("CardLayout Demonstration");
        Container contentPane = getContentPane();
        contentPane.setLayout(new BorderLayout());

        deckPanel = new JPanel();
        dealer = new CardLayout();
        deckPanel.setLayout(dealer);

        JPanel startCardPanel = new JPanel();
        startCardPanel.setLayout(new FlowLayout());
        startCardPanel.setBackground(Color.LIGHT_GRAY);
        JLabel startLabel = new JLabel("Hello");
        startCardPanel.add(startLabel);
        deckPanel.add("start", startCardPanel);
```

图7-12　CardLayout管理器

```
        JPanel greenCardPanel = new JPanel();
        greenCardPanel.setLayout(new FlowLayout());
        greenCardPanel.setBackground(Color.GREEN);
        JLabel goLabel = new JLabel("Go");
        greenCardPanel.add(goLabel);
        deckPanel.add("green", greenCardPanel);

        JPanel redCardPanel = new JPanel();
        redCardPanel.setLayout(new FlowLayout());
        redCardPanel.setBackground(Color.RED);
        JLabel stopLabel = new JLabel("Stop");
        redCardPanel.add(stopLabel);
        deckPanel.add("red", redCardPanel);

        contentPane.add(deckPanel, BorderLayout.CENTER);

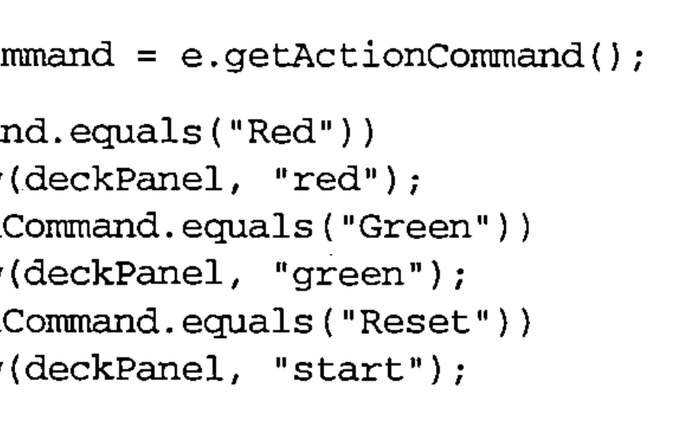
        JPanel buttonPanel = new JPanel();
        buttonPanel.setBackground(Color.WHITE);
        buttonPanel.setLayout(new FlowLayout());
        JButton stopButton = new JButton("Red");
        stopButton.addActionListener(this);
        buttonPanel.add(stopButton);
        JButton goButton = new JButton("Green");
        goButton.addActionListener(this);
        buttonPanel.add(goButton);
        JButton resetButton = new JButton("Reset");
        resetButton.addActionListener(this);
        buttonPanel.add(resetButton);

        contentPane.add(buttonPanel, BorderLayout.SOUTH);
        dealer.first(deckPanel);//Optional
    }
    public void actionPerformed(ActionEvent e)
    {
        String actionCommand = e.getActionCommand();

        if (actionCommand.equals("Red"))
            dealer.show(deckPanel, "red");
        else if (actionCommand.equals("Green"))
            dealer.show(deckPanel, "green");
        else if (actionCommand.equals("Reset"))
            dealer.show(deckPanel, "start");
        else
            System.out.println("Error in CardLayout Demo.");
    }

    public static void main(String[] args)
    {
        CardLayoutDemo demoGui = new CardLayoutDemo();
        demoGui.setVisible(true);
    }
}
```

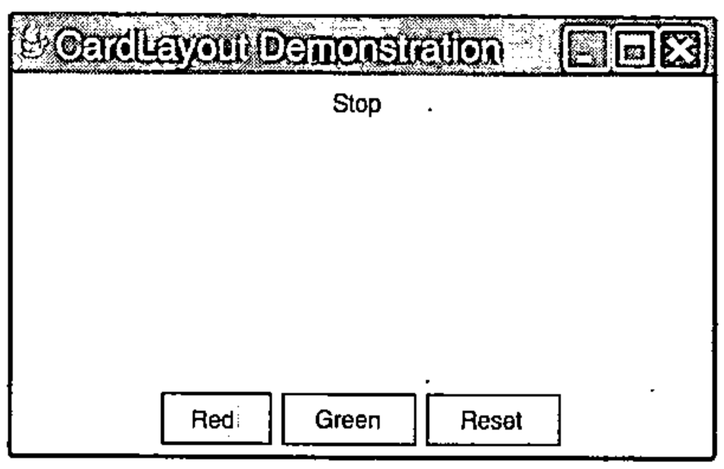
显示的第一个GUI



这是点击Reset按钮时生成的视图。

图7-12 （续）

点击Red按钮时显示的GUI



点击Green按钮时显示的GUI

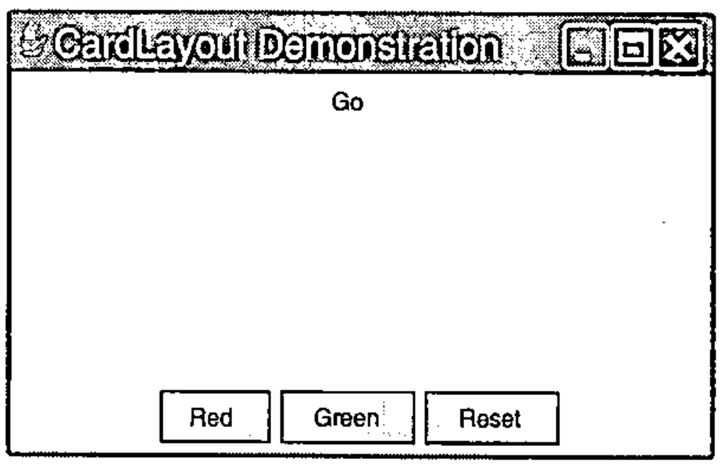


图7-12　（续）

创建了一个CardLayout类的对象，并像任意其他布局管理器类一样，将这个布局管理器分配给一个容器。来自图7-12的下列代码行显示了这个过程，为deckPanel面板分配了一个CardLayout管理器：

```
deckPanel = new JPanel();
dealer = new CardLayout();
deckPanel.setLayout(dealer);
```

注意，没有为setLayout方法使用匿名CardLayout实参。下列代码行虽然是合法的，但却没什么用处：

```
deckPanel.setLayout(new CardLayout());  //Do not use this!
```

CardLayout管理器需要一个名字（dealer，庄家），这样就可以用它来改变要显示的那张“牌”了。而且，需要在多个方法中引用CardLayout管理器，因此，要将dealer设置为一个CardLayout类型的实例变量。

如GUI代码所示，向一个使用了CardLayout管理器的容器中添加元素时，要将字符串作为第一个实参，将组件作为第二个实参。第一个实参中的字符串是作为组件的名字使用的：

```
deckPanel.add("start", startCardPanel);
...
deckPanel.add("green", greenCardPanel);
...
deckPanel.add("red", redCardPanel);
```

第一次显示GUI时，它显示的是第一个添加的组件。为了进入另一个视图，代码使用了方法show。例如，考虑下列来自图7-12actionPerformed定义中的代码行。

```
dealer.show(deckPanel, "red");
```

执行这条语句时，当前显示的视图（“牌”）就变成了名为"red"的那个视图。在图7-12的GUI

中，这就意味着，当用户点击标记为"Red"的按钮时，就会显示名为redCardPanel的面板。注意，容器是作为show的第一个实参给出的。（毕竟，布局管理器可以管理多个容器。）

另外两种进入新视图（新“牌”）的方法是first和next。来自图7-12构造器末尾的下列代码行显示了添加到容器中去的第一个组件：

```
dealer.first(deckPanel); //Optional
```

如单词Optional所示，如果省略对first的这个特定调用，GUI的表现将是一样的。这是因为容器总是以视图中的第一个组件（第一张“牌”）开始的。但是，在其他一些地方，这个方法会改变显示的内容。

方法next会按照组件的添加顺序进入下一个视图（下一张“牌”）。在最后一个组件之后，next会回到第一个组件上去。图7-12中没有使用下列代码行，但如果用了，就会将视图从当前组件移到下一个组件上去。

```
dealer.next(deckPanel);
```

图7-13对CardLayout类的一些方法进行了总结。

`public void first(Container theContainer)`
显示theContainer中的第一张“牌”。

`public void last(Container theContainer)`
显示theContainer中的最后一张“牌”。

`public void next(Container theContainer)`
显示theContainer中的下一张“牌”。下一张牌是在当前显示的牌之后添加的那张牌。如果当前显示的牌是最后一张牌，next就会显示第一张牌。

`public void previous(Container theContainer)`
显示theContainer中的前一张“牌”。前一张牌是在当前显示的牌之前添加的那张牌。如果当前显示的是第一张牌，previous就会显示最后一张牌。

`public void show(Container theContainer, String cardName)`
显示以名字cardName添加的那张“牌”。

图7-13 CardLayout管理器类中的一些方法

快速参考：CardLayout管理器类

CardLayout管理器用图7-13中给出的方法显示了添加到其容器中去的组件，一次显示一个。每个组件被添加到容器中去时，都被分配了一个字符串名，如下所示：

语法：

Container.add(*String_Name*, *Component*);

举例：

```
deckPanel.add("start", startCardPanel);
```

自测题

26. 在图7-12中，按下列形式将布局管理器添加到面板deckPanel中去。

```
dealer = new CardLayout();
deckPanel.setLayout(dealer);
```

为什么不用下列代码添加布局管理器呢？

```
deckPanel.setLayout(new CardLayout());
```

27. 编写会显示出图7-12 deckPanel中下一张“牌”的Java代码。
28. 编写会显示出图7-12 deckPanel中前一张“牌”的Java代码。
29. 假设dealer是CardLayout类的一个对象，deckPanel是一个将dealer作为其布局管理器的JPanel（如图7-12所示）。如果deckPanel当前显示的牌是第一张牌，那么执行了下面的语句之后显示的是哪张牌？

```
dealer.previous(deckPanel);
```

7.4　内部类

隐藏很深的东西一定是在什么东西的背后。

——阿尔伯特·爱因斯坦，《纽约时代杂志》引用的话（1964年8月2日）

内部类（inner class）是在其他类中定义的类。它们的用途并不局限于那些使用Swing的程序和类。但是，在对Swing编程时常常会用到它们。尽管对内部类的完整描述已经超出了本书的范围，但对内部类的简单使用是非常容易而且有用的。我们将讨论一种最有用的内部类应用，即将内部类作为辅助类使用。

内部类是在第3章讨论的，但由于有些读者可能会在阅读第3章之前阅读本章，所以并不假设你已经读过第3章了。（如果读过第3章了，至少还应该读读本节的这些例子。）

辅助类

所有的Swing窗口都是用（图5-4中的）WindowDestroyer类来关闭窗口的。用这种方式实现时，必须得有WindowDestroyer类才行。内部类就是把WindowDestroyer这样的辅助类的定义移到类定义内部中去。例如，图7-14中的类简单演示了如何将内部类作为一个窗口侦听器使用。这个GUI显示的内容本质上与第一个Swing GUI类（图5-2）显示的内容相同，但它使用了内部类InnerDestroyer而不是WindowDestroyer类。一定要注意，InnerDestroyer类的定义位于InnerClassDemo类（图7-14）的内部，这样，整个程序就位于一个（内部有另外一个类定义的）类定义内部了。

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class InnerClassDemo extends JFrame
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;

    /**
     Creates and displays a window of the class InnerClassDemo.
    */
    public static void main(String[] args)
    {
```

图7-14　内部类

```
        InnerClassDemo sampleGUI = new InnerClassDemo();
        sampleGUI.setVisible(true);
    }

    public InnerClassDemo()
    {
        setSize(WIDTH, HEIGHT);
        setTitle("Inner Class Demo");
        Container contentPane = getContentPane();
        contentPane.setLayout(new BorderLayout());

        JLabel label = new JLabel("Please don't click that button!");
        contentPane.add(label, BorderLayout.CENTER);

        addWindowListener(new InnerDestroyer());
    }

    //An inner class with the same functionality
    //as the class WindowDestroyer.
    private class InnerDestroyer extends WindowAdapter
    {
        public void windowClosing(WindowEvent e)
        {
            System.exit(0);
        }
    }
}
```

一个内部类

得到的GUI

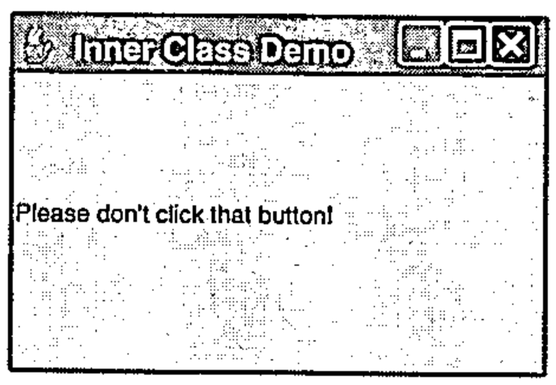


图7-14 （续）

内部类有两个很大的优点。第一，因为它们是在一个类的内部定义的，所以可以使包含了内部类的外部类成为自包含的，或者使其比不使用内部类时的自包含性更高一些。第二个优点在图7-14中没有用到，但这个优点可能是很有用的。定义一个内部类时，内部类中的方法可以访问定义它的那个类中的所有实例变量和方法，即使它们是私有的也一样。由于第二个原因，内部类是很有用的辅助类，因为它们做某些事情的时候，会比一个在所要辅助的类之外定义的类更加方便。由于这些便利的特性，内部类通常会被用作侦听器来处理由其外部类或外部类的一个组件激发的事件。更长的例子，以及更多地使用了内部类特性的例子，请参见Web上的InnerClassDemo2.java程序。

内部类还有一个优点。内部类的名字是局部于定义它的那个类的。在图7-14中那个类的外面还可以定义另一个名为InnerDestroyer的类，而图7-14中的程序会完全忽略那个名为

InnerDestroyer的类。类似的，在类之外定义的名为InnerDestroyer的类也会完全忽略同名的内部类。

将这个内部类InnerDestroyer设置为私有的，是因为它是一个辅助类。也可能有公有的内部类，但使用公有内部类时会存在一些微妙之处。在本书中不对此话题进行讨论。

记住：调用外部类的一个方法

用Java在一个内部类定义中调用外部类的一个方法是很容易的。如果有一个方法调用，而内部类中没有叫做那个名字的方法，但外部类中确实有一个那个名字的方法，Java就会将这个调用解释为对那个外部对象方法的调用（也就是说，调用对象是外部类的this，而不是内部类的this。）Web上的InnerClassDemo2.java对这种调用进行了演示。

7.5　更多关于事件和侦听器的内容

一个人就是没有眼睛，也可以看见这世界的丑恶。用你的耳朵瞧着吧……

——威廉·莎士比亚，《李尔王》

7.5.1　WindowListener接口

将按钮放入一个窗口中时，将窗口自身作为按钮侦听器类。另一方面，希望窗口侦听器对关闭窗口事件进行响应时，就将窗口侦听器作为一个单独的、名为WindowDestroyer的类（或者一个名为InnerDestroyer的内部类）。本节将展示如何使窗口自身成为窗口侦听器。

窗口通过实现名为ActionListener的接口，已经使其自身成为一个按钮侦听器了。还可以用类似的方法使用一个名为WindowListener的接口。例如，在图5-14中，通过下列形式添加了implements ActionListener，使窗口类ButtonDemo成为了一个动作侦听器：

```
public class ButtonDemo extends JFrame implements ActionListener
```

要使ButtonDemo成为一个窗口侦听器而不是动作侦听器，定义就应该以下列代码行开始。

```
public class ButtonDemo extends JFrame implements WindowListener
```

更可能出现的情况是，希望ButtonDemo既成为动作侦听器，又成为窗口侦听器，那就应该以下列代码行作为开始。

```
public class ButtonDemo extends JFrame implements ActionListener, WindowListener
```

注意，当一个类实现多个接口时，只使用一次关键字implements，并用逗号分隔接口的名字。

避免将一个窗口作为其自身的窗口侦听器的原因是，当一个类实现一个接口，比如ActionListener或WindowListener接口时，必须要包含那个接口指定的所有方法的定义。接口ActionListener只有一个方法actionPerformed，在这种情况下，这个要求并不是很棘手。但是，图7-15给出了接口WindowListener的7个方法。这些方法是第5章讨论过的WindowAdapter类的10个方法中的前7个。如果一个类实现了WindowListener接口，就必须包含所有这7个方法的定义。如果不是所有方法都需要，就可以将不需要的那些方法的主体定义为空，如下所示：

```
public void windowDeiconified(WindowEvent e)
{}
```

尽管这种要求实现起来并不困难，但如果只需要WindowListener接口中的一个或两个方法，这种要求就有点儿烦人了。

```
public void windowOpened(WindowEvent e)
```
窗口已经被打开时调用。

```
public void windowClosing(WindowEvent e)
```
在关闭一个窗口的过程中调用。点击关闭窗口按钮会引发对这个方法的调用。

```
public void windowClosed(WindowEvent e)
```
窗口已经被关闭时调用。

```
public void windowIconified(WindowEvent e)
```
窗口被图标化时调用。点击JFrame上的最小化按钮时，窗口就被图标化了。

```
public void windowDeiconified(WindowEvent e)
```
窗口解除图标化时调用。激活一个最小化窗口时，就解除了它的图标化。

```
public void windowActivated(WindowEvent e)
```
窗口被激活时调用。在一个窗口中点击时，它就成了活动窗口。其他动作也可以激活一个窗口。

```
public void windowDeactivated(WindowEvent e)
```
窗口被解除激活时调用。一个窗口被激活时，所有其他的窗口都是解除激活的。其他动作也可以解除一个窗口的激活。

图7-15 WindowListener接口中的方法

使用WindowListener接口有一些好处。如果将窗口类作为JFrame类的派生类，并使其实现WindowListener接口，那么，在窗口侦听器类的内部调用窗口类中的方法就很简单了（因为它们在同一个类中）。

例如，图7-16中的程序就是第5章中给出的程序（图5-14）的变体。在图7-16中，GUI类是其自身的窗口侦听器。这就使它可以在方法windowClosing中拥有下列调用：

```
this.dispose();
```

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class WindowListenerDemo extends JFrame
                    implements ActionListener, WindowListener
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;

public static void main(String[] args)
{
    WindowListenerDemo demoWindow = new WindowListenerDemo();
    demoWindow.setVisible(true);
}

public WindowListenerDemo()
{
    setSize(WIDTH, HEIGHT);

    addWindowListener(this);
```

这个版本不需要WindowDestroyer类或任何类似的类。

图7-16 窗口侦听器

```
        setTitle("Window Listener Demonstration");
        Container content = getContentPane();
        content.setBackground(Color.BLUE);

        content.setLayout(new FlowLayout());

        JButton stopButton = new JButton("Red");
        stopButton.addActionListener(this);
        content.add(stopButton);

        JButton goButton = new JButton("Green");
        goButton.addActionListener(this);
        content.add(goButton);
    }

    public void actionPerformed(ActionEvent e)
    {
       Container content = getContentPane();
       if (e.getActionCommand().equals("Red"))
           content.setBackground(Color.RED);
       else if (e.getActionCommand().equals("Green"))
           content.setBackground(Color.GREEN);
       else
           System.out.println("Error in WindowListenerDemo.");
    }

    public void windowOpened(WindowEvent e)
    {}

    public void windowClosing(WindowEvent e)
    {
        this.dispose();
        System.exit(0);
    }

    public void windowClosed(WindowEvent e)
    {}

    public void windowIconified(WindowEvent e)
    {}

    public void windowDeiconified(WindowEvent e)
    {}

    public void windowActivated(WindowEvent e)
    {}

    public void windowDeactivated(WindowEvent e)
    {}
}
```

得到的GUI

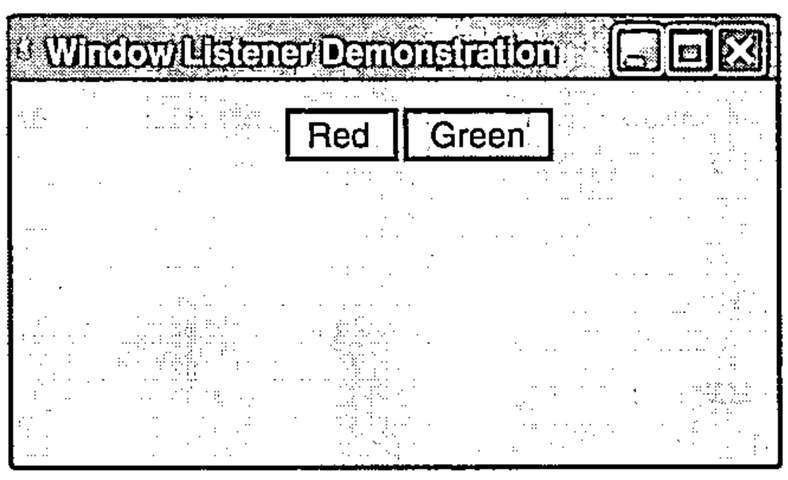


图7-16　（续）

方法dispose是JFrame类中的一个方法，WindowListenerDemo类是JFrame类的一个派生类，因此它继承了方法dispose。方法dispose释放了窗口使用的所有资源。在这么简单的一个程序中，并不真正需要调用dispose，因为程序终止时资源会被自动地释放，但是如果程序有多个窗口，只是要关闭这一个窗口，而使程序运行更长的时间，这时对dispose的调用（不调用System.exit）就显得很有用了。使用dispose只是想列举一个简单的例子。（在Java提示“对关闭窗口按钮编程”中，提供了一个更实际的dispose方法应用。）

为什么既要有WindowListener又要有WindowAdapter呢？原因在于WindowAdapter完全是为了便于程序员使用而提供的WindowListener的一个变体。WindowAdapter是一个实现了WindowListener接口的类，它为每个方法都给出了一个空主体。这样的话，定义WindowAdapter的派生类时，就不需要在派生类中放入那些空定义了。那为什么不总是使用WindowAdapter呢？通常，都希望一个侦听器类是某个类（比如JFrame，以及一个窗口侦听器）的派生类。一个类不能是两个类（比如JFrame和WindowAdapter）的派生类。因此在这种情况下，要将类作为JFrame类的派生类，并使其实现WindowListener接口。一个类只能是一个类的派生类，但它还可以实现一个或多个接口。

快速参考：WindowListener接口

可以像第5章讨论的那样，将一个类作为WindowAdapter类的派生类使其成为窗口侦听器，也可以通过实现WindowListener接口使其成为窗口侦听器。作为其自身关闭窗口按钮的窗口侦听器的JFrame类是实现WindowListener接口的类的一种常见实例。但是，任意类型的类都可以实现WindowListener接口。不一定非得是JFrame类。

要让一个类实现WindowListener接口，这个类必须将下列代码放在类定义的起始处（如果这个类是一个JFrame类的话，就要放在extends JFrame之后）：

```
implements WindowListener
```

这个类还必须实现图7-15中给出的所有方法。

快速参考：dispose方法

JFrame类有一个名为dispose的方法，会在不结束程序的情况下关闭JFrame。调用dispose时，JFrame消耗的资源被返回以重新利用，这样JFrame就消失了，但程序并没有结束。通常会在拥有多个窗口的程序中使用方法dispose，以关闭一个窗口而不结束整个程序。

语法：

JFrame_Object.dispose();

*JFrame_Object*通常就是隐式的this。在图7-16中可以看到使用dispose的完整实例。

自测题

30. 定义一个类，并使其实现WindowListener接口时，必须定义哪些方法？如果并不想要某个方法执行什么特殊动作的话，该怎么办？
31. 一个类可以是多个类的派生类吗？
32. 一个类可以实现多个接口吗？

33. 如果希望一个Swing程序完全结束，可以调用方法System.exit。如果希望一个窗口消失，但不希望结束程序该怎么办呢？可以调用什么方法呢？

□ Java提示：对关闭窗口按钮编程

可以像图7-16那样，通过使用WindowListener接口对关闭窗口按钮进行编程，或者可以像大多数Swing类定义中所做的那样，使用方法WindowDestroyer，来结束一个Swing程序。但有的时候，当用户点击关闭窗口按钮时，会希望发生其他一些情况。

如果想对JFrame类的关闭窗口按钮编程，使其不去关闭窗口或结束程序，而是做其他一些事情，就应该将下列对方法setDefaultCloseOperation的调用添加到其构造器中去。

```
setDefaultCloseOperation(WindowConstants.DO_NOTHING_ON_CLOSE);
```

图7-17的类定义中使用了这样一个调用。那个类对关闭窗口按钮进行编程，使得当用户点击关闭窗口按钮时，出现第二个窗口，询问用户是否想结束程序。如果用户点击"No"，第二个窗口就消失，但第一个窗口会保留下来。如果用户点击"Yes"，程序结束，这两个窗口都消失。我们来看看图7-17类定义中的一些细节。

方法setDefaultCloseOperation完成了其名字所隐含的功能。它对用户点击关闭窗口按钮时的默认动作进行了设置。如果不调用setDefaultCloseOperation，默认的动作是窗口消失，但不结束程序。注意，仅仅重新对方法windowClosing编程并不能撤销默认的动作。如果对windowClosing重新编程，而不调用方法setDefaultCloseOperation，那么，当用户点击关闭窗口关闭按钮时，窗口可能会执行方法windowClosing中编写的内容，但窗口也会消失。如果不希望窗口消失，就必须先用方法setDefaultCloseOperation对默认的动作进行重置。

除了调用setDefaultCloseOperation之外，还要对（图7-17中）CloseWindowDemo类的关闭窗口按钮进行编程，通过CloseWindowDemo构造器中的下列代码，将内部类InnerDestroyer注册为一个窗口侦听器：

```
addWindowListener(new InnerDestroyer());
```

要想弄清楚对哪些动作进行了编程，就要去看看内部类InnerDestroyer中方法windowClosing的定义。现将那个方法定义复制如下：

```
public void windowClosing(WindowEvent e)
{
    ConfirmWindow askWindow = new ConfirmWindow();
    askWindow.setVisible(true);
}
```

用户点击外部类CloseWindowDemo的关闭窗口按钮时，会调用windowClosing方法。然后，这个方法会创建一个ConfirmWindow类的、名为askWindow的对象，并使其成为可见的。

ConfirmWindow类的actionPerformed方法对窗口askWindow的行为进行了描述。相关代码如下所示：

```
if (e.getActionCommand().equals("Yes"))
   System.exit(0);
else if (e.getActionCommand().equals("No"))
   dispose(); //Destroys only the ConfirmWindow.
```

如前面的代码所示，如果用户点击按钮"Yes"，程序结束，所有窗口消失。如果用户点击按钮"No"，dispose调用会关闭第二个窗口，但第一个窗口会保留下来。

（ConfirmWindow类是CloseWindowDemo类的内部类。这就使得CloseWindowDemo类成为自包含的。但是，如果将ConfirmWindow类作为CloseWindowDemo类之外一个独立的类来定义，程序的表现会是完全相同的。）

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

/**
 Demonstration of programming the close-window button.
*/
public class CloseWindowDemo extends JFrame
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;

    public static void main(String[] args)
    {
        CloseWindowDemo gui = new CloseWindowDemo();
        gui.setVisible(true);
    }

    public CloseWindowDemo()
    {
        setSize(WIDTH, HEIGHT);
        setDefaultCloseOperation(WindowConstants.DO_NOTHING_ON_CLOSE);
        addWindowListener(new InnerDestroyer());
        setTitle("Close Window Demo");
        Container contentPane = getContentPane();
        contentPane.setLayout(new BorderLayout());

        JLabel message = new JLabel("Please don't click that button.");
        contentPane.add(message, BorderLayout.CENTER);
    }

    //Displays a window that checks if the user wants to exit.
    private class InnerDestroyer extends WindowAdapter
    {
        public void windowClosing(WindowEvent e)
        {
            ConfirmWindow askWindow = new ConfirmWindow();
            askWindow.setVisible(true);
        }
    }

    //Designed to be used with the inner class InnerDestroyer.
    //Checks if the user wants to exit.
    private class ConfirmWindow extends JFrame implements ActionListener
    {
        public static final int WIDTH = 200;
        public static final int HEIGHT = 100;

        public ConfirmWindow()
        {
            setSize(WIDTH, HEIGHT);
            Container confirmContent = getContentPane();
            confirmContent.setBackground(Color.WHITE);
```

一个内部类

另一个内部类

图7-17 对关闭窗口按钮编程

```
        confirmContent.setLayout(new BorderLayout());

        JLabel msgLabel = new JLabel("Are you sure you want to exit?");
        confirmContent.add(msgLabel, BorderLayout.CENTER);

        JPanel buttonPanel = new JPanel();
        buttonPanel.setLayout(new FlowLayout());

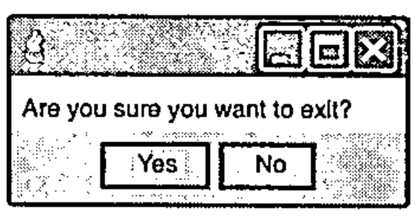
        JButton exitButton = new JButton("Yes");
        exitButton.addActionListener(this);
        buttonPanel.add(exitButton);

        JButton cancelButton = new JButton("No");
        cancelButton.addActionListener(this);
        buttonPanel.add(cancelButton);

        confirmContent.add(buttonPanel, BorderLayout.SOUTH);
    }

    public void actionPerformed(ActionEvent e)
    {
        if (e.getActionCommand().equals("Yes"))
            System.exit(0);
        else if (e.getActionCommand().equals("No"))
            dispose();//Destroys only the ConfirmWindow.
        else
            System.out.println("Error in Confirm Window.");
        }
    }
}
```

得到的GUI



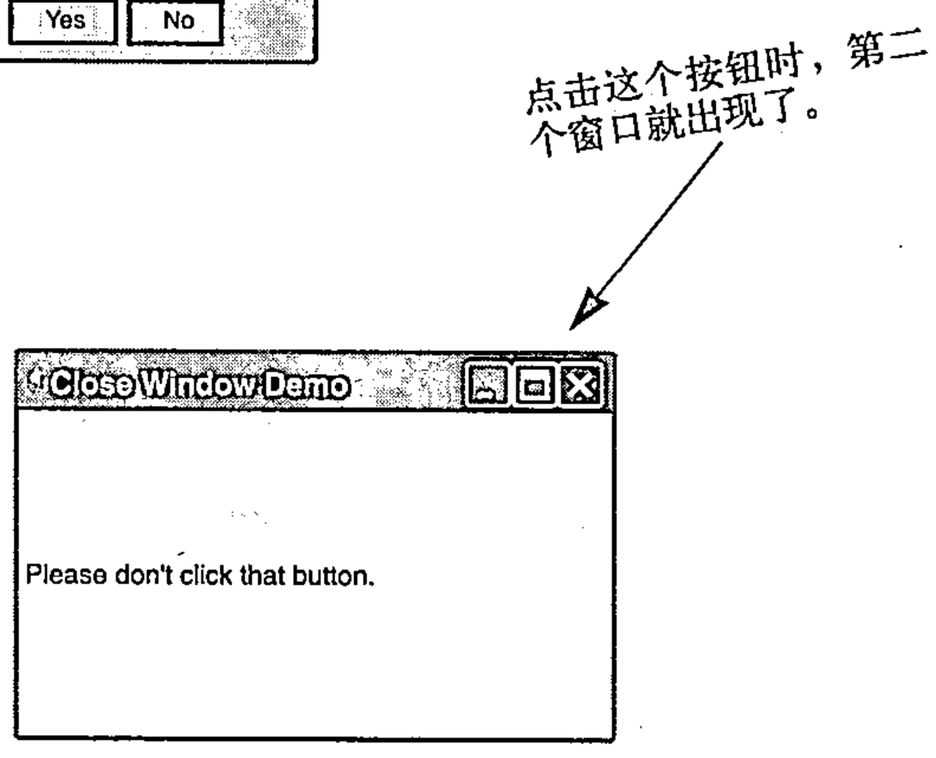


图7-17　（续）

快速参考：对关闭窗口按钮编程

如果想对JFrame的关闭窗口按钮编程，使其做一些比结束程序或使窗口消失更复杂的事情，就需要将下列调用添加到构造器中去（或者将其添加到某个与添加到构造器中具有相同效果的地方去）：

```
setDefaultCloseOperation(WindowConstants.DO_NOTHING_ON_CLOSE);
```

□ Java提示：更多关于setDefaultCloseOperation的内容

可以对方法setDefaultCloseOperation使用下列任意常量。

WindowConstants.DO_NOTHING_ON_CLOSE：什么都不做。程序员应该在已注册WindowListener对象的窗口类的windowClosing方法定义中对任何期望的动作进行编程。

WindowConstants.HIDE_ON_CLOSE：当调用任何已注册的WindowListener对象后，自动地隐藏窗口。

WindowConstants.DISPOSE_ON_CLOSE：当调用任何已注册的WindowListener对象后，自动地隐藏并关闭窗口。

JFrame.EXIT_ON_CLOSE：用System.exit方法退出应用程序。

有时，对JFrame的关闭窗口按钮编程时，唯一需要做的工作就是对setDefaultCloseOperation使用这些实参中的一个。

如果没有调用setDefaultCloseOperation，那么默认的动作就是WindowConstants.HIDE_ON_CLOSE执行的动作。

几乎可以对所有的Swing GUI重新编程，通过下列对setDefaultCloseOperation的调用，使其不带有窗口侦听器，而关闭窗口按钮仍然能够正确动作：

```
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

可以用这个调用来取代

```
addWindowListener(new WindowDestroyer());
```

但是，使用窗口侦听器的技术提供了更多的工具，这样就可以对任意其他动作进行编程了。

正如刚才所讨论的那样，在接下来的程序实例中，用

```
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

取代了WindowDestroyer。

自测题

34. 假设定义了一个视窗GUI，没有说明点击关闭窗口按钮时应如何处理。那么当用户点击关闭窗口按钮时，会发生什么情况呢？
35. 如果要对JFrame的关闭窗口按钮进行编程，使其不关闭窗口或结束程序，而做些其他的事情，需要向构造器中添加哪个方法调用？
36. 假设要将一个JFrame设置为窗口侦听器，使其可以侦听它自己的关闭窗口按钮。假设按下列形式定义了JFrame的windowClosing方法：

```
public void windowClosing(WindowEvent e)
{
    dispose();
}
```

应该将作为自测题35答案的那个调用添加到JFrame的构造器中去吗？

37. 什么是WindowConstants.DO_NOTHING_ON_CLOSE？它是一个方法调用？是一个类？还是其他什么东西？

编程示例：可见性可变的组件

图7-18给出了一个带有组件（在这个例子中，就是标签）的Swing GUI实例，其中的组件可以从可见的变成不可见的，然后再变回来。点击适当的按钮时，一个标签消失，另一个标签出现。点击标记为"Up"的按钮时，标签"Here I am up here!"会在窗口的顶端成为可见的，标签"Here I am down here!"消失。点击标记为"Down"的按钮时，标签"Here I am down here!"会在窗口的底端成为可见的，标签"Here I am up here!"消失。

这两个标签是在构造器中添加，并被设置为不可见的。下面重现了图7-18中的相关代码：

```
upLabel = new JLabel("Here I am up here!");
contentPane.add(upLabel, BorderLayout.NORTH);
upLabel.setVisible(false);
downLabel = new JLabel("Here I am down here!");
contentPane.add(downLabel, BorderLayout.SOUTH);
downLabel.setVisible(false);
```

注意，可以在整个GUI可见的情况下，使一个组件成为不可见的。标签upLabel和downLabel是私有实例变量，因此在构造器和方法actionPerformed中都可以引用它们。

在这个GUI中，点击一个特定按钮时，标签就变成可见或不可见的。例如，下列来自图7-18中actionPerformed方法的代码决定了点击带有文本"Up"的按钮时会发生什么情况：

```
if (e.getActionCommand().equals("Up"))
{
    upLabel.setVisible(true);
    downLabel.setVisible(false);
    validate();
}
```

这两条语句

```
upLabel.setVisible(true);
downLabel.setVisible(false);
```

使计算机上GUI中的upLabel成为可见的，downLabel成为不可见的。但是，要将这种变化显示在屏幕上还需要一个validate调用。

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class VisibilityDemo extends JFrame
                            implements ActionListener
{
    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;

    private JLabel upLabel;
    private JLabel downLabel;

    public VisibilityDemo()
    {
        setSize(WIDTH, HEIGHT);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setTitle("Visibility Demonstration");
        Container contentPane = getContentPane();
        contentPane.setLayout(new BorderLayout());
```

图7-18　不可见的标签

```
        contentPane.setBackground(Color.WHITE);

        upLabel = new JLabel("Here I am up here!");
        contentPane.add(upLabel, BorderLayout.NORTH);
        upLabel.setVisible(false);
        downLabel = new JLabel("Here I am down here!");
        contentPane.add(downLabel, BorderLayout.SOUTH);
        downLabel.setVisible(false);

        JPanel buttonPanel = new JPanel();
        buttonPanel.setBackground(Color.WHITE);
        buttonPanel.setLayout(new FlowLayout());
        JButton upButton = new JButton("Up");

        upButton.addActionListener(this);
        buttonPanel.add(upButton);
        JButton downButton = new JButton("Down");
        downButton.addActionListener(this);
        buttonPanel.add(downButton);
        contentPane.add(buttonPanel, BorderLayout.CENTER);
    }

    public void actionPerformed(ActionEvent e)
    {
       if (e.getActionCommand().equals("Up"))
       {
           upLabel.setVisible(true);
           downLabel.setVisible(false);
           validate();
       }
           else if (e.getActionCommand().equals("Down"))
           {
               downLabel.setVisible(true);
               upLabel.setVisible(false);
               validate();
           }
           else
               System.out.println("Error in VisibilityDemo interface.");
       }

       public static void main(String[] args)
       {
           VisibilityDemo demoGui = new VisibilityDemo();
           demoGui.setVisible(true);
       }
}
```

得到的GUI（同一个GUI的两种视图）

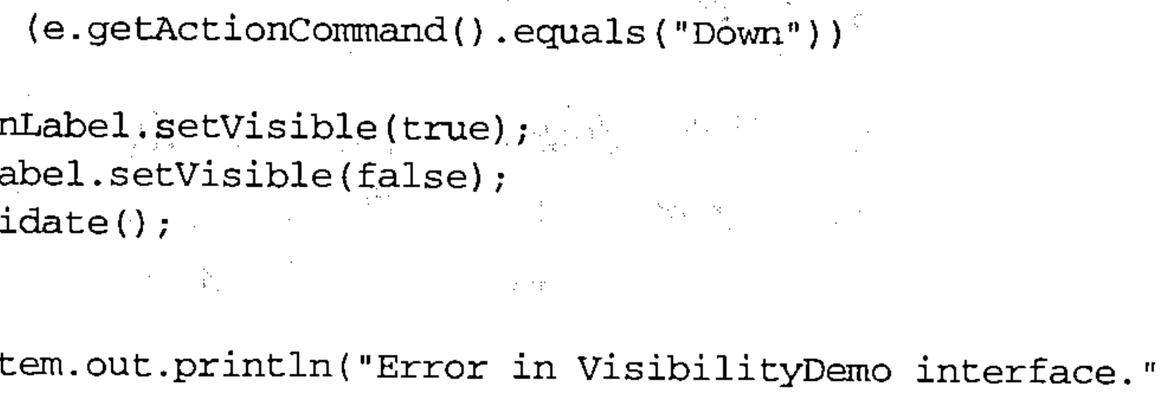


图7-18 （续）

每个容器类中都有方法validate。validate调用会使容器再次对其组件进行排列。这是一种使

组件中的变化实际出现在屏幕上的“更新”方法。对Swing GUI所作的一些简单修改，比如改变颜色或改变一个文本域中的文本，是自动发生的。其他修改，比如添加组件或改变可见性，则需要调用validate（或者其他一些“更新”方法。）尽管在不需要调用validate时错误地包含一个额外的validate调用会降低程序的效率，但这样做不会有什么其他的不利影响。

快速参考：validate方法

Container类，以及所有的容器类，都有一个名为validate的方法，这是一个用来更新容器的方法。validate调用会使容器再次对其组件进行排列，并将其重新显示在屏幕上。正如在正文中讨论的那样，对Swing GUI进行的某些修改需要validate调用，以更新屏幕内容，并将变化显示在屏幕上。

语法：

Container_Object.validate();

*Container_Object*通常就是个隐式的this。图7-18中有一个完整的validate使用实例。

自测题

38. 在图7-18中，通过使标签可见和不可见，将文本显示或隐藏起来。这个过程需要对validate方法的调用。假设用一个TextField类对象取代了标签，并修改了文本域中的文本，使其看起来好像一个单词消失，另一个单词出现了一样。要使文本域的变化出现在屏幕上，需要调用validate吗？
39. 重新编写图7-18中的程序，使得点击"Up"按钮时，"Up"按钮变成不可见的，"Down"按钮变成可见的，类似地，点击"Down"按钮时，"Down"变成不可见的，而"Up"按钮变成可见的。首次显示GUI时，这两个按钮都是可见的。（这个程序很长，但并不需要向图7-18的程序中添加很多代码。）

7.5.2　更多与更新GUI有关的细节

对Swing来说，对GUI视窗系统的大部分改动都是自动更新，以使其在屏幕上可见。这项操作是由一个名为**重画管理器**（repaint manager）的对象完成的。重画管理器是自动工作的，甚至都不需要知道它的存在。但是，有一些更新重画管理器是不会自动完成的。你已经看到了，如图7-18所示，当GUI改变一个组件的可见性时，需要调用方法validate来更新屏幕。

查阅Swing代码时经常会见到的另外两个更新方法为pack和repaint。方法pack会调整窗口的大小，通常会将其调整的更小，但更精确地说，是将窗口调整为近似于一个被称为优选尺寸（preferred size）的尺寸。（是的，优选尺寸是可以修改的，但在这几章中无法涵盖Swing库中的所有内容。）方法repaint会重画窗口。

本书没有足够的空间来讨论在屏幕上更新GUI的所有细节，但这几点备注会使那些更高级的书中的代码看起来不那么令人困惑。

7.6　Swing类的层次关系

所有的人都是会死的。
亚里士多德是人。
所以，亚里士多德是会死的。

——常见的三段论

按钮、菜单和抽象按钮

图7-19显示了Swing类的部分层次关系。注意，JButton类和JMenuItem类都是从

AbstractButton类中派生来的。JButton和JMenuItem类中所有的基本属性和方法都是从AbstractButton类中继承来的。这就是为什么JButton类的对象和JMenuItem类的对象那么相似的原因了。图7-3中列出的（除构造器之外的）JButton类的所有方法都是从AbstractButton类继承而来的。JMenuItem类也从AbstractButton类继承了所有这些方法，因此，JMenuItem类也具有图7-3中列出的（除构造器之外的）JButton类的所有方法。（这些方法中有一部分是AbstractButton类从JComponent类中继承来的，因此，有时可以看到某些方法会被列为“从JComponent继承”的。）

AbstractButton类是一个抽象类。正如第5章所说的那样，不能直接创建一个抽象类的对象。定义AbstractButton类的唯一原因就是，这样的话，JButton和JMenuItem这样的类就可以从它派生出来了。这种设置方式为JButton对象和JMenuItem对象提供了统一的视图，这就意味着Swing的设计者们不需要为了JButton类和JMenuItem类编写重复的代码了，只为AbstractButton类编写一次代码就行了。

正如原来所说的那样，JLabel类和JButton类也非常相似。但是，它们没有JButton类和JMenuItem类那么相似。在图7-19显示的Swing类层次关系中也反映出了这种相似度。这三个类都是JComponent类的子孙类，但只有JButton和JMenuItem是AbstractButton类的子孙类。

从图7-19显示的层次关系中还可以看出很多其他东西。注意，JMenu是从JMenuItem中派生出来的。这就意味着菜单也是一个菜单条目，这也是为什么可以像将普通的JMenuItem添加到JMenu中去那样，将一个JMenu添加到另一个JMenu中去的原因。这种情形使得嵌套式菜单成为可能。

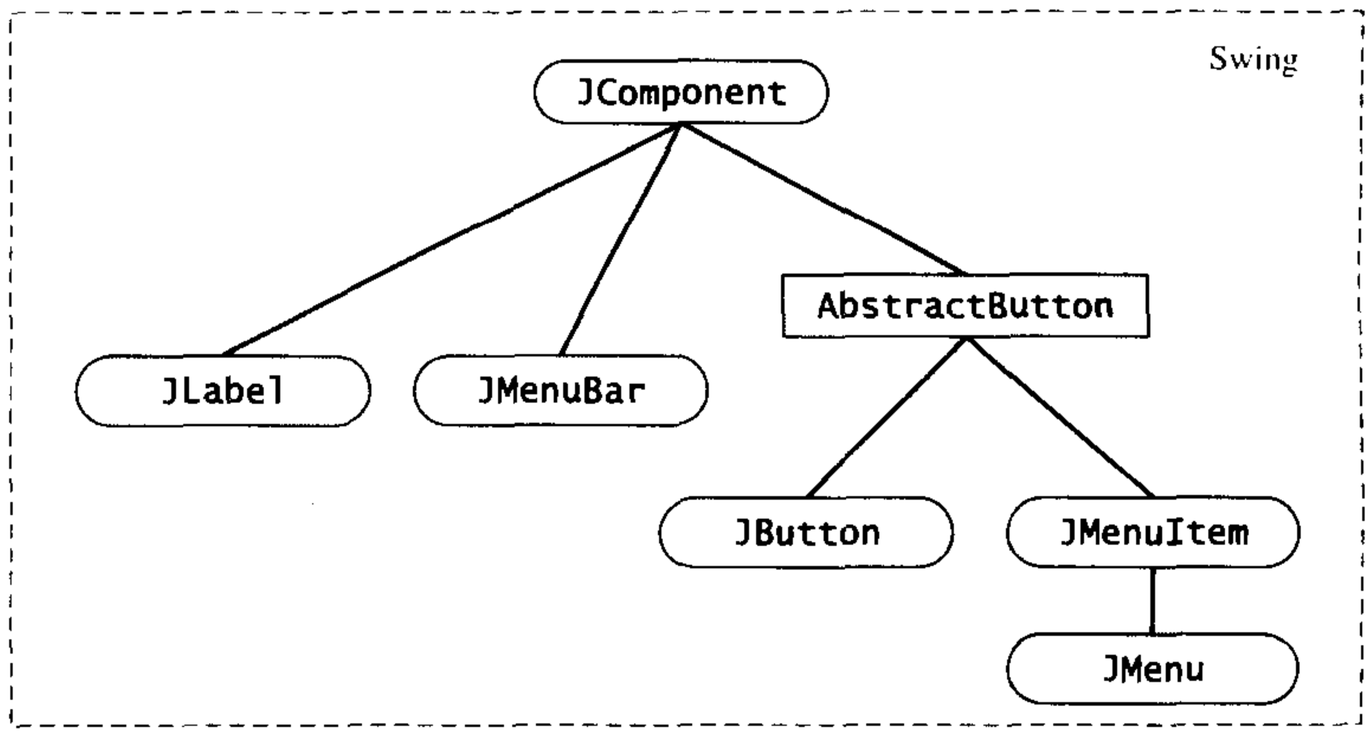


图7-19 Swing类的部分层次结构

还要注意，JMenuBar类是JComponent类的派生类。这就意味着可以像添加任何其他JComponent（比如一个标签或按钮）那样，向一个容器（比如一个面板或JFrame的内容面板）中，添加JMenuBar。因此，通过适当的布局管理器，想拥有多少JMenuBar，就可以拥有多少，而且几乎可以将它们放在容器中的任何位置上。

□ Java提示：更多JMenuItem类的构造器

正如在7.1节所讨论的那样，JMenuItem类拥有图7-3中为JButton类列出的（除构造器之外的）所有方法。JMenuItem类还拥有类似于图7-3中为JButton类列出的构造器。将那些构造器列在下面：

```
public JMenuItem();
```

创建一个不带有文本或图标的菜单条目。（大概稍后会对菜单条目使用setText或setIcon。）

```
public JMenuItem(String text);
```

创建一个带有文本text的菜单条目。

```
public JMenuItem(ImageIcon picture)
```

创建一个带有图标picture的菜单条目。

```
public JMenuItem(String text, ImageIcon picture)
```

创建一个既带有文本text，又带有图标picture的菜单条目。

□ Java提示：关于Swing类和方法的内容还有很多

在第5章和本章中，已经学习了很多关于Swing的知识，这些内容已经足够实现想实现的大部分基本功能了。但是，Swing这个话题太广泛了，无法涵盖其详尽的细节。通常一本写给有经验的Java程序员的、关于Swing的书都比这本书要厚，而且通常那样的书也是不完整的。如果觉得Swing中应该包含某些方法、类、或其他的实用工具，而在本书中未曾讲到，它们很可能是在Swing中某个地方的。在大多数情况下，只给出了一个类中最重要的方法，而没有给出它所有的方法。

Java 2所带的文档是研究新的Swing类及方法的一个很好的地方。（需要版本1.4或更高的版本。）如果已经下载了Java 2，那你的机器中很可能已经有这份文档了。如果没有这份文档，可以从Sun的Java Web站点上获取。在本书付印的时候，下载的Web地址如下所示：

http://java.sun.com/j2se/1.4/download.html

在线阅读的网址如下所示：

http://java.sun.com/j2se/1.4/docs/api

如果这些地址不对了，可以找到Sun的Java Web站点，并沿着那些超链找到文档。

自测题

40. 在Java中，下列代码行合法吗？

```
AbstractButton b = new AbstractButton();
```

41. JButton是AbstractButton吗？JLabel是AbstractButton吗？JMenuItem是AbstractButton吗？JMenu是AbstractButton吗？（提示：最后一个问题比其他三个都要难一点儿。）

小　结

□ 菜单条目是JMenuItem类的对象，菜单是JMenu类的对象，菜单条是JMenuBar类的对象。
□ 可以向JButton、JLabel和JMenuItem添加图标。
□ 可以用方法setJMenuBar将JMenuBar添加到一个JFrame中去。还可以像添加任何其他组件那样，用方法add将其添加进去。
□ 按钮和菜单条目都可以激发动作事件，因此，它们都应该有一个注册的动作侦听器，以响应这些事件。
□ 可以用JScrollPane类向文本区添加滚动条。
□ 可以通过让一个类实现WindowListener接口使其成为一个窗口侦听器类。
□ 可以对JFrame的关闭窗口按钮进行重新编程，以实现任何其他按钮可以实现的功能。但是，如果希望它完成除了关闭窗口或结束程序之外的其他功能，就必须使用setDefaultCloseOperation方法。
□ 如果改变了组件的可见性（或做了某些其他修改），就应该使用方法validate来更新GUI。

✓自测题答案

1. 点击JMenuItem会激发一个动作事件（即，一个ActionEvent类的对象）。这种情况与JButton一样。从这个角度来说，这两者之间没有区别。
2. 要修改JMenuItem的动作命令，可以像为JButton使用方法setActionCommand那样使用这个方法。
3. 是的，完全合法。实际上，向菜单中添加菜单是很常见的。这就是获得嵌套式菜单的方式。
4. 想要多少就可以有多少。用方法setJMenuBar只能添加一个，但可以用add方法向内容面板中添加任意多个。
5. setJMenuBar
6. `JMenuItem mItem = new JMenuItem("Choose Me!");`

   或者，使用等价的

   ```
   JMenuItem mItem;
   mItem = new JMenuItem("Choose Me!");
   ```
7. 菜单条目会激发ActionEvent类的事件。ActionEvent对象的侦听器是动作侦听器。注意，这种设置方式与JButton类对象的设置方式相同。
8. `m.add(mItem);`

   ```
   mBar.add(m);
   setJMenuBar(mBar);
   ```

   可以用下列代码取代setJMenuBar：

   ```
   getContentPane().add(mBar);
   ```

   如果你为没有提到MenusGalore类而感到奇怪的话，这是因为它是隐式命名的。所有这些都发生在一个名为MenusGalore的构造器中。
9. 列出了三种解决方案。还可以有其他的解决方案。第一种方案为：

   ```
   JButton myButton = new JButton("Push Me");
   ImageIcon aliceIcon = new ImageIcon("alice.gif");
   myButton.setIcon(aliceIcon);
   ```

   下列方案也能工作：

   ```
   JButton myButton = new JButton("Push Me");
   myButton.setIcon(new ImageIcon("alice.gif"));
   ```

   下列方案也能工作：

   ```
   ImageIcon aliceIcon = new ImageIcon("alice.gif");
   JButton myButton = new JButton(aliceIcon);
   myButton.setIcon("Push Me");
   ```
10. 列出了两种解决方案。还可以有其他的解决方案。第一种方案为：

    ```
    ImageIcon aliceIcon = new ImageIcon("alice.gif");
    JLabel alicePicture = new JLabel(aliceIcon);
    picturePanel.add(alicePicture);
    ```

    下列方案也能工作：

    ```
    picturePanel.add(new JLabel(new ImageIcon("alice.gif")));
    ```
11. ```
    ImageIcon aliceIcon = new ImageIcon("alice.gif");
    JButton aliceButton = new JButton(aliceIcon);
    aliceButton.setActionCommand("Curiouser and curiouser!");
    ```

    下列方案也能工作：

    ```
    JButton aliceButton = new JButton(new ImageIcon("alice.gif"));
    aliceButton.setActionCommand("Curiouser and curiouser!")
    ```
12. 不能。对Java来说，方法setMinimumSize只是建议性的。
13. 不是。可以调用这两个方法，调用任意一个而不调用另一个，或者两个都不调用。
    ```

14. 不是，这不是类型违规。JTextArea类是Component类的子孙类，因此每个JTextArea都是Component类型的。如果这个概念不太清楚的话，图5-18及相关文本会帮助你澄清这些问题的。
15. 可以向JScrollPane添加JTextArea。可以将JTextArea作为JScrollPane构造器的实参来实现这一点。
16. 是的，可以用EmptyBorder类在组件周围添加一些间隔。例如，下列代码行在组件b的左边和右边添加了30个像素的间隔：
```
b.setBorder(new EmptyBorder(0, 30, 0, 30));
```
17. 不行，需要使用
```
import javax.swing.border.*;
```
18. `myPanel.setLayout(new BoxLayout(myPanel, BoxLayout.Y_AXIS));`
19. 组件的添加方式没有区别，组件的顺序也没有区别，但组件间的间隔有区别。对BoxLayout管理器来说，不会在strut周围添加额外的间隔，但对FlowLayout管理器来说，它就会在其周围添加额外的间隔。还会出现组件间隔和尺寸上的其他一些区别。
20. `Component horizontalSpace = Box.createHorizontalStrut(20);`
21. strut组件在某个维度（宽度或高度）上有固定的尺寸，而glue组件在任意维度上的尺寸都是可以修改的。
22. `Box vBox = new Box(BoxLayout.Y_AXIS);`

    及与之等价的
```
Box vBox = Box.createVerticalBox();
```
23. 不用。Box容器会自动获得BoxLayout管理器。但确实需要在创建Box容器时说明它获得的是哪种类型的BoxLayout管理器（垂直或水平的）。
24. strut和glue只能用于BoxLayout管理器（包含Box容器）。如果将其用于其他布局管理器，不会出现编译错误，但对其他布局管理器来说，它们的行为可能是不正确的。
25. 除了BoxLayout管理器之外，方法setHgap和setVgap可以用于本书中讨论的所有布局管理器。
26. 因为需要为布局管理器起个名字，这样就可以对显示出来的那张“牌”进行切换了，如方法actionPerformed定义中的下列代码行所示：
```
dealer.show(deckPanel, "red");
```
27. `dealer.next(deckPanel);`
28. `dealer.previous(deckPanel);`
29. 会显示最后一张牌。
30. 必须定义图7-15中给出的所有方法。而且，方法参数的数量和类型（以及顺序）都必须与图7-15所示的完全一样。如果不想定义一个做什么特殊事情的方法，可以为其设置一个空的主体，如下所示：
```
public void windowIconified(WindowEvent e)
{}
```
31. 不行。
32. 可以，只要在关键字implements后面列出所有的接口名，并以逗号分隔它们就行了，如下所示：
```
public class WindowListenerDemo extends JFrame
                                implements ActionListener, WindowListener
```
    还需要实现所有接口中的所有方法。
33. 可以用希望消失的那个对象作为调用对象来调用dispose()。
34. 窗口消失，但程序不会结束。如果程序中没有其他窗口的话，这会是个问题，因为这样就没什么好办法来终止程序了。
35. `setDefaultCloseOperation(WindowConstants.DO_NOTHING_ON_CLOSE);`

36. 不用，不需要这么做。（尽管把它加上，也不会有什么严重的问题。）

37. 它是在WindowConstants类中定义的一个名为DO_NOTHING_ON_CLOSE的常量。应该可以根据它的书写方式和使用方式将其区分出来。（顺便说一下，WindowConstants类位于javax.swing包中。）因此，要使用这个类（以及这个常量），需要的唯一一条import语句为。

```
import javax.swing.*;
```

38. 不用。用来设置文本域或文本区中文本的方法setText会自动更新GUI。

39. 需要为那些按钮起名字。因此要添加下面两个实例变量：

```
private JButton upButton;
private JButton downButton;
```

对构造器所要做的唯一的修改就是用

```
upButton = new JButton("Up");
```

取代

```
JButton upButton = new JButton("Up");
```

并用

```
downButton = new JButton("Down");
```

取代

```
JButton downButton = new JButton("Down");
```

另外一个变化就是将方法actionPerformed中的嵌套if-then-else语句改成下列形式：

```
if (e.getActionCommand().equals("Up"))
{
    upLabel.setVisible(true);
    downLabel.setVisible(false);
    upButton.setVisible(false);
    downButton.setVisible(true);
    validate();
}
else if(e.getActionCommand().equals("Down"))
{
    downLabel.setVisible(true);
    upLabel.setVisible(false);
    downButton.setVisible(false);
    upButton.setVisible(true);
    validate();
}
else
    System.out.println("Error in VisibilityDemoExercise interface.");
```

完整的程序可以在Web上的文件VisibilityDemoExercise.java中找到。

40. 不，这是不合法的。AbstractButton是一个抽象类，不能直接创建一个抽象类的对象。如果想要AbstractButton对象，它必须是一个更具体的类的对象，比如JButton或JMenuItem类的对象。

41. JButton是一个AbstractButton。JLabel不是一个AbstractButton。JMenuItem是一个AbstractButton。JMenu是一个AbstractButton。前三个答案可以直接从图7-19中得到。要弄清楚JMenu是一个AbstractButton，可以回想一下，JMenu是一个JMenuItem，而JMenuItem是一个AbstractButton。但是，不总是能对JMenu使用AbstractButton所有的方法。因此，JMenu不是“典型的”AbstractButton。

○ 编程项目

1. 编写一个GUI，使用户可以选取图标。它应该有一个名为Icons的菜单，提供三个选项：Smiley

Face、Duke Waving和Duke Standing。用户选择一个菜单条目时，显示相应的图标（不显示其他图标）。首次显示GUI时，没有图标是可见的。Web上的源代码中有这些图标的图片文件。

2. 通过下列所有途径增强图7-1中备注保存器GUI的功能。
   - 添加另一个名为Scroll Bars的菜单。
   - 名为Scroll Bars的菜单要为用户提供三个选项：Never、Always和As Needed。用户做了选择之后，根据用户的选择来显示滚动条。
   - 用户点击关闭窗口按钮时，像图7-17那样，弹出另一个窗口，并询问用户是否确定要结束程序。
3. 增强第2章编程项目7编写的程序的功能，给它一个GUI界面，允许用户向一个文件写入或从其中读出PetRecord类型的记录（《Java程序设计与问题解决：基础篇》中第5章的图5-20）。要包含一个菜单，菜单中有一些选项，用以创建新文件、读入一个已存在的文件、或者退出。创建两个名为dogPetRecords和catPetRecords的PetRecord文件，并将其列在读选项下的一个子菜单中。（提示：可以用这个程序来创建dogPetRecords和catPetRecords文件。这个项目需要两个测试阶段，一个在创建dogPetRecords和catPetRecords之前，一个在此之后，但这并不是主要问题。）
4. 编写一个GUI，使用户可以选取边界。要包含一个名为Borders的、提供了三个选项——斜角边界（beveled border）、蚀刻边界（etched border），和线边界（line border）——的菜单，以及带有下列选项的子菜单。
   - 斜角边界选项：升高或降低。
   - 蚀刻边界选项：升高或降低。
   - 线边界选项：小、中或大。
   - 线边界的每个子菜单都应该有一个带有三个颜色选项黑、红和蓝的子菜单。

   将边界放在一个JLabel周围，这个JLabel中包含了描述边界的文本，比如"Raised Border"、"Lowered Etched Border"等。将蚀刻边界选项的高亮和阴影颜色固定设置为任意喜欢的颜色，并将小的线边界设置为5个像素宽，中等的设为10个像素宽，大的设置为20个像素宽。
5. 重新或首次实现本章的编程项目3，但这次，使用BoxLayout管理器（或者Box容器类）来创建GUI，并放入两个按钮，一个用来向文件写入，另一个用来从文件读出，这两个按钮垂直排列在左边。
6. 编写一个实现了WindowListener接口的“框架”GUI程序。为图7-15中所列的每个方法编写代码，用以简单地打印一条消息说明发生的是哪个事件。在文本域中将消息打印出来。注意，点击关闭窗口按钮时程序不会结束（只是向文本域发送一条消息说明windowClosing方法被调用了）。要包含一个标记为Exit的按钮，用户点击它就可以结束程序了。

TURING 图灵计算机科学丛书

PEARSON Prentice Hall

Java程序设计与问题解决

高级篇（第4版）

Java: An Introduction to Problem Solving and Programming

Fourth Edition

[美] Walter Savitch 著
陈 涓 赵振平 译



人民邮电出版社
POSTS & TELECOM PRESS



Java程序设计与问题解决
高级篇（第4版）

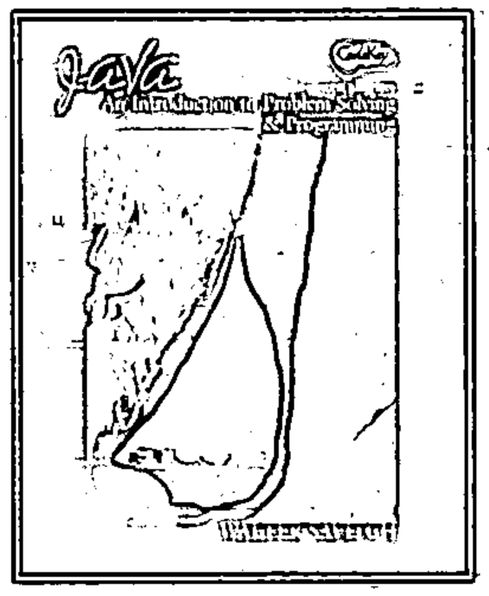
“本书作者的教学功力深厚。书中对面向对象概念的阐述堪称完美。包括Deitel、Horton、Schildt在内的许多作者都没有达到这种水平。”

——Amazon.com

本书是一部享有盛誉的Java教程，已被全球包括哈佛大学、普度大学、得克萨斯大学奥斯汀分校、密歇根大学、加州大学洛杉矶分校等名校在内的200多所高校采用为教材。

书中作者在讲授Java程序设计语言基础知识的同时，还将问题求解的技能、编程技巧和良好的编程实践融会其中。流畅易读的语言，透彻清晰的叙述，精心编排的版式和内容，充分体现了作者数十年计算机教学的经验和功力。

新版覆盖了Java 5.0。书中内容主要包括异常处理、流和文件I/O、动态数据结构与泛型，以及递归等高级主题，其中还包含丰富的自测题、编程技巧提示、编程项目和常见编程错误。而且内容组织方式非常灵活，可以根据实际教学需要调整授课内容和讲授顺序，适合各种教学方式，也是极好的自学教程。



Java
An Introduction to Problem Solving and Programming
Fourth Edition



Walter Savitch

[redacted]

作之一。

PEARSON Prentice Hall

2007

www.PearsonEd.com

ISBN 978-7-115-16922-8

9 787115 169228 >

ISBN 978-7-115-16922-8/TP

定价：49.00 元

[redacted] http://www.turingbook.com

[redacted] uringbook.com

上架建议 计算机/程序设计/Java

人民邮电出版社网址 www.ptpress.com.cn

PEARSON Education

写给教师的前言

本书是为程序设计及计算机科学方面的第一门课程而撰写的。书中涵盖了一些编程技巧，以及Java程序设计语言的基本概念。适用于短到半学年的课程，或长到一个学年的课程。学生不需要事先具备编程经验，除了一点代数知识外，也不需要更多的数学知识。本书也可以在为那些已经学过其他程序设计课程的学生开设的Java课程中使用。

本书只使用了Java中的标准类（这些类是Java的一部分），不需要额外的类。

本书所有代码都用Sun公司的Java JDK 5.0 beta 2版测试过。为了保持与本书的全面兼容，使用的Java必须是5.0版或更高的版本[①]。

本版中的修改

如果没有用过本书的第3版，就可以跳过本节。如果用过第3版，本节会告诉你第4版和第3版有哪些不同。

对教师来说，从第3版到第4版的转换是很容易的。在讲授相同的课程时，可以以相同的顺序讲授基本上相同的主题，只是在涵盖的材料上有极少量的修改。这个版本最大的修改就是用Java 5.0版中提供的新的Scanner类取代SavitchIn类[②]。如果想对课程做进一步的修改，本版中还包含了图形编程补充小节，可以选择尽早开始讲述图形编程的内容。本版还添加了对带有类型参数的泛型编程概念的介绍。

第1章和第2章[③]都以一个图形编程补充小节结束，这一小节涵盖了使用applet以及JFrame的图形应用程序和GUI。这些图形编程补充小节是可选的。如果在第2章之后，希望获得更多有关图形和GUI的知识，可以直接跳到第5章去，从该章开始对Swing进行了详细的讨论。

本版中其他的重要修改都是为了与Java 5.0版相匹配：为键盘输入使用了新的Scanner类；解释并使用了自动装箱和拆箱；对向量内容进行升级，以使用泛型类型参数。第3章给出了对类型参数和泛型的介绍。

最新的Java特性

本版已经进行了更新，使用了Java 5.0版的最新特性。特别地，为键盘输入使用了新的Scanner类，涵盖并使用了自动装箱和拆箱，用类型参数使用向量，还介绍了使用类型参数的泛型编程。编写本书时，我们用Sun公司的Java 5.0 beta 2版检测了所有代码。

---

① Sun公司修改了它的版本编号方式。5.0版原来被称为1.5版，在某些地方可能会发现它还被称为1.5版。

② SavitchIn类在附录中提供，而且源代码也可以从网上下载。但是，在本书的正文中没有使用SavitchIn。

③ 本书主要包含*Java™: An Introduction to Problem Solving and Programming, Fourth Edition*的第8章~第14章的内容，其第1章~第7章的内容包含在本书的姊妹篇《Java程序设计与问题解决：基础篇（第4版）》中，附录及索引内容可从图灵网站（www.turingbook.com）下载。——编者注

灵活性

如果你是一名教师，本书会适应你的教学方法，而不需要你来适应本书。本书没有严格规定所授课程必须涵盖的主题之间的次序，可以很容易地改变所要讲授的章节间的次序。每章开始的“预备知识”部分对重新组织材料的详情进行了解释。

本书没有使用专门的库，只使用标准Java库中的标准类。

更早的图形编程

本书前两章以可选的图形编程补充小节作为章节的结尾。这样，从课程一开始，你就可以选择是否讲授图形和GUI编程内容。图形编程补充小节的重点是applet，但也涵盖了用JFrame类构建的GUI。你也可以一开始就跳转到讲述GUI编程的主要章节（第5章～第7章）。或者继续讲到第2章的图形编程补充小节，这样就可以继续进行一些结合了图形编程方式和传统编程方式的练习。希望推迟讲授图形编程内容的教师可以推迟讲授或跳过这些图形编程补充小节。

问题求解及编程技术的内容

本书以讲解基本的问题求解方法及编程技术为设计目标，而不仅仅是一本有关Java语法的书。书中包含了大量的案例学习和编程提示，以及很多讲解了重要问题求解方法及编程技术的小节，比如循环设计技术、调试技术、编程风格、抽象数据类型和基本的面向对象编程技术，包括UML和事件驱动编程，以及使用类型参数的泛型编程。

面向对象技术与传统技术

任何实际讲授Java的课程都必须很早就讲授类的概念，因为Java中所有的概念都涉及了类。一个Java程序是一个类，字符串数据类型是一个类，甚至等于运算符（==）的行为都取决于它是在与类中的对象进行比较，还是在与一个较简单的数据进行比较。除非利用那些极长且复杂的“魔法公式”，否则，类是无法避免的。本书很早就引入了类的概念。所有关于类的基本信息，包括继承在内，都在《Java程序设计与问题解决：基础篇》的第7章结束之前进行了交代（即使略过该书的第6章也是如此）。但是，可以将某些与类有关的主题，包括继承，推迟到课程的后期或高级课程中介绍。

尽管本书很早就引入了类的概念，但它并没有忽略自顶向下的设计方法和循环设计技术这样的传统编程技术。这些较老的主题可能不再那么吸引人，但仍然是所有初学者必须掌握的内容。

泛型编程

类型参数的概念是在第3章学习向量时介绍的。在基本向量类型中包含了类型参数的就是最新版本的向量。然后，向学生们展示了如何定义自己的、包含了类型参数的类。

语言细节及示例代码

本书讲授的是编程技术，而不仅仅是Java语言。但是，对于一门不讲授编程语言的入门性编程课程，无论是学生还是教师都不会感到满意的。在你帮助学生克服对语言细节的恐惧之前，通常是无法将他们的注意力集中到更大的问题上的。因此，本书给出了对Java语言特性的完整解释，以及大量的示例代码。书中提供了完整的程序，以及作为示例的输入和输出。在很多情况下，除了正文中那些完整的示例之外，还可以从因特网上获得一些额外的完整示例。

自测题

每章都配有自测题。这些问题的难度级别跨度很大。有些只需要用一句话来回答，而有些则需要读者编写一个完整的、重要的程序。在每章的结尾都给出了所有自测题的完整答案，包括那些需要完整程序的题目。

经过课堂检验

书中所有资料都经过完整的课堂检验。根据检验结果对很多资料和表达方式进行了修改。

配套资料

可以从出版商或者因特网上获取以下的配套资料。

配套资源

在本书的配套网站http://www.prenhall.com/savitch上提供了本书的源代码、额外的编程示例以及下载Java编译器和编程环境的链接。

教师资源指南

教师工具中包含各章的教师资源指南，指南中包含了大量的授课提示、带有答案的测试题、很多编程练习的答案、PowerPoint幻灯片以及其他一些授课资源。教师可以与Prentice Hall销售代表联系，以获取访问教师网站的相关信息。可以通过网址http://www.prenhall.com与Prentice Hall联系，获取销售代表的名字和编号。

Walter Savitch
wsavitch@ucsd.edu
http://www.cse.ucsd.edu/users/savitch

写给学生的前言

本书的目的是讲授Java程序设计语言，不要求学生事先具备编程经验，除了一些简单的代数知识之外，也不需要更多的数学知识。为了最大程度地从本书获益，在机器上应该装有一个可用的Java版本，这样就可以练习书中给出的示例和技巧了。应该安装Java 5.0版（或更高的版本）。（Sun公司修改了它的版本编号方式。5.0版原来被称为1.5版，在某些地方可能会发现它仍然被称为1.5版。如果有一个名为“1.5版”的Java的副本，也应该可以。）

如果以前编写过程序

使用本书时，不要假设Java和习惯使用的编程语言是一样的。所有的语言都是不同的，它们之间的区别即使很小，也足以给你带来一些问题。

如果你以前曾经用C或C++编写过程序，那么，向Java的转换可能会比较麻烦。乍一看，Java好像和C或C++基本上是一样的。但是，Java和这些语言有很大的不同，你必须认识到这些区别。附录K对Java和C++进行了比较，以帮助你了解它们之间的区别。

正文中的程序代码

本书中所有的程序和其他软件示例都可以从本书的网站上下载，这样，不用将这些示例输入计算机，就可以用它们进行练习了。

获取Java环境

在本书的配套网站上有可以下载Java编译器和编程环境的链接。对于初学者来说，推荐使用Sun公司的Java SDK作为Java编译器及相关软件，推荐TextPad作为简单的编辑环境来编写Java代码。下载Java SDK时，一定要下载5.0版或更新的版本。

本书的配套网站

在本书的配套网站http://www.prenhall.com/savitch上提供了本书中的源代码、额外的编程示例以及下载Java编译器和编程环境的链接。

自测题

每章中都配有大量的自测题。在每章的结尾都给出了这些问题的完整答案。要对所学的知识进行练习，最好的方法就是在看答案之前做这些自测题。

本书也是一本参考书

除了将本书作为教材之外，还可以并且应该将它当作一本参考书。“快速参考”中会有一个很短的条目，给出了所有与那个主题有关的基本内容。可以用这种方法来查看Java语言及编程技术方面的细节。

每章的“小结”都对该章的主要内容进行了简要的总结。可以根据这些小结中的内容对该章进行复习，或者根据这些内容来查看Java语言的一些细节。

我们期待读者的意见

本书是为你编写的，我们期望听到你对本书的任何评论。你可以用电子邮件通过下面的地址与我联系：

wsavitch@ucsd.edu

遗憾的是，不能提供编程练习的答案。只有采用本书作为教材的教师才可以从出版商那里获取部分答案。要想获取编程练习方面的帮助，只能与你的老师联系。但是，每章的结尾都有所有自测题的答案。

Walter Savitch
http://www.cse.ucsd.edu/users/savitch

致　　谢

我要感谢我所在的加州大学圣迭戈分校的计算机科学与工程系，本书的很多素材在那里得到了检验。在我授课的班级中，很多学生都帮助我对本书的初稿进行了校对。这些学生的建议和在课堂上试用本书的教师的建议对形成本书的最终版本有着极大的帮助。我要特别感谢加州大学萨克拉门托分校的Carole McNamee和加州大学圣迭戈分校的Paul Kube，他们对本书的早期版本提出了详细的反馈意见，并在课堂进行试用。我还要特别感谢杨百翰大学的Robert Burton，他为本版的草稿准备了详细的分析材料。他们的建议对本书的最终成型提供了巨大的帮助。

我要感谢所有花时间阅读本书早期版本书稿的审阅者。他们提供了非常宝贵的、详细的评论和建议，这个新的版本也继续从这些评论和建议中受益。按照字母顺序，这些审阅者是：

Jim Buffenbarger——爱达荷州立大学（Idaho State University）

Robert P. Burton——杨百翰大学（Brigham Young University）

Steve Cater——凯特林大学（Kettering University）

Martin Chelten——穆尔帕克社区学院（Moorpark Community College）

Michael Clancy——加州大学伯克利分校（University of California, Berkeley）

Tom Cortina——纽约州立大学石溪分校（SUNY, Stony Brook）

Prasun Dewan——北卡罗来纳大学（University of North Carolina）

Laird Dornan——Sun公司（Sun Microsystems, Inc.）

H.E. Dunsmore——普度大学拉斐特分校（Purdue University, Lafayette）

Adel Elmaghraby——路易斯维尔大学（University of Louisville）

Gobi Gopinath——萨福克县社区学院（Suffolk County Community College）

Le Gruenwald——俄克拉何马大学（University of Oklahoma）

Gopal Gupta——得克萨斯大学达拉斯分校（University of Texas, Dallas）

Ricci Heishman ——北弗吉尼亚社区学院（North Virginia Community College）

Robert Herrmann——Sun公司（Sun Microsystems, Inc., Java Soft）

Robert Holloway——威斯康星大学麦迪逊分校（University of Wisconsin, Madison）

Lily Hou——卡内基-梅隆大学（Carnegie Mellon University）

Rob Kelly——纽约州立大学石溪分校（SUNY, Stony Brook）

Michele Kleckner——埃隆学院（Elon College）

Mike Litman——西伊利诺伊大学（Western Illinois University）

Blayne Mayfield——俄克拉何马州立大学（Oklahoma State University）
John Motil——加利福尼亚州立大学北岭分校（California State University, Northridge）
Michael Olan——斯托克顿州立大学（Stockton State）
James Roberts——卡内基-梅隆大学（Carnegie Mellon University）
Alan Saleski——罗耀拉大学芝加哥分校（Loyola University, Chicago）
Nan C. Schaller——罗切斯特理工学院（Rochester Institute of Technology）
Ryan Shoemaker——Sun公司（Sun Microsystems, Inc.）
Ken Slonneger——艾奥瓦大学（University of Iowa）
Donald E. Smith——罗杰斯大学（Rutgers University）
Boyd Trolinger——巴特学院（Butte College）
Subramanian Vijayarangam——麻省大学洛厄尔分校（University of Massachusetts, Lowell）

我还要感谢本版的审阅者们：

Robert Burton——杨百翰大学（Brigham Young University）
Ed Gellenbeck——中华盛顿大学（Central Washington University）
Anthony Larrain——德保大学（Depaul University）
Michael Long——加利福尼亚州立大学（California State University）
Drew McDermott——耶鲁大学（Yale University）
Ken Slonneger——艾奥瓦大学（University of Iowa）
Navabi Tadayon——亚利桑那州立大学（Arizona State University）
Richard Whitehouse——亚利桑那州立大学（Arizona State University）
Michael Young——俄勒冈大学（University of Oregon）

我还要感谢Prentice Hall所有负责组织本书审阅和出版工作的人员。我尤其要感谢Toni Holm、Patrick Lindner和Irwin Zucker。我要特别感谢我的出版人Alan Apt，感谢他在本书的编写和出版过程中提供的非常宝贵的支持和建议。这些优秀的人都做得很棒！

Lew Rakocy编写了教师指南的编程答案，Robert Burton制作了PowerPoint幻灯片。我要感谢他们细心的工作。

我要感谢Sun公司允许我在很多GUI例子中使用Duke图标。

Walter Savitch

版权声明

Authorized translation from the English language edition, entitled *Java™: An Introduction to Problem Solving and Programming, Fourth Edition*, 0131492020 by Walter Savitch, published by Pearson Education, Inc., publishing as Prentice Hall, Copyright © 1999, 2001, 2004, 2005 by Pearson Education, Inc.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from Pearson Education, Inc.

CHINESE SIMPLIFIED language edition published by PEARSON EDUCATION ASIA LTD. and POSTS & TELECOM PRESS Copyright © 2007.

本书中文简体字版由Pearson Education Asia Ltd. 授权人民邮电出版社独家出版。未经出版者书面许可，不得以任何方式复制或抄袭本书内容。

本书封面贴有Pearson Education（培生教育出版集团）激光防伪标签，无标签者不得销售。

版权所有，侵权必究。

图书在版编目（CIP）数据

Java 程序设计与问题解决：第 4 版．高级篇 /（美）萨维奇（Savitch, W. ）著；陈涓，赵振平译．—北京：人民邮电出版社，2007.12
（图灵计算机科学丛书）
ISBN 978-7-115-16922-8

Ⅰ．J… Ⅱ．①萨…②陈…③赵… Ⅲ．JAVA 程序—程序设计—教材 Ⅳ．TP312

中国版本图书馆 CIP 数据核字（2007）第 151549 号

内 容 提 要

本书是国外高校采用率很高的Java程序设计经典教材。书中不但讲解了Java的知识，还提供了一些编程技巧。本书主要讲述高级内容，包括异常处理、流和文件I/O、动态数据结构与泛型、递归等高级主题。本书使用了J2SE 5.0的最新特性，并且包含新的、可选讲的图形编程内容和贯穿全书的GUI材料。

本书可作为高等院校计算机及相关专业本科生或研究生程序设计课程的教材，也适合相关技术人员作为技术提高之用。

图灵计算机科学丛书

Java 程序设计与问题解决：高级篇（第 4 版）

◆ 著 [美] Walter Savitch
译 陈 涓 赵振平
责任编辑 杨海玲 刘 静

◆ 人民邮电出版社出版发行 北京市崇文区夕照寺街 14 号
邮编 100061 电子函件 315@ptpress.com.cn
网址 http://www.ptpress.com.cn
北京鸿佳印刷厂印刷
新华书店总店北京发行所经销

◆ 开本：787 × 1092 1/16
印张：21.25
字数：517 千字 2007 年 12 月第 1 版
印数：1–4 000 册 2007 年 12 月北京第 1 次印刷

著作权合同登记号 图字：01-2007-2669 号

ISBN 978-7-115-16922-8/TP

定价：49.00 元

读者服务热线：(010)88593802 印装质量热线：(010)67129223